

Revisit to CPE233 (Digital Design)

Authors:
WhisperingRock

Professor:

Class
Quarter
August 30, 2026

Contents

Contents	1
0.1 Hardware	2
0.1.1 HW1 - Program ROM	2
0.1.2 HW2 - Program Counter	5
0.1.3 HW3 - Register File	8
0.1.4 HW4 - ALU + ImmGen	9
0.1.5 HW5 - Real Memory and Branch	12
0.1.6 HW6 - Control Unit	15
0.1.7 HW7 - Otter Wrapper	26
0.1.8 HW8 - Interrupts	27
0.2 Software	34
0.2.1 SW1 - Intro to Assembly Lang	34
0.2.2 SW2 - Conditional Statements	43
0.2.3 SW3 - Loops	47
0.2.4 SW4 - Arrays	52
0.2.5 SW5 - Division and Stacks	58
0.2.6 SW6 - Subroutines	64
0.2.7 SW7 - Interrupts	70
0.3 Otter Code & Test Sims	72
0.3.1 Mux(s)	72
0.3.2 Program Counter and Additives	75
0.3.3 Register File	77
0.3.4 Arithmetic Logic Unit	80
0.3.5 Immediate Generator	85
0.3.6 Memory	87
0.3.7 Branch Address Generator	90
0.3.8 Branch Condition Generator	93
0.3.9 Control Unit Decoder	95
0.3.10 Control Unit Finite State Machine	104
0.3.11 MicroController	115
0.3.12 Seven Segment	120
0.3.13 Peripheral Fabric	124
0.3.14 Interrupts	130

Note : All assignments are in the CPE 233 Lab Manual.

0.1 Hardware

0.1.1 HW1 - Program ROM

Reverse Engineer’d Table

0x0000_0000:	11000537
0x0000_0004:	00a00f13
0x0000_0008:	00051783
0x0000_000C:	41e7da33
0x0000_0010:	00fa4633
0x0000_0014:	04c52023
0x0000_0018:	ff1ff06f

Figure 1: Which Table? This table

RISC-V Instruction Set Summary

<table><tr><td colspan="2">31:25</td><td colspan="2">24:20</td><td colspan="2">19:15</td><td colspan="2">14:12</td><td colspan="2">11:7</td><td colspan="2">6:0</td></tr><tr><td colspan="2">funct7</td><td>rs2</td><td>rs1</td><td colspan="2">funct3</td><td colspan="2">rd</td><td colspan="2">op</td><td colspan="2" rowspan="2">R-Type</td></tr><tr><td colspan="2">imm11,0</td><td></td><td>rs1</td><td colspan="2">funct3</td><td colspan="2">rd</td><td colspan="2">op</td></tr><tr><td colspan="2">imm11,5</td><td>rs2</td><td>rs1</td><td colspan="2">funct3</td><td colspan="2">imm4,0</td><td colspan="2">op</td><td colspan="2">I-Type</td></tr><tr><td colspan="2">imm12,10,5</td><td>rs2</td><td>rs1</td><td colspan="2">funct3</td><td colspan="2">imm4,1,11</td><td colspan="2">op</td><td colspan="2">S-Type</td></tr><tr><td colspan="2">imm31,12</td><td colspan="6"></td><td colspan="2">rd</td><td colspan="2">op</td><td>B-Type</td></tr><tr><td colspan="2">imm20,10,1,11,19,12</td><td colspan="6"></td><td colspan="2">rd</td><td colspan="2">op</td><td>U-Type</td></tr><tr><td colspan="2">fs3</td><td>funct2</td><td>fs2</td><td>fs1</td><td colspan="2">funct3</td><td colspan="2">fd</td><td colspan="2">op</td><td>J-Type</td></tr><tr><td colspan="2">5 bits</td><td colspan="2">2 bits</td><td colspan="2">5 bits</td><td colspan="2">3 bits</td><td colspan="2">5 bits</td><td colspan="2">7 bits</td><td>R4-Type</td></tr></table>										31:25		24:20		19:15		14:12		11:7		6:0		funct7		rs2	rs1	funct3		rd		op		R-Type		imm11,0			rs1	funct3		rd		op		imm11,5		rs2	rs1	funct3		imm4,0		op		I-Type		imm12,10,5		rs2	rs1	funct3		imm4,1,11		op		S-Type		imm31,12								rd		op		B-Type	imm20,10,1,11,19,12								rd		op		U-Type	fs3		funct2	fs2	fs1	funct3		fd		op		J-Type	5 bits		2 bits		5 bits		3 bits		5 bits		7 bits		R4-Type	- imm: signed immediate in imm_{11,0} - uimm: 5-bit unsigned immediate in imm_{4,0} - upimm: 20 upper bits of a 32-bit immediate, in imm_{31,12} - Address: memory address: rs1 + SignExt(imm_{11,0}) [Address]: data at memory location Address - BTA: branch target address: PC + SignExt(imm_{12,1}, 1'b0)) - JTA: jump target address: PC + SignExt(imm_{20,1}, 1'b0)) - label: text indicating instruction address - SignExt: value sign-extended to 32 bits - ZeroExt: value zero-extended to 32 bits - csr: control and status register
31:25		24:20		19:15		14:12		11:7		6:0																																																																																																													
funct7		rs2	rs1	funct3		rd		op		R-Type																																																																																																													
imm11,0			rs1	funct3		rd		op																																																																																																															
imm11,5		rs2	rs1	funct3		imm4,0		op		I-Type																																																																																																													
imm12,10,5		rs2	rs1	funct3		imm4,1,11		op		S-Type																																																																																																													
imm31,12								rd		op		B-Type																																																																																																											
imm20,10,1,11,19,12								rd		op		U-Type																																																																																																											
fs3		funct2	fs2	fs1	funct3		fd		op		J-Type																																																																																																												
5 bits		2 bits		5 bits		3 bits		5 bits		7 bits		R4-Type																																																																																																											
Figure B.1 RISC-V 32-bit instruction formats																																																																																																																							

Table B.1 RV32I: RISC-V integer instructions											
op	funct3	funct7	Type	Instruction		Description	Operation				
0000011 (3)	000	–	I	lb	rd, imm(rs1)	load byte	rd = SignExt([Address] _{7:0})				
0000011 (3)	001	–	I	lh	rd, imm(rs1)	load half	rd = SignExt([Address] _{15:0})				
0000011 (3)	010	–	I	lw	rd, imm(rs1)	load word	rd = [Address] _{31:0}				
0000011 (3)	100	–	I	lbu	rd, imm(rs1)	load byte unsigned	rd = ZeroExt([Address] _{7:0})				
0000011 (3)	101	–	I	lhu	rd, imm(rs1)	load half unsigned	rd = ZeroExt([Address] _{15:0})				
0010011 (19)	000	–	I	addi	rd, rs1, imm	add immediate	rd = rs1 + SignExt(imm)				
0010011 (19)	001	0000000*	I	slli	rd, rs1, uimm	shift left logical immediate	rd = rs1 << uimm				
0010011 (19)	010	–	I	slti	rd, rs1, imm	set less than immediate	rd = (rs1 < SignExt(imm))				
0010011 (19)	011	–	I	sltiu	rd, rs1, imm	set less than imm. unsigned	rd = (rs1 < SignExt(imm))				
0010011 (19)	100	–	I	xori	rd, rs1, imm	xor immediate	rd = rs1 ^ SignExt(imm)				
0010011 (19)	101	0000000*	I	srlri	rd, rs1, uimm	shift right logical immediate	rd = rs1 >> uimm				
0010011 (19)	101	0100000*	I	srai	rd, rs1, uimm	shift right arithmetic imm.	rd = rs1 >>> uimm				
0010011 (19)	110	–	I	ori	rd, rs1, imm	or immediate	rd = rs1 SignExt(imm)				
0010011 (19)	111	–	I	andi	rd, rs1, imm	and immediate	rd = rs1 & SignExt(imm)				
0010111 (23)	–	–	U	auipc	rd, upimm	add upper immediate to PC	rd = (upimm, 12'b0) + PC				
0100011 (35)	000	–	S	sb	rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}				
0100011 (35)	001	–	S	sh	rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}				
0100011 (35)	010	–	S	sw	rs2, imm(rs1)	store word	[Address] _{31:0} = rs2				
0110011 (51)	000	0000000	R	add	rd, rs1, rs2	add	rd = rs1 + rs2				
0110011 (51)	000	0100000	R	sub	rd, rs1, rs2	sub	rd = rs1 – rs2				
0110011 (51)	001	0000000	R	sll	rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}				
0110011 (51)	010	0000000	R	slt	rd, rs1, rs2	set less than	rd = (rs1 < rs2)				
0110011 (51)	011	0000000	R	sltu	rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)				
0110011 (51)	100	0000000	R	xor	rd, rs1, rs2	xor	rd = rs1 ^ rs2				
0110011 (51)	101	0000000	R	srl	rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}				
0110011 (51)	101	0100000	R	sra	rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}				
0110011 (51)	110	0000000	R	or	rd, rs1, rs2	or	rd = rs1 rs2				
0110011 (51)	111	0000000	R	and	rd, rs1, rs2	and	rd = rs1 & rs2				
0110111 (55)	–	–	U	lui	rd, upimm	load upper immediate	rd = (upimm, 12'b0)				
1100011 (99)	000	–	B	beq	rs1, rs2, label	branch if =	if (rs1 == rs2) PC = BTA				
1100011 (99)	001	–	B	bne	rs1, rs2, label	branch if ≠	if (rs1 ≠ rs2) PC = BTA				
1100011 (99)	100	–	B	blt	rs1, rs2, label	branch if <	if (rs1 < rs2) PC = BTA				
1100011 (99)	101	–	B	bge	rs1, rs2, label	branch if ≥	if (rs1 ≥ rs2) PC = BTA				
1100011 (99)	110	–	B	bltu	rs1, rs2, label	branch if < unsigned	if (rs1 < rs2) PC = BTA				
1100011 (99)	111	–	B	bgeu	rs1, rs2, label	branch if ≥ unsigned	if (rs1 ≥ rs2) PC = BTA				
1100111 (103)	000	–	I	jalr	rd, rs1, imm	jump and link register	PC = rs1 + SignExt(imm), rd = PC + 4				
1101111 (111)	–	–	J	jal	rd, label	jump and link	PC = JTA, rd = PC + 4				

*Encoded in instr1,259 the upper seven bits of the immediate field

ProgROM Addr	Machine Code	Type	Assembly Instruction
0000	0x1100 0537		
	0001 0001 0000 0000 0000 0101 0 011 0111	(55) U	lui rd, upimm
	0001 0001 0000 0000 0000 0101 0 011 0111	(55) U	lui x10, 0x11000
0004	0x00A0 0F13		
	0000 0000 1010 0000 0 000 1111 0 001 0011	(19)I	addi rd, rs1, imm
	0000 0000 1010 0000 0 000 1111 0 001 0011	(19)I	addi x30, x0, 0x00A
0008	0x0005 1783		
	0000 0000 0000 0101 0 001 0111 1 000 0011	(3)I	lh rd, imm(rs1)
	0000 0000 0000 0101 0 001 0111 1 000 0011	(3)I	lh x15, 0(x10)
000C	0x41E7 DA33		
	0100 000 1 1110 0111 1 101 1010 0 011 0011	(51)R	sra rd, rs1, rs2
	0100 000 1 1110 0111 1 101 1010 0 011 0011	(51)R	sra x20, x15, x30
0010	0x00FA 4633		
	0000 0000 1111 1010 0 100 0110 0 011 0011	(51)R	xor rd, rs1, rs2
	0000 000 0 1111 1010 0 100 0110 0 011 0011	(51)R	xor x12, x20, x15
0014	0x04C5 2023		
	0000 0100 1100 0101 0 010 0000 0 010 0011	(35)S	sw rs2, imm(rs1)
	0000 010 0 1100 0101 0 010 0000 0 010 0011	(35)S	sw x12, 0x040(x10)
0018	0xFF1F F06F		
	1111 1111 0001 1111 1111 0000 0 110 1111	(111)J	jal rd, label
	1111 1111 0001 1111 1111 0000 0 110 1111	(111)J	jal x0, -16

Table 1: Reverse Engineering *otter_memory.mem* segment

For the last command, some explanation is required here. Consider

- $Imm[20] = B31 = 1$
- $Imm[10 : 1] = B30 : B21 = 1111111000$
- $Imm[11] = B20 = 1$
- $Imm[19 : 12] = B19 : B12 = 11111111$
- $Imm[0]$ is always zero
- Therefore, $Imm = 0x1FFFF0$

To find the decimal representation, consider adding our large negative number to the next highest max.

$$\begin{aligned}
 2^{21} - 0x1FFFF0 &= 0x200000 - 0x1FFFF0 \\
 &= -0x10 \\
 &= -16
 \end{aligned}$$

BUT WAIT! jal doesn't take immediates, only labels.

True, our label must be 4 spaces prior to this jump - fill this portion in with a label in the asm and respect the spacing.

Assembly Code

Listing 1: Assembly of HW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Confirm decoded machine code (by hand)
4 #           functionality matches what the RARS compiler
5 #           produces.
6
7
8
9 .data
10 A: .word 0x1234    # dummy value to see change
11
12 .global _start
13
14 .text
15 .equ MMIO, 0x11000000
16
17 _start:
18 # ~~~~ init registers ~~~~
19 li t1, MMIO
20 lw t2, A
21 sw t2, 0(t1)
22
23 # ~~~~ decoded instr ~~~~

```

```
24  #lui x10, 0x11000
25  #addi x30, x0, 0x00A
26  #loop:
27  #lh x15, 0(x10)
28  #sra x20, x15, x30
29  #xor x12, x20, x15
30  #sw x12, 0x040(x10)
31  #jal x0, loop    #loop label is 4 spaces back
32
33  lui a0, 0x11000    # a0 = x10
34  addi t5, zero, 0x00A    # zero = x0, t5 = x30
35  loop:
36  lh a5, 0(a0)        # a5 = x15
37  sra s4, a5, t5        # s4 = x20
38  xor a2, s4, a5        # a2 = x12
39  sw a2, 64(a0)
40  jal zero, loop
41
42  # ~~~~ syscall exit status ~~~~
43  li a7, 10    # load syscall call num for exit()
44  li a0, 0     # EXIT_SUCCESS code
45  ecall       # make the system call
```

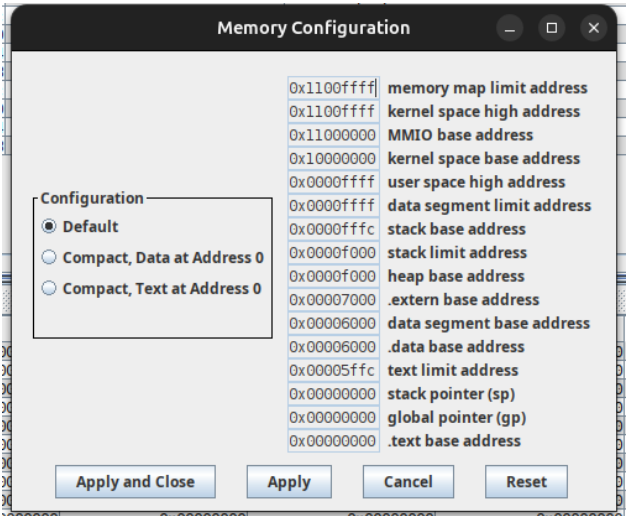


Figure 2: Segment Address Config

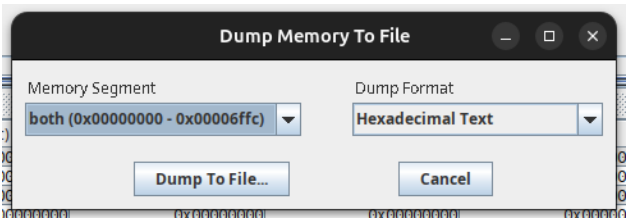


Figure 3: Memory Dump Config

Text Segment				
Bkpt	Address	Code	Basic	Source
	0x00000000	0x11000337	lui x6, 0x00011000	19: li t1, 0x10000000
	0x00000004	0x00030313	addi x6, x6, 0	
	0x00000008	0x00006397	auipc x7, 6	20: lw t2, A
	0x0000000c	0xff83a383	lw x7, 0xfffffff8(x7)	
	0x00000010	0x00732023	sw x7, 0(x6)	21: sw t2, 0(t1)
	0x00000014	0x11000537	lui x10, 0x00011000	33: lui a0, 0x11000 # a0 = x10
	0x00000018	0x00a00f13	addi x30, x0, 10	34: addi t5, zero, 0x00A # zero = x0, t5 = x30
	0x0000001c	0x00051783	lh x15, 0(x10)	36: lh a5, 0(a0) # a5 = x15
	0x00000020	0x41e7da33	sra x20, x15, x30	37: sra s4, a5, t5 # s4 = x20
	0x00000024	0x00fa4633	xor x12, x20, x15	38: xor a2, s4, a5 # a2 = x12
	0x00000028	0x04c52023	sw x12, 0x00000040(x10)	39: sw a2, 64(a0)
	0x0000002c	0xff1ff06f	jal x0, 0xfffffff0	40: jal zero, loop
	0x00000030	0x00a00893	addi x17, x0, 10	43: li a7, 10 # load syscall call num for exit()
	0x00000034	0x00000513	addi x10, x0, 0	44: li a0, 0 # EXIT_SUCCESS code
	0x00000038	0x00000073	ecall	45: ecall # make the system call

Figure 4: RARS instruction addresses alongside code

Vivado Simulation

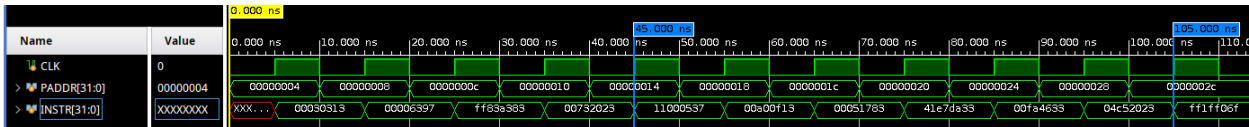


Figure 5: Simulation waveform of ProgROM_tb.sv running otter_memory_hw1.mem

By inspection, the instructions in the waveform 5 match those in RARS4 and our decoded table 1

Intended Behavior(s)

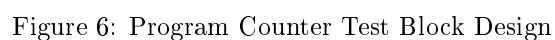


Figure 7: RTL Design of PC_TestBlock.sv

Figure 8: Synthesis Errors of PC_TextBlock.sv

```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 07/19/2026 09:39:05 AM
7 // Design Name:
8 // Module Name: PC_TestBlock_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
```

```

13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_TestBlock_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic reset, write, clk;
27     logic[2:0] sel;
28     logic[5:0][31:0] d;           // d[0] is unused b/c its p+4 on mux
29     logic[31:0] pc, pc_plus_4;
30
31     // ~~~~ instances ~~~~
32     PC_TestBlock UUT(
33         .PC_RST(reset),
34         .PC_WE(write),
35         .CLK(clk),
36         .PC_SEL(sel),
37         .JALR(d[1]),
38         .BRANCH(d[2]),
39         .JAL(d[3]),
40         .MTVEC(d[4]),
41         .MEPC(d[5]),
42         .PC_COUNT(pc),
43         .PC_PLUS_FOUR(pc_plus_4)
44     );
45
46     // ~~ action : CLK (10ns period) ~~
47     always begin
48         #5;
49         clk <= !clk;
50     end
51
52     // ~~~~ action : data ~~~~
53     initial begin
54
55         // ~~ init ~~
56         clk = 1; reset = 0; write = 0; sel = 0;
57         d[1] = 32'h1234_5678;
58         d[2] = 32'hBAAC_4444;
59         d[3] = 32'h0000_01A1;
60         d[4] = 32'hBEEF_BEEF;
61         d[5] = 32'hDEAD_DEAD;
62         pc = 0; pc_plus_4 = 0;
63         #10;
64
65         // ~~ TC1 : sel all w/ write disabled ~~
66         for (int i = 0; i < 16; i++) begin
67             sel = i;
68             #10;
69             assert(pc === 0) else $error("TC1: sel failed (pc)");
70             assert(pc_plus_4 === 4) else $error("TC1: sel failed (pc+4)");
71         end
72
73
74
75         // ~~ TC2 : sel all w/ write enabled ~~
76         reset = 1; sel = 0; #10; reset = 0; #10;
77         write = 1; #10;
78
79         assert(pc === 4) else $error("TC2: sel(0) failed on pc");           // Not zero b/c d[0] is 0+4 already
80         assert(pc_plus_4 === 8) else $error("TC2: sel(0) failed on pc+4");
81
82         for(int i=1; i < 6; i++) begin
83             sel = i; #10;
84             assert(pc === d[i]) else $error("TC2: failed on pc");
85             assert(pc_plus_4 === (d[i]+4) ) else $error("TC2: failed on pc+4");
86         end
87
88
89         // ~~ TC3 : inc pc ~~
90         reset = 1; sel = 0; #10; reset = 0;
91         for(int i=1; i < 256; i++) begin
92             #10;
93             assert(pc === (4*i)) else $error("TC3: failed on pc");
94             assert(pc_plus_4 === (4+(4*i)) ) else $error("TC3: failed on pc+4");
95         end
96
97
98         // ~ test conclusion ~
99         reset = 1;
100     end
101 endmodule

```

For the 6 Input Mux, see [19](#)

For the Program Counter register and test file, see [21](#) and [22](#).

SystemVerilog Code

Listing 3: PC_TestBlock.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 07:34:25 PM
7  // Design Name:
8  // Module Name: PC_TestBlock
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_TestBlock(
24     input logic PC_RST, PC_WE, CLK,
25     input logic[2:0] PC_SEL,
26     input logic[31:0] JALR, BRANCH, JAL, MTVEC, MEPC,
27     output logic[31:0] PC_COUNT, PC_PLUS_FOUR
28 );
29
30 // ~~~~ inits and wires ~~~~
31 logic[31:0] MUX_O_WIRE;
32
33 // ~~~~ instances ~~~~
34 Mux_6N #(32) Mux(
35     .D0(PC_PLUS_FOUR),
36     .D1(JALR),
37     .D2(BRANCH),
38     .D3(JAL),
39     .D4(MTVEC),
40     .D5(MEPC),
41     .S(PC_SEL),
42     .Y(MUX_O_WIRE)
43 );
44
45
46 PC_Reg PC(
47     .CLK(CLK),
48     .PC_RST(PC_RST),
49     .PC_WE(PC_WE),
50     .PC_DIN(MUX_O_WIRE),
51     .PC_COUNT(PC_COUNT)
52 );
53
54 Plus4_1N #(32) P4(
55     .IN(PC_COUNT),
56     .OUT(PC_PLUS_FOUR)
57 );
58
59
60 endmodule
```

0.1.3 HW3 - Register File

Intended Behavior(s)

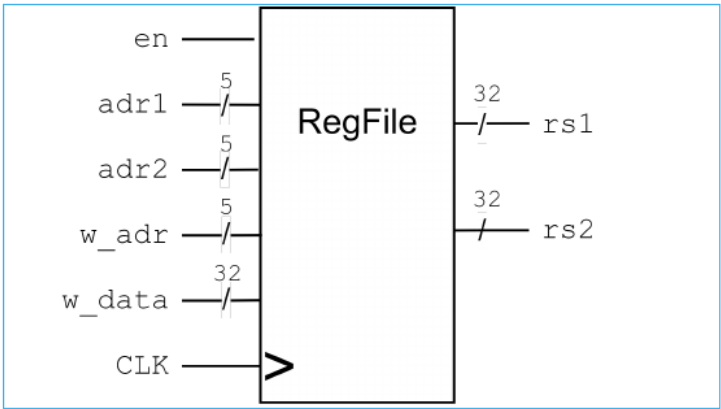


Figure 9: Top Level Register File

The RAM register file must ...

- store 32 32-bit registers
- dual asynch fetch the values at ADR1 / ADR2 and place them in RS1, RS2
- single synch write W_DATA to W_ADR if EN is high.
- prevent x0 "zero" register from being overwritten.

Structural Design(s)

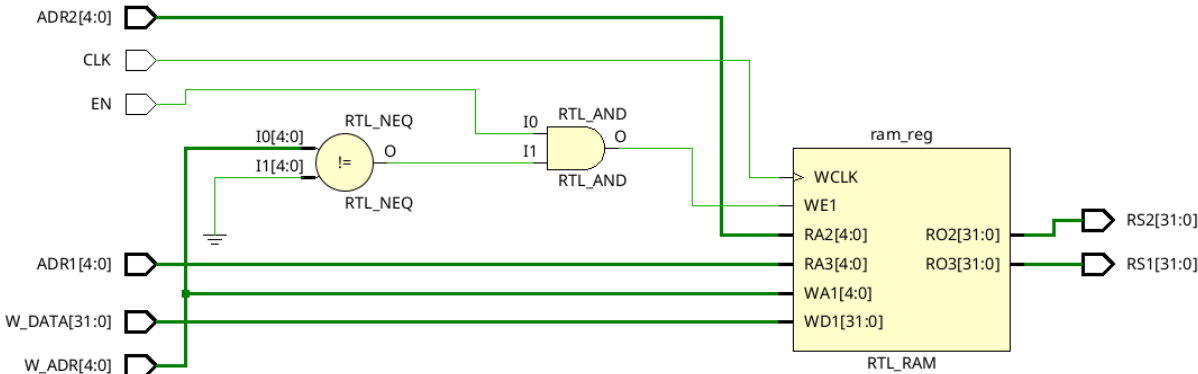


Figure 10: RTL Design of RegFile.sv

Notice we only overwrite the W_ADR if EN is high AND W_ADR is not 0 (for x0).

Synthesis Warning(s)

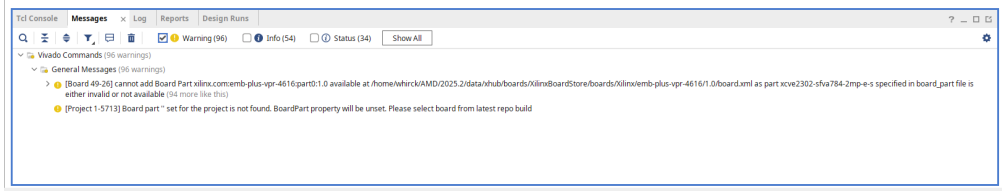


Figure 11: Synthesis Errors of RegFile.sv

Verification

All test cases passed in RegFile_tb.sv 25 passed in simulation along with each subcomponent test bench.

SystemVerilog Code

See 24 for the RegFile.sv.

0.1.4 HW4 - ALU + ImmGen

ALU

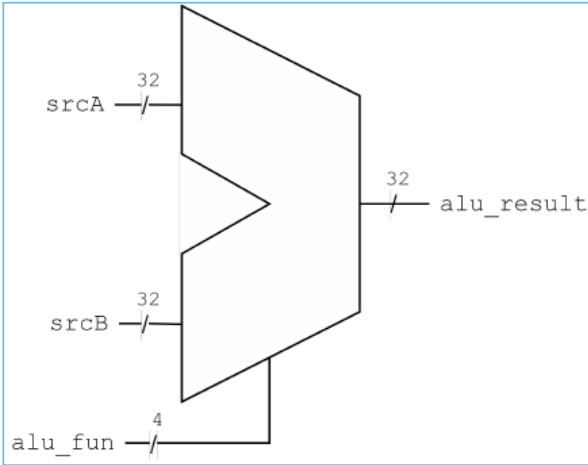


Figure 12: Top Level ALU

The ALU is designed to perform the arithmetic and logic operations required by the RISC-V CPU. Note the 11 we implemented here.

ALU_FUN	Instruction
0000	ADD
1000	SUB
0110	OR
0111	AND
0100	XOR
0101	SRL

ALU_FUN	Instruction
0001	SLL
1101	SRA
0010	SLT
0011	SLTU
1001	LUI-COPY

Figure 13: Instruction Operations of the ALU

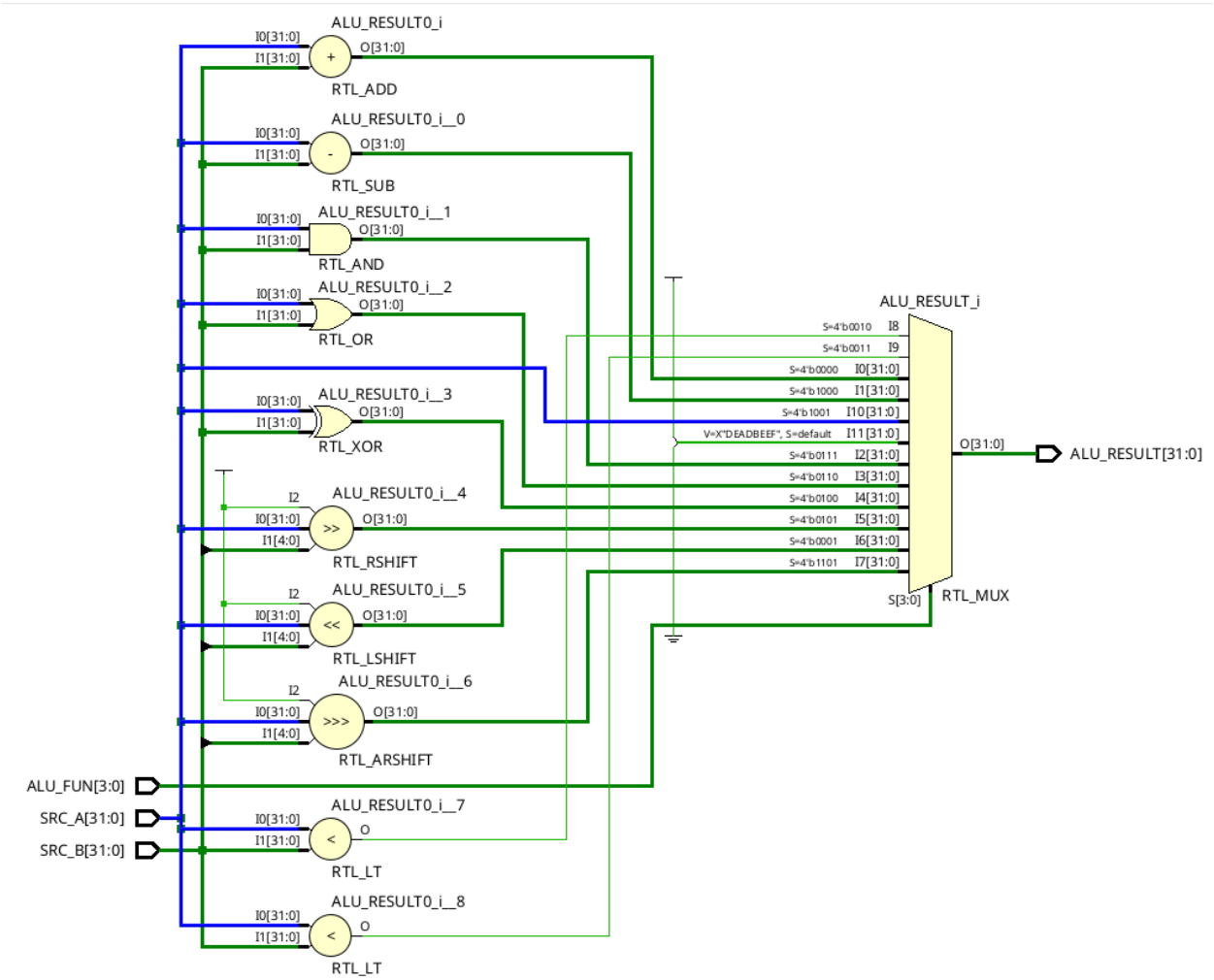


Figure 14: RTL Design of the ALU

Function	ALU_FUN	Arg A	Arg B	Expected Result
ADD	0x0000	0xA50F_96C3	0x5AF0_693C	0xFFFF_FFFF
		0x8410_5F21	0x7B10_5FDE	0xFF20_BEFF
		0xFFFF_FFFF	0x0000_0001	0x0000_0000
SUB	0x1000	0x0000_0000	0x0000_0001	0xFFFF_FFFF
		0xAA80_6355	0x5501_62AA	0x557F_00AB
		0x5501_62AA	0xAA80_6355	0xAA80_FF55
AND	0x0111	0xA55A_00FF	0x5A5A_FFFF	0x005A_00FF
		0xC3C3_F966	0xFF66_9F5A	0xC342_9942
OR	0x0110	0x9A9A_C300	0x65A3_CC0F	0xFFBB_CF0F
		0xC3C3_F966	0xFF66_9F5A	0xFFE7_FF7E
XOR	0x0100	0xAA55_00FF	0x5AA5_0FF0	0xF0F0_0F0F
		0xA5A5_6C6C	0xFF00_C6FF	0x5AA5_AA93
SRL	0x0101	0x805A_6CF3	0x0000_0010	0x0000_805A
		0x705A_6CF3	0x0000_0005	0x0382_D367
		0x805A_6CF3	0x0000_0000	0x805A_6CF3
		0x805A_6CF3	0x0000_0100	0x805A_6CF3
SLL	0x0001	0x805A_6CF3	0x0000_0010	0x6CF3_0000
		0x805A_6CF3	0x0000_0005	0x0B4D_9E60
		0x805A_6CF3	0x0000_0100	0x805A_6CF3
SRA	0x1101	0x805A_6CF3	0x0000_0010	0xFFFF_805A
		0x705A_6CF3	0x0000_0005	0x0382_D367
		0x805A_6CF3	0x0000_0000	0x805A_6CF3
		0x805A_6CF3	0x0000_0100	0x805A_6CF3
SLT	0x0010	0x7FFF_FFFF	0x8000_0000	0x0000_0000
		0x8000_0000	0x0000_0001	0x0000_0001
		0x0000_0000	0x0000_0000	0x0000_0000
		0x5555_5555	0x5555_5555	0x0000_0000
SLTU	0x0011	0x7FFF_FFFF	0x8000_0000	0x0000_0001
		0x8000_0000	0x0000_0001	0x0000_0000
		0x0000_0000	0x0000_0000	0x0000_0000
		0x55AA_55AA	0x55AA_55AA	0x0000_0000
LUI COPY (MV?)	0x1001	0x0123_4567	0x7654_3210	0x0123_4567
		0xFEDC_BA98	0x89AB_CDEF	0xFEDC_BA98

Table 2: ALU Required Test Cases for Verification

See `ALU.sv` 26 and `ALU_tb.sv` 27 for passing simulation test cases from table ??.

Immediate Generator

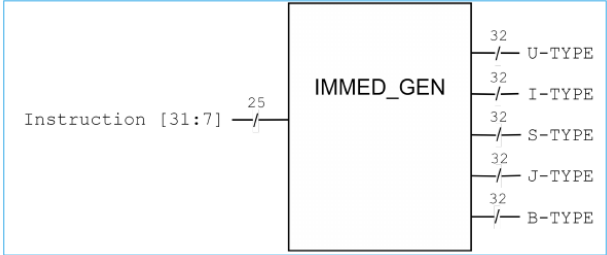


Figure 15: Top Level ImmedGen

The Immediate Generator is a feed though bit parser for different assembly instructions, continuously out-putting all types per instruction.

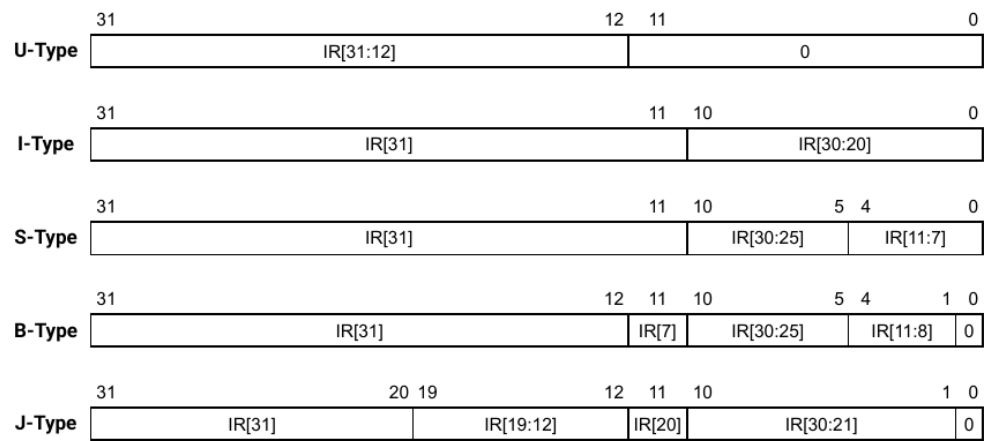


Figure 16: Instruction types of the Immediate Generator

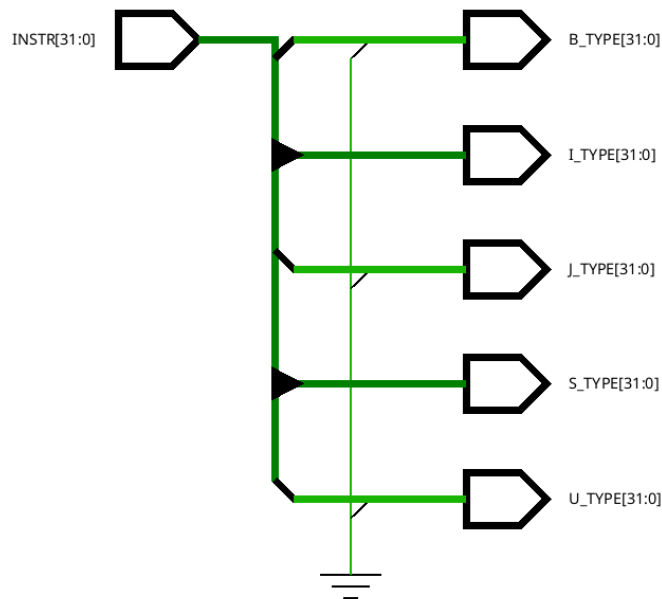


Figure 17: RTL Design of the ImmedGen

See `ImmedGen.sv` [28](#) and `ImmedGen_tb.sv` [29](#) for passing simulation test cases

0.1.5 HW5 - Real Memory and Branch

Memory

The memory segment is designed to dual synch output an instruction and a data value with architecture for pulling data from the MMIO addresses. They are on separate addresses physically such that they are not on the stack but rather beyond what could be the heap (if we implement one).

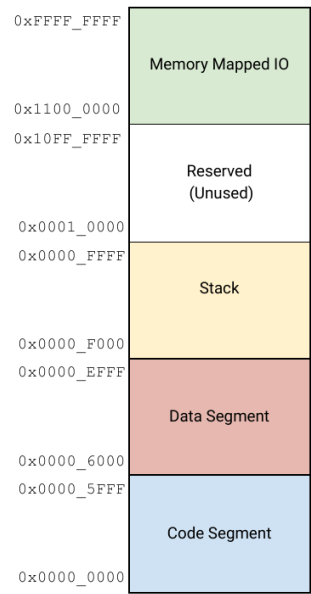


Figure 5.1: OTTER Memory Map

Figure 18: Memory Map for OTTER

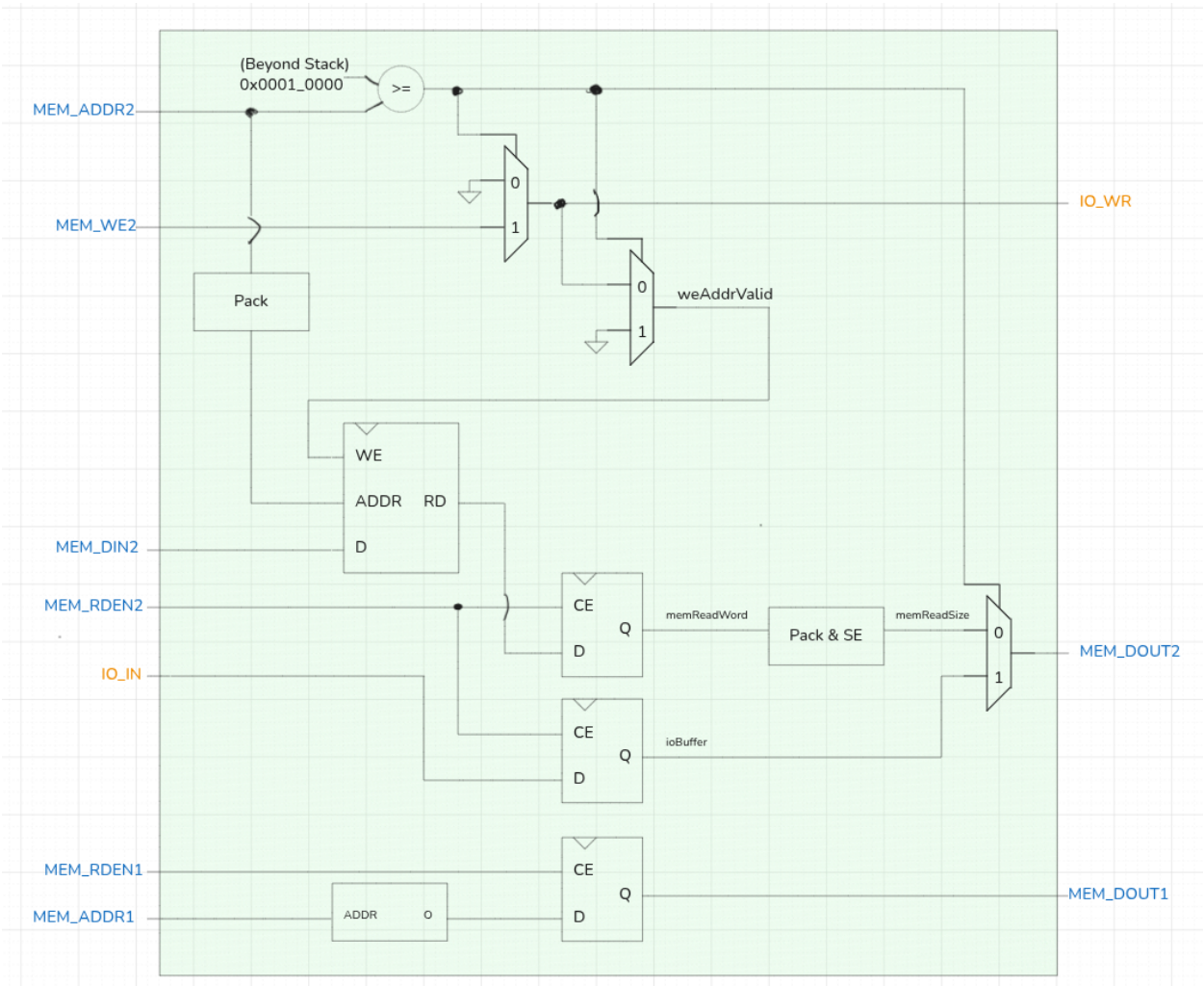


Figure 19: **ROUGH** Flow Diagram for Memory

Signal	Size(bits)	Purpose
MEM_CLK	1	Required for synchronous logic
MEM_RDEN1	1	Request an instr read at ADDR1, to register, to DOUT1
MEM_RDEN2	1	Request a value read at ADDR2 with path to update DOUT2 (but not entire path)
MEM_WE2	1	Request DIN2 overwrite register connected to DOUT wire to IO_WR
MEM_ADDR1	14	Instruction address
MEM_ADDR2	32	Data Address
MEM_DIN2	32	Data value to overwrite one at ADDR2
MEM_SIZE	2	Indicate (Byte, Half, Word) size of data going to ADDR2
MEM_SIGN	1	Indicate (signed, unsigned) data going to ADDR2
IO_IN	32	Incoming data from IO
IO_WR	1	Feedthrough of MEM_WE2 if ADDR2 is in MMIO range
MEM_DOUT1	32	Instr1 output
MEM_DOUT2	32	Data2 output

Table 3: Memory Module Signals

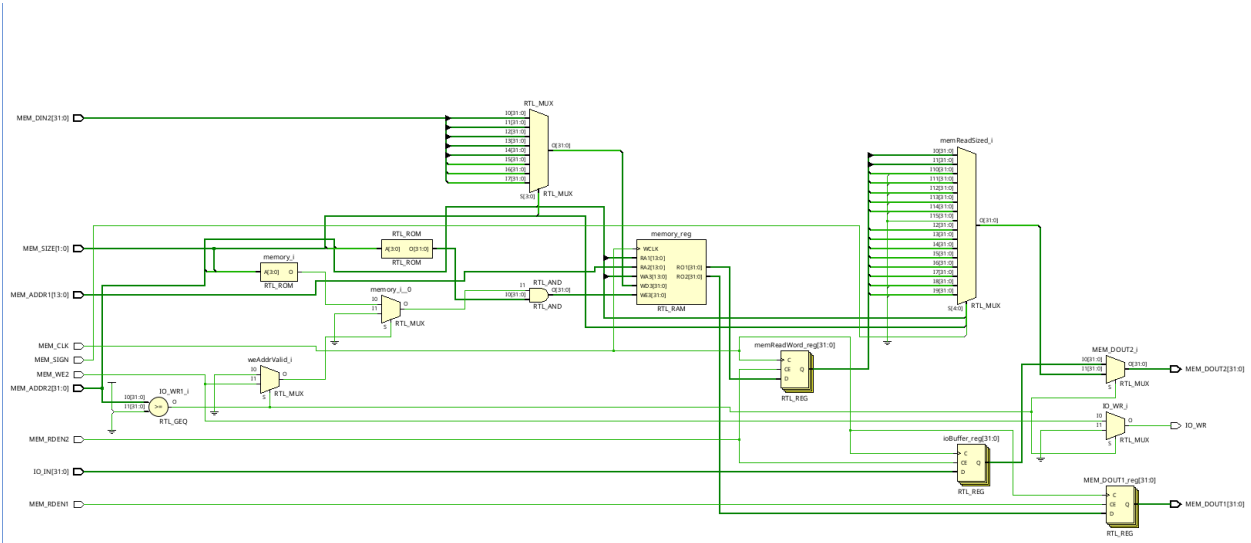


Figure 20: Lint RTL Schematic of Memory

See `otter_memory_v1_07.sv` [30](#).

Branch Condition Generator

The BCG compares two register values and generates a command signal for equality, less than (signed), and less than (unsigned). This is needed to perform the logic evaluations of the differing branch types s.a. `beq`, `bltu`, ...

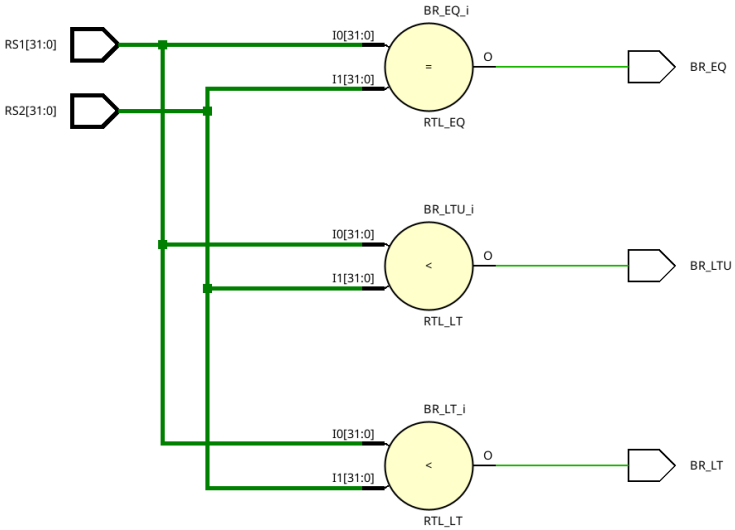


Figure 21: Lint RTL Schematic of BCG

See `BranchConditionGen.sv` [33](#) and sim test `BranchConditionGen_tb.sv` [34](#).

Branch Address Generator

The BAG is used to modify the program counter! We want this module s.t. we can jump (J-Type), branch (B-Type), and jump-return (using I-Type). These instructions are useful for control flow.

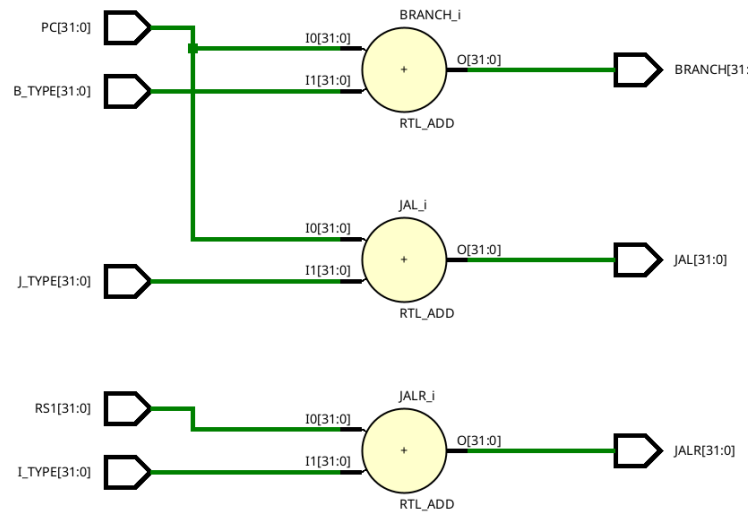


Figure 22: Lint RTL Schematic of BAG

See [BranchAddrGen.sv](#) [31](#) and sim test [BranchAddrGen_tb.sv](#) [32](#).

0.1.6 HW6 - Control Unit

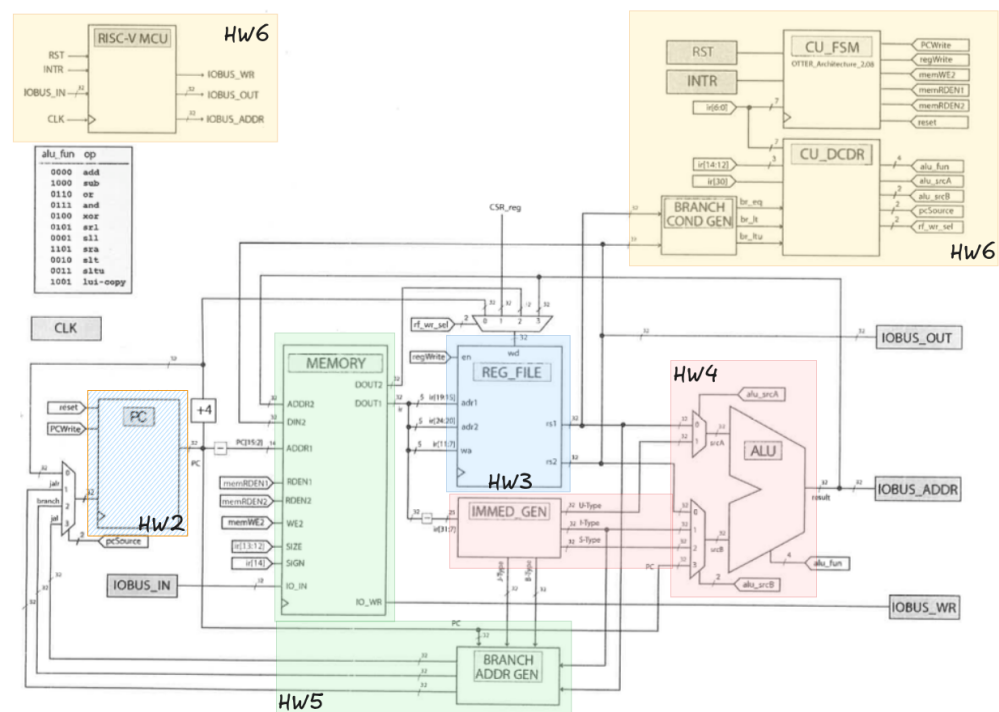


Figure 23: MultiCycle Otter after HW6

Data and control mapping is different for each type of instruction. Therefore, I've provided the various configurations per instruction type below.

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	-	I	lb rd, imm(rs1)	load byte	rd ← SignExt([Address] _{7:0})
0000011 (3)	001	-	I	lh rd, imm(rs1)	load half	rd ← SignExt([Address] _{15:0})
0000011 (3)	010	-	I	lw rd, imm(rs1)	load word	rd ← [Address] _{31:0}
0000011 (3)	100	-	I	lbu rd, imm(rs1)	load byte unsigned	rd ← ZeroExt([Address] _{7:0})
0000011 (3)	101	-	I	lhu rd, imm(rs1)	load half unsigned	rd ← ZeroExt([Address] _{15:0})

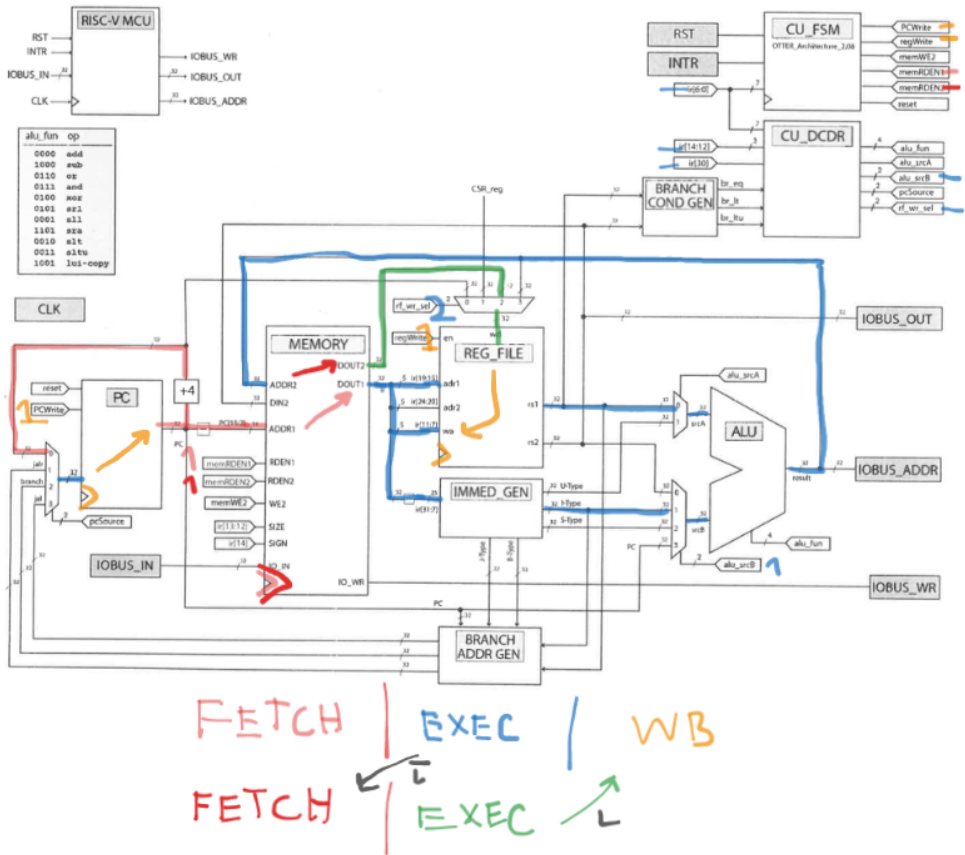


Figure 24: MultiCycle Otter Pathing for load

0010011 (19)	000	-	I	addi rd, rs1, imm	add immediate	$rd = rs1 + \text{SignExt}(imm)$
0010011 (19)	001	0000000	I	slli rd, rs1, imm	shift left logical immediate	$rd = rs1 \ll imm$
0010011 (19)	010	-	I	slti rd, rs1, imm	set less than immediate	$rd = (rs1 < \text{SignExt}(imm))$
0010011 (19)	011	-	I	sltiu rd, rs1, imm	set less than imm. unsigned	$rd = (rs1 < \text{SignExt}(imm))$
0010011 (19)	100	-	I	xori rd, rs1, imm	xor immediate	$rd = rs1 \wedge \text{SignExt}(imm)$
0010011 (19)	101	0000000	I	srlr rd, rs1, imm	shift right logical immediate	$rd = rs1 \gg imm$
0010011 (19)	101	0100000	I	srair rd, rs1, imm	shift right arithmetic imm.	$rd = rs1 \ggg imm$
0010011 (19)	110	-	I	ori rd, rs1, imm	or immediate	$rd = rs1 \text{SignExt}(imm)$
0010011 (19)	111	-	I	andi rd, rs1, imm	and immediate	$rd = rs1 \& \text{SignExt}(imm)$

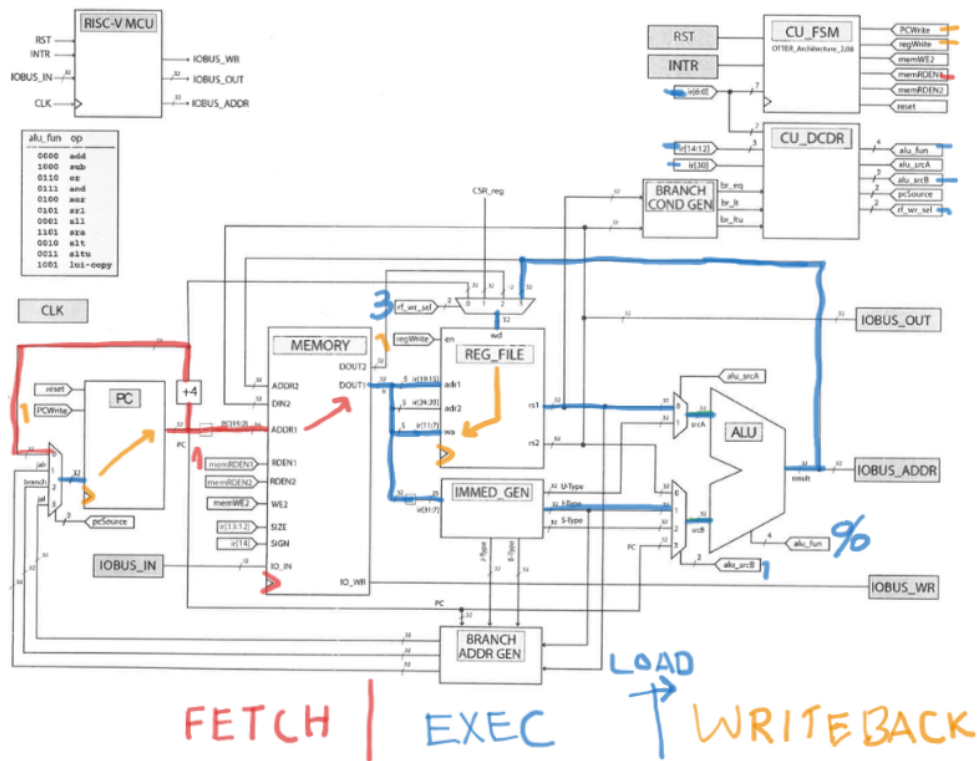


Figure 25: MultiCycle Otter Pathing for immed ops

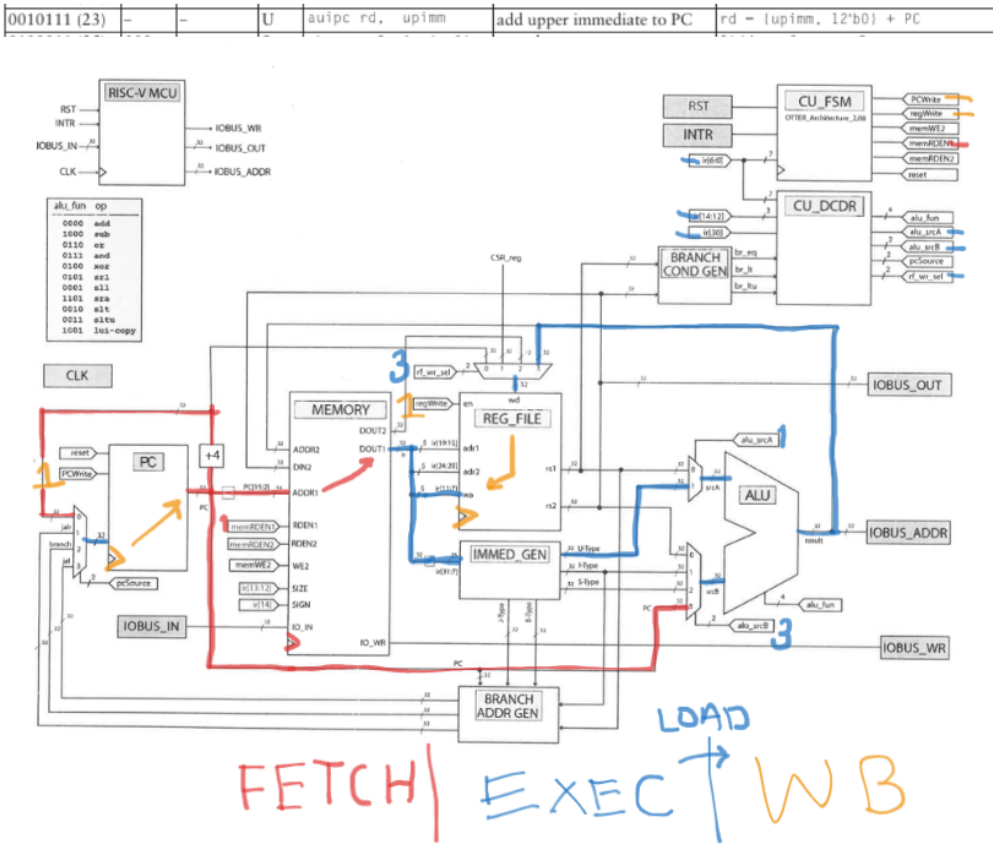


Figure 26: MultiCycle Otter Pathing for auipcc

0100011 (35)	000	—	S	sb	rs2, imm(rs1)	store byte	[Address] _{7:0} = rs2 _{7:0}
0100011 (35)	001	—	S	sh	rs2, imm(rs1)	store half	[Address] _{15:0} = rs2 _{15:0}
0100011 (35)	010	—	S	sw	rs2, imm(rs1)	store word	[Address] _{31:0} = rs2

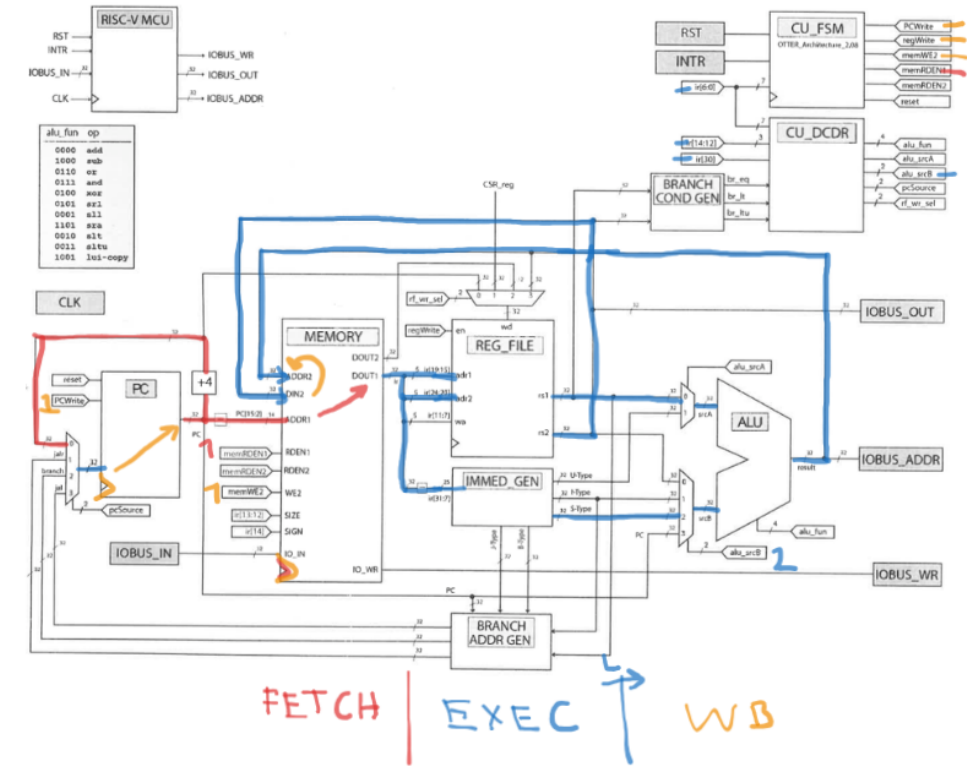


Figure 27: MultiCycle Otter Pathing for store

0110011 (51)	000	0000000	R	add	rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub	rd, rs1, rs2	sub	rd = rs1 - rs2
0110011 (51)	001	0000000	R	sll	rd, rs1, rs2	shift left logical	rd = rs1 << rs2 _{4:0}
0110011 (51)	010	0000000	R	slt	rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu	rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor	rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl	rd, rs1, rs2	shift right logical	rd = rs1 >> rs2 _{4:0}
0110011 (51)	101	0100000	R	sra	rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> rs2 _{4:0}
0110011 (51)	110	0000000	R	or	rd, rs1, rs2	or	rd = rs1 rs2
0110011 (51)	111	0000000	R	and	rd, rs1, rs2	and	rd = rs1 & rs2

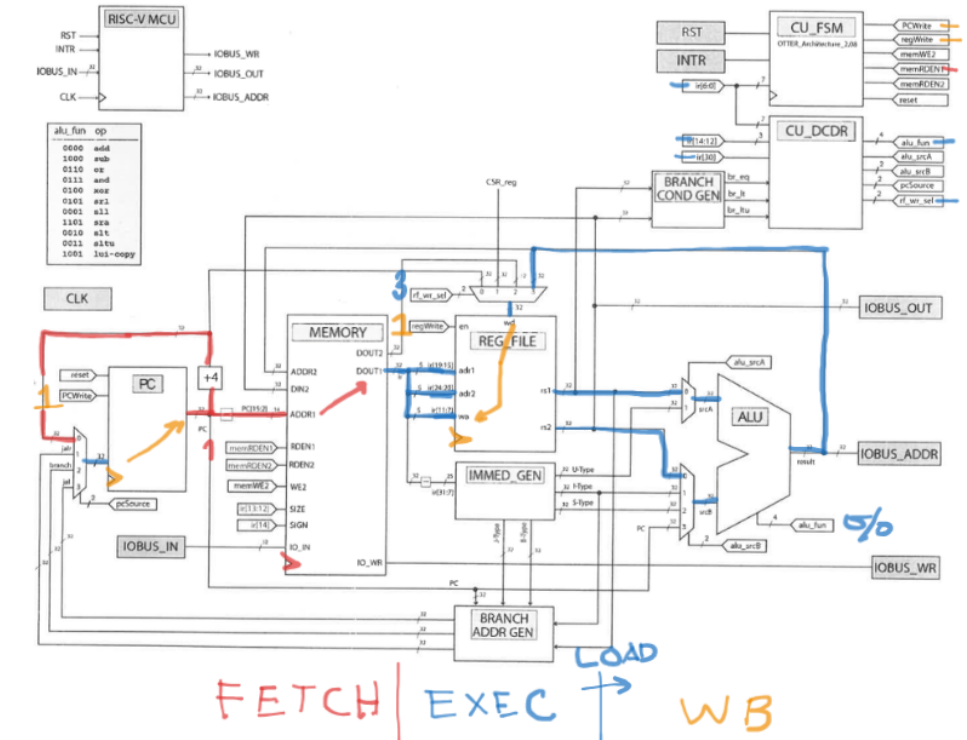


Figure 28: MultiCycle Otter Pathing for dual register ops

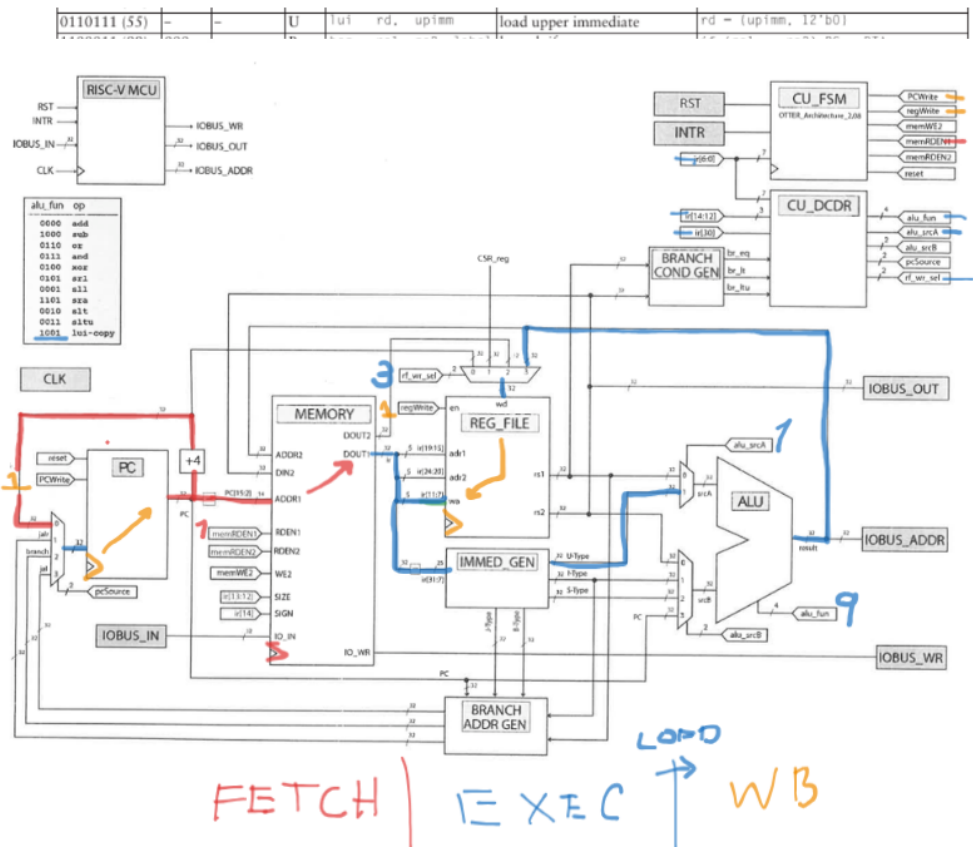


Figure 29: MultiCycle Otter Pathing for lui

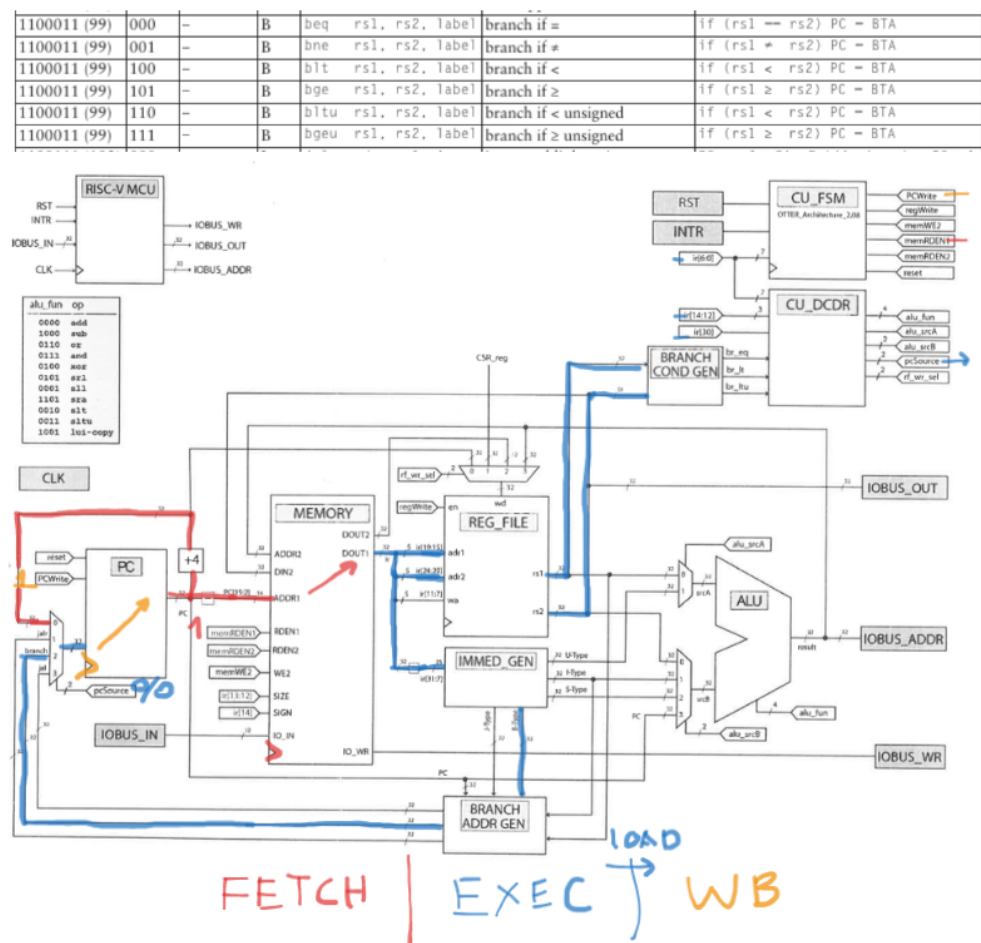


Figure 30: MultiCycle Otter Pathing for branch

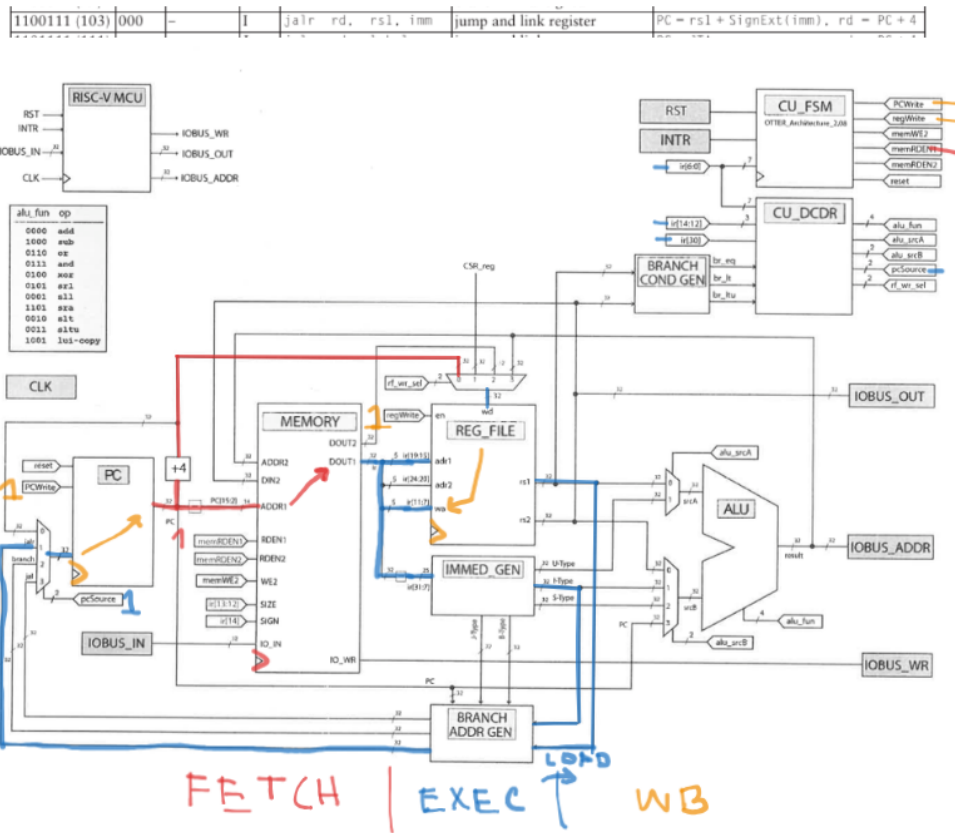


Figure 31: MultiCycle Otter Pathing for jalr

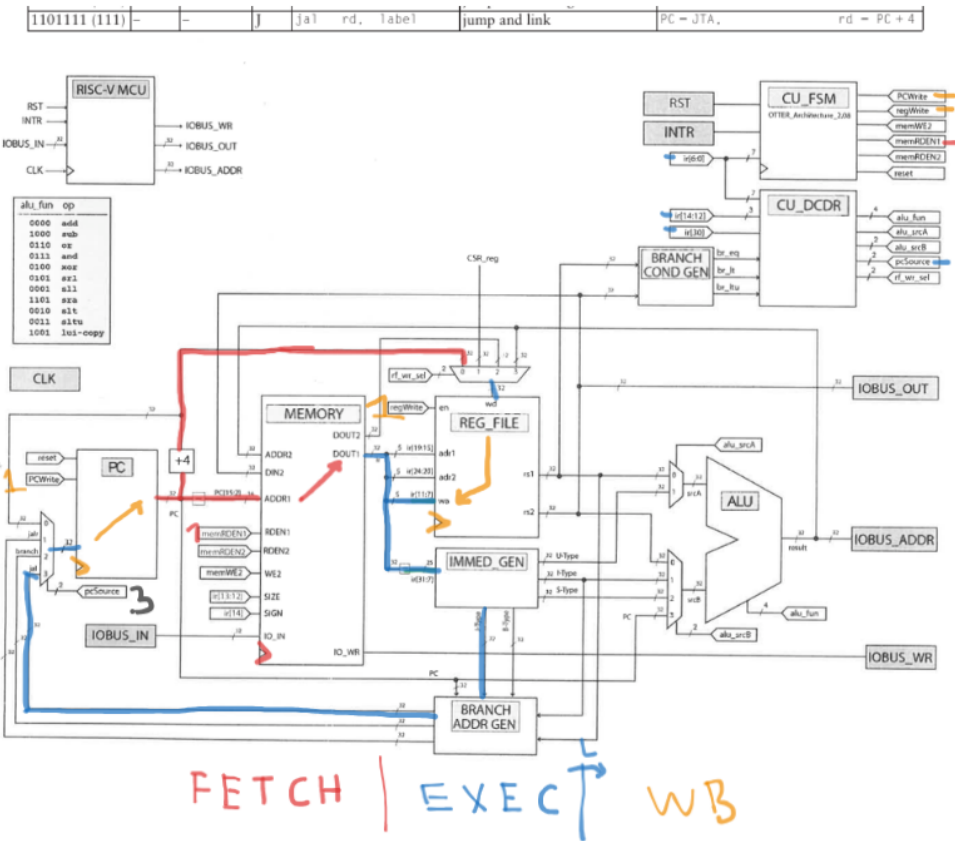


Figure 32: MultiCycle Otter Pathing for jal

Control Unit Decoder

After mapping the data and control signals, we need a way to control data flowing into the ALU, register file, program counter, as well as the ALU itself. We depend on the Decoder to do this, writing to the selectors for each mux. Note that the decoder can only make an intelligent switch during and after the EXEC cycle s.t. the opcode is available to process.

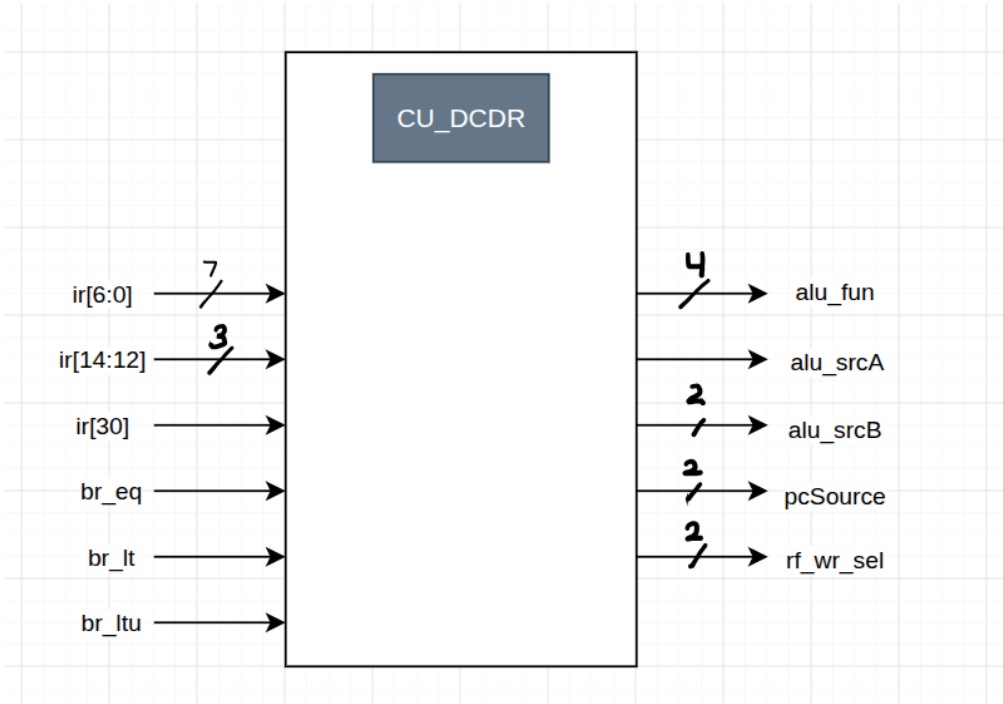


Figure 33: Control Unit Decoder Top Level

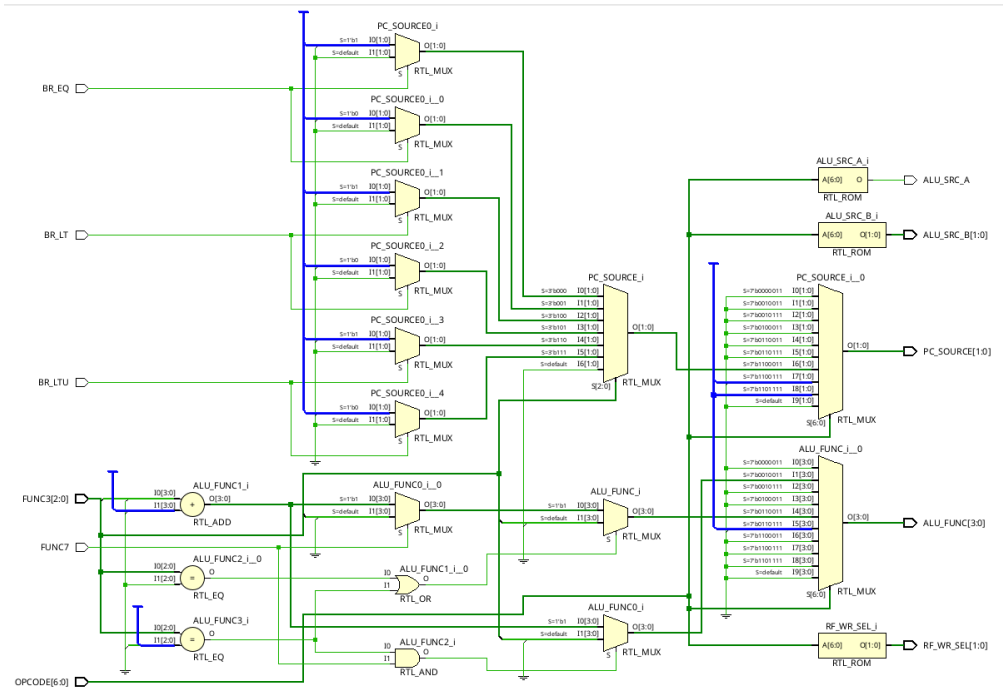


Figure 34: Control Unit Decoder RTL

See CUnit_Decoder.sv [35](#) and sim test cases CUnit_Decoder_tb.sv [36](#).

Control Unit FSM

With the decoder controlling the data/mux selectors, we need a module to issue the control signals for read/writing. The FSM allows us to do just that, toggling read signals for memory and PC while also controlling when to write contents in memory and registers. We’ve confined our FSM to 4 stages (INIT, FETCH, EXEC, WRITEBACK).

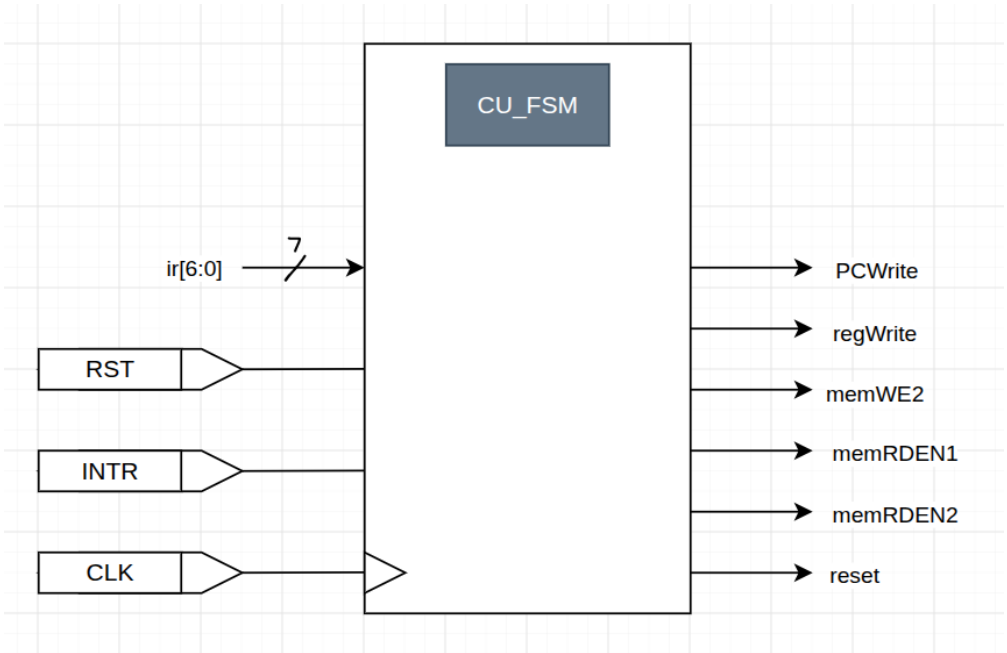


Figure 35: Control Unit FSM Top Level

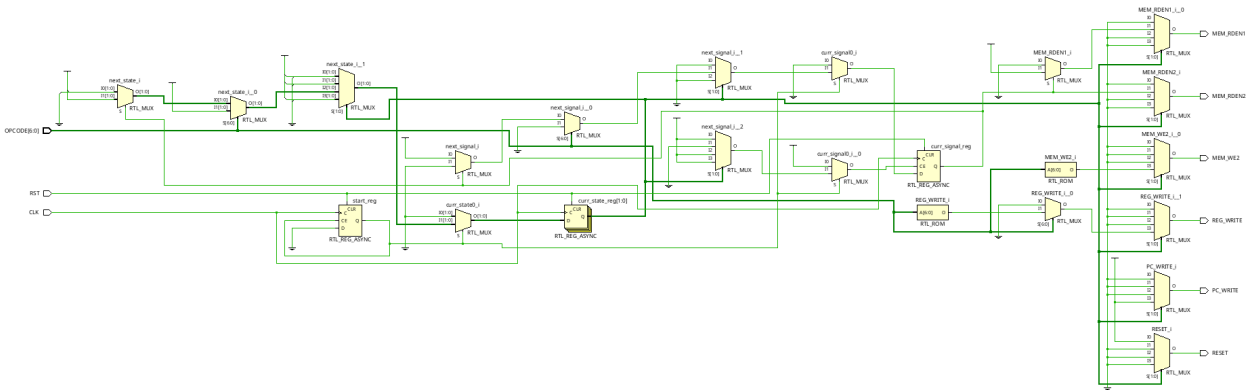


Figure 36: Control Unit FSM RTL

Stages :

- **INIT** : stage upon startup or reset that zeros out all signals.
- **FETCH** : PC is available and this stage processes the contents at that PC address, tucked away inside the memory module. By the end, DOUT1 should be available (and maybe DOUT2 depending on the instr).
- **EXEC** : stage is used to give instr operations runway to compute and arrive to their respective mux channels.
- **WRITEBACK** : stage is used to overwrite a desired PC, memory, or register.

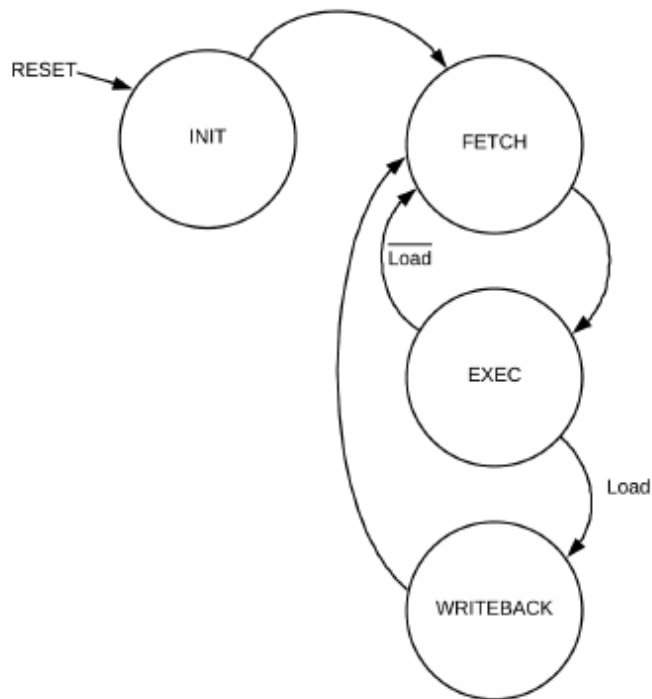


Figure 37: FSM Diagram

See `CUnit_FSM.sv` [37](#) and sim test cases `CUnit_FSM_tb.sv` [38](#).

Otter Micro-Controller

At last, we've reached a point where we can combine all created modules from HW2-6 and begin to process memory files with RISC-V instructions.

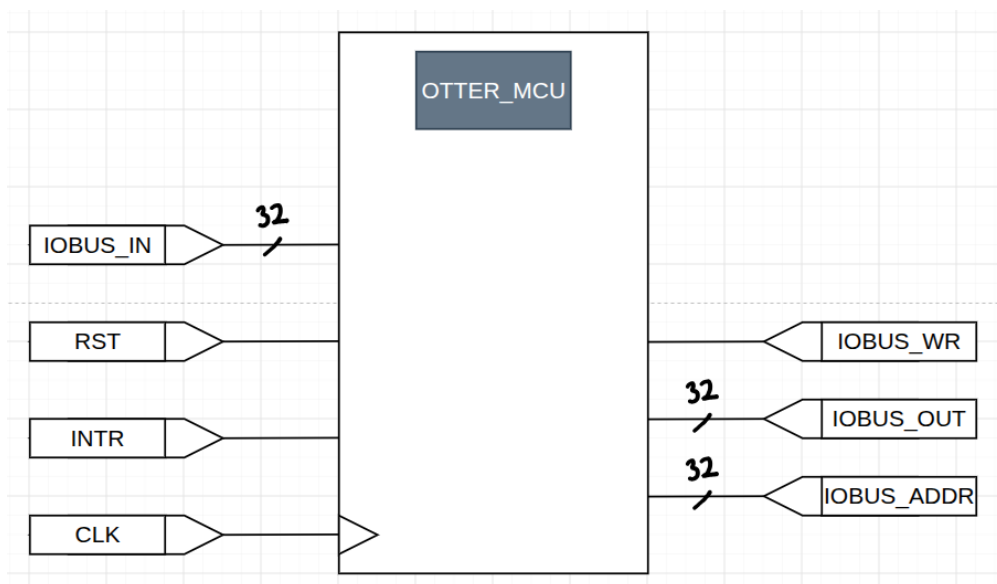


Figure 38: OTTER MCU : Non-Interrupt

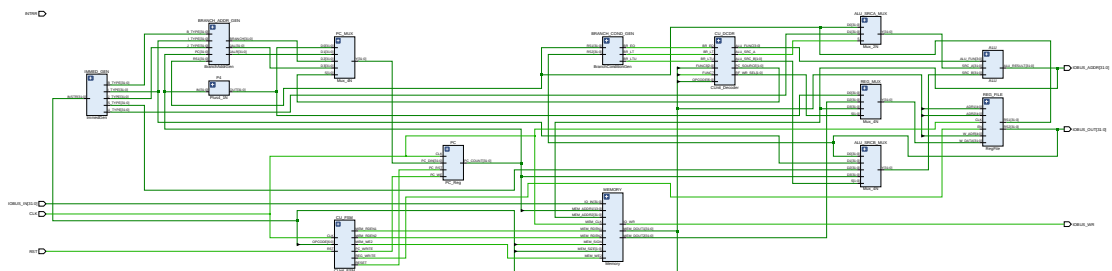


Figure 39: MCU RTL

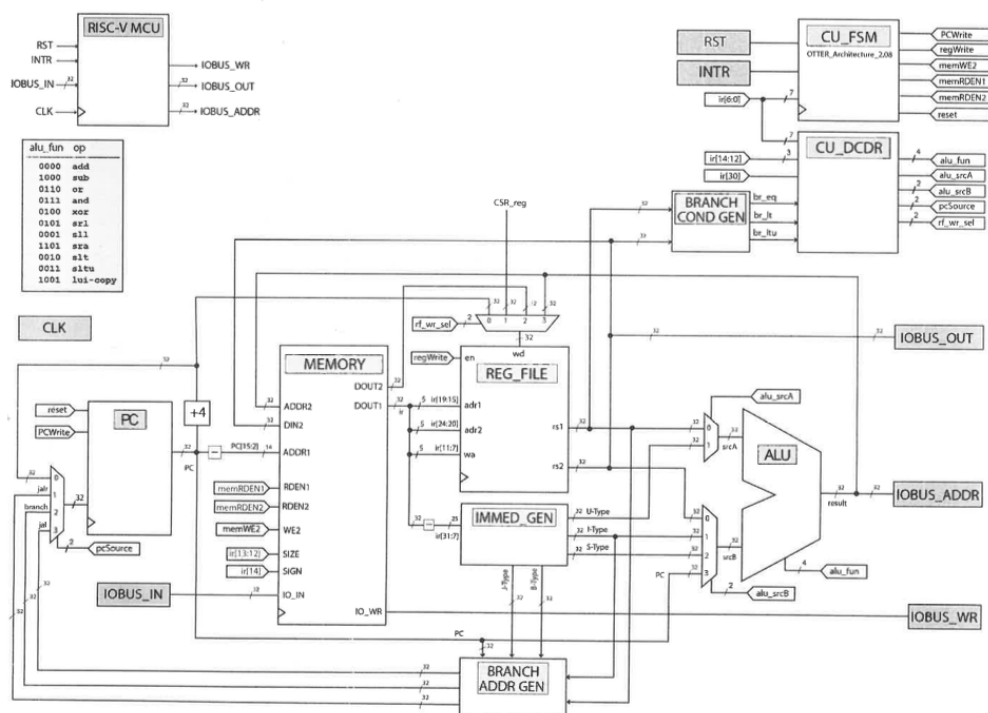


Figure 40: Non-Interrupt MultiCycle RISCv Otter

To perform a small validation for the MCU, a small assembly file was created, compiled under RARS tool for RISC-V compatibility, and the memory dump (`otter_hw6sample_test.mem`) injected into the memory portion of the OTTER MCU Memory.

Listing 4: Simple Test Script

```

1 # Author : WhisperingRock
2 #
3 # Purpose : machine code dump for which to compare results on OTTER MCU
4 #
5 # Input :
6 #
7 # Output :
8
9 main:
10     lui     x5,    0xAA055        # x5 = 0xAA05_5000
11     addi    x8,    x5,    0x765    # x8 = 0xAA05_5000 + 0x0000_0765 = 0xAA05_5765
12     slli    x10,   x8,    3        # x10 = 0x502A_BB28
13     slt     x12,   x5,    x8       # x12 = 1
14     xor     x13,   x8,    x10      # x13 = 0xFA2F_EC4D
15     beq     x0,    x0,    main     # 0 == 0, return to main

```

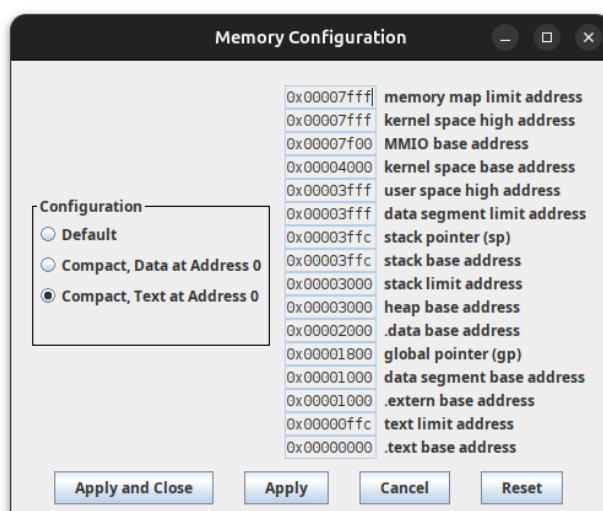


Figure 41: Memory config for RARS

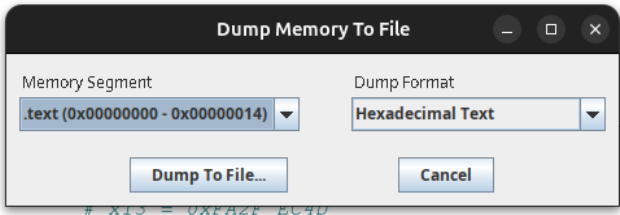


Figure 42: Memory dump config for RARS

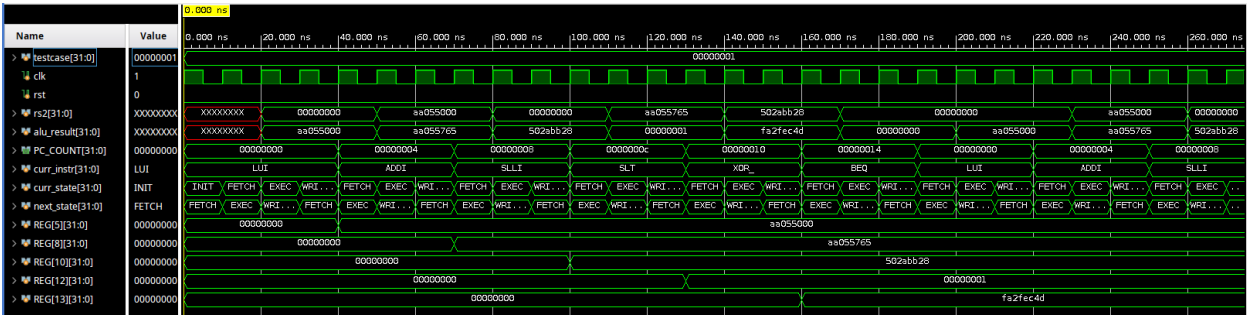


Figure 43: Verification Waveforms in simulation testing

See Otter_MCU.sv 39 and sim test cases Otter_MCU_tb.sv 40.

0.1.7 HW7 - Otter Wrapper

The Otter Wrapper expanded the MCU I/O BUS capabilities to accept the onboard switches as input as well as LED/7SEG as outputs (as defined in the constraints file 46). In integrating the new peripherals, we’ve injected a testall memory dump diego_Test_All.mem that ideally exercises all RISC-V-I instructions and displays the results on actual hardware, giving a visual inspection of the hardware as well as the internals. All of these tests passed accordingly.

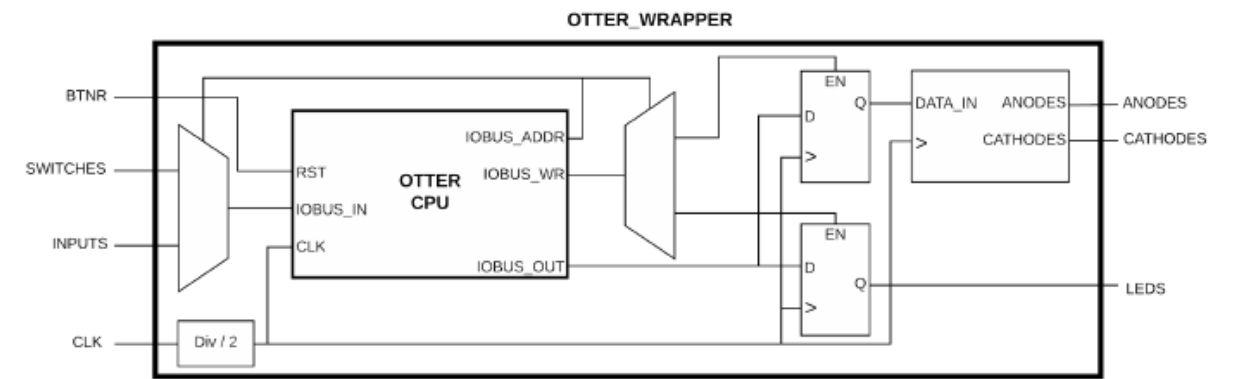


Figure7. 1: OTTER_WRAPPER Block Diagram

Figure 44: Otter Wrapper Schematic

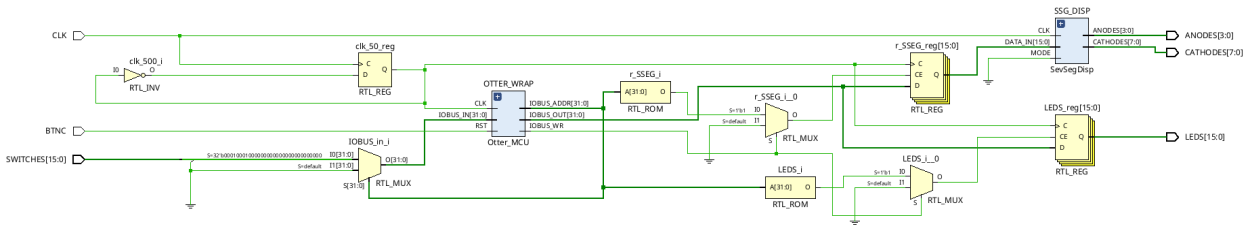


Figure 45: Otter Wrapper RTL

See 44 and 45 for wrapper code and test sim.

0.1.8 HW8 - Interrupts

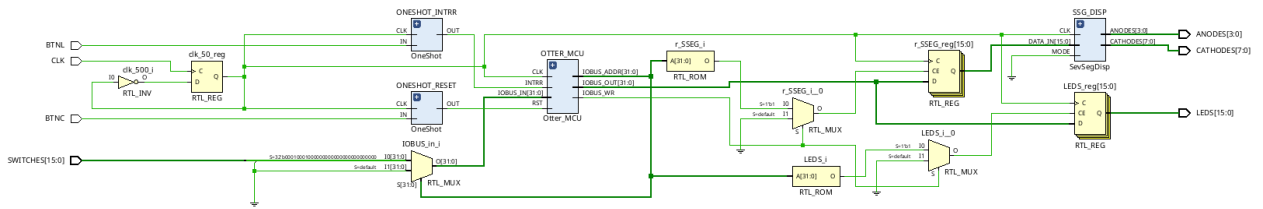


Figure 46: Otter Wrapper RTL w/ Interrupts Enabled

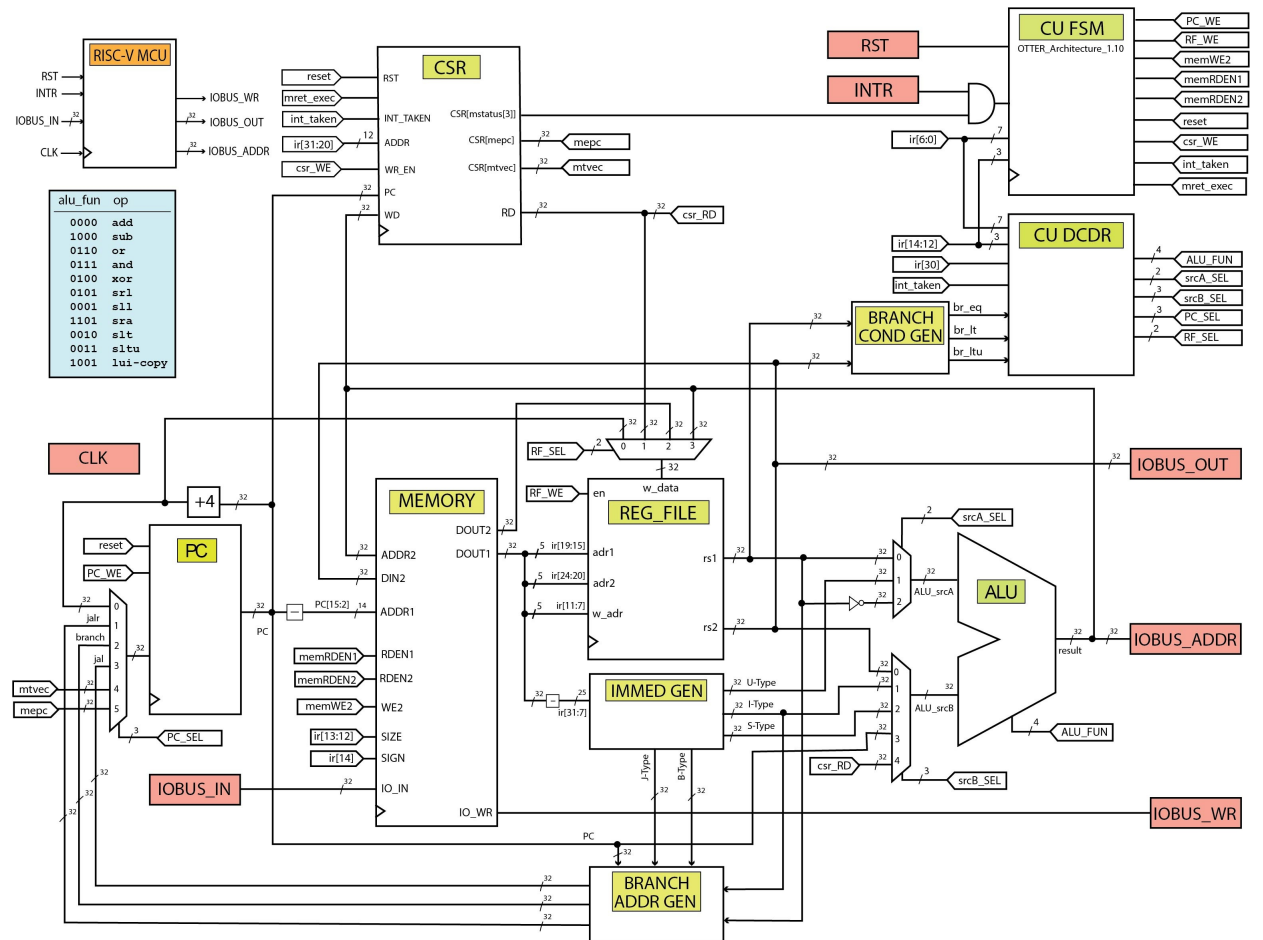


Figure 47: Schematic of Otter MCU w/ Interrupts Enabled

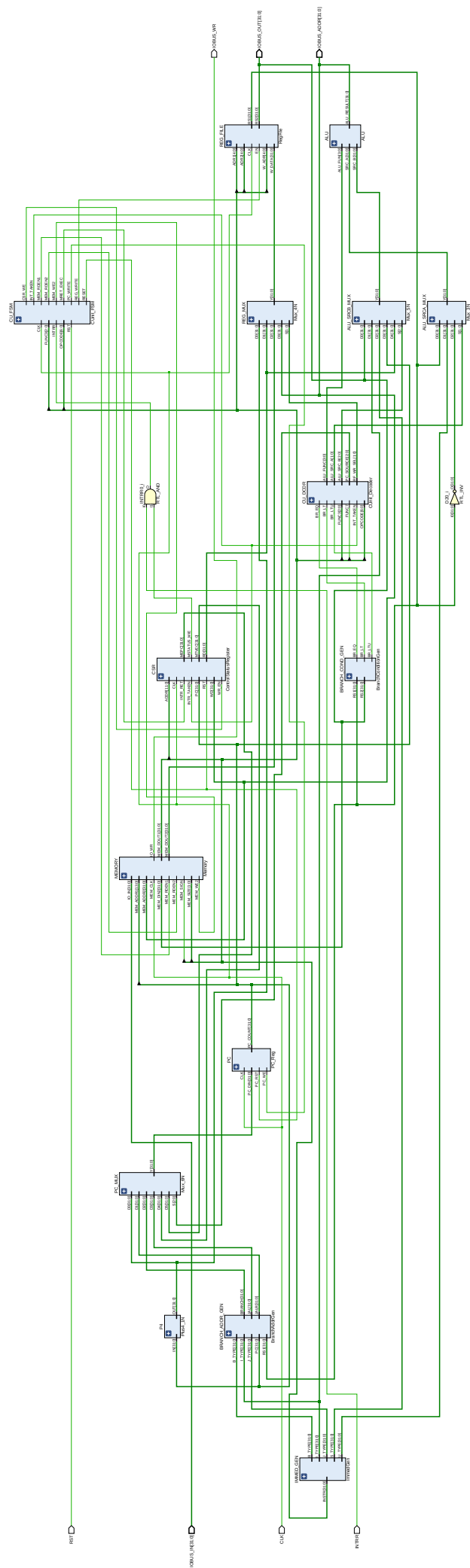


Figure 48: Otter Wrapper RTL w/ Interrupts Enabled

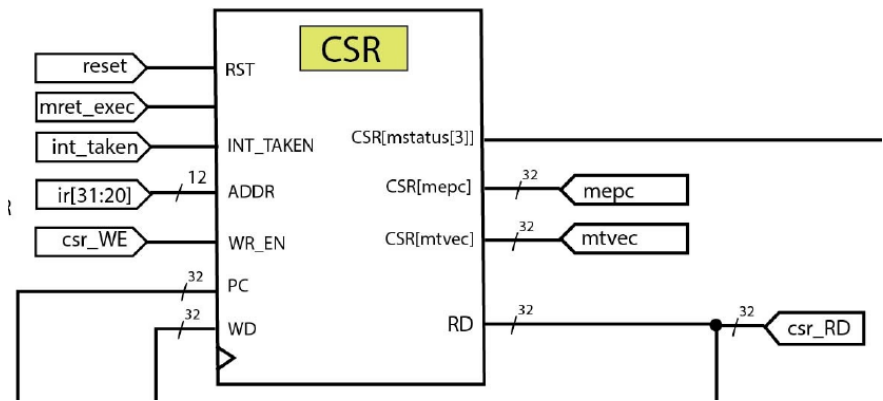


Figure 49: Otter's Control Status Register

The Control Status Register (CSR) is a register file that enables Interrupts and other service routines. In our sparse use, we use one register, the MTVEC, to store and load the address to our single ISR location. We use another register, the MEPC, to temporarily store and load the PC when entering and exiting an ISR. Lastly, we use a register, the MSTATUS, to store the interrupt enable setting as well as prevent other interrupt calls during an interrupt routine. It should be noted that there are write and read calls for the 12-bit registers stored in the CSR but we're only using three (as mentioned above). The CSR stores and loads the registers mentioned above on HIGH activation of signals `int_taken` and `mret_exec` which are generated by the Control Unit FSM.

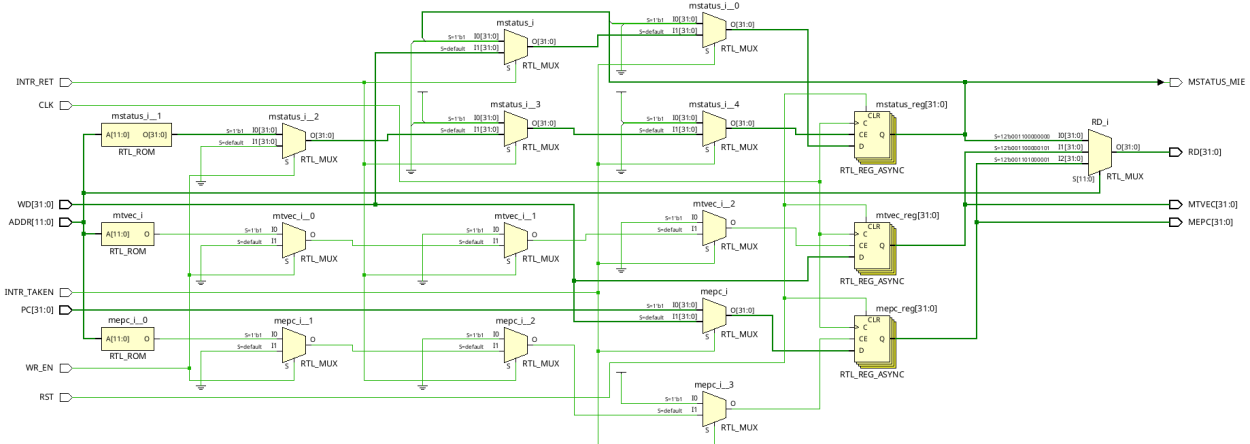


Figure 50: CSR RTL

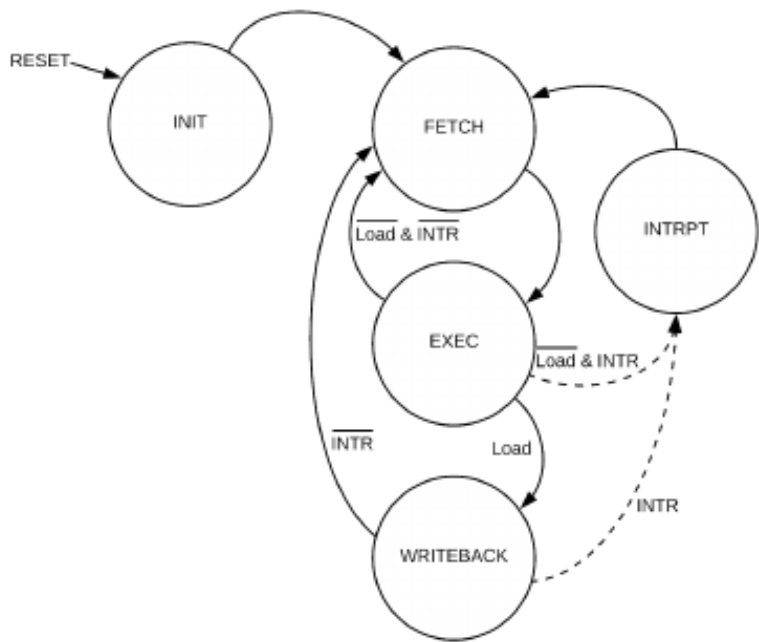
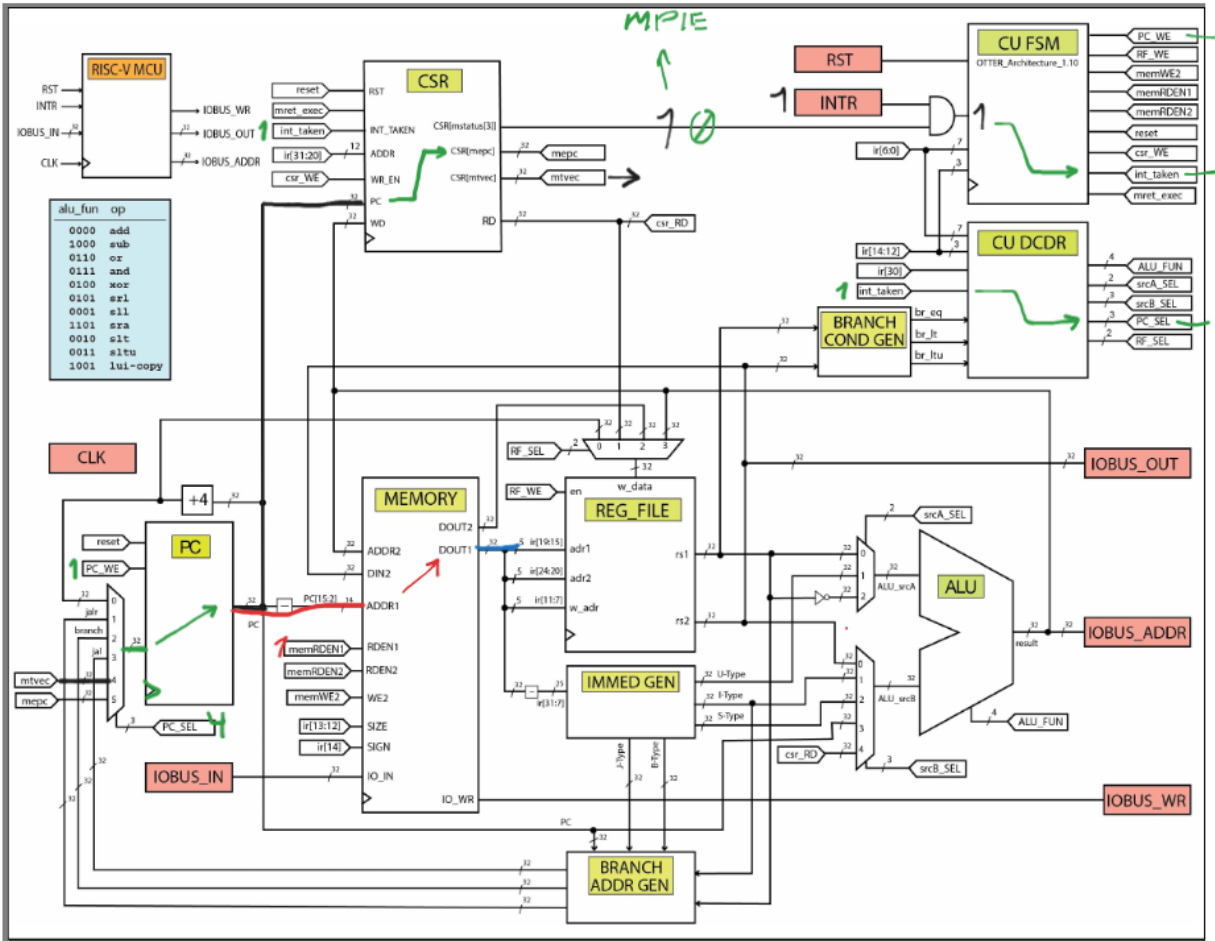


Figure 51: Interrupt-Updated FSM for the Control Unit



ISR | FETCH | EXEC |

Figure 52: Dataflow entering the ISR

opcode	rd, csr, imm	branch if not equal to zero	$PC \leftarrow PC + sext(imm) \text{ if } (X(rs1) \neq 0)$	$imm \neq \text{value}$
call	label	branch to subroutine WARNING: overwrites X6	$X[rd] \leftarrow PC + 8; PC \leftarrow \&\text{symbol}$ (rd=X1 if rd omitted)	imm ≠ value overwrites X6
csrrc	rd, csr, rs1	control & status reg read and bit clear	$X[rd] \leftarrow CSR[csr]; CSR[csr] \leftarrow CSR[csr] \& \sim rs1$	clears part of reg
csrrs	rd, csr, rs1	control & status reg read and bit set	$X[rd] \leftarrow CSR[csr]; CSR[csr] \leftarrow CSR[csr] rs1$	sets part of reg
csrrw	rd, csr, rs1	control & status register read & write	$X[rd] \leftarrow CSR[csr]; CSR[csr] \leftarrow rs1$	writes entire reg
csrw	rs1, csr	control & status register write	$CSR[csr] \leftarrow rs1$	
j	imm	unconditional branch	$PC \leftarrow PC + sext(imm)$	imm ≠ value

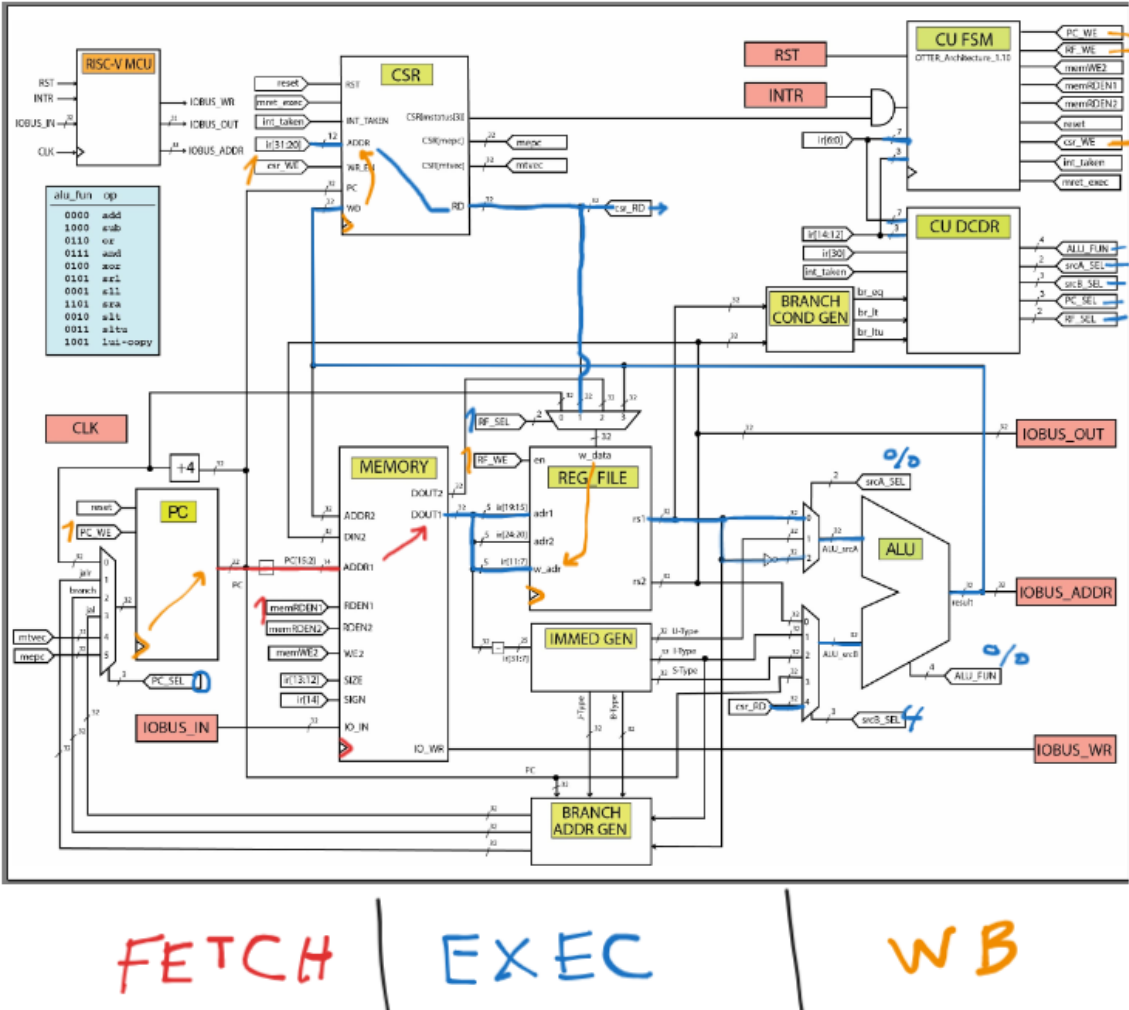


Figure 53: Dataflow reading/writing the ISR registers

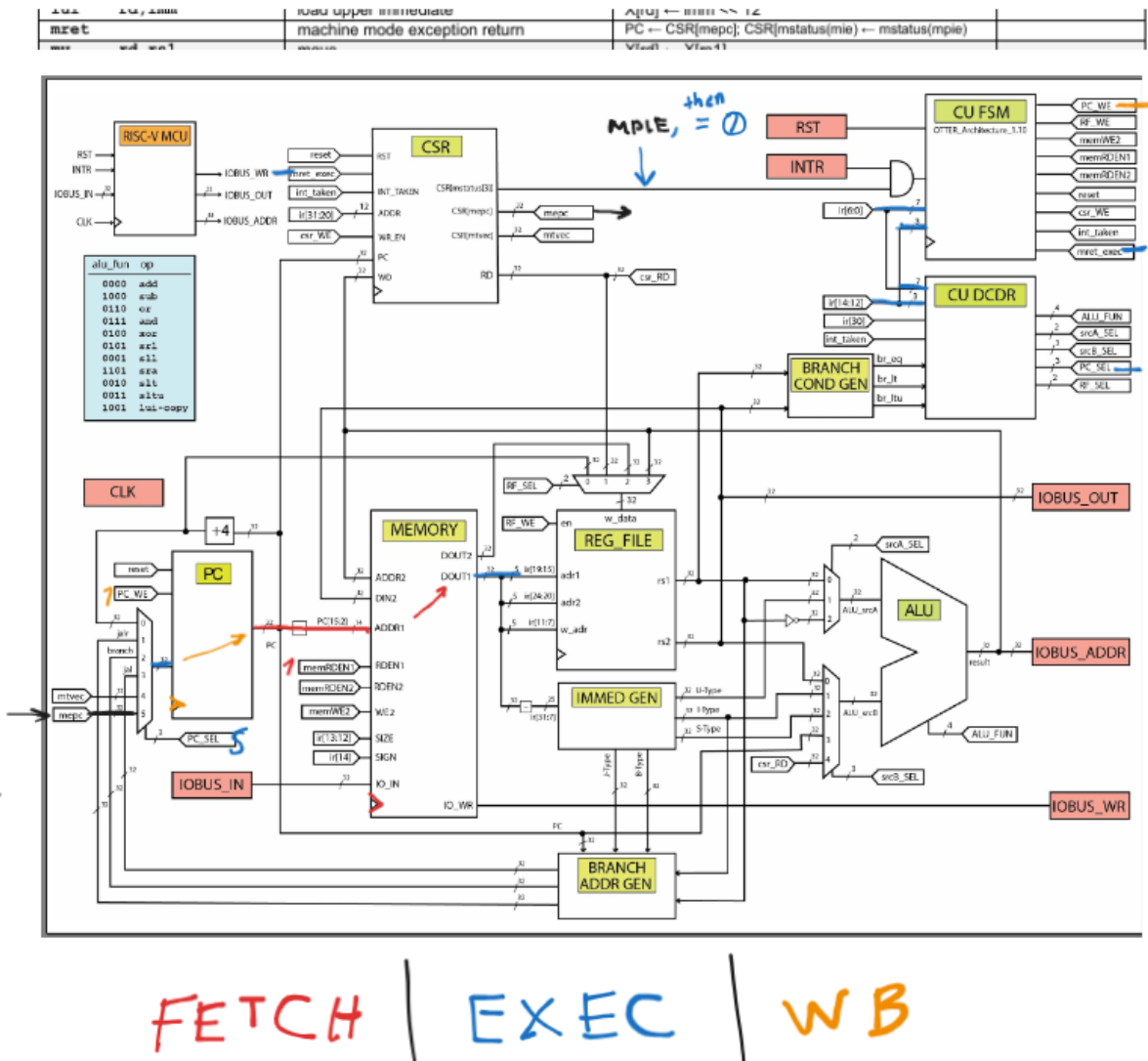


Figure 54: Dataflow exiting the ISR

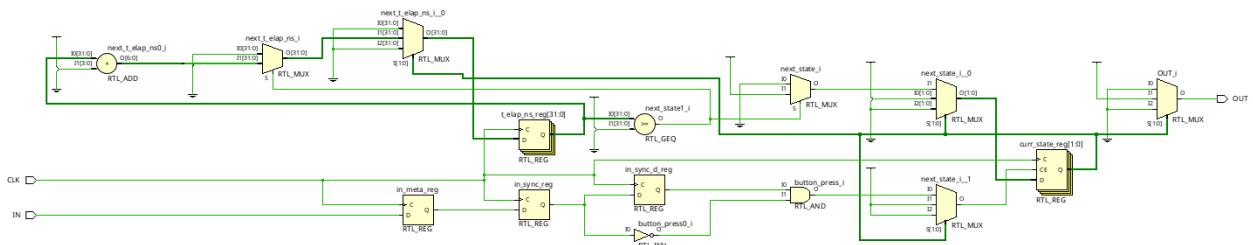


Figure 55: Debounce OneShot RTL

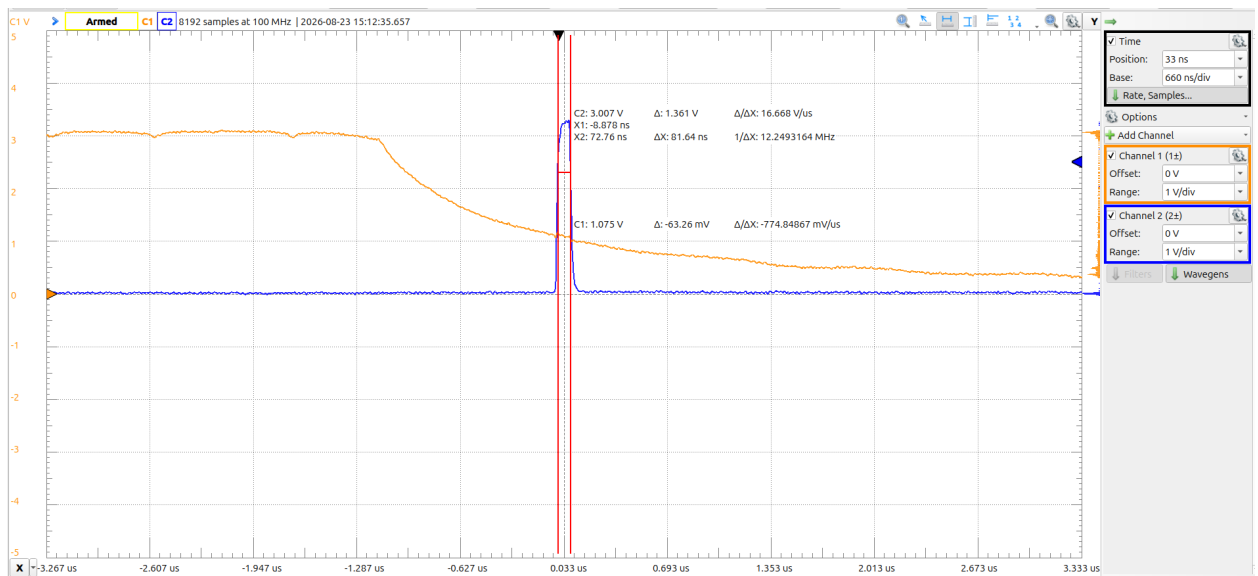


Figure 56: 80ns debounce measurement on hardware

- OTTER WRAPPER rtl [44](#) and test [45](#)

- OTTER MCU rtl [39](#) and test [40](#)
- OTTER CSR rtl [49](#) and test [50](#)
- OTTER OneShot debounce rtl [47](#) and test [48](#)
- Asm hardware test file for Interrupt OTTER WRAPPER [51](#).

0.2 Software

0.2.1 SW1 - Intro to Assembly Lang

Note : Assume caller/callee does not need to preserve their respective registers in order to use them. KISS method applied for assignment 1.

Part 1

"Read three 16-bit unsigned values consecutively from SWITCHES (address 0x11000000), add them together, and output the result to LEDS (address 0x11000020). Assume the input values are 16-bit unsigned values. (Assume it is possible for the switches to change between each time they are read)"

Assuming the LEDS input value is limited to 16-bit unsigned as well.

Flowchart

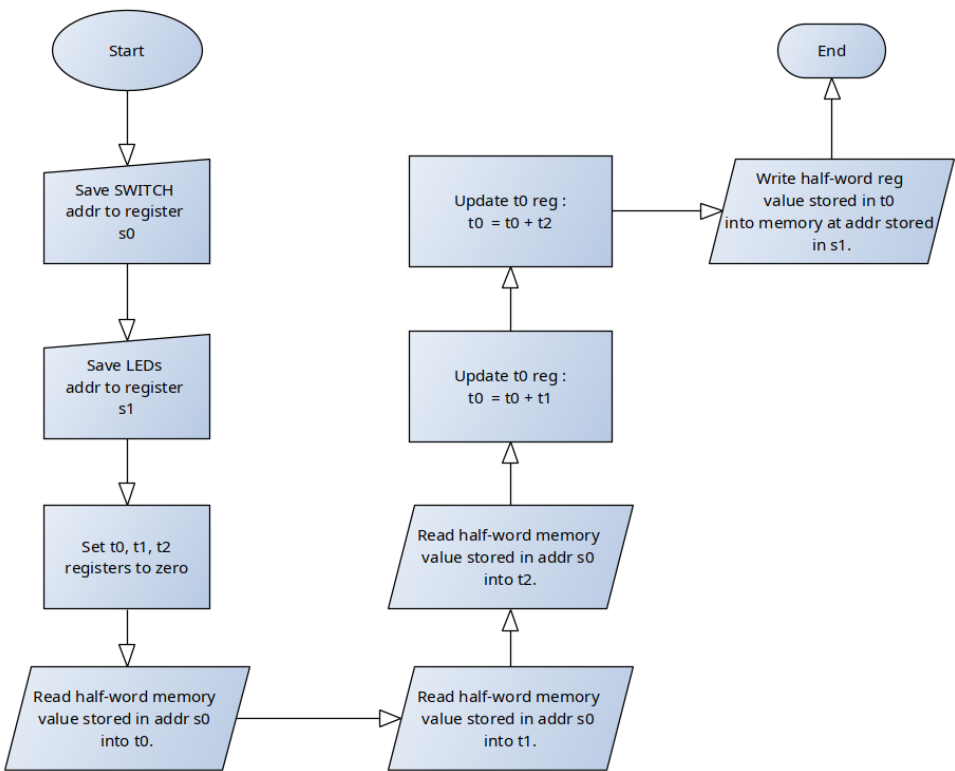


Figure 57: Flow Chart for Part 1

Verification Simulations

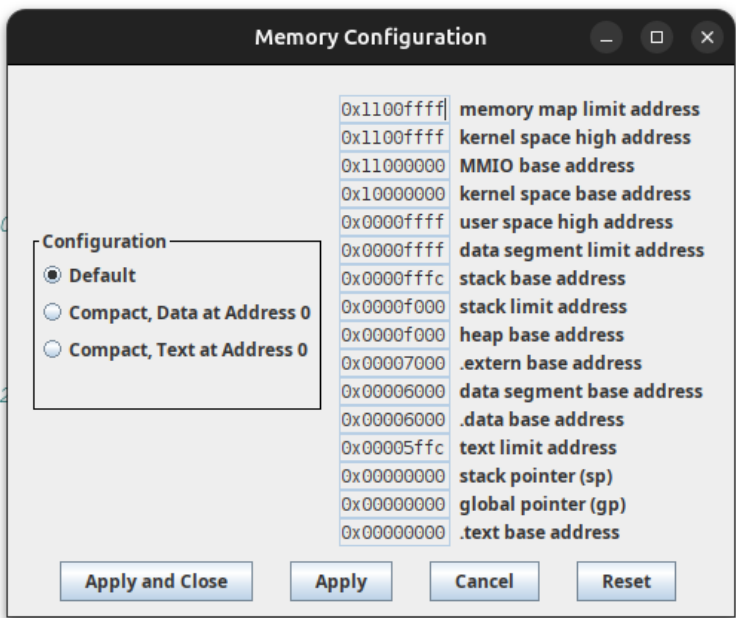


Figure 58: RARS Memory Configuration

```

20
21 #.equ SWITCHES_H,      0x11000 # upper 20 bits
22 #.equ SWITCHES_L,      0x000    # lower 12 bits
23 .equ SWITCHES_H,       0x00006 # upper 20 bits (data seg at 0x00006000)
24 .equ SWITCHES_L,       0x000    # lower 12 bits
25
26 #.equ LEDS_H,          0x11000 # upper 20 bits
27 #.equ LEDS_L,          0x020    # lower 12 bits
28 .equ LEDS_H,           0x00006 # upper 20 bits (data seg at 0x00006020)
29 .equ LEDS_L,           0x020    # lower 12 bits
30

```

Figure 59: Address Correction to work with RARS

- Test Case 1

```

33 # TC1 : Word Clip
34 .equ READ1_H,          0x00001 # switch first read (0x1234)
35 .equ READ1_L,          0x234
36
37
38 .equ READ2_H,          0x12345 # switch second read (0x5678)
39 .equ READ2_L,          0x678
40
41
42 .equ READ3_H,          0x11111 # switch third read (0x1111)
43 .equ READ3_L,          0x111
44 #   answer= 0x79BD

```

Figure 60: TC1 given inputs

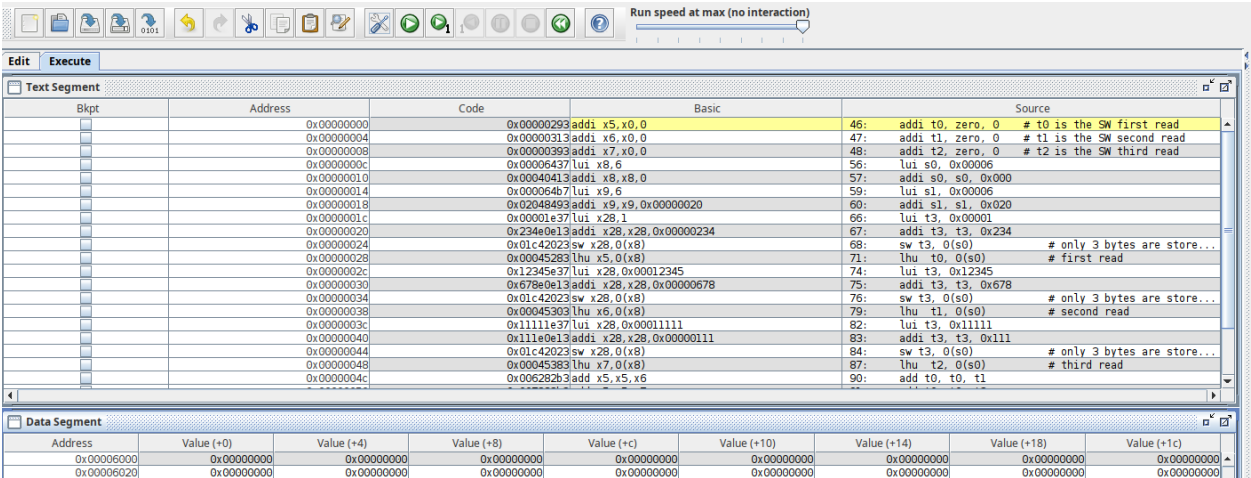


Figure 61: Data segment before executing instructions

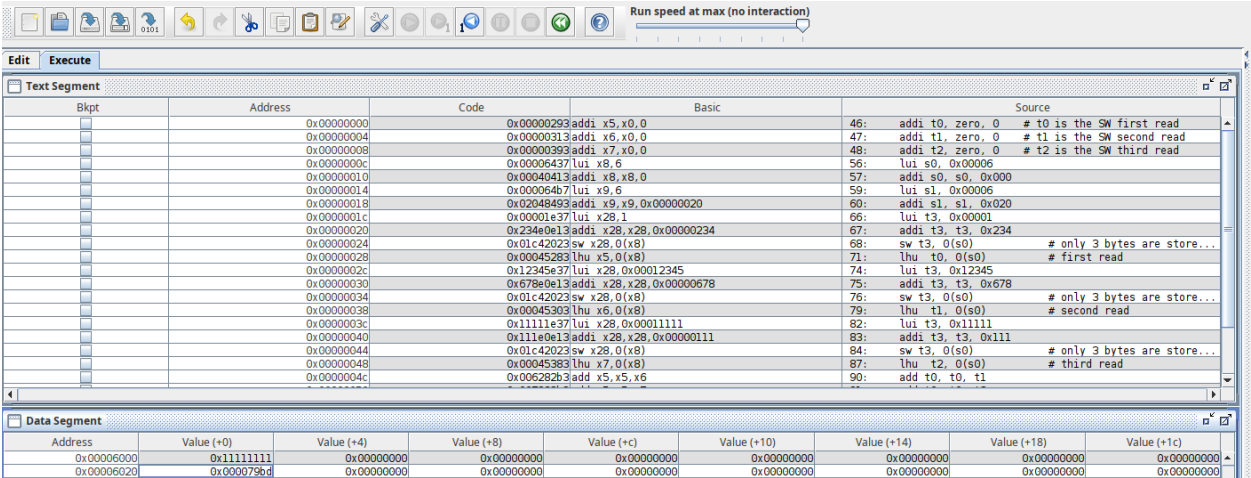


Figure 62: Data segment after executing instructions

Result at LEDS address in 62 matches the answer in 60.

- Test Case 2

```

46 # TC2 : Maxed
47 .equ READ1_H,      0x00005 # switch first read (0x5555)
48 .equ READ1_L,      0x555
49
50 .equ READ2_H,      0x00005 # switch second read (0x5555)
51 .equ READ2_L,      0x555 # sign extended
52
53
54 .equ READ3_H,      0x00005 # switch third read (0x5555)
55 .equ READ3_L,      0x555
56 # answer= 0xFFFF

```

Figure 63: TC2 given input

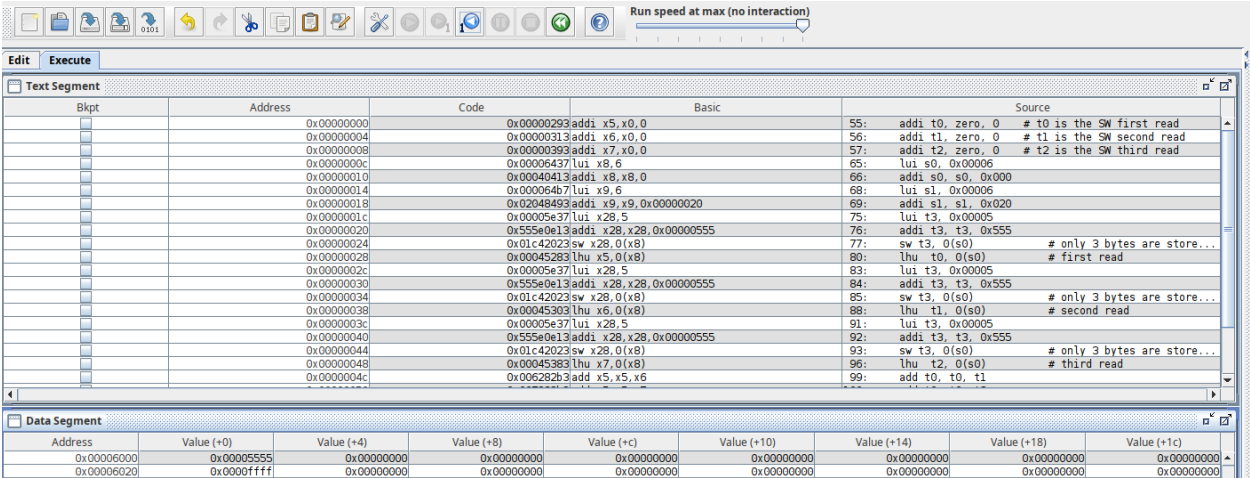


Figure 64: Data segment after executing instructions

Result at LEDS address in 64 matches the answer in 63.

- Test Case 3

```

59 # TC3 : Overflow
60 .equ READ1_H,      0x00005 # switch first read (0x5555)
61 .equ READ1_L,      0x555
62
63 .equ READ2_H,      0x00005 # switch second read (0x5555)
64 .equ READ2_L,      0x555 # sign extended
65
66
67 .equ READ3_H,      0x00005 # switch third read (0x5557)
68 .equ READ3_L,      0x557
69 # answer= 0x0001

```

Figure 65: TC3 given input

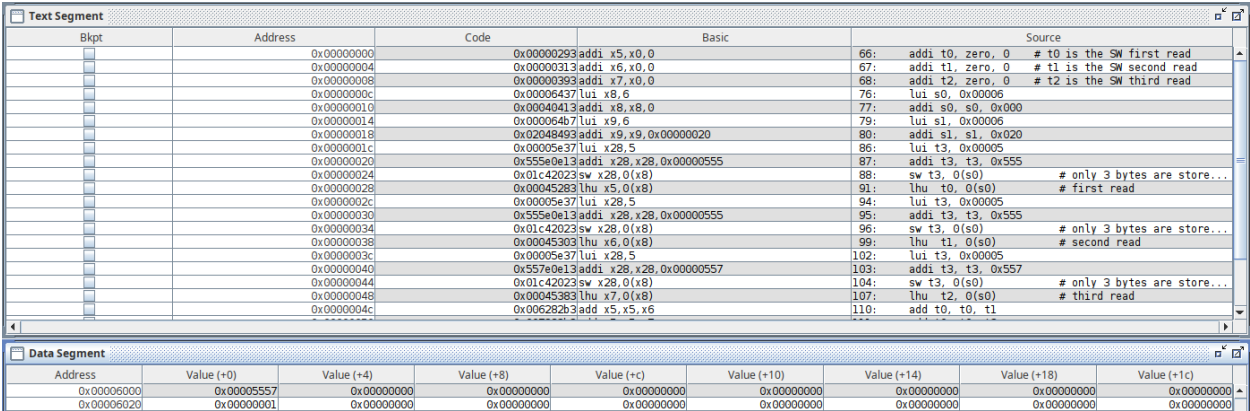


Figure 66: Data segment after executing instructions

Result at LEDS address in 66 matches the answer in 65.

- Test Case 4

2

Figure 67: TC4 given input

[illegible]

Figure 68: Data segment after executing instructions

Result at LEDS address in 68 matches the answer in 67.

- **Test Case 5**

94 #

Figure 69: TC5 given input

[illegible]

Figure 70: Data segment after executing instructions

Result at LEDS address in 70 matches the answer in 69.

- **Test Case 6**

```

96
97 # TC6 : All highs into full
98 .equ READ1_H,      0x00008 # switch first read
99 .equ READ1_L,      0x000
100
101 .equ READ2_H,      0x00008 # switch second read
102 .equ READ2_L,      0x000
103
104 .equ READ3_H,      0x00008 # switch third read
105 .equ READ3_L,      0x000
106
107 #   answer= 0x8000 (0x18000 as full word though)

```

Figure 71: TC6 given input

Text Segment								
Bkpt	Address	Code	Basic	Source				
	0x00000000	0x00000293	addi x5,x0,0	99: addi t0, zero, 0 # t0 is the SW first read				
	0x00000004	0x00000313	addi x6,x0,0	100: addi t1, zero, 0 # t1 is the SW second read				
	0x00000008	0x00000393	addi x7,x0,0	101: addi t2, zero, 0 # t2 is the SW third read				
	0x0000000c	0x00006437	lui x8,6	109: lui s0, 0x00006				
	0x00000010	0x00040413	addi x8,x8,0	110: addi s0, s0, 0x000				
	0x00000014	0x000064b7	lui x9,6	112: lui s1, 0x00006				
	0x00000018	0x02048493	addi x9,x9,0x00000020	113: addi s1, s1, 0x020				
	0x0000001c	0x00008e37	lui x28,8	119: lui t3, 0x00008				
	0x00000020	0x000e6e13	ori x28,x28,0	120: ori t3, t3, 0x000				
	0x00000024	0x01c42023	sw x28,0(x8)	121: sw t3, 0(s0) # only 3 bytes are store...				
	0x00000028	0x00045283	lhu x5,0(x8)	124: lhu t0, 0(s0) # first read				
	0x0000002c	0x00008e37	lui x28,8	127: lui t3, 0x00008				
	0x00000030	0x000e6e13	ori x28,x28,0	128: ori t3, t3, 0x000				
	0x00000034	0x01c42023	sw x28,0(x8)	129: sw t3, 0(s0) # only 3 bytes are store...				
	0x00000038	0x00045303	lhu x6,0(x8)	132: lhu t1, 0(s0) # second read				
	0x0000003c	0x00008e37	lui x28,8	135: lui t3, 0x00008				
	0x00000040	0x000e6e13	ori x28,x28,0	136: ori t3, t3, 0x000				
	0x00000044	0x01c42023	sw x28,0(x8)	137: sw t3, 0(s0) # only 3 bytes are store...				
	0x00000048	0x00045383	lhu x7,0(x8)	140: lhu t2, 0(s0) # third read				
	0x0000004c	0x006282b3	add x5,x5,x6	143: add t0, t0, t1				
Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00005000	0x00008000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x00008000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 72: Data segment after executing instructions

Result at LEDS address in 72 matches the answer in 71.

Assembly Source Code

Listing 5: Question 1 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory three consecutive
4 #           times, summing the unsigned half-word reads and writing
5 #           to the LEDS addr.
6 #
7 # Notes :
8 #   - Assume caller/calle has no need to preserve reg values.
9 #     KISS method for this assignment
10 #
11 #   - Split 32-bit addresses according to instr imm handling.
12
13
14
15
16 .global main
17
18 .equ SWITCHES_H, 0x11000 # upper 20 bits
19 .equ SWITCHES_L, 0x000 # lower 12 bits
20 .equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
21 .equ SWITCHES_L, 0x000 # lower 12 bits
22
23 .equ LEDS_H, 0x11000 # upper 20 bits
24 .equ LEDS_L, 0x020 # lower 12 bits
25 .equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
26 .equ LEDS_L, 0x020 # lower 12 bits
27
28 # ~~~~~ Test Cases ~~~~~
29 # TC1 : Word Clip
30 .equ READ1_H, 0x00001 # switch first read (0x1234)
31 .equ READ1_L, 0x234
32
33 .equ READ2_H, 0x12345 # switch second read (0x5678)
34 .equ READ2_L, 0x678
35
36 .equ READ3_H, 0x11111 # switch third read (0x1111)
37 .equ READ3_L, 0x111
38 # answer= 0x79BD
39
40 # TC2 : Maxed
41 .equ READ1_H, 0x00005 # switch first read (0x5555)

```

```

42 #.equ READ1_L,      0x555
43
44 #.equ READ2_H,      0x00005 # switch second read (0x5555)
45 #.equ READ2_L,      0x555 # sign extended
46
47 #.equ READ3_H,      0x00005 # switch third read (0x5555)
48 #.equ READ3_L,      0x555
49 # answer= 0xFFFF
50
51 # TC3 : Overflow
52 #.equ READ1_H,      0x00005 # switch first read (0x5555)
53 #.equ READ1_L,      0x555
54
55 #.equ READ2_H,      0x00005 # switch second read (0x5555)
56 #.equ READ2_L,      0x555 # sign extended
57
58 #.equ READ3_H,      0x00005 # switch third read (0x5557)
59 #.equ READ3_L,      0x557
60 # answer= 0x0001
61
62 # TC4 : zero
63 #.equ READ1_H,      0x00000 # switch first read (0)
64 #.equ READ1_L,      0x0
65
66 #.equ READ2_H,      0x00000 # switch second read (0)
67 #.equ READ2_L,      0x0 # sign extended
68
69 #.equ READ3_H,      0x00000 # switch third read (0)
70 #.equ READ3_L,      0x0
71 # answer= 0x0000
72
73 # TC5 : MAXXXED
74 #.equ READ1_H,      0x0000F # switch first read (0xFFFF)
75 #.equ READ1_L,      -1
76
77 #.equ READ2_H,      0xFFFFF # switch second read (0xFFFF)
78 #.equ READ2_L,      -1
79
80 #.equ READ3_H,      0x00000 # switch third read (0)
81 #.equ READ3_L,      0
82 # answer= 0xFFFFE
83
84 # TC6 : All highs into full
85 .equ READ1_H,      0x00008 # switch first read
86 .equ READ1_L,      0x000
87
88 .equ READ2_H,      0x00008 # switch second read
89 .equ READ2_L,      0x000
90
91 .equ READ3_H,      0x00008 # switch third read
92 .equ READ3_L,      0x000
93 # answer= 0x8000 (0x18000 as full word though)
94 # ~~~~~
95
96 main:
97
98 # ~~~~ register init ~~~~
99 addi t0, zero, 0 # t0 is the SW first read
100 addi t1, zero, 0 # t1 is the SW second read
101 addi t2, zero, 0 # t2 is the SW third read
102 # t3 is a dummy variable for testing
103
104 # s0 contains the SWITCH addr
105 # s1 contains the LEDS addr
106
107
108 # ~~~~ load addr into registers ~~~~
109 lui s0, SWITCHES_H
110 addi s0, s0, SWITCHES_L
111
112 lui s1, LEDS_H
113 addi s1, s1, LEDS_L
114
115
116 # ~~~~ read SWITCHES and sum ~~~~
117
118 # ~~ behind the scenes : switch mem changes value ~~
119 lui t3, READ1_H
120 ori t3, t3, READ1_L
121 sw t3, 0(s0) # only 3 bytes are stored (0x234)
122 # ~~~~~
123
124 lhu t0, 0(s0) # first read
125
126 # ~~ behind the scenes : switch mem changes value ~~
127 lui t3, READ2_H
128 ori t3, t3, READ2_L
129 sw t3, 0(s0) # only 3 bytes are stored (0x678)
130 # ~~~~~
131
132 lhu t1, 0(s0) # second read

```

```
133
134 # ~~~~ behind the scenes : switch mem changes value ~~~~
135 lui t3, READ3_H
136 ori t3, t3, READ3_L
137 sw t3, 0(s0)      # only 3 bytes are stored (0x111)
138 # ~~~~~
139
140 lhu t2, 0(s0)      # third read
141
142 # ~~~~ sum values ~~~~
143 add t0, t0, t1
144 add t0, t0, t2
145
146
147 # ~~~~ Write the unsigned half word to LEDS mem addr
148 sh t0, 0(s1)
149
150
151 # ~~~~ return with exit status ~~~~
152 li a7, 10 # load system call number for exit()
153 li a0, 0  # Load the exit status code ( EXIT_SUCCESS )
154 ecall    # Make the system call
```

Part 2

"Read an input from SWITCHES (address 0x11000000). Assume the input is a 16-bit signed value in 2's complement (RC). Change the sign of the input and output the result to LEDS (address 0x11000020). The result should also be a 16-bit signed value in 2's complement (RC)."

Flowchart

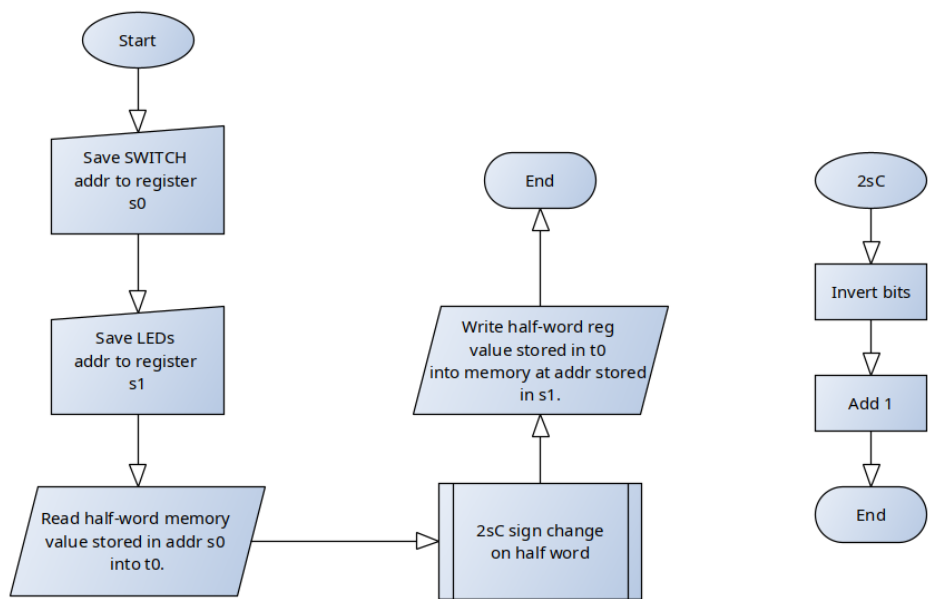


Figure 73: Flow Chart for Part 2

Verification Simulations

Test cases passed with definitions defined in code below.

```
16 # TC1 : zero
17 #A: .half 0
18 # answer= 0x0000
```

Data Segment	
Address	Value (+0)
0x00006000	0x00000000
0x00006020	0x00000000

Figure 74: IO and Data Segment of TC1

```
21 # TC2 : positive
22 #A: .half 0x5
23 # answer= 0xFFFFB (-5)
```

Data Segment	
Address	Value (+0)
0x00006000	0x00000005
0x00006020	0x0000ffffb

Figure 75: IO and Data Segment of TC2

25	# TC3 : negative		
26	#A: .half 0xFFFB	# -5	
27	# answer= 0x0005	# 5	
28			

Data Segment	
Address	Value (+0)
0x00006000	0x0000ffff
0x00006020	0x00000005

Figure 76: IO and Data Segment of TC3

30	# TC4 : positive		
31	#A: .half 0x5678	# 22136	
32	# answer= 0xA988	# -22136	

Data Segment	
Address	Value (+0)
0x00006000	0x00005678
0x00006020	0x0000a988

Figure 77: IO and Data Segment of TC4

34	# TC5 : reverse		
35	#A: .half 0xA988		
36	# answer= 0x5678		
37			

Data Segment	
Address	Value (+0)
0x00006000	0x0000a988
0x00006020	0x00005678

Figure 78: IO and Data Segment of TC5

39	# TC6 : negative		
40	#A : .half 0xFFFF #(-1)		
41	# answer= 1		
42			

Data Segment	
Address	Value (+0)
0x00006000	0x0000ffff
0x00006020	0x00000001

Figure 79: IO and Data Segment of TC6

43	# TC7 : negative 2		
44	#A : .half 0xFFFE #(-2)		
45	# answer= 2		
46			

Data Segment	
Address	Value (+0)
0x00006000	0x0000fffe
0x00006020	0x00000002

Figure 80: IO and Data Segment of TC7

48	# TC7 : negative 32 bit		
49	#A : .word 0x89ABCDEF	#(0xCDEF = -12817)	
50	# answer= 0x3211	# 12817	
51			

Data Segment	
Address	Value (+0)
0x00006000	0x89abcdef
0x00006020	0x00003211

Figure 81: IO and Data Segment of TC8

Assembly Code

Listing 6: Question 2 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory a unsigned half-word
4 #           changing the sign, then writing to the LEDS addr.
5 #
6 # Notes :
7 #     - Assume caller/callee has no need to preserve reg values.
8 #     - KISS method for this assignment
9 #
10 #     - Split 32-bit addresses according to instr imm handling.
11
12 .data
13 # ~~~~~ Test Cases ~~~~~
14 # TC1 : zero
15 #A: .half 0
16 # answer= 0x0000
17
18 # TC2 : positive
19 #A: .half 0x5
20 # answer= 0xFFFB (-5)
21
22 # TC3 : negative
23 #A: .half 0xFFFB # -5
24 # answer= 0x0005 # 5
25
26 # TC4 : positive
27 #A: .half 0x5678 # 22136

```

```

28 # answer= 0xA988      # -22136
29
30 # TC5 : reverse
31 #A: .half 0xA988
32 # answer= 0x5678
33
34 # TC6 : negative
35 #A : .half 0xFFFF #(-1)
36 # answer= 1
37
38 # TC7 : negative 2
39 #A : .half 0xFFFE #(-2)
40 # answer= 2
41
42 # TC7 : negative 32 bit
43 #A : .word 0x89ABCDEF    #(0xCDEF = -12817)
44 # answer= 0x3211      # 12817
45
46 # ~~~~~
47
48
49 .global _start
50 .text
51 #.equ SWITCHES_H, 0x11000 # upper 20 bits
52 #.equ SWITCHES_L, 0x000 # lower 12 bits
53 #.equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
54 #.equ SWITCHES_L, 0x000 # lower 12 bits
55
56 #.equ LEDS_H, 0x11000 # upper 20 bits
57 #.equ LEDS_L, 0x020 # lower 12 bits
58 #.equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
59 #.equ LEDS_L, 0x020 # lower 12 bits
60
61
62
63 _start:
64
65 # ~~~~ register init ~~~~
66 addi t0, zero, 0 # t0 is the SW first read
67 # t3 is a dummy variable for testing
68
69 # s0 contains the SWITCH addr
70 # s1 contains the LEDS addr
71
72 # ~~~~ load addr into registers ~~~~
73 lui s0, SWITCHES_H
74 addi s0, s0, SWITCHES_L
75
76 lui s1, LEDS_H
77 addi s1, s1, LEDS_L
78
79
80 # ~~~~ read SWITCHES and sum ~~~~
81
82 # ~~ behind the scenes : switch value ~~
83 lw t3, A
84 sw t3, 0(s0)
85 # ~~~~~
86
87 lhu t0, 0(s0) # first read
88
89
90 # ~~~~ invert the sign (2sC) ~~~~
91 not t0, t0
92 addi t0, t0, 1
93
94 # ~~~~ Write the unsigned half word to LEDS mem addr
95 sh t0, 0(s1)
96
97
98 # ~~~~ return with exit status ~~~~
99 li a7, 10 # load system call number for exit()
100 li a0, 0 # Load the exit status code ( EXIT_SUCCESS )
101 ecall # Make the system call

```

0.2.2 SW2 - Conditional Statements

Assignment:
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input is greater than or equal to 32,768, the value is divided by 4. You can ignore any remainder. If the value is less than 32,768, the value is multiplied by 2. The result should be written to the 7 SEGMENT (address 0x11000040).
- 2. Read a 32-bit unsigned value from SWITCHES (address 0x11000000). If the input value is a multiple of 4, all of the bits should be inverted, otherwise, if the input value is odd, add 4095 and divide the result by 2, otherwise subtract 1 from the value. The result should be written to the 7 SEGMENT (address 0x11000040).

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 82: Questions to address

Part 1

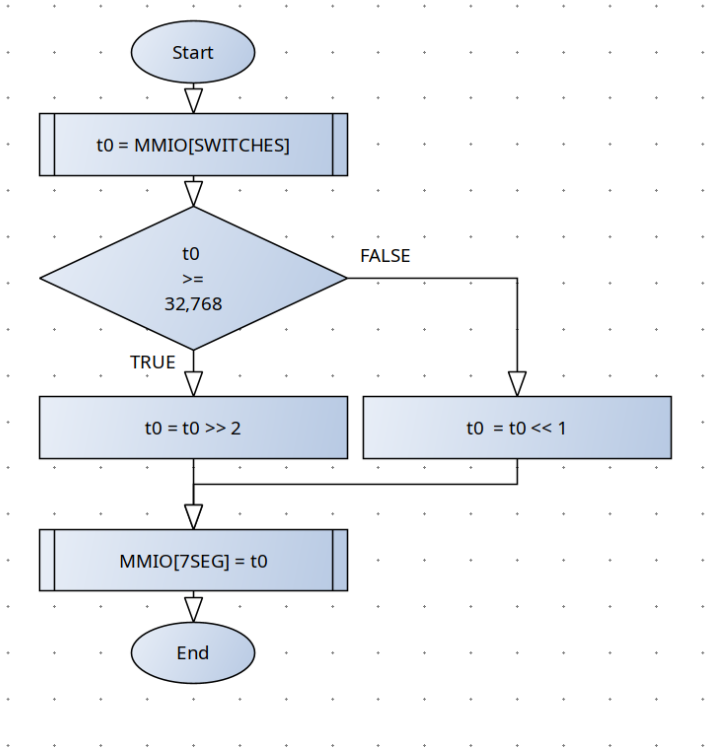


Figure 83: Flow chart for part 1

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Lowest unsigned val	0	0	Passed
2	Below boundary	0xFFF	0x1FFE	Passed
3	1 before boundary	0x7FFF	0xFFFE	Passed
4	On boundary	0x8000	0x2000	Passed
5	1 above boundary	0x8001	0x2000	Passed
6	Above boundary	0x12345678	0x048D159E	Passed
7	32-bit unsigned ceiling	0xFFFFFFFF	0x3FFFFFFF	Passed

Table 4: Test cases for part 1

Listing 7: sw2_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Conditionals, branching, jumping
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #     constant CONST
8 #
9 # Output :
10 #     Write to 7SEG addr
11
12
13 .data
14 # ~~~~ Test Cases ~~~~
15
16 # ~~ TC1 : Zero ~~
17 #TC:      .word 0
18 # answer = 0 (verified)
19
20 # ~~ TC2 : Under ~~
21 #TC:      .word 0xFFFF
22 # answer = 0x1FFE (verified)
23
24 # ~~ TC3 : One before ~~
25 #TC:      .word 0x07FFF
26 # answer = 0xFFFE (verified)
27
28 # ~~ TC4 : On boundary ~~
29 #TC:      .word 0x08000
30 # answer = 0x2000 (verified)
31
32 # ~~ TC5 : Above boundary ~~
33 #TC:      .word 0x08001
34 # answer = 0x2000 (verified)
35
36 # ~~ TC6 : Above boundary 2 ~~
37 #TC:      .word 0x12345678
38 # answer = 0x048D159E (verified)
39
40 # ~~ TC7 : 32-bit unsigned ceiling ~~
41 #TC:      .word 0xFFFFFFFF
42 # answer = 0x3FFFFFFF (verified)
43
44 .global _start
45
46 .text
47 .equ MMIO_SWITCHES, 0x11000000
48 .equ MMIO_SEVSEG, 0x11000040
49 .equ CONST, 32768
50
51 _start:
52 # ~~~~ init reg ~~~~
53 li t1, MMIO_SWITCHES    # t1 is the addr for MMIO SWITCHES
54 li t2, CONST            # t2 is the constant 32768
55 li t3, MMIO_SEVSEG      # t3 is the addr for MMIO 7SEG
56                          # t4 is a test case variable
57
58 # ~~~~ TC injection ~~~~
59 lw t4, TC
60 sw t4, 0(t1)
61
62 # ~~~~ read SWITCHES ~~~~
63 lw t0, 0(t1)            # load SW value into t0
64
65 # ~~~~ unsigned branch ~~~~
66 bgeu t0, t2, Div4
67
68 # ~~ else (false) case ~~
69 slli t0, t0, 1          # multiply by 2
70 j End
71
72 # ~~ if (true) case ~~
73 Div4:
74 srli t0, t0, 2          # logical shift for unsigned
75
76 # ~~~~ write to 7SEG ~~~~
77 End:
78 sw t0, 0(t3)
79
80 # ~~~~ syscall exit status ~~~~
81 li a7, 10              # load syscall num for exit()
82 li a0, 0               # EXIT_SUCCESS code
83 ecall                 # execute sys call
84
85

```

Figure 84: Assembly code for SW2 part 1

Part 2

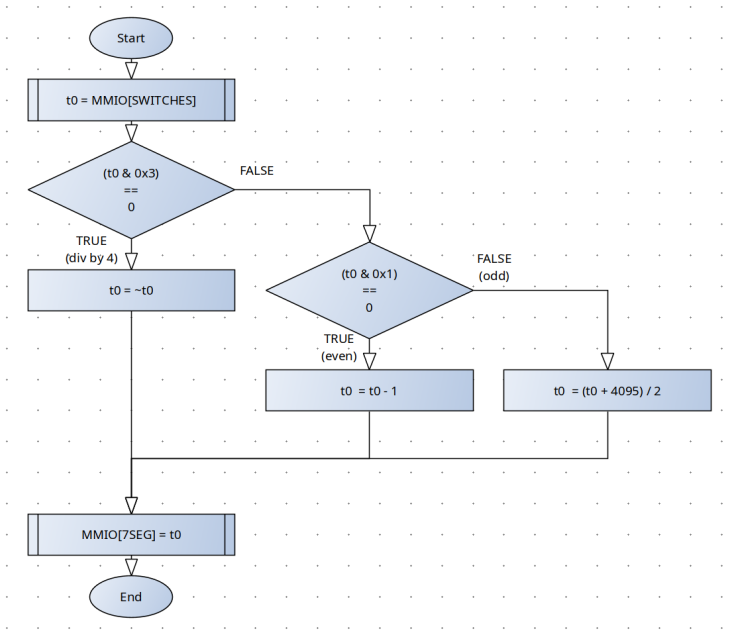


Figure 85: Flow chart for part 2

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Zero is div by 4	0	0xFFFFFFFF	Passed
2	Two is even	0x2	0x1	Passed
3	One is odd	0x1	0x0800	Passed
4	32-bit unsinged ceiling	0xFFFFFFFF	0x07FF	Passed
5	Div by 4 near ceiling	0xFFFFFFFFC	0x3	Passed
6	Odd near ceiling	0xFFFFFFFDD	0x07FE	Passed
7	Even near ceiling	0xFFFFFFFEE	0xFFFFFFFDD	Passed

Table 5: Test cases for part 1

Listing 8: sw2_p2.asm

```

1  # Author : WhisperingRock
2  #
3  # Purpose : Conditionals, branching, jumping
4  # Input :
5  #     SWITCHES MMIO address to 32-bit unsigned value
6  #     7SEG MMIO address to 8-digits, each of 4-bits
7  #     constant CONST
8  #
9  # Output :
10 #     Write to 7SEG addr
11
12
13 .data
14 # ~~~~ Test Cases ~~~~
15
16 # ~~ TC1 : Zero (Div4) ~~
17 #TC:      .word 0
18 # answer = 0xFFFF_FFFF (verified)
19
20 # ~~ TC2 : Two (Div2 = even) ~~
21 #TC:      .word 2
22 # answer = 0x1 (verified)
23
24 # ~~ TC3 : 1 (odd) ~~
25 #TC:      .word 0x1
26 # answer = 0x0800 (verified)
27
28 # ~~ TC4 : 32-bit unsigned ceiling ~~
29 #TC:      .word 0xFFFFFFFF
30 # answer = 0x0000_07FF | 2047 (verified)
31
32 # ~~ TC5 : unsigned ceiling Div4 ~~
33 #TC:      .word 0xFFFF_FFC
34 # answer = 0x03 (verified)
35
36 # ~~ TC6 : unsigned ceiling odd ~~
37 #TC:      .word 0xFFFF_FFD
38 # answer = 0x07FE (verified)
39
40 # ~~ TC6 : unsigned ceiling even ~~
41 #TC:      .word 0xFFFF_FFE
42 # answer = 0xFFFF_FFD (verified)
43
44 .global _start
45
46 .text
47 .equ MMIO_SWITCHES, 0x11000000
48 .equ MMIO_SEVSEG, 0x11000040
49
50
51 _start:
52 # ~~~~ init reg ~~~~
53 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
54 li s2, MMIO_SEVSEG   # s2 is the addr for MMIO 7SEG
55                       # t0 is the working variable for SWITCHES value
56                       # t1 is a logic case variable
57 li t2, 4095          # t2 is reserved for const
58
59 # ~~~~ TC injection ~~~~
60 lw t0, TC
61 sw t0, 0(s1)
62
63 # ~~~~ read SWITCHES ~~~~
64 lw t0, 0(s1) # load SW value into t0
65
66 # ~~~~ branch : div by 4 ~~~~
67 andi t1, t0, 0x3
68 beqz t1, Div4
69
70 # ~~~~ branch : div by 2 ~~~~
71 andi t1, t0, 0x1
72 beqz t1, Div2
73
74 # ~~ else case ~~
75 add t0, t0, t2 # add const 4095
76 srli t0, t0, 1 # then scale
77 j End
78
79 Div4:
80 not t0, t0 # invert bits
81 j End
82
83 Div2:
84 addi t0, t0, -1
85
86 End: # ~~~~ write to 7SEG ~~~~
87 sw t0, 0(s2)
88
89
90
91 # ~~~~ syscall exit status ~~~~
92 li a7, 10 # load syscall num for exit()
93 li a0, 0 # EXIT_SUCCESS code
94 ecalls # execute sys call
95
96

```

0.2.3 SW3 - Loops

Assignment:

First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Read a 32-bit value from SWITCHES (address 0x11000000). Assume the 32-bit value is the combination of two 16-bit unsigned values. Split the 32-bit input into two 16-bit values (the upper and lower 16-bits). Assume both of the 16-bit values are unsigned. Multiply the two 16-bit values together and output the result as a 32-bit unsigned value to 7 SEGMENT (address 0x11000040).
- 2. Read a 32-bit value from SWITCHES (address 0x11000000), delay for 0.5 seconds, and then output the value to 7 SEGMENT (address 0x11000040). The OTTER MCU operates at 25 MHz so each instruction takes 40 ns to run. (Hint: Use loops.) **There is no method to verify the run time in RARS and RARS does not execute at the same speed as the OTTER.** You will need to verify your timing by showing your calculation for the number of instructions being performed between the load (reading from the switches) and store (writing to the 7 segment display) instructions.

Assume the BASYS3 SWITCHES are expanded to 32 individual switches and 7 SEGMENT is expanded to 8 digits.

Figure 87: Questions to address

Part 1

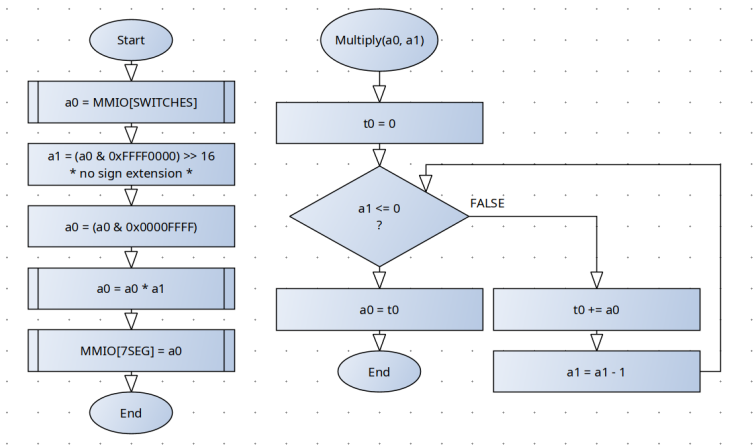


Figure 88: Flow chart for part 1

TC#	Desc	Given MMIO[SWITCHES]	Correct MMIO[7SEG]	Result
1	Zero	0x0000_0000	0x0000_0000	Passed
2	Zero + One	0x0000_0001	0x0000_0000	Passed
3	One + Zero	0x0001_0000	0x0000_0000	Passed
4	One + One	0x0001_0001	0x0000_0001	Passed
5	num1	0x0013_0004	0x0000_004C	Passed
6	num1 reverse	0x0004_0013	0x0000_004C	Passed
7	shift 1	0xFFFF_FFFF	0xFFFE_0001	Passed
8	shift 2	0x0010_FFFF	0x000F_FFF0	Passed
9	shift 3	0x0100_FFFF	0x00FF_FF00	Passed

Table 6: Test cases for part 1

Listing 9: sw3_p1.asm

```
1 # Author : WhisperingRock
2 #
3 # Purpose : Half word multiply
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #
8 # Output :
9 #     Write to 7SEG addr
10
11
12 .data
13 # ~~~~ Test Cases ~~~~
14
15 # ~~ TC1 : Zero ~~
16 #TC:      .word 0
17 # answer = 0      (verified)
```

```

18
19 # ~~ TC2 : Zero + One ~~
20 #TC:      .word 0x00000001
21 # answer = 0      (verified)
22
23 # ~~ TC3 : One + Zero ~~
24 #TC:      .word 0x00010000
25 # answer = 0      (verified)
26
27 # ~~ TC4 : One ~~
28 #TC:      .word 0x00010001
29 # answer = 1      (verified)
30
31 # ~~ TC5 : num1 ~~
32 #TC:      .word 0x00130004
33 # answer = 0x0000_004C (verified)
34
35 # ~~ TC6 : num1 reverse ~~
36 #TC:      .word 0x00040013
37 # answer = 0x0000_004C (verified)
38
39 # ~~ TC7 : shift1 ~~
40 #TC:      .word 0xFFFFFFFF
41 # answer = 0xFFFE_0001 (verified)
42
43 # ~~ TC8 : shift2 ~~
44 #TC:      .word 0x0010FFFF
45 # answer = 0x000F_FFF0 (verified)
46
47 # ~~ TC9 : shift3 ~~
48 #TC:      .word 0x0100FFFF
49 # answer = 0x00FF_FF00 (verified)
50
51 .global _start
52
53 .text
54 #.equ MMIO_SWITCHES, 0x11000000 # moved to Compact(Text @ 0) config
55 #.equ MMIO_SEVSEG, 0x11000040 # sp moved tp 0x00003FFC
56 #.equ MMIO_SWITCHES, 0x00007F00
57 #.equ MMIO_SEVSEG, 0x00007F40
58 #.equ UPPER_WORD, 0xFFFF0000
59 #.equ LOWER_WORD, 0x0000FFFF
60
61 _start:
62 # ~~~~ init reg ~~~~
63 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
64 li s2, MMIO_SEVSEG # s2 is the addr for MMIO 7SEG
65 # a0 is a working variable for input and return
66 # a1 is a working var for upper half of word
67 # a2 is a working var for lower half of word
68 # t0 is single use TC injection var
69
70 # ~~~~ TC injection ~~~~
71 lw t0, TC
72 sw t0, 0(s1)
73
74 # ~~~~ read SWITCHES ~~~~
75 lw a0, 0(s1)
76
77 # ~~~~ parse input val (and load args for func) ~~~~
78 # ~~ a1 = (a0 & 0xFFFF0000) >> 16 ~~
79 li t0, UPPER_WORD
80 and a1, a0, t0
81 srli a1, a1, 16
82 # ~~ a0 = a0 & 0x0000FFFF ~~
83 li t0, LOWER_WORD
84 and a0, a0, t0
85
86 # ~~~~ caller(_start) saves non-preserved ~~~~
87 # "Not needed b/c we dont need t0"
88 jal Multiply
89
90 # ~~~~ write return to 7SEG ~~~~
91 sw a0, 0(s2)
92
93 # ~~~~ syscall exit ~~~~
94 li a7, 10 # load syscall num for exit()
95 li a0, 0 # EXIT_SUCCESS code
96 ecall # execute sys call
97
98
99 # ~~~~~
100 # Purpose : Multiply two unsigned 32-bit numbers
101 # Input :
102 #     a0 : 32-bit unsigned word
103 #     a1 : 32-bit unsigned word
104 # Temps :
105 #     t0 :
106 # Note :
107 #     - consumes a1
108 # Return:

```

```

109 #      a0 : product of a0 * a1
110 Multiply:
111
112 # ~~~~ calle(Multiply) saves preserved ~~~~
113 # ~~ 1. calle(Multiply) allocates space on stack ~~
114 addi sp, sp, -12
115
116 # ~~ 2. calle(Multiply) stores reg values on stack ~~
117 sw ra, 8(sp)
118 sw s1, 4(sp)
119 sw s2, 0(sp)
120
121 # ~~ 3. calle(Multiply) function execution ~~
122 # ~ sum(t0) init ~
123 addi t0, zero, 0
124 # ~ while a1 <= 0
125 While:
126 bleu a1, zero, EndWhile
127 add t0, t0, a0
128 addi a1, a1, -1
129 j While
130 EndWhile:
131 addi a0, t0, 0
132
133 # ~~ 4. calle(Multiply) restores preserved from stack ~~
134 lw ra, 8(sp)
135 lw s1, 4(sp)
136 lw s2, 0(sp)
137
138 # ~~ 5. calle(Multiply) deallocates space on stack ~~
139 addi sp, sp, 12
140
141 # ~~ calle(Multiply) return ~~
142 jr ra
143 # ~~~~~~

```

Part 2

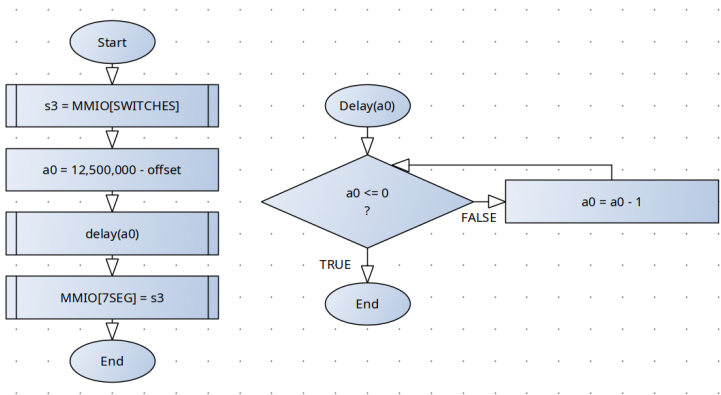


Figure 89: Flow chart for part 2

Consider the following calculations

$$\begin{aligned}
 12.5 \cdot 10^6 \text{ instr} &= 0.5s \cdot \frac{\text{instr}}{40ns} \cdot \frac{10^9 ns}{1s} \\
 &= 0x00BE_BC20
 \end{aligned}$$

- There are 13 instructions, after reading from SWITCHES, that are only executed once. Lets refer to these instructions as bias s.t. $b = 13$.
- There are 3 instruction in our delay loop s.t. $m = 3$.

Now lets relate our instructions to the instruction layout s.t. we identify how many times we actually need to loop.

$$\begin{aligned}
 3m + b &= 12.5 \cdot 10^9 \\
 m &= \frac{12,500,000 - 13}{3} \\
 &= 4,166,663
 \end{aligned}$$

Rather than use this number in directly, I opted to write the constant in terms of total instructions needed against how many loops we need.

$$\begin{aligned}
\text{offset} &= 12.5 \cdot 10^9 - m \\
&= 8,333,337 \\
&= 0x007F_2819
\end{aligned}$$

Listing 10: Assembly code for sw3_p2.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Delay
4 # Input :
5 #     SWITCHES MMIO address to 32-bit unsigned value
6 #     7SEG MMIO address to 8-digits, each of 4-bits
7 #
8 # Output :
9 #     Write to 7SEG addr
10
11
12 .data
13 # ~~~~ Test Cases ~~~~
14
15 # ~~ TC1 : simple ~~
16 TC:  .word 0x12345678
17 # answer = 0      (verified)
18
19 .global _start
20
21 .text
22 #.equ MMIO_SWITCHES, 0x11000000 # moved to Compact(Text @ 0) config
23 #.equ MMIO_SEVSEG, 0x11000040 # sp moved tp 0x00003FFC
24 .equ MMIO_SWITCHES, 0x00007F00
25 .equ MMIO_SEVSEG, 0x00007F40
26 .equ HALF_SEC_DELAY, 0x00BEC20 # 12,500,000 instr for 0.5s delay
27 .equ OFFSET, 0x007F2819 # loop and bias compensation (see notes)
28
29 _start:
30 # ~~~~ init reg ~~~~
31 li s1, MMIO_SWITCHES # s1 is the addr for MMIO SWITCHES
32 li s2, MMIO_SEVSEG # s2 is the addr for MMIO 7SEG
33 # s3 is a working variable for input and return
34 li a0, HALF_SEC_DELAY # a0 is reserved for const
35 li a1, OFFSET # a1 is reserved for offset
36 # t0 is single use TC injection var
37
38 # ~~~~ TC injection ~~~~
39 lw t0, TC
40 sw t0, 0(s1)
41
42 # ~~~~ read SWITCHES ~~~~
43 lw s3, 0(s1)
44
45 # ~~~~ caller(_start) saves non-preserved ~~~~
46 # "Not needed b/c we dont need t0"
47 sub a0, a0, a1 # a0 = delay - offset ; +1 bias
48 jal Delay # +1 bias
49
50 # ~~~~ write return to 7SEG ~~~~
51 sw s3, 0(s2)
52
53 # ~~~~ syscall exit ~~~~
54 li a7, 10 # load syscall num for exit()
55 li a0, 0 # EXIT_SUCCESS code
56 ecall # execute sys call
57
58
59 # ~~~~~~
60 # Purpose : delay(busy wait) of 0.5s
61 # Input :
62 #     a0 : [unsigned 32-bit] delay amount in instruction count
63 # Note :
64 #     - Consuming a0 in the process
65 # Return:
66 #
67
68 Delay:
69
70 # ~~~~ calle(Delay) saves preserved ~~~~
71 # ~~ 1. calle(Delay) allocates space on stack ~~
72 addi sp, sp, -16 # +1 bias
73
74 # ~~ 2. calle(Delay) stores reg values on stack ~~
75 sw ra, 12(sp) # +1 bias
76 sw s1, 8(sp) # +1 bias
77 sw s2, 4(sp) # +1 bias
78 sw s3, 0(sp) # +1 bias
79
80

```

```

81
82 # ~~ 3. calle(Delay) function execution ~~
83 # ~ while a0 <= 0
84 While: # loop = 3x instr
85     bleu a0, zero, EndWhile
86     addi a0, a0, -1
87     j While
88 EndWhile:
89
90 # ~~ 4. calle(Delay) restores preserved from stack ~~
91 lw ra, 12(sp)           # +1 bias
92 lw s1, 8(sp)           # +1 bias
93 lw s2, 4(sp)           # +1 bias
94 lw s3, 0(sp)           # +1 bias
95
96 # ~~ 5. calle(Delay) deallocates space on stack ~~
97 addi sp, sp, 16        # +1 bias
98
99 # ~~ calle(Delay) return ~~
100 jr ra                 # +1 bias
101 # ~~~~~~

```

0.2.4 SW4 - Arrays

Assignment:
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Use register alternative names (ie s0, t0) instead of register numbers (ie x5, x10). Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.

- 1. Using `.data`, define an array in the data segment that contains the first 25 values of the Fibonacci sequence (starting with 0 and ending with 46368). Write a program that progresses through the array and calculates the difference between values that are 3 spots away from each other. For example, the first value (0) and the 4th value (2) is a difference of 2. The next calculation would be between the 2nd (1) and 5th (3) values. When no item exists 3 spots away, no difference can be calculated. Each difference should be output to LEDS (address 0x11000020).
- 2. Create an array of 10 32-bit values. Read 10 32-bit values consecutively from SWITCHES (address 0x11000000) saving them in the array. After the 10th value has been saved, the array should be sorted from least to greatest. The sorted array should be output to LEDS (address 0x11000020) in order from least to greatest.

Assume the BASYS3 SWITCHES and LEDS are expanded to 32 individual switches and LEDs.

Figure 90: Questions to address

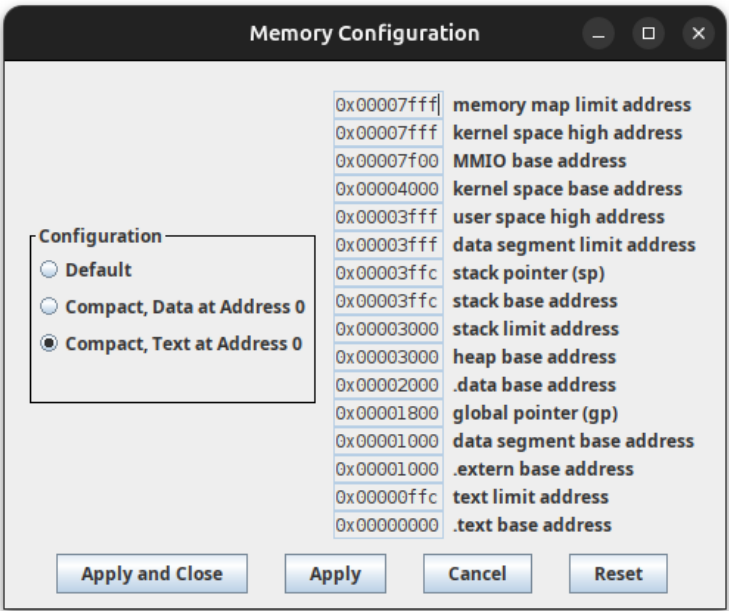


Figure 91: Memory Map for this Assignment

Part 1

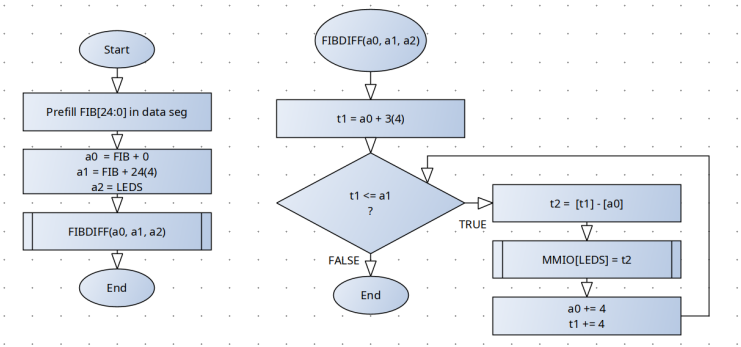


Figure 92: Flow chart for part 1

Listing 11: sw4_p1.asm

```
1 # Author : WhisperingRock
2 #
3 # Purpose : Difference between array elems
4 # Input :
5 #     Hardcoded values of fibonacci in array
6 #     LEDS MMIO address to 32 bits
7 #
8 # Output :
```

```

9  #       Write to LEDS addr
10
11
12 .data
13 FIB:  .word 0, 1, 1, 2, 3
14       .word 5, 8, 13, 21, 34
15       .word 55, 89, 144, 233, 377
16       .word 610, 987, 1597, 2584, 4181
17       .word 6765, 10946, 17711, 28657, 46368
18
19 # answer = 2, 2, 4, 6, 10, 16, 26, 42, 68, 110, 178,
20 #       288, 466, 754, 1220, 1974, 3194, 5168, 8362
21 #       13530, 21892, 35422
22 # Verified - values match and does not exceed bounds
23
24 .global _start
25
26 .text
27 #.equ MMIO_SWITCHES, 0x00007F00
28 #.equ MMIO_SEVSEG, 0x00007F40
29 #.equ MMIO_LEDS, 0x00007F20
30 #.equ LAST_ELEM, 96 # 25th elem, 4 bytes apart
31
32 _start:
33 # ~~~~ init reg ~~~~
34 la a0, FIB # a0 is the addr for head of FIB
35 addi a1, a0, LAST_ELEM # a1 is the addr for end of FIB
36 li a2, MMIO_LEDS # a2 is the addr for MMIO LEDS
37
38 # ~~~~ caller(_start) saves non-preserved ~~~~
39 # ~~ 1. caller(_start) allocates space on stack ~~
40 addi sp, sp, -24
41 # ~~ 2. caller(_start) stores reg values on stack ~~
42 sw t2, 20(sp)
43 sw t1, 16(sp)
44 sw t0, 12(sp)
45 sw a2, 8(sp)
46 sw a1, 4(sp)
47 sw a0, 0(sp)
48 # ~~ 3. caller(_start) function execution ~~
49 jal FIBDIFF
50 # ~~ 4. caller(_start) restores preserved from stack ~~
51 lw t2, 20(sp)
52 lw t1, 16(sp)
53 lw t0, 12(sp)
54 lw a2, 8(sp)
55 lw a1, 4(sp)
56 lw a0, 0(sp)
57 # ~~ 5. caller(_start) deallocates space on stack ~~
58 addi sp, sp, 24
59
60 # ~~~~ syscall exit ~~~~
61 li a7, 10 # load syscall num for exit()
62 li a0, 0 # EXIT_SUCCESS code
63 ecall # execute sys call
64
65
66 # ~~~~~~
67 # Purpose : For all elements of the array, write (TO LEDS) the difference
68 # between a elem and its left (3 over) neighbor, if they exist.
69 #
70 # Input :
71 # a0 : [unsigned 32-bit] head address of array
72 # a1 : [unsigned 32-bit] tail address of array
73 # a2 : [unsigned 32-bit] LEDS MMIO addr
74 #
75 # Return:
76 #
77
78 FIBDIFF:
79
80 # ~~~~ calle(FIBDIFF) saves preserved ~~~~
81 # ~~ 1. calle(FIBDIFF) allocates space on stack ~~
82 addi sp, sp, -4
83
84 # ~~ 2. calle(FIBDIFF) stores reg values on stack ~~
85 sw ra, 0(sp)
86
87 # ~~ 3. calle(FIBDIFF) function execution ~~
88
89 # ~ init vars ~
90 addi t1, a0, 12 # t1 is the reference index and right neighbor
91 addi t2, zero, 0 # t2 is the window difference
92 addi t3, zero, 0 # t3 is a working variable
93
94 # ~ while the sliding window hasn't hit the right limit ~
95 While: # while t1 <= a1
96 bgtu t1, a1, EndWhile
97
98 # t2 = *(t1) - *(t0)
99 lw t2, 0(t1) # t2 = *(t1) (right neighbor)

```

```

100 lw t3, 0(a0) # t3 = *(a0) (left neighbor)
101 sub t2, t2, t3 # t2 = t2 - t3
102
103 # MMIO[LEDS] = t2
104 sw t2, 0(a2)
105
106 # slide window right
107 addi a0, a0, 4
108 addi t1, t1, 4
109
110 j While
111 EndWhile:
112
113 # ~~ 4. calle(FIBDIFF) restores preserved from stack ~~
114 lw ra, 0(sp)
115
116 # ~~ 5. calle(FIBDIFF) deallocates space on stack ~~
117 addi sp, sp, 4
118
119 # ~~ calle(FIBDIFF) return ~~
120 jr ra
121 # ~~~~~

```

Part 2

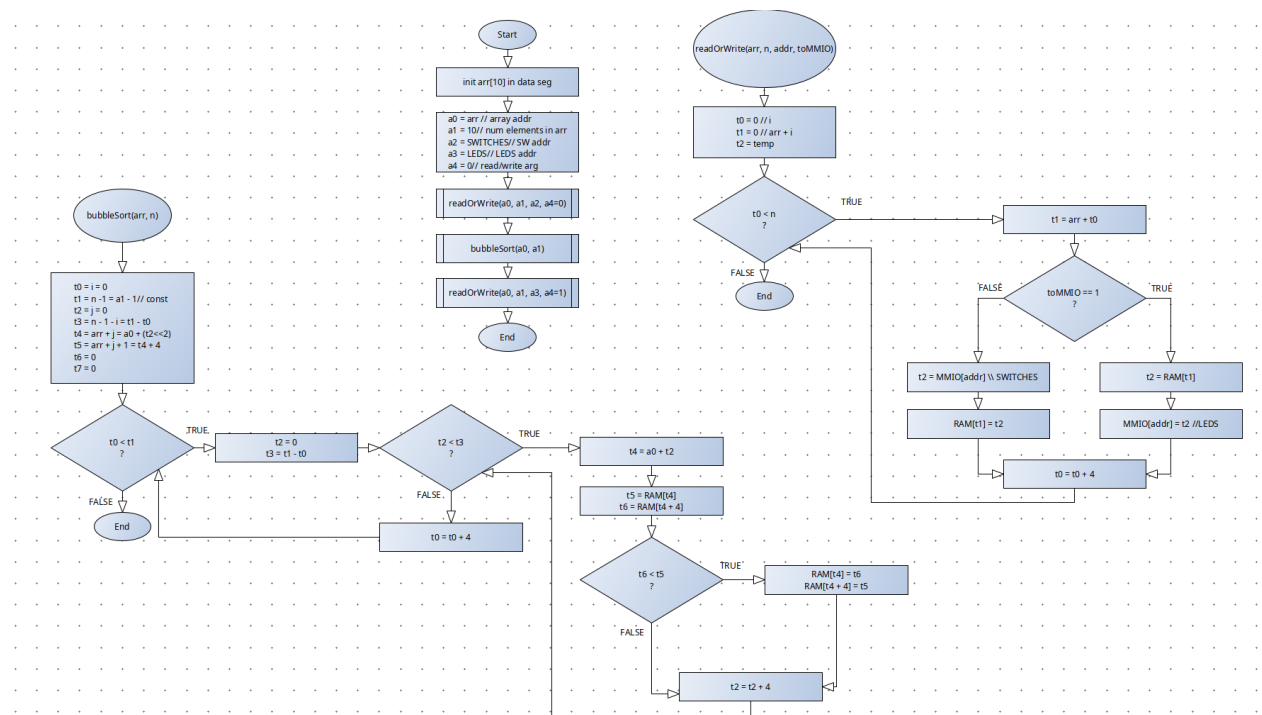


Figure 93: Flow chart for part 2

Listing 12: Assembly code for sw4_p2.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read 10 conseq inputs from SWITCHES into array,
4 #           sort array in ASC order,
5 #           then write the the entire array to LEDS consecutively
6 # Input :
7 #         SWITCHES MMIO address to 32 bits
8 #         LEDS MMIO address to 32 bits
9 #
10 # Output :
11 #         Write 10 to LEDS addr
12
13
14 .data
15 ARR: .word -7:10
16
17 # ~~~~ Test Cases ~~~~
18 # ~~ TC1: worst bubble case ~~
19 #TC: .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
20 # answer = 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
21 # verified r/w and sort
22
23 # ~~ TC2: negatives ~~
24 #TC: .word 1, -5, 2, 8, 3, 3, -1000, 1000, 0, 10
25 # answer = -1000, -5, 0, 1, 2, 3, 3, 8, 10, 1000
26 # verified r/w and sort
27
28 # ~~ TC3: mirror ~~
29 TC: .word 0x7FFFFFFF:4
30 .word 0
31 .word 0xEFFFFFFF:5

```

```

32 # answer = 0xEFFFFFFF:5 , 0x7FFFFFFF:4
33 # verified r/w and sort
34
35 .global _start
36
37 .text
38 .equ MMIO_SWITCHES, 0x00007F00
39 #.equ MMIO_SEVSEG, 0x00007F40
40 .equ MMIO_LEDS, 0x00007F20
41 .equ ARR_N_ELEM, 10
42
43 _start:
44 # ~~~~ init reg ~~~~
45 la s0, ARR # s0 is the addr for head of ARR
46 addi s1, zero, ARR_N_ELEM # s1 is the const num elem in ARR
47 li s2, MMIO_SWITCHES # s2 is the addr for MMIO SWITCHES
48 li s3, MMIO_LEDS # s3 is the addr for MMIO LEDES
49
50
51 # ~~~~ caller(_start) saves non-preserved ~~~~
52 # ~~ 1. caller(_start) allocates space on stack ~~
53 # ~~ 2. caller(_start) stores reg values on stack ~~
54 # ~~ 3. caller(_start) function execution ~~
55 # ~ read from MMIO[SWITCHES] ~~
56 addi a0, s0, 0 # a0 is the base addr for ARR
57 addi a1, s1, 0 # a1 is the num of elem in ARR
58 addi a2, s2, 0 # a2 is the r/w base address (set to SW to read)
59 addi a3, zero, 0 # a3 is the "toMMIO" set to 0 for read
60 jal ReadOrWrite
61 # ~~ 4. caller(_start) restores preserved from stack ~~
62 # ~~ 5. caller(_start) deallocates space on stack ~~
63
64 # ~~~~ caller(_start) saves non-preserved ~~~~
65 # ~~ 1. caller(_start) allocates space on stack ~~
66 # ~~ 2. caller(_start) stores reg values on stack ~~
67 # ~~ 3. caller(_start) function execution ~~
68 # ~ sort array ~~
69 addi a0, s0, 0 # a0 is the base addr for ARR
70 addi a1, s1, 0 # a1 is the num of elem in ARR
71 jal BubbleSort
72 # ~~ 4. caller(_start) restores preserved from stack ~~
73 # ~~ 5. caller(_start) deallocates space on stack ~~
74
75 # ~~~~ caller(_start) saves non-preserved ~~~~
76 # ~~ 1. caller(_start) allocates space on stack ~~
77 # ~~ 2. caller(_start) stores reg values on stack ~~
78 # ~~ 3. caller(_start) function execution ~~
79 # ~ write to MMIO[LEDES] ~~
80 addi a0, s0, 0 # a0 is the base addr for ARR
81 addi a1, s1, 0 # a1 is the num of elem in ARR
82 addi a2, s3, 0 # a2 is the r/w base address (set to LED to write)
83 addi a3, zero, 1 # a3 is the "toMMIO" set to 1 for write
84 jal ReadOrWrite
85 # ~~ 4. caller(_start) restores preserved from stack ~~
86 # ~~ 5. caller(_start) deallocates space on stack ~~
87
88
89 # ~~~~ syscall exit ~~~~
90 li a7, 10 # load syscall num for exit()
91 li a0, 0 # EXIT_SUCCESS code
92 ecall # execute sys call
93
94
95 # ~~~~~~ReadOrWrite()~~~~~
96 # Purpose : Read from from the MMIO address n times and place into array
97 # or write the contents of array onto MMIO addr conseq n times.
98 #
99 #
100 # Input :
101 # a0 : [unsigned 32-bit] base address of array
102 # a1 : [unsigned 32-bit] num elem in array
103 # a2 : [unsigned 32-bit] base addres of MMIO
104 # a3 : [unsigned 32-bit] r/w command (0-read, 1-write)
105 # t3 : [unsigned 32-bit] base addr for test case array (if needed)
106 #
107 # Note :
108 # - modifying a1 to save a register (caller preserve)
109 # Return:
110 #
111
112 ReadOrWrite:
113
114 # ~~~~ calle(READORWRITE) saves preserved ~~~~
115 # ~~ 1. calle(READORWRITE) allocates space on stack ~~
116 addi sp, sp, -20
117
118 # ~~ 2. calle(READORWRITE) stores reg values on stack ~~
119 sw s3, 16(sp)
120 sw s2, 12(sp)
121 sw s1, 8(sp)
122 sw s0, 4(sp)

```

```

123     sw ra, 0(sp)
124
125     # ~~ 3. calle(READORWRITE) function execution ~~
126
127     # ~ init vars ~
128     addi t0, zero, 0      # t0 = i = 0
129     addi t1, zero, 0      # t1 = arr + i
130                                # t2 is local temp
131     la t3, TC             # t3 is a test case var (array base addr)
132                                # t4 is a test case var (dummy)
133
134     slli a1, a1, 2        # shift s.t. we dont use reg for i and 4i
135
136     # ~ for(i=0; i < 10; i++) ~
137 For_ROW:
138     bgeu t0, a1, End_For_ROW
139     add t1, a0, t0        # t1 = arr + 4i
140
141
142     beqz a3, ReadMMIO
143     # ~ write to MMIO addr ~
144     lw t2, 0(t1)         # t2 = *(arr + 4i)
145     sw t2, 0(a2)         # MMIO[LEDS]= t2
146     j End_ReadMMIO
147
148
149 ReadMMIO:
150     # ~~~~ TC injection point ~~~~
151     addi t4, t0, 0        # t4 = 4i
152     add t4, t4, t3        # t4 = TC + 4i
153     lw t4, 0(t4)         # t4 = *(TC + 4i)
154     sw t4, 0(a2)         # MMIO[SWITCHES] = t4
155     # ~~~~~~
156     # ~ read from MMIO addr ~
157     lw t2, 0(a2)         # read SW : t2 = MMIO[SWITCHES]
158     sw t2, 0(t1)         # store in arr : *(arr+4i) = t2
159 End_ReadMMIO:
160
161
162     addi t0, t0, 4        # i++
163     j For_ROW
164 End_For_ROW:
165
166     # ~~ 4. calle(READORWRITE) restores preserved from stack ~~
167     lw s3, 16(sp)
168     lw s2, 12(sp)
169     lw s1, 8(sp)
170     lw s0, 4(sp)
171     lw ra, 0(sp)
172
173     # ~~ 5. calle(READORWRITE) deallocates space on stack ~~
174     addi sp, sp, 20
175
176     # ~~ calle(READORWRITE) return ~~
177     jr ra
178 # ~~~~~~
179
180 # ~~~~~~BubbleSort()~~~~~
181 # Purpose : Sort array in ascending order
182 #
183 #
184 # Input :
185 #     a0 : [unsigned 32-bit] base address of array
186 #     a1 : [unsigned 32-bit] num elem in array
187 #
188 # Note :
189 #
190 # Return:
191 #
192
193 BubbleSort:
194
195     # ~~~~ calle(BubbleSort) saves preserved ~~~~
196     # ~~ 1. calle(BubbleSort) allocates space on stack ~~
197     addi sp, sp, -20
198
199     # ~~ 2. calle(BubbleSort) stores reg values on stack ~~
200     sw s3, 16(sp)
201     sw s2, 12(sp)
202     sw s1, 8(sp)
203     sw s0, 4(sp)
204     sw ra, 0(sp)
205
206     # ~~ 3. calle(BubbleSort) function execution ~~
207
208     slli a1, a1, 2        # shift s.t. we dont use reg for i and 4i
209
210     # ~ init vars ~
211     addi t0, zero, 0      # t0 = i = 0 (4i)
212     addi t1, a1, -4       # t1 = n - 1
213                                # t2 = j = 0

```

```

214             # t3 = n - 1 - i
215             # t4 = arr + j
216             # t5, t6 = dummy
217
218 For_i_Bubble: # for(int i = 0; i < n-1; i++)
219     bgeu t0, t1, End_For_i_Bubble
220
221
222     addi t2, zero, 0           # j = 0
223     sub t3, t1, t0            # n - 1 - i
224 For_j_Bubble: # for(int j = 0; j < n-1-i; j++)
225     bgeu t2, t3, End_For_j_Bubble
226
227     add t4, a0, t2            # t4 = arr + j
228     lw t5, 0(t4)              # t5 = *(arr + j)
229     lw t6, 4(t4)              # t6 = *(arr + j + 1)
230
231     # if(t6 < t5)
232     bgt t6, t5, NoSwap
233     sw t6, 0(t4)              # arr[j] = t6
234     sw t5, 4(t4)              # arr[j+1] = t5
235 NoSwap:
236
237     addi t2, t2, 4            # j++
238     j For_j_Bubble
239 End_For_j_Bubble:
240
241     addi t0, t0, 4            # i++
242     j For_i_Bubble
243 End_For_i_Bubble:
244
245 # ~~ 4. calle(BubbleSort) restores preserved from stack ~~
246 lw s3, 16(sp)
247 lw s2, 12(sp)
248 lw s1, 8(sp)
249 lw s0, 4(sp)
250 lw ra, 0(sp)
251
252 # ~~ 5. calle(BubbleSort) deallocates space on stack ~~
253 addi sp, sp, 20
254
255 # ~~ calle(BubbleSort) return ~~
256 jr ra
257 # ~~~~~

```

0.2.5 SW5 - Division and Stacks

- Assignment:**
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Use register alternative names (ie s0, t0) instead of register numbers (ie x5, x10). Make sure your code is formatted properly including comments. Use the RARS simulator to ensure the program performs as desired.
1. Read two 16-bit unsigned values from an array in the data segment. Divide the first value by the 2nd and output the quotient (result) to 7 SEGMENT (address 0x11000040) and the remainder to LEDS (address 0x11000020).
 2. Read 32-bit values repeatedly from SWITCHES (address 0x11000000). Each value should be added to the stack until the value 0xFFFFFFFF is read. When 0xFFFFFFFF is input the program will not add it to the stack but instead will output all of the saved values to LEDS (address 0x11000020) in the reverse order they were input. Specify the limits of your program. (Assume it is possible for the switches to change between each time they are read)

Assume the BASYS3 SWITCHES and LEDS are expanded to 32 individual switches and LEDs.

Figure 94: Questions to address

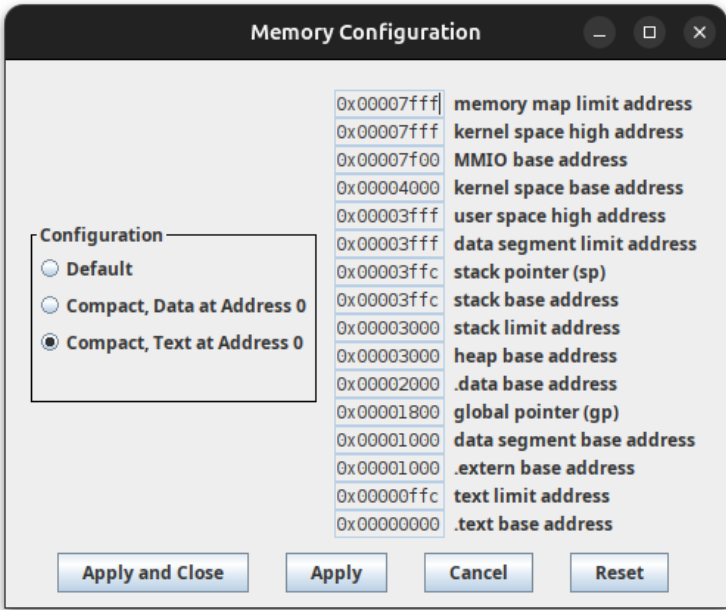


Figure 95: Memory Map for this Assignment

Part 1

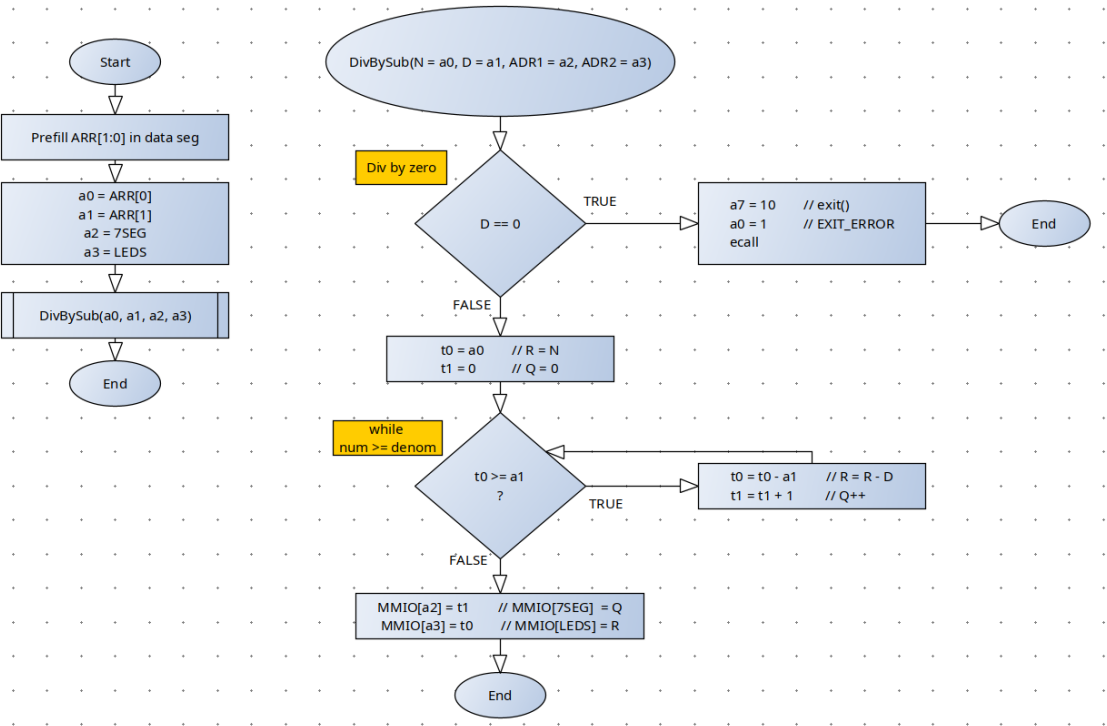


Figure 96: Flow chart for part 1

Listing 13: sw5_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Division by repeat subtraction
4 # Input :
5 #     - array of two unsigned 16-bit elems
6 #     - 7SEG & LEDS MMIO address to 32 bits
7 #
8 # Output :
9 #     Write to 7SEG and LEDS addr
10
11
12 .data
13 # ~~~~ Test Cases ~~~~
14
15 # ~~ TC1 : 0 / 1 ~~
16 #TC:      .half 0, 1
17 # answer : *verified*
18 # Q = 0
19 # R = 0
20
21 # ~~ TC2 : 1 / 0 ~~
22 #TC:      .half 1, 0
23 # answer : *verified*
24 # RaiseError
25 #
26
27 # ~~ TC3 : 1 / 1 ~~
28 #TC:      .half 1, 1
29 # answer : *verified*
30 # Q = 1
31 # R = 0
32
33 # ~~ TC4 : 2 / 1 ~~
34 #TC:      .half 2, 1
35 # answer : *verified*
36 # Q = 2
37 # R = 0
38
39 # ~~ TC5 : 1 / 2 ~~
40 #TC:      .half 1, 2
41 # answer : *verified*
42 # Q = 0
43 # R = 1
44
45 # ~~ TC6 : half max'd ~~
46 #TC:      .half 0xFFFF, 0xFFFF
47 # answer : *verified*
48 # Q = 1
49 # R = 0
50
51 # ~~ TC7 : quotient max'd ~~
52 #TC:      .half 0xFFFF, 1
53 # answer : *verified*
54 # Q = 0xFFFF
55 # R = 0
56
57 # ~~ TC8 : prime1 ~~
58 #TC:      .half 7919, 13
59 # answer : *verified*
60 # Q = 609
61 # R = 2
62
63 # ~~ TC9 : prime2 ~~
64 #TC:      .half 9973, 241
65 # answer : *verified*
66 # Q = 41
67 # R = 92
68
69 # ~~ TC10 : prime3 ~~
70 #TC:      .half 0xFFFF, 149
71 # answer : *verified*
72 # Q = 439
73 # R = 124
74
75 # ~~ TC11 : 0 / 0 ~~
76 #TC:      .half 0, 0
77 # answer : *verified*
78 # RaiseError
79 #
80
81 # ~~ TC12 : denom max'd ~~
82 #TC:      .half 1, 0xFFFF
83 # answer : *verified*
84 # Q = 0
85 # R = 1
86
87
88 .global _start
89

```

```

90 .text
91 #.equ MMIO_SWITCHES, 0x00007F00
92 #.equ MMIO_SEVSEG, 0x00007F40
93 #.equ MMIO_LEDS, 0x00007F20
94
95 _start:
96 # ~~~~ init reg + TC injection ~~~~
97 la s0, TC # s0 is the addr for ARR
98 li s1, MMIO_SEVSEG # a2 is the addr for MMIO 7SEG
99 li s2, MMIO_LEDS # a2 is the addr for MMIO LEDES
100
101 # ~~~~ caller(_start) saves non-preserved ~~~~
102 # ~~ 1. caller(_start) allocates space on stack ~~
103 # ~~ 2. caller(_start) stores reg values on stack ~~
104 # ~~ 3. caller(_start) function execution ~~
105 lhu a0, 0(s0) # a0 is N
106 lhu a1, 2(s0) # a1 is D; NOTE: halfword spacing
107 addi a2, s1, 0 # a2 is the addr for MMIO 7SEG
108 addi a3, s2, 0 # a3 is the addr for MMIO LEDES
109 jal DivBySub
110 # ~~ 4. caller(_start) restores preserved from stack ~~
111 # ~~ 5. caller(_start) deallocates space on stack ~~
112
113 # ~~~~ syscall exit ~~~~
114 li a7, 10 # load syscall num for exit()
115 li a0, 0 # EXIT_SUCCESS code
116 ecall # execute sys call
117
118
119 # ~~~~~~
120 # Purpose : Perform division by subtraction, writing the quotient to
121 # addr1 and the remainder to addr2
122 #
123 #
124 # Input :
125 # a0 : [unsigned 16-bit] Numerator variable N
126 # a1 : [unsigned 16-bit] Denominator variable D
127 # a2 : [unsigned 32-bit] addr1
128 # a3 : [unsigned 32-bit] addr2
129 #
130 # Return:
131 #
132
133 DivBySub:
134
135 # ~~~~ calle(DivBySub) saves preserved ~~~~
136 # ~~ 1. calle(DivBySub) allocates space on stack ~~
137 addi sp, sp, -16
138
139 # ~~ 2. calle(DivBySub) stores reg values on stack ~~
140 sw s2, 12(sp)
141 sw s1, 8(sp)
142 sw s0, 4(sp)
143 sw ra, 0(sp)
144
145 # ~~ 3. calle(DivBySub) function execution ~~
146
147 # ~ error ~
148 blez a1, RaiseError
149
150 # ~ init ~
151 addi t0, a0, 0 # R = t0 = N
152 addi t1, zero, 0 # Q = t1 = 0
153
154 While: # ~ while div produces integers s.t. (R >= D) ~
155 # (R>= D) = (D > R)
156 bgtu a1, t0, EndWhile
157 sub t0, t0, a1 # R = R - D
158 addi t1, t1, 1 # Q++
159 j While
160 EndWhile:
161 # ~ write to MMIOs ~
162 sw t1, 0(a2) # Q goes to addr1
163 sw t0, 0(a3) # R goes to addr2
164
165 # ~~ 4. calle(DivBySub) restores preserved from stack ~~
166 lw s2, 12(sp)
167 lw s1, 8(sp)
168 lw s0, 4(sp)
169 lw ra, 0(sp)
170
171 # ~~ 5. calle(DivBySub) deallocates space on stack ~~
172 addi sp, sp, 16
173
174 # ~~ calle return ~~
175 jr ra
176 # ~~~~~~
177
178 RaiseError:
179 # ~~~~ syscall exit ~~~~
180 li a7, 10 # load syscall num for exit()

```

```

181  li a0, 1      # EXIT_FAILURE code
182  ecall        # execute sys call

```

Part 2

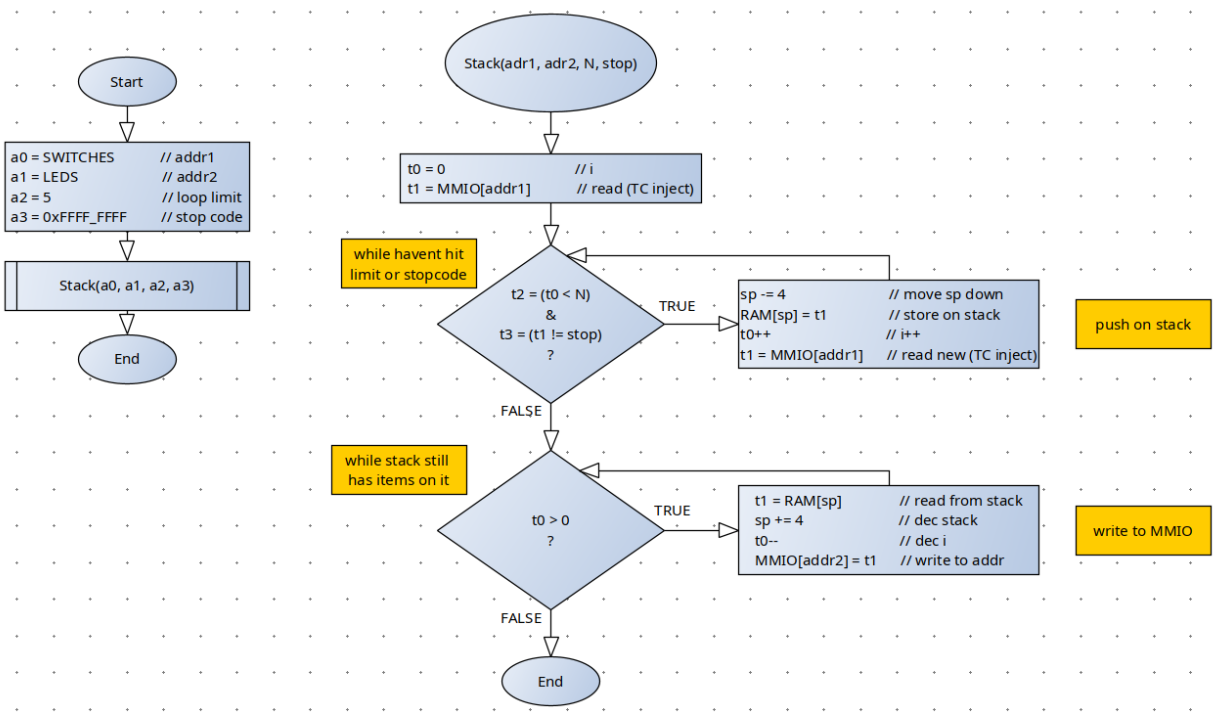


Figure 97: Flow chart for part 2

Listing 14: Assembly code for sw5_p2.asm

```

1  # Author : WhisperingRock
2  #
3  # Purpose : Stack using the stack
4  # Input :
5  #     - SWITCHES & LEDES MMIO address to 32 bits
6  #
7  # Output :
8  #     - conseq write to LEDES addr
9
10
11 .data
12 # ~~~~ Test Cases ~~~~
13
14 # ~~ TC1 : loop limited 1 ~~
15 #TC:      .word 5, 4, 3, 2, 1
16 # answer : 1, 2, 3, 4, 5 (conseq writes)
17 # verified!
18
19 # ~~ TC2 : stopcode limited 1 ~~
20 #TC:      .word 5, 4, 0xFFFFFFFF, 2, 1
21 # answer : 4, 5 (conseq writes)
22 # verified!
23
24 # ~~ TC3 : immediate stopcode ~~
25 #TC:      .word 0xFFFFFFFF, 5, 4, 3, 2, 1
26 # answer : *no writes*
27 # verified!
28
29 # ~~ TC4 : loop limit is zero ~~
30 # ~ NOTE : change LIMIT to 0 ~
31 #TC:      .word 5, 4, 3, 2, 1
32 # answer : *no writes*
33 # verified!
34
35 # ~~ TC5 : near ceiling ~~
36 #TC:      .word 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0
37 # answer : 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, 0xFFFFFFF, (conseq writes)
38 # verified!
39
40 # ~~ TC6 : stopcode and loop limits at same time ~~
41 #TC:      .word 5, 4, 3, 2, 1, 0xFFFFFFFF, 999
42 # answer : 1, 2, 3, 4, 5 (conseq writes)
43 # verified!
44
45 # ~~ TC7 : beyond loop limit ~~
46 #TC:      .word 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
47 # answer : 6, 7, 8, 9, 10 (conseq writes)
48 # verified!
49
50 .global _start

```

```

51
52 .text
53
54 .equ MMIO_SWITCHES,    0x00007F00
55 #.equ MMIO_SEVSEG,     0x00007F40
56 .equ MMIO_LEDS,        0x00007F20
57
58 .equ LIMIT, 5
59 .equ STOPCODE, 0xFFFFFFFF
60
61 _start:
62 # ~~~~ init reg + TC injection ~~~~
63 la s0, TC           # s0 is the addr for TC array
64 li s1, MMIO_SWITCHES # a2 is the addr for MMIO SWITCHES
65 li s2, MMIO_LEDS     # a2 is the addr for MMIO LEDs
66
67 # ~~~~ caller(_start) saves non-preserved ~~~~
68 # ~~ 1. caller(_start) allocates space on stack ~~
69 # ~~ 2. caller(_start) stores reg values on stack ~~
70 # ~~ 3. caller(_start) function execution ~~
71 addi a0, s1, 0      # addr1
72 addi a1, s2, 0      # addr2
73 li a2, LIMIT        # loop limit
74 li a3, STOPCODE     # stop value
75 jal Stack
76 # ~~ 4. caller(_start) restores preserved from stack ~~
77 # ~~ 5. caller(_start) deallocates space on stack ~~
78
79 # ~~~~ syscall exit ~~~~
80 li a7, 10           # load syscall num for exit()
81 li a0, 0             # EXIT_SUCCESS code
82 ecall               # execute sys call
83
84
85 # ~~~~~~
86 # Purpose : Read values from addr1 onto the stack until the loop limit
87 #           or stop code is reached. Then write the values, from the stack
88 #           to addr2.
89 #
90 #
91 # Input :
92 #   a0 : [unsigned 32-bit] read address
93 #   a1 : [unsigned 32-bit] write address
94 #   a2 : [unsigned 32-bit] loop limit
95 #   a3 : [unsigned 32-bit] stop code
96 #
97 # Note :
98 #   - using s0 as a global for TC injection
99 # Return:
100 #
101
102 Stack:
103
104 # ~~~~ calle(Stack) saves preserved ~~~~
105 # ~~ 1. calle(Stack) allocates space on stack ~~
106 addi sp, sp, -16
107
108 # ~~ 2. calle(Stack) stores reg values on stack ~~
109 sw s2, 12(sp)
110 sw s1, 8(sp)
111 sw s0, 4(sp)
112 sw ra, 0(sp)
113
114 # ~~ 3. calle(Stack) function execution ~~
115 # ~ init ~
116 addi t0, zero, 0      # t0 = i = 0
117                        # t1, t2, t3 = read and logic temps
118                        # t4 = TC arr index variable
119 # ---- TC Injection ----
120 slli t4, t0, 2         # t4 = 4i (TC)
121 add t4, t4, s0         # t4 = TC + 4i
122 lw t4, 0(t4)          # t4 = *(TC + 4i)
123 sw t4, 0(a0)          # MMIO[addr1] = t4
124 # -----
125 lw t1, 0(a0)          # t1 = MMIO[addr1]
126 ReadWhile:
127 sltu t2, t0, a2        # t2 = (i < limit)
128 seqz t2, t2
129 xor t3, t1, a3         # t3 = (t1 != stopcode)
130 seqz t3, t3
131 or t3, t3, t2          # t3 = t3 && t2
132 bgt t3, zero, End_ReadWhile # while (i < limit) && (t1 != stopcode)
133 addi sp, sp, -4        # sp -= 4
134 sw t1, 0(sp)          # RAM[sp] = read
135 addi t0, t0, 1        # i++
136 # ---- TC Injection ----
137 slli t4, t0, 2         # t4 = 4i (TC)
138 add t4, t4, s0         # t4 = TC + 4i
139 lw t4, 0(t4)          # t4 = *(TC + 4i)
140 sw t4, 0(a0)          # MMIO[addr1] = t4
141 # -----

```

```

142     lw t1, 0(a0)           # t1 = MMIO[addr1]
143     j ReadWhile
144 End_ReadWhile:
145 WriteWhile:
146     beqz t0, End_WriteWhile # while(i > 0)
147     lw t1, 0(sp)           # t1 = RAM[sp]
148     addi sp, sp, 4         # sp += 4
149     addi t0, t0, -1        # i--
150     sw t1, 0(a1)           # MMIO[addr2] = t1
151     j WriteWhile
152 End_WriteWhile:
153
154 # ~~ 4. calle(Stack) restores preserved from stack ~~
155 lw s2, 12(sp)
156 lw s1, 8(sp)
157 lw s0, 4(sp)
158 lw ra, 0(sp)
159
160 # ~~ 5. calle(Stack) deallocates space on stack ~~
161 addi sp, sp, 16
162
163 # ~~ calle return ~~
164 jr ra
165 # ~~~~~~
166
167 RaiseError:
168 # ~~~~ syscall exit ~~~~
169 li a7, 10      # load syscall num for exit()
170 li a0, 1       # EXIT_FAILURE code
171 ecall         # execute sys call

```

0.2.6 SW6 - Subroutines

- 1. Create a subroutine that will divide a value by 10. The subroutine should utilize appropriate registers for passing parameters, returning results, and only affect temporaries. Write a main program that reads a 16-bit unsigned value from an array in the data segment and converts the value into a 5 digit, BCD (binary coded decimal) value using the divide by 10 subroutine. For example, if the input value is 0x3A6C (14956 decimal), it should be split into 5 nibbles (4-bits) for 1, 4, 9, 5, and 6, so the final answer would be 0x14956. The resulting 20-bit BCD can be written as a 32-bit word to another array in the data segment. The arrays can be of any length.
- 2. Create a subroutine that implements the following greatest common denominator function given below. The subroutine should utilize appropriate registers for passing parameters, returning results, and only affect temporaries. Write a main program that reads two 16-bit unsigned values from an array in the data segment, uses the subroutine to find the GCD, and saves the result to another array in the data segment. *Do not replace the subroutine call with a loop.* You can assume all values are unsigned and greater than 0. Specify any limits of your program.

```
int gcd(int a0, int a1){
    if (a0 > a1)
        a0 = gcd(a0-a1,a1);
    else if (a0 < a1)
        a0 = gcd(a0,a1-a0);
    return a0;
}
```

Code S6.1: GCD Subroutine Code

Figure 98: Questions to address

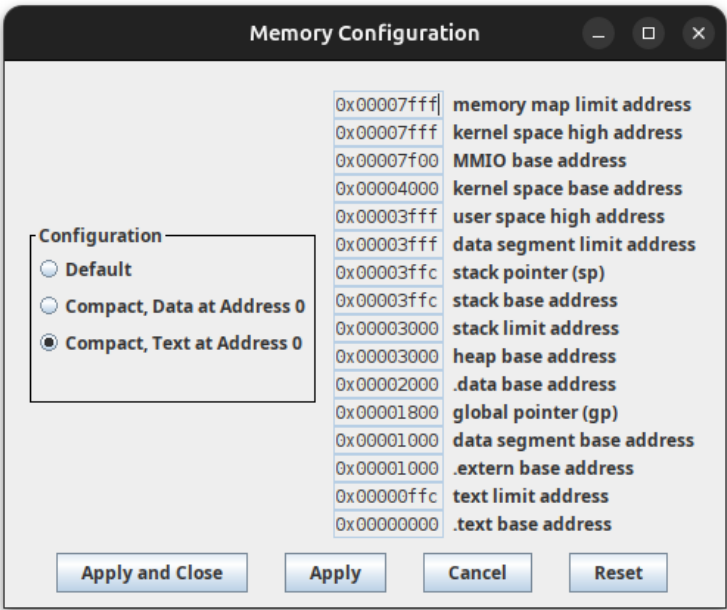


Figure 99: Memory Map for this Assignment

Part 1

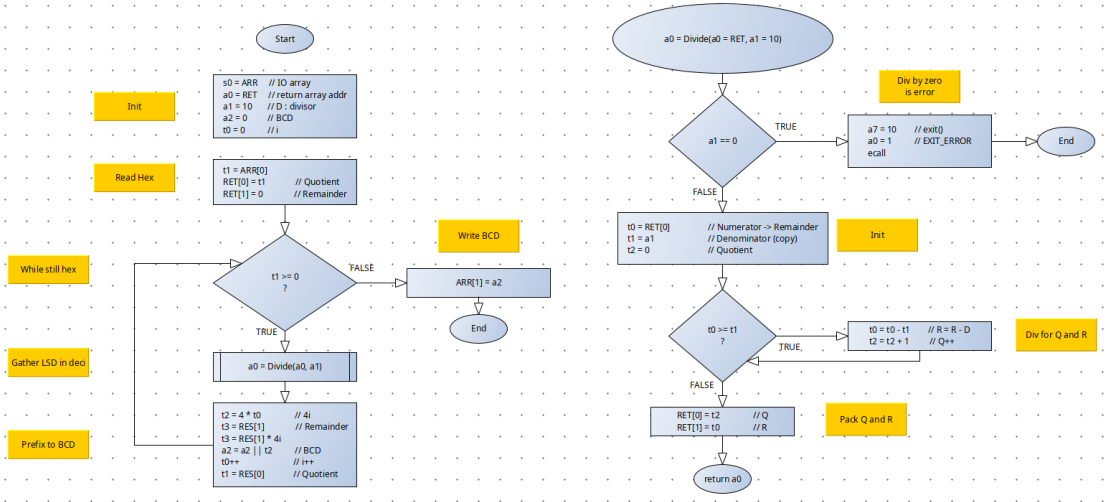


Figure 100: Flow chart for part 1

Listing 15: sw6_p1.asm

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Convert an unsigned hex to BCD, where each hex char
4 #           in the result is actually base 10 decimal.
5 # Input :
6 #         - A single read from an unsigned 16-bit array
7 #
8 # Output :
9 #         - A single 32-bit write to a 32-bit array
10
11
12 .data
13 DIVSET:    .word 0, 0      # Quotient[0] and Remainder[1]
14
15 # ~~~~ Test Cases ~~~~
16 # ~~ TC1 : one decimal digit ~~
17 #TC:      .half    0x7
18 #RES:     .word 0
19 # answer : 0x00007    *verified*
20
21 # ~~ TC2 : two decimal digit ~~
22 #TC:      .half    0xA
23 #RES:     .word 0
24 # answer : 0x00010    *verified*
25
26 # ~~ TC3 : given ~~
27 #TC:      .half    0x3A6C
28 #RES:     .word 0
29 # answer : 0x14956    *verified*
30
31 # ~~ TC4 : max input for 16 unsign bit~~
32 #TC:      .half    0xFFFF
33 #RES:     .word 0
34 # answer : 0x65535    *verified*
35
36 # ~~ TC5 : zero ~~
37 #TC:      .half    0x0
38 #RES:     .word 0
39 # answer : 0x0        *verified*
40
41 # ~~ TC6 : lucky ~~
42 #TC:      .half    0x309
43 #RES:     .word 0
44 # answer : 0x777      *verified*
45
46 .global _start
47
48 .text
49 #.equ MMIO_SWITCHES, 0x00007F00
50 #.equ MMIO_SEVSEG, 0x00007F40
51 #.equ MMIO_LEDS, 0x00007F20
52 .equ DIVISOR, 10
53
54 _start:
55 # ~~~~ init reg + TC injection ~~~~
56 # ~~ preserved ~~
57 la s0, TC      # s0 is the addr for read array
58 la s1, RES     # s1 is the addr for write array
59 # ~~ non-preserved ~~
60 la a0, DIVSET  # a0 is an array for passing args
61 addi a1, zero, DIVISOR # a1 is the divisor arg (const)
62 addi a2, zero, 0 # a2 is working version of BCD
63 addi t0, zero, 0 # t0 = i = 0
64 # t1 is working var
65
66 # ~~~~ read and load hex from array ~~~~
67 lhu t1, 0(s0)
68 sw t1, 0(a0)   # Numerator is initial quotient
69 sw zero, 4(a0) # Remainder
70
71 # ~~~~ while (numerator >= 0) s.t. hex can be further converted ~~~~
72 MainWhile:
73 blez t1, End_MainWhile
74
75 # ~~~~ caller(_start) saves non-preserved ~~~~
76 # ~~ 1. caller(_start) allocates space on stack ~~
77 addi sp, sp, -20
78 # ~~ 2. caller(_start) stores reg values on stack ~~
79 sw t1, 16(sp)
80 sw a0, 12(sp)
81 sw a1, 8(sp)
82 sw a2, 4(sp)
83 sw t0, 0(sp)
84 # ~~ 3. caller(_start) function execution ~~
85 # a0 remains addr (const)
86 # a1 remains const (10)
87 jal DivBySub
88 # return a0 should remain as above
89 # ~~ 4. caller(_start) restores preserved from stack ~~

```

```

90  lw t1, 16(sp)
91  lw a0, 12(sp)
92  lw a1, 8(sp)
93  lw a2, 4(sp)
94  lw t0, 0(sp)
95  # ~~ 5. caller(_start) deallocates space on stack ~~
96  addi sp, sp, 20
97
98  # ~~~~ prefix hex to BCD ~~~~
99  slli t2, t0, 2          # t2 = 4i
100 lw t3, 4(a0)           # t3 = RES[1] = Remainder
101 sll t3, t3, t2          # remainder shifted to correct hex spot
102 or a2, a2, t3          # prefix remainder to BCD
103 addi t0, t0, 1         # i++
104 lw t1, 0(a0)           # t1 = RES[0] = Quotinet
105
106 j MainWhile
107 End_MainWhile:
108
109 sw a2, 0(s1)           # write BCD to result array
110
111 # ~~~~ syscall exit ~~~~
112 li a7, 10              # load syscall num for exit()
113 li a0, 0                # EXIT_SUCCESS code
114 ecall                  # execute sys call
115
116
117 # ~~~~~~
118 # Purpose : Perform division by subtraction, writing the quotient and
119 #           remainder to a single array.
120 #
121 # Input :
122 #   a0 : [signed 32-bit] Array of two elements
123 #       a[0] is Numerator
124 #   a1 : [signed 32-bit] Denominator
125 #
126 # Return:
127 #   address to array stored in a0, with
128 #       arr[0] storing the quotient
129 #       arr[1] storing the remainder
130
131 DivBySub:
132
133 # ~~~~ calle(DivBySub) saves preserved ~~~~
134 # ~~ 1. calle(DivBySub) allocates space on stack ~~
135 addi sp, sp, -12
136
137 # ~~ 2. calle(DivBySub) stores reg values on stack ~~
138 sw s1, 8(sp)
139 sw s0, 4(sp)
140 sw ra, 0(sp)
141
142 # ~~ 3. calle(DivBySub) function execution ~~
143
144 # ~ Edge case : div by zero ~
145 blez a1, RaiseError
146
147 # ~ init ~
148 lw t0, 0(a0)           # R = N = RES[0]
149 addi t1, a1, 0         # D = a1 = 10
150 addi t2, zero, 0       # Q = 0
151
152 DivWhile: # ~ while div produces integers s.t. (R >= D) ~
153 # (R>= D) = (D > R)
154 bgtu t1, t0, End_DivWhile
155 sub t0, t0, t1         # R = R - D
156 addi t2, t2, 1         # Q++
157
158 j DivWhile
159 End_DivWhile:
160
161 # ~ write to return array ~
162 sw t2, 0(a0)           # Q saved in a0[0]
163 sw t0, 4(a0)           # R saved in a0[1]
164
165 # ~~ 4. calle(DivBySub) restores preserved from stack ~~
166 lw s1, 8(sp)
167 lw s0, 4(sp)
168 lw ra, 0(sp)
169
170 # ~~ 5. calle(DivBySub) deallocates space on stack ~~
171 addi sp, sp, 12
172
173 # ~~ calle return ~~
174 jr ra
175 # ~~~~~~
176
177 RaiseError:
178 # ~~~~ syscall exit ~~~~
179 li a7, 10              # load syscall num for exit()
180 li a0, 1                # EXIT_FAILURE code

```

Part 2

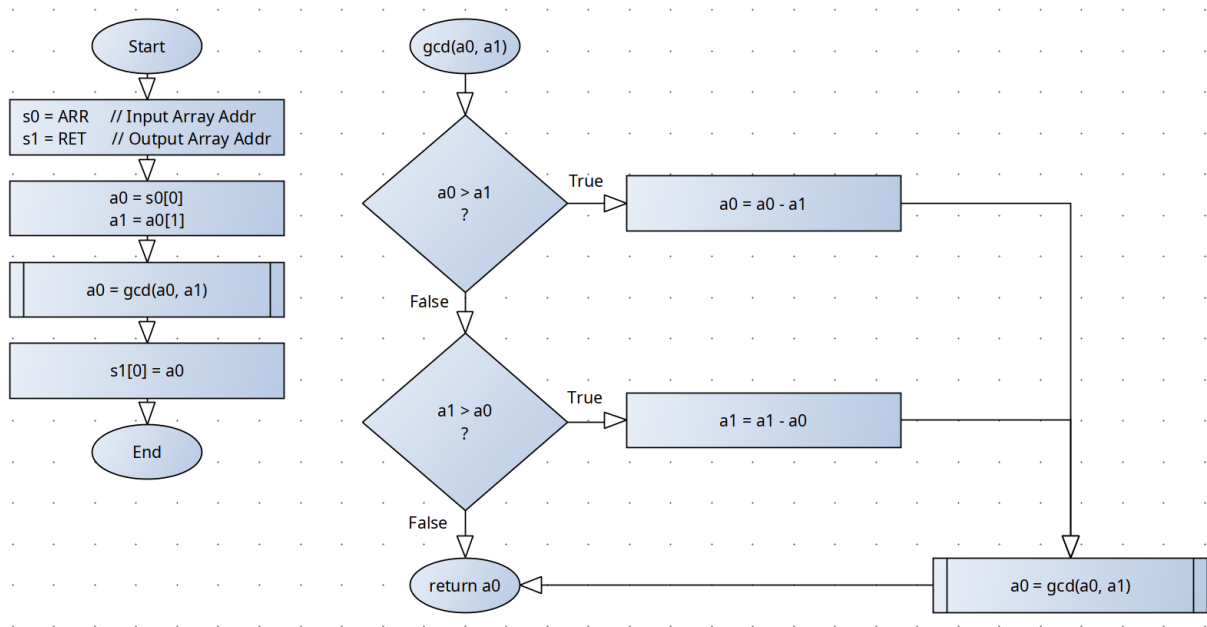


Figure 101: Flow chart for part 2

Listing 16: Assembly code for sw6_p2.asm

```
1 # Author : WhisperingRock
2 #
3 # Purpose : Find the greatest common denominator using recursion
4 #
5 # Assumptions : All input unsigned and greater than zero.
6 #
7 # Input :
8 #   - TC : [16-bit unsigned] array with at least 2 values
9 #
10 # Output :
11 #   - the gcd between the two numbers, written to a result array
12
13
14 .data
15
16 # ~~~~ Test Cases ~~~~
17 # ~~ TC1 : one decimal digit 1 ~~
18 #TC:    .half    8, 16
19 #RES: .word 0
20 # answer : 8 *verified*
21
22 # ~~ TC2 : one decimal digit rev ~~
23 #TC:    .half    16, 8
24 #RES: .word 0
25 # answer : 8 *verified*
26
27 # ~~ TC3 : same | floor ~~
28 #TC:    .half    1, 1
29 #RES: .word 0
30 # answer : 1 *verified*
31
32 # ~~ TC4 : same | ceiling ~~
33 #TC:    .half    0xFF, 0xFF
34 #RES: .word 0
35 # answer : 0xFF *verified*
36
37 # ~~ TC5 : not input 1 ~~
38 #TC:    .half    255, 102
39 #RES: .word 0
40 # answer : 51 *verified*
41
42 # ~~ TC6 : not input 1 rev ~~
43 #TC:    .half    102, 255
44 #RES: .word 0
45 # answer : 51 *verified*
46
47 # ~~ TC7 : one #1~~
48 #TC:    .half    1, 255
49 #RES: .word 0
50 # answer : 1 *verified*
51
52 # ~~ TC8 : one rev #2 ~~
53 #TC:    .half    255, 1
54 #RES: .word 0
```

```

55 # answer : 1 *verified*
56
57 .global _start
58
59 .text
60 #.equ MMIO_SWITCHES, 0x00007F00
61 #.equ MMIO_SEVSEG, 0x00007F40
62 #.equ MMIO_LEDS, 0x00007F20
63
64 _start:
65 # ~~~~ init reg + TC injection ~~~~
66 # ~~ preserved ~~
67 la s0, TC # s0 is the addr for read array
68 la s1, RES # s1 is the addr for write array
69 # ~~ non-preserved ~~
70 # a0 first arg (consumed)
71 # a1 second arg (consumed)
72
73 # ~~~~ read from array (TC injection) ~~~~
74 lhu a0, 0(s0)
75 lhu a1, 2(s0)
76
77 # ~~~~ caller(_start) saves non-preserved ~~~~
78 # ~~ 1. caller(_start) allocates space on stack ~~
79 # ~~ 2. caller(_start) stores reg values on stack ~~
80 # ~~ 3. caller(_start) function execution ~~
81 jal GCD
82 # ~~ 4. caller(_start) restores preserved from stack ~~
83 # ~~ 5. caller(_start) deallocates space on stack ~~
84
85 sw a0, 0(s1) # write GCD to result array
86
87 # ~~~~ syscall exit ~~~~
88 li a7, 10 # load syscall num for exit()
89 li a0, 0 # EXIT_SUCCESS code
90 ecall # execute sys call
91
92
93 # ~~~~~~
94 # Purpose : Find the GCD by recursively subtracting til equal
95 #
96 #
97 # Input :
98 # a0 : [unsigned 16-bit]
99 # a1 : [unsigned 16-bit]
100 #
101 # Return:
102 # a0 : [unsigned 16-bit] gcd between a0 and a1
103
104 GCD:
105
106 # ~~~~ calle(GCD) saves preserved ~~~~
107 # ~~ 1. calle(GCD) allocates space on stack ~~
108 addi sp, sp, -4
109 # ~~ 2. calle(GCD) stores reg values on stack ~~
110 sw ra, 0(sp)
111 # ~~ 3. calle(GCD) function execution ~~
112
113 bltu a1, a0, GCD_If
114 bltu a0, a1, GCD_ElseIf
115 j GCD_IfEnd
116
117 GCD_If:
118 sub a0, a0, a1
119
120 # a0 and a1 don't need to be preserved
121 # ~~~~ caller(GCD) saves non-preserved ~~~~
122 # ~~ 1. caller(GCD) allocates space on stack ~~
123 # ~~ 2. caller(GCD) stores reg values on stack ~~
124 # ~~ 3. caller(GCD) function execution ~~
125 jal GCD
126 # ~~ 4. caller(GCD) restores preserved from stack ~~
127 # ~~ 5. caller(GCD) deallocates space on stack ~~
128
129 j GCD_IfEnd
130
131 GCD_ElseIf:
132 sub a1, a1, a0
133 # a0 and a1 don't need to be preserved
134 # ~~~~ caller(GCD) saves non-preserved ~~~~
135 # ~~ 1. caller(GCD) allocates space on stack ~~
136 # ~~ 2. caller(GCD) stores reg values on stack ~~
137 # ~~ 3. caller(GCD) function execution ~~
138 jal GCD
139 # ~~ 4. caller(GCD) restores preserved from stack ~~
140 # ~~ 5. caller(GCD) deallocates space on stack ~~
141
142 GCD_IfEnd:
143
144 # ~~ return a0 as itself ~~
145

```

```

146 # ~~ 4. calle() restores preserved from stack ~~
147 lw ra, 0(sp)
148 # ~~ 5. calle() deallocates space on stack ~~
149 addi sp, sp, 4
150
151 # ~~ calle return ~~
152 jr ra
153 # ~~~~~~
154
155 RaiseError:
156 # ~~~~ syscall exit ~~~~
157 li a7, 10      # load syscall num for exit()
158 li a0, 1       # EXIT_FAILURE code
159 ecall          # execute sys call

```

0.2.7 SW7 - Interrupts

Awkwardly enough, RARS doesn't support interrupts. This leaves me at an impasse such that a polling 'main' function could be implemented using the *Keyboard and Display MMIO Simulator* setting but that won't use the CSR instructions I suspect what we want to use here. BUT using CSR instructions ultimately means that the program wont be able to be tested by RARS but only compiled into memory. For brevity, I've opted to skip the RARS portion and map out the flow charts for now...

- Assignment:**
First, create a flowchart that will perform the following behavior. Then implement the flowchart with OTTER MCU assembly language using only the instructions in the OTTER assembly manual. Use register alternative names (ie s0, t0) and utilize registers according to standard use practices. Make sure your code is formatted properly including comments.
- 1. Write an interrupt driven program that will turn the LEDs (address 0x11000020) on or off. Every interrupt will alternately toggle certain LEDs on or off. The LEDs that change (toggle) are the ones corresponding to the value on the SWITCHES (address 0x11000000). When an interrupt occurs, if the switch for a specific bit is 1, the corresponding bit on the LEDs will toggle (change from on to off or off to on).
- Example:

LEDS start as

Interrupt occurs

SWITCHES are

LEDS change to

0101 0011 1010 1100

1111 0000 0110 1001

1010 0011 1100 0101
- 2. Write an interrupt driven program that toggles the LEDs (address 0x11000020) based on the SWITCHES (address 0x11000000) when an interrupt occurs just like problem 1 above. However, if on two consecutive interrupts the switches have the same value, the program will turn all of the LEDs off until BUTTON 0 is pressed. Button 0 is connected to bit 0 of address 0x11000200. *Button 0 is not connected to any interrupts.* This means the LEDs will all stay off until button 0 is pressed. When button 0 is pressed, the program will light up the LEDs that were turned on before the duplicate switches were detected and return to the previous functionality of toggling the LEDs as before. You can assume the same switch values will never repeat on more than two consecutive interrupts. (*Hint: The data segment can be used to save a program state or pass values to / from an ISR*)

Figure 102: Questions to address

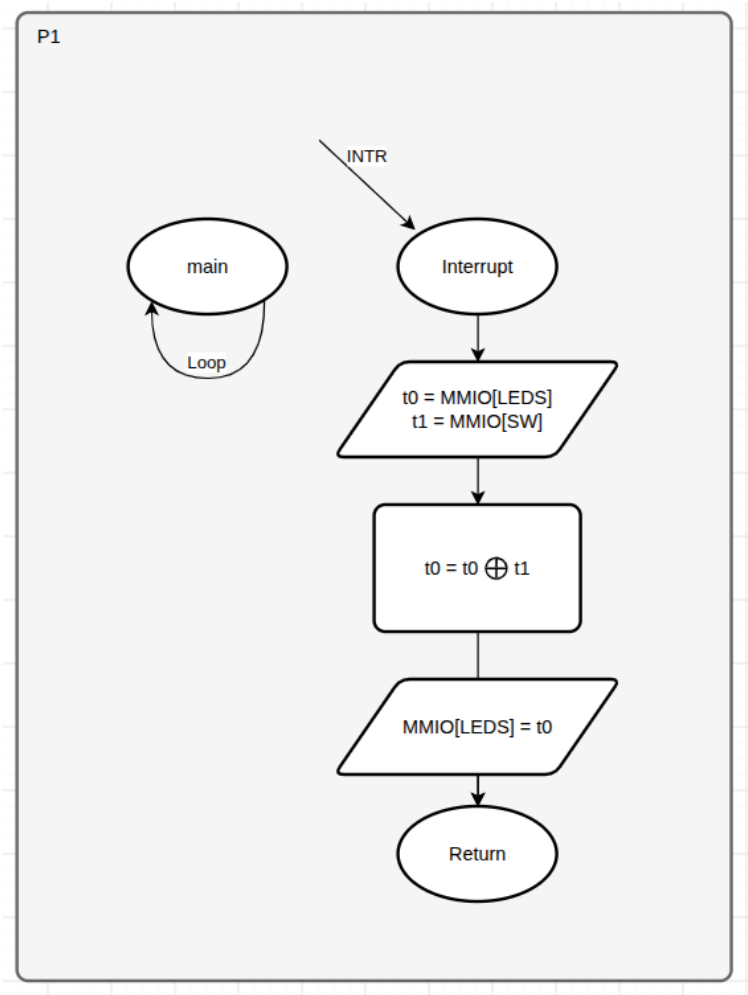


Figure 103: Flow chart for part 1

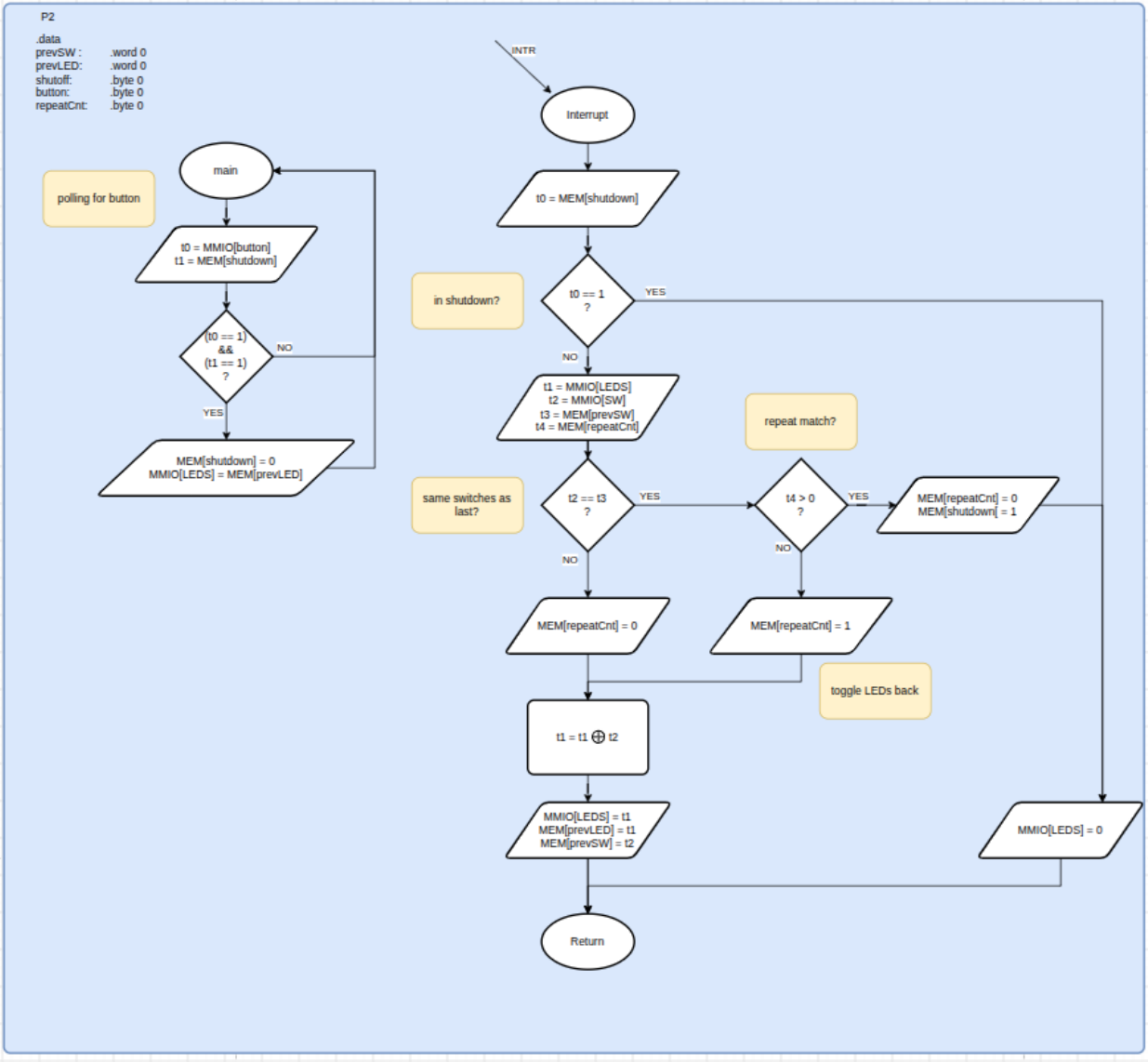


Figure 104: Flow chart for part 2

0.3 Otter Code & Test Sims

0.3.1 Mux(s)

Listing 17: 2 input Mux Mux_2N.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/19/2026 10:35:06 AM
7  // Design Name:
8  // Module Name: Mux_2N
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21 module Mux_2N
22 #(parameter width = 32)
23 (
24     input logic[width-1:0] D0, D1,
25     input logic S,
26     output logic[width-1:0] Y
27 );
28
29     always_comb begin
30         Y = (S == 1'b1) ? D1 : D0;
31     end
32
33 endmodule
```

Listing 18: 4 input Mux Mux_4N.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/19/2026 09:15:04 AM
7  // Design Name:
8  // Module Name: Mux_4N
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21 module Mux_4N
22 #(parameter width = 32)
23 (
24     input logic[width-1:0] D0, D1, D2, D3,
25     input logic[1:0] S,
26     output logic[width-1:0] Y
27 );
28
29     logic[width-1:0] ERR = 0;
30
31     always_comb
32     begin
33         case(S)
34             0: Y = D0;
35             1: Y = D1;
36             2: Y = D2;
37             3: Y = D3;
38             default Y = (~ERR) -13 ;
39         endcase
40     end
41 endmodule
```

Listing 19: 6 input Mux Mux_6N.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company:
4  // Engineer:
```

```

5 //
6 // Create Date: 07/17/2026 04:24:06 PM
7 // Design Name:
8 // Module Name: Mux_6N
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Mux_6N
24 #(parameter width = 32)                                // default bits
25 (
26     input logic[width-1:0] D0, D1, D2, D3, D4, D5,      // data channels
27     input logic[2:0] S,                                  // selector
28     output logic[width-1:0] Y
29 );
30
31     logic[width-1:0] ERR = 0;
32
33     always_comb
34         case(S)
35             0: Y = D0;
36             1: Y = D1;
37             2: Y = D2;
38             3: Y = D3;
39             4: Y = D4;
40             5: Y = D5;
41             default Y = (~ERR) -13 ;
42         endcase
43
44 endmodule

```

Listing 20: TestSim for 6 input Mux Mux_6N_tb.sv

```

1 `timescale 1ns / 1ps
2 ///////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 07/17/2026 04:26:27 PM
7 // Design Name:
8 // Module Name: Mux_6N_tb
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Mux_6N_tb();
24
25     // ~~ local variables ~~
26     logic CLK;
27     logic[31:0] d0, d1, d2, d3, d4, d5;
28     logic[3:0] sel;
29     logic[31:0] out;
30
31
32     // ~~ high-level module instances (and hooks) ~~
33     Mux_6N #(32) UUT (
34         .D0(d0),
35         .D1(d1),
36         .D2(d2),
37         .D3(d3),
38         .D4(d4),
39         .D5(d5),
40         .S(sel),
41         .Y(out)
42     );
43
44     // ~~ action : CLK (10ns period) ~~
45     always begin
46         #5;
47         CLK <= !CLK;
48     end

```

```

49
50 // ~~ action : Data ~~
51 initial begin
52
53     // ~ init ~
54     CLK = 0;
55     d0 = 32'h0000_0000;
56     d1 = 32'h1234_0000;
57     d2 = 32'h0000_5678;
58     d3 = 32'h1111_1111;
59     d4 = 32'hFFFF_FFFF;
60     d5 = 32'hDEAD_BEEF;
61
62     // ~ selector test ~
63     sel = 0; #10
64     assert(out === d0) else $error("sel = 0 failed");
65
66     sel = 1; #10
67     assert(out === d1) else $error("sel = 1 failed");
68
69     sel = 2; #10
70     assert(out === d2) else $error("sel = 2 failed");
71
72     sel = 3; #10
73     assert(out === d3) else $error("sel = 3 failed");
74
75     sel = 4; #10
76     assert(out === d4) else $error("sel = 4 failed");
77
78     sel = 5; #10
79     assert(out === d5) else $error("sel = 5 failed");
80
81     sel = 0; #10
82     assert(out === d0) else $error("sel = 0 failed");
83
84     sel = 2; #10
85     assert(out === d2) else $error("sel = 2 failed");
86
87     sel = 0; #10
88     assert(out === d0) else $error("sel = 0 failed");
89
90     sel = 4; #10
91     assert(out === d4) else $error("sel = 4 failed");
92
93     // ~ edge case : input changes ~
94     sel = 3; #10
95     assert(out === d3) else $error("sel = 3 failed");
96
97     d3 = 32'hF000_0000; #10
98     assert(out === 32'hF000_0000) else $error("h7 failed");
99
100    d3 = 32'h0F00_0000; #10
101    assert(out === 32'h0F00_0000) else $error("h6 failed");
102
103    d3 = 32'h00F0_0000; #10
104    assert(out === 32'h00F0_0000) else $error("h5 failed");
105
106    d3 = 32'h000F_0000; #10
107    assert(out === 32'h000F_0000) else $error("h4 failed");
108
109    d3 = 32'h0000_F000; #10
110    assert(out === 32'h0000_F000) else $error("h3 failed");
111
112    d3 = 32'h0000_0F00; #10
113    assert(out === 32'h0000_0F00) else $error("h2 failed");
114
115    d3 = 32'h0000_00F0; #10
116    assert(out === 32'h0000_00F0) else $error("h1 failed");
117
118    d3 = 32'h0000_000F; #10
119    assert(out === 32'h0000_000F) else $error("h0 failed");
120
121    // ~ default case ~
122    sel = 6;
123    while(sel < 15) begin
124        #10;
125        assert(out === (32'hFFFF_FFFF - 13)) else $error("default %d failed", sel);
126        sel = sel + 1;
127    end
128
129 end
130
131
132 endmodule

```

0.3.2 Program Counter and Additives

Listing 21: Program Counter PC_Reg.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: WhisperingRock
5  //
6  // Create Date: 07/17/2026 03:36:51 PM
7  // Design Name:
8  // Module Name: PC_Reg
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description: Program Counter (register) holds PC_COUNT
13 //                constant unless a reset has occurred or
14 //                a write enable overwrites PC_COUNT with
15 //                PC_DIN
16 //
17 // Dependencies:
18 //
19 // Revision:
20 // Revision 0.01 - File Created
21 // Additional Comments:
22 //
23 ///////////////////////////////////////////////////////////////////
24
25
26 module PC_Reg(
27     input CLK,
28     input PC_RST,           // (highest-priority) active high synch reset s.t. 1 -> PC=0
29     input PC_WE,           // active high synch load value from PC_DIN into PC
30     input[31:0] PC_DIN,    // potential unsigned 32-bit PC
31     output logic [31:0] PC_COUNT // current value of PC (used as addr for ProgROM)
32 );
33
34     always_ff@(posedge CLK) // changes on clk demand sequential logic (use nonblocking)
35     begin
36         if(PC_RST) PC_COUNT <= 0;
37         else if (PC_WE) PC_COUNT <= PC_DIN;
38     end
39
40 endmodule
```

Listing 22: TestSim for Program Counter PC_Reg_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 06:36:03 PM
7  // Design Name:
8  // Module Name: PC_Reg_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module PC_Reg_tb();
24
25     // ~~ local variables ~~
26     logic clk, reset, write;
27     logic[31:0] din, dout;
28     logic[31:0] answer;
29
30
31     // ~~ high-level module instances (and hooks) ~~
32     PC_Reg UUT (
33         .CLK(clk),
34         .PC_RST(reset),
35         .PC_WE(write),
36         .PC_DIN(din),
37         .PC_COUNT(dout)
38     );
39
40     // ~~ action : CLK (10ns period) ~~
41     always begin
42         #5;
43         clk <= !clk;
```

```

44     end
45
46     // ~~ action : Data ~~
47     initial begin
48
49         // ~ init ~
50         clk = 1; reset = 0; write = 0;
51         reset = 1; #10;                                // zero out register
52
53         // ~ test cases ~
54         answer = 32'hDEAD_BEEF;
55         din = answer;
56
57         // ~ TC1 : write ~
58         reset = 0; #10;
59         assert(dout === 0) else $error("TC1: init failed");
60         write = 1; #10;
61         assert(dout === answer) else $error("TC1: write failed");
62
63         // ~ TC2: reset after write ~
64         reset = 1; #10;
65         assert(dout === 0) else $error("TC2: reset failed");
66
67         // ~ TC3: reset priority over write ~
68         write = 1; reset = 1; #10;
69         assert(dout === 0) else $error("TC3: reset failed");
70
71
72         // ~ TC4 : write within range ~
73         write = 1; reset = 0; #10;
74         answer = 32'h0246_8ACE;
75         for (int i = 0; i < 8; i++) begin
76             din = answer; #10;
77             assert(dout === answer) else $error("TC4: write failed");
78             answer = answer << 4;
79         end
80
81         // ~ TC5 : Rollover ~
82         din = 32'hFFFF_FFFF; #10;
83         assert(dout === din) else $error("TC5: write failed");
84         din = din + 1; #10;
85         assert(dout === 0) else $error("TC5: rollover failed");
86
87
88
89     end
90
91
92
93 endmodule

```

Listing 23: 4+ PC Incrementer Plus4_1N.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/17/2026 07:28:53 PM
7  // Design Name:
8  // Module Name: Plus4_1N
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Plus4_1N
24     #(parameter width = 32)                                // default bits
25     (
26         input logic[width-1:0] IN,
27         output logic[width-1:0] OUT
28     );
29
30     always_comb
31         OUT = IN + 4;
32
33 endmodule

```

0.3.3 Register File

Listing 24: Register File RegFile.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: WhisperingRock
5  //
6  // Create Date: 07/21/2026 03:41:49 PM
7  // Design Name:
8  // Module Name: RegFile
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //           A register file is a block of RAM, not ROM,
14 //           that contains the values of all 32 32-bit
15 //           registers x0:x31, with x0 held constant to
16 //           zero.
17 //
18 //           This register is a dual asynch fetch s.t.
19 //           the values at ADR1 and ADR2 are fetched
20 //           and forwarded immediately to RS1, RS2.
21 //
22 //           This register is single synch write s.t.
23 //           if en is high, W_DATA will overwrite the
24 //           value in W_ADR (excluding x0)
25 //
26 // Dependencies:
27 //
28 // Revision:
29 // Revision 0.01 - File Created
30 // Additional Comments:
31 //
32 ///////////////////////////////////////////////////////////////////
33 parameter BITWIDTH      = 32;
34
35 parameter N_REGS        = 32;
36 parameter REG_BUSWIDTH  = 5;
37
38 module RegFile(
39     input EN, CLK,
40     input[REG_BUSWIDTH-1:0] ADR1, ADR2,    // asynch fetch addr(s)
41     input[REG_BUSWIDTH-1:0] W_ADR,        // synch write addr
42     input[BITWIDTH-1:0] W_DATA,          // synch write value
43     output[BITWIDTH-1:0] RS1, RS2        // asynch fetch'd values
44 );
45
46
47 // ~~~~ alloc mem ~~~~
48 logic [BITWIDTH-1:0] ram [0:N_REGS-1];    // memory uses big-endian
49
50 // ~~~~ init mem ~~~~
51 initial
52 begin
53     for(int i = 0; i < N_REGS; i++)
54     begin
55         ram[i] = 0;
56     end
57 end
58
59 // ~~~~ asynch fetch ~~~~
60 assign RS1 = ram[ADR1];
61 assign RS2 = ram[ADR2];
62
63
64 // ~~~~ synch write ~~~~
65 always_ff@(posedge CLK)                // changes on clk demand sequential logic (use nonblocking)
66 begin
67     if((EN == 1'b1) && (W_ADR != 5'b00000))
68     begin
69         ram[W_ADR] <= W_DATA;
70     end
71 end
72
73 endmodule
```

Listing 25: TestSim for Register File RegFile_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: WhisperingRock
5  //
6  // Create Date: 07/21/2026 03:42:37 PM
7  // Design Name:
8  // Module Name: RegFile_tb
9  // Project Name:
10 // Target Devices:
```

```

11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22 module RegFile_tb();
23
24 // ~~~~ init local vars ~~~~
25 logic enable, clk; // input
26 logic[4:0] addr1, addr2, w_addr; // input
27 logic[31:0] w_val; // input
28 logic[31:0] out1, out2; // output
29
30
31 // ~~~~ instances ~~~~
32 RegFile UUT(
33     .EN(enable),
34     .CLK(clk),
35     .ADR1(addr1),
36     .ADR2(addr2),
37     .W_ADR(w_addr),
38     .W_DATA(w_val),
39     .RS1(out1),
40     .RS2(out2)
41 );
42
43 // ~~ predefined tasks ~~
44 task fill_ram(input int bias);
45     enable = 1;
46     for(int i = 0; i < N_REGS; i++)
47     begin
48         w_addr = i;
49         w_val = i + bias;
50         #10;
51     end
52     enable = 0;
53 endtask
54
55 // ~~ action : CLK (10ns period) ~~
56 always begin
57     #5;
58     clk <= !clk;
59 end
60
61 // ~~~~ action : data ~~~~
62 initial begin
63
64     // ~~ init ~~
65     clk = 1; enable = 0;
66     addr1 = 0; addr2 = 0;
67     w_addr = 0; w_val = 0;
68     #10;
69     fill_ram(0);
70     #10
71
72     // ~~ TC1 : same asynch fetch ~~
73     for(int i = 0; i < 32; i++)
74     begin
75         addr1 = i;
76         addr2 = i;
77         #10;
78         assert(out1 == i) else $error("TC1: rs1 failed");
79         assert(out2 == i) else $error("TC1: rs2 failed");
80     end
81
82     // ~~ TC2 : prevent write to x0 ~~
83     addr1 = 0; enable = 1;
84     #10;
85     for(int i = 0; i < 100; i++)
86     begin
87         w_addr = 0;
88         w_val = $random();
89         #10;
90         assert(out1 == 0) else $error("TC2: x0 prevent failed");
91     end
92
93     // ~~ TC3 : write not enabled on same addr ~~
94     fill_ram(0); // cheap sanitizer
95     addr1 = 1; addr2 = 1; w_addr = 1; // vals should be 1 in reg
96     enable = 0;
97     #10;
98     for(int i = 0; i < 100; i++)
99     begin
100         w_val = $random();
101         #10;

```

```

102         assert(out1 === 1) else $error("TC3: write prevent failed");
103         assert(out2 === 1) else $error("TC3: write prevent failed");
104     end
105
106
107     // ~~ TC4 : reading the register being written to ~~
108     addr1 = 14; addr2 = 14; w_addr = 14;
109     enable = 1;
110     #10;
111     for(int i = 0; i < 500; i++)
112     begin
113         w_val = $random();
114         #10;
115         assert(out1 === w_val) else $error("TC4: r/w failed");
116         assert(out2 === w_val) else $error("TC4: r/w failed");
117     end
118
119 end
120
121
122 endmodule

```

0.3.4 Arithmetic Logic Unit

Listing 26: ALU ALU.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: WhisperingRock
5  //
6  // Create Date: 07/24/2026 09:28:51 PM
7  // Design Name:
8  // Module Name: ALU
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //     A Purely combinational operation execution(er).
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21 //parameter BITWIDTH      = 32;
22 parameter FUNWIDTH        = 4;
23
24
25 module ALU(
26     input logic [FUNWIDTH-1:0] ALU_FUN,
27     input logic [BITWIDTH-1:0] SRC_A, SRC_B,
28     output logic[BITWIDTH-1:0] ALU_RESULT
29 );
30
31     // ~~ local vars ~~
32     logic[BITWIDTH-1:0] ERR = 32'hDEAD_BEEF;
33
34     always_comb
35     begin
36         case(ALU_FUN)
37             // ~~ arithmetic operators~~
38             4'b0000: begin ALU_RESULT = SRC_A + SRC_B; end           // ADD
39             4'b1000: begin ALU_RESULT = SRC_A - SRC_B; end           // SUB
40
41             // ~~ bitwise operators ~~
42             4'b0111: begin ALU_RESULT = SRC_A & SRC_B; end           // AND
43             4'b0110: begin ALU_RESULT = SRC_A | SRC_B; end           // OR
44             4'b0100: begin ALU_RESULT = SRC_A ^ SRC_B; end           // XOR
45
46             // ~~ shift operators ~~
47             // Note : shifting only uses the 5 LSBs of SRC_B (see manual)
48             // Note : arithmetic shifting has its own systemverilog operator
49             4'b0101: begin ALU_RESULT = SRC_A >> SRC_B[4:0]; end       // SRL
50             4'b0001: begin ALU_RESULT = SRC_A << SRC_B[4:0]; end       // SLL
51             4'b1101: begin ALU_RESULT = $signed(SRC_A) >>> $signed(SRC_B[4:0]); end // SRA
52
53             // ~~ set operators ~~
54             4'b0010: begin ALU_RESULT = $signed(SRC_A) < $signed(SRC_B); end // SLT
55             4'b0011: begin ALU_RESULT = SRC_A < SRC_B; end           // SLTU
56
57             // ~~ copy operator ~~
58             4'b1001: begin ALU_RESULT = SRC_A; end                   // LUI-COPY (MV?)
59
60             // ~~ error ~~
61             default: begin ALU_RESULT = ERR; end
62         endcase
63     end
64
65 endmodule
```

Listing 27: TestSim for ALU ALU_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/24/2026 10:13:51 PM
7  // Design Name:
8  // Module Name: ALU_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
```

```

19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module ALU_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic[4:0] sel;
27     logic[31:0] a, b;
28     logic[31:0] result;
29
30     logic[31:0] testcase;    // testing var
31
32     // ~~~~ instances ~~~~
33     ALU UUT
34     (
35         .ALU_FUN(sel),
36         .SRC_A(a),
37         .SRC_B(b),
38         .ALU_RESULT(result)
39     );
40
41     // ~~~~ testing ~~~~
42     initial
43     begin
44
45         // ~~~~ ADD ~~~~
46         sel = 4'b0000;
47
48         // ~~ TC1 : ADD1 ~~
49         #5;
50         a = 32'hA50F_96C3;
51         b = 32'h5AF0_693C;
52         testcase = 1;
53         #5;
54         assert(result == 32'hFFFF_FFFF) else $error("TC1 : ADD failed");
55
56         // ~~ TC2 : ADD2 ~~
57         #5;
58         a = 32'h8410_5F21;
59         b = 32'h7B10_5FDE;
60         testcase++;
61         #5;
62         assert(result == 32'hFF20_BEFF) else $error("TC2 : ADD failed");
63
64         // ~~ TC3 : ADD3 ~~
65         #5;
66         a = 32'hFFFF_FFFF;
67         b = 32'h0000_0001;
68         testcase++;
69         #5;
70         assert(result == 32'h0000_0000) else $error("TC3 : ADD failed");
71
72
73
74         // ~~~~ SUB ~~~~
75         sel = 4'b1000;
76
77         // ~~ TC4 : SUB4 ~~
78         #5;
79         a = 32'h0000_0000;
80         b = 32'h0000_0001;
81         testcase++;
82         #5;
83         assert(result == 32'hFFFF_FFFF) else $error("TC4 : SUB failed");
84
85         // ~~ TC5 : SUB5 ~~
86         #5;
87         a = 32'hAA80_6355;
88         b = 32'h5501_62AA;
89         testcase++;
90         #5;
91         assert(result == 32'h557F_00AB) else $error("TC5 : SUB failed");
92
93         // ~~ TC6 : SUB6 ~~
94         #5;
95         a = 32'h5501_62AA;
96         b = 32'hAA80_6355;
97         testcase++;
98         #5;
99         assert(result == 32'hAA80_FF55) else $error("TC6 : SUB failed");
100
101
102
103         // ~~~~ AND ~~~~
104         sel = 4'b0111;
105
106         // ~~ TC7 : AND7 ~~
107         #5;
108         a = 32'hA55A_00FF;
109         b = 32'h5A5A_FFFF;

```

```

110     testcase++;
111     #5;
112     assert(result === 32'h005A_00FF) else $error("TC7 : AND failed");
113
114     // ~~ TC8 : AND8 ~~
115     #5;
116     a = 32'hC3C3_F966;
117     b = 32'hFF66_9F5A;
118     testcase++;
119     #5;
120     assert(result === 32'hC342_9942) else $error("TC8 : AND failed");
121
122
123     // ~~~~ OR ~~~~
124     sel = 4'b0110;
125
126     // ~~ TC9 ~~
127     #5;
128     a = 32'h9A9A_C300;
129     b = 32'h65A3_CC0F;
130     testcase++;
131     #5;
132     assert(result === 32'hFFBB_CF0F) else $error("TC9 : OR failed");
133
134     // ~~ TC10 ~~
135     #5;
136     a = 32'hC3C3_F966;
137     b = 32'hFF66_9F5A;
138     testcase++;
139     #5;
140     assert(result === 32'hFFE7_FF7E) else $error("TC10 : OR failed");
141
142
143     // ~~~~ XOR ~~~~
144     sel = 4'b0100;
145
146     // ~~ TC11 ~~
147     #5;
148     a = 32'hAA55_00FF;
149     b = 32'h5AA5_0FF0;
150     testcase++;
151     #5;
152     assert(result === 32'hF0F0_0F0F) else $error("TC11 : XOR failed");
153
154     // ~~ TC12 ~~
155     #5;
156     a = 32'hA5A5_6C6C;
157     b = 32'hFF00_C6FF;
158     testcase++;
159     #5;
160     assert(result === 32'h5AA5_AA93) else $error("TC12 : XOR failed");
161
162
163
164     // ~~~~ SRL ~~~~
165     sel = 4'b0101;
166
167     // ~~ TC13 ~~
168     #5;
169     a = 32'h805A_6CF3;
170     b = 32'h0000_0010;
171     testcase++;
172     #5;
173     assert(result === 32'h0000_805A) else $error("TC13 : SRL failed");
174
175     // ~~ TC14 ~~
176     #5;
177     a = 32'h705A_6CF3;
178     b = 32'h0000_0005;
179     testcase++;
180     #5;
181     assert(result === 32'h0382_D367) else $error("TC14 : SRL failed");
182
183     // ~~ TC15 ~~
184     #5;
185     a = 32'h805A_6CF3;
186     b = 32'h0000_0000;
187     testcase++;
188     #5;
189     assert(result === 32'h805A_6CF3) else $error("TC15 : SRL failed");
190
191     // ~~ TC16 ~~
192     #5;
193     a = 32'h805A_6CF3;
194     b = 32'h0000_0100;
195     testcase++;
196     #5;
197     assert(result === 32'h805A_6CF3) else $error("TC16 : SRL failed");
198
199
200

```

```

201 // ~~~~ SLL ~~~~
202 sel = 4'b0001;
203
204 // ~~ TC17 ~~
205 #5;
206 a = 32'h805A_6CF3;
207 b = 32'h0000_0010;
208 testcase++;
209 #5;
210 assert(result === 32'h6CF3_0000) else $error("TC17 : SLL failed");
211
212 // ~~ TC18 ~~
213 #5;
214 a = 32'h805A_6CF3;
215 b = 32'h0000_0005;
216 testcase++;
217 #5;
218 assert(result === 32'h0B4D_9E60) else $error("TC18 : SLL failed");
219
220 // ~~ TC19 ~~
221 #5;
222 a = 32'h805A_6CF3;
223 b = 32'h0000_0100;
224 testcase++;
225 #5;
226 assert(result === 32'h805A_6CF3) else $error("TC19 : SLL failed");
227
228
229 // ~~~~ SRA ~~~~
230 sel = 4'b1101;
231
232 // ~~ TC20 ~~
233 #5;
234 a = 32'h805A_6CF3;
235 b = 32'h0000_0010;
236 testcase++;
237 #5;
238 assert(result === 32'hFFFF_805A) else $error("TC20 : SRA failed");
239
240 // ~~ TC21 ~~
241 #5;
242 a = 32'h705A_6CF3;
243 b = 32'h0000_0005;
244 testcase++;
245 #5;
246 assert(result === 32'h0382_D367) else $error("TC21 : SRA failed");
247
248 // ~~ TC22 ~~
249 #5;
250 a = 32'h805A_6CF3;
251 b = 32'h0000_0000;
252 testcase++;
253 #5;
254 assert(result === 32'h805A_6CF3) else $error("TC22 : SRA failed");
255
256 // ~~ TC23 ~~
257 #5;
258 a = 32'h805A_6CF3;
259 b = 32'h0000_0100;
260 testcase++;
261 #5;
262 assert(result === 32'h805A_6CF3) else $error("TC23 : SRA failed");
263
264
265 // ~~~~ SLT ~~~~
266 sel = 4'b0010;
267
268 // ~~ TC24 ~~
269 #5;
270 a = 32'h7FFF_FFFF;
271 b = 32'h8000_0000;
272 testcase++;
273 #5;
274 assert(result === 32'h0000_0000) else $error("TC24 : SLT failed");
275
276 // ~~ TC25 ~~
277 #5;
278 a = 32'h8000_0000;
279 b = 32'h0000_0001;
280 testcase++;
281 #5;
282 assert(result === 32'h0000_0001) else $error("TC25 : SLT failed");
283
284 // ~~ TC26 ~~
285 #5;
286 a = 32'h0000_0000;
287 b = 32'h0000_0000;
288 testcase++;
289
290
291

```

```

292     #5;
293     assert(result === 32'h0000_0000) else $error("TC26 : SLT failed");
294
295     // ~~ TC27 ~~
296     #5;
297     a = 32'h5555_5555;
298     b = 32'h5555_5555;
299     testcase++;
300     #5;
301     assert(result === 32'h0000_0000) else $error("TC27 : SLT failed");
302
303
304     // ~~~~ SLTU ~~~~
305     sel = 4'b0011;
306
307     // ~~ TC28 ~~
308     #5;
309     a = 32'h7FFF_FFFF;
310     b = 32'h8000_0000;
311     testcase++;
312     #5;
313     assert(result === 32'h0000_0001) else $error("TC28 : SLTU failed");
314
315     // ~~ TC29 ~~
316     #5;
317     a = 32'h8000_0000;
318     b = 32'h0000_0001;
319     testcase++;
320     #5;
321     assert(result === 32'h0000_0000) else $error("TC29 : SLTU failed");
322
323     // ~~ TC30 ~~
324     #5;
325     a = 32'h0000_0000;
326     b = 32'h0000_0000;
327     testcase++;
328     #5;
329     assert(result === 32'h0000_0000) else $error("TC30 : SLTU failed");
330
331     // ~~ TC31 ~~
332     #5;
333     a = 32'h55AA_55AA;
334     b = 32'h55AA_55AA;
335     testcase++;
336     #5;
337     assert(result === 32'h0000_0000) else $error("TC31 : SLTU failed");
338
339
340     // ~~~~ LUI-COPY (MV?) ~~~~
341     sel = 4'b1001;
342
343     // ~~ TC32 ~~
344     #5;
345     a = 32'h0123_4567;
346     b = 32'h7654_3210;
347     testcase++;
348     #5;
349     assert(result === 32'h0123_4567) else $error("TC32 : LUI-COPY failed");
350
351     // ~~ TC33 ~~
352     #5;
353     a = 32'hFEDC_BA98;
354     b = 32'h89AB_CDEF;
355     testcase++;
356     #5;
357     assert(result === 32'hFEDC_BA98) else $error("TC33 : LUI-COPY failed");
358
359     end
360
361 endmodule

```

0.3.5 Immediate Generator

Listing 28: Immediate Gen ImmedGen.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/25/2026 10:59:49 AM
7  // Design Name:
8  // Module Name: ImmedGen
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module ImmedGen(
24     input logic [BITWIDTH-1:0] INSTR,
25     output logic[BITWIDTH-1:0] U_TYPE, I_TYPE, S_TYPE, J_TYPE, B_TYPE
26 );
27
28     //~~~~~ immediates are immediate! ~~~~~
29     assign U_TYPE = { INSTR[31:12], 12'b0 };
30
31     assign I_TYPE = { {21{INSTR[31]}} , INSTR[30:20] };
32
33     assign S_TYPE = { {21{INSTR[31]}} , INSTR[30:25], INSTR[11:7] };
34
35     assign B_TYPE =
36     {
37         {20{INSTR[31]}} ,
38         INSTR[7],
39         INSTR[30:25],
40         INSTR[11:8],
41         1'b0
42     };
43
44     assign J_TYPE =
45     {
46         {12{INSTR[31]}} ,
47         INSTR[19:12],
48         INSTR[20],
49         INSTR[30:21],
50         1'b0
51     };
52
53
54
55 endmodule
```

Listing 29: TestSim for ImmedGen ImmedGen_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/25/2026 11:41:08 AM
7  // Design Name:
8  // Module Name: ImmedGen_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module ImmedGen_tb();
24
25     // ~~~~~ init local vars ~~~~~
26     logic[31:0] instr;
27     logic[31:0] u, i, s, b, j;
28
```

```

29 logic[31:0] testcase; // testing var
30
31 // ~~~~ instances ~~~~
32 ImmedGen UUT
33 (
34     .INSTR(instr),
35     .U_TYPE(u),
36     .I_TYPE(i),
37     .S_TYPE(s),
38     .J_TYPE(j),
39     .B_TYPE(b)
40 );
41
42 // ~~~~ testing ~~~~
43 initial
44 begin
45
46     testcase = 1;
47
48     // ~~ TC1 : all zero ~~
49     #5;
50     testcase++;
51     instr = 0;
52     #5;
53     assert(u === 0) else $error("TC1 : U failed");
54     assert(i === 0) else $error("TC1 : I failed");
55     assert(s === 0) else $error("TC1 : S failed");
56     assert(b === 0) else $error("TC1 : B failed");
57     assert(j === 0) else $error("TC1 : J failed");
58
59
60     // ~~ TC2 : all one ~~
61     #5;
62     testcase++;
63     instr = 32'hFFFF_FFFF;
64     #5;
65     assert(u === 32'hFFFF_F000) else $error("TC2 : U failed");
66     assert(i === 32'hFFFF_FFFF) else $error("TC2 : I failed");
67     assert(s === 32'hFFFF_FFFF) else $error("TC2 : S failed");
68     assert(b === 32'hFFFF_FFFE) else $error("TC2 : B failed");
69     assert(j === 32'hFFFF_FFFE) else $error("TC2 : J failed");
70
71
72     // ~~ TC3 : hex1 ~~
73     #5;
74     testcase++;
75     instr = 32'h5A5A_5A5A;
76     #5;
77     assert(u === 32'h5A5A_5000) else $error("TC3 : U failed");
78     assert(i === 32'h0000_05A5) else $error("TC3 : I failed");
79     assert(s === 32'h0000_05B4) else $error("TC3 : S failed");
80     assert(b === 32'h0000_05B4) else $error("TC3 : B failed");
81     assert(j === 32'h000A_5DA4) else $error("TC3 : J failed");
82
83
84     // ~~ TC4 : hex2 ~~
85     #5;
86     testcase++;
87     instr = 32'hF0F0_F0F0;
88     #5;
89     assert(u === 32'hF0F0_F000) else $error("TC4 : U failed");
90     assert(i === 32'hFFFF_FF0F) else $error("TC4 : I failed");
91     assert(s === 32'hFFFF_FF01) else $error("TC4 : S failed");
92     assert(b === 32'hFFFF_FF00) else $error("TC4 : B failed");
93     assert(j === 32'hFFF0_FF0E) else $error("TC4 : J failed");
94
95
96     // ~~ TC5 : hex3 ~~
97     #5;
98     testcase++;
99     instr = 32'h3CA5_5A3C;
100     #5;
101     assert(u === 32'h3CA5_5000) else $error("TC5 : U failed");
102     assert(i === 32'h0000_03CA) else $error("TC5 : I failed");
103     assert(s === 32'h0000_03D4) else $error("TC5 : S failed");
104     assert(b === 32'h0000_03D4) else $error("TC5 : B failed");
105     assert(j === 32'h0005_53CA) else $error("TC5 : J failed");
106
107
108     end
109
110
111 endmodule

```

0.3.6 Memory

Listing 30: Memory module otter_memory_v1_07.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: J. Callenes, P. Hummel
5  //
6  // Create Date: 01/27/2019 08:37:11 AM
7  // Module Name: OTTER_mem
8  // Project Name: Memory for OTTER RV32I RISC-V
9  // Tool Versions: Xilinx Vivado 2019.2
10 // Description: 64k Memory, dual access read single access write. Designed to
11 //              purposely utilize BRAM which requires synchronous reads and write
12 //              ADDR1 used for Program Memory Instruction. Word addressable so it
13 //              must be adapted from byte addresses in connection from PC
14 //              ADDR2 used for data access, both internal and external memory
15 //              mapped IO. ADDR2 is byte addressable.
16 //              RDEN1 and EDEN2 are read enables for ADDR1 and ADDR2. These are
17 //              needed due to synchronous reading
18 //              MEM_SIZE used to specify reads as byte (0), half (1), or word (2)
19 //              MEM_SIGN used to specify unsigned (1) vs signed (0) extension
20 //              IO_IN is data from external IO and synchronously buffered
21 //
22 // Memory OTTER_MEMORY (
23 //     .MEM_CLK      (),
24 //     .MEM_RDEN1    (),
25 //     .MEM_RDEN2    (),
26 //     .MEM_WE2      (),
27 //     .MEM_ADDR1    (),
28 //     .MEM_ADDR2    (),
29 //     .MEM_DIN2     (),
30 //     .MEM_SIZE     (),
31 //     .MEM_SIGN     (),
32 //     .IO_IN        (),
33 //     .IO_WR        (),
34 //     .MEM_DOUT1    (),
35 //     .MEM_DOUT2    () );
36 //
37 // Revision:
38 // Revision 0.01 - Original by J. Callenes
39 // Revision 1.02 - Rewrite to simplify logic by P. Hummel
40 // Revision 1.03 - changed signal names, added instantiation template
41 // Revision 1.04 - added default to write case statement
42 // Revision 1.05 - changed MEM_WD to MEM_DIN2, changed default to save nothing
43 // Revision 1.06 - removed typo in instantiation template
44 // Revision 1.07 - remove unused wordAddr1 signal
45 //
46 ///////////////////////////////////////////////////////////////////
47
48 module Memory (
49     input          MEM_CLK,
50     input          MEM_RDEN1, // read enable : Instruction    (addr1)
51     input          MEM_RDEN2, // read enable : Data      (addr2)
52     input          MEM_WE2,   // write enable.
53     input [13:0]   MEM_ADDR1, // Addr : Instruction Memory (Connect to PC[15:2])
54     input [31:0]   MEM_ADDR2, // Addr : Data Memory
55     input [31:0]   MEM_DIN2,  // Data to save
56     input [1:0]    MEM_SIZE,   // {0-Byte, 1-Half, 2-Word}
57     input          MEM_SIGN,   // 1-unsigned 0-signed
58     input [31:0]   IO_IN,      // Data from IO
59     output logic   IO_WR,      // IO 1-write 0-read
60     output logic [31:0] MEM_DOUT1, // Instruction
61     output logic [31:0] MEM_DOUT2 // Data
62     //output ERR              // only used for testing
63 );
64
65 // ~~~~ local variables ~~~~
66 logic [13:0] wordAddr2;
67 logic [31:0] memReadWord, ioBuffer, memReadSized;
68 logic [1:0] byteOffset;
69 logic weAddrValid; // active when saving (WE) to valid memory address
70 localparam [31:0] MMIO_ADDR = 32'h0001_0000;
71
72 // ~~~~ attributes (for memory) ~~~~
73 (* rom_style="{distributed | block}" *)
74 (* ram_decomp = "power" *) logic [31:0] memory [0:16383];
75
76 // ~~~~ memory injection ~~~~
77 initial
78 begin
79     //$readmemh("tc3x3.mem", memory, 0, 16383);
80     //$readmemh("tc10x10.mem", memory, 0, 16383);
81     //$readmemh("tc16x16.mem", memory, 0, 16383);
82
83     //$readmemh("otter_hw6sample_test.mem", memory, 0, 16383); // cpe233 : hw6
84     //$readmemh("diego_Test_All.mem", memory, 0, 16383); // cpe233 : hw7
85     $readmemh("InterruptTestProg_defaultmap.mem", memory, 0, 16383); // cpe233 : hw8
86
87 end
```

```

88
89 // ~~~~ todo ~~~~
90 assign wordAddr2      = MEM_ADDR2[15:2];
91 assign byteOffset     = MEM_ADDR2[1:0];      // byte offset of memory address
92
93 // NOT USED IN OTTER
94 //Check for misaligned or out of bounds memory accesses
95 //assign ERR = ((MEM_ADDR1 >= 2**ACTUAL_WIDTH)|| (MEM_ADDR2 >= 2**ACTUAL_WIDTH)
96 //              || MEM_ADDR1[1:0] != 2'b0 || MEM_ADDR2[1:0] !=2'b0)? 1 : 0;
97
98 // ~~~~ buffer the IO input for reading ~~~~
99 always_ff @(posedge MEM_CLK)
100 begin
101     if(MEM_RDEN2)
102         ioBuffer <= IO_IN;
103 end
104
105 // ~~~~ BRAM requires all reads and writes to occur synchronously ~~~~
106 always_ff @(posedge MEM_CLK)
107 begin
108
109     // ~~ save data (WD) to memory (ADDR2) ~~
110     if (weAddrValid == 1)                                // write enable and valid address space (not past sta
111     begin
112         case({MEM_SIZE,byteOffset})
113             // ~~ byte(00) ~~
114             4'b00_00: memory[wordAddr2][7:0]    <= MEM_DIN2[7:0];        // sb instr
115             4'b00_01: memory[wordAddr2][15:8]   <= MEM_DIN2[7:0];
116             4'b00_10: memory[wordAddr2][23:16]  <= MEM_DIN2[7:0];
117             4'b00_11: memory[wordAddr2][31:24]  <= MEM_DIN2[7:0];
118             // ~~ half(01) ~~
119             4'b01_00: memory[wordAddr2][15:0]   <= MEM_DIN2[15:0];      // sh instr
120             4'b01_01: memory[wordAddr2][23:8]   <= MEM_DIN2[15:0];
121             4'b01_10: memory[wordAddr2][31:16]  <= MEM_DIN2[15:0];
122             // ~~ word(10) ~~
123             4'b10_00: memory[wordAddr2]         <= MEM_DIN2;            // sw instr
124
125             //default: memory[wordAddr2]         <= 32'b0                // unsupported size, byte offset
126             // removed to avoid mistakes causing memory to be zeroed.
127         endcase
128     end
129
130     // ~~ read all data synchronously required for BRAM ~~
131     if (MEM_RDEN1)                                // need EN for extra load cycle to not change instru
132         MEM_DOUT1 <= memory[MEM_ADDR1];
133
134     if (MEM_RDEN2)                                // Read word from memory
135         memReadWord <= memory[wordAddr2];
136 end
137
138 // ~~~~ Change the data word, from ADR2, into sized bytes and sign extend ~~~~
139 always_comb
140 begin
141     // ~~ sign(1 bit), size(2 bits), offset(2 bits) ~~
142     case({MEM_SIGN,MEM_SIZE,byteOffset})
143
144         // ~~ signed (0) : extend extensions ~~~
145         // ~ byte(00) : extend for 3 bytes and 1 byte of data ~
146         5'b0_00_11: memReadSized = {{24{memReadWord[31]}},memReadWord[31:24]};
147         5'b0_00_10: memReadSized = {{24{memReadWord[23]}},memReadWord[23:16]};
148         5'b0_00_01: memReadSized = {{24{memReadWord[15]}},memReadWord[15:8]};
149         5'b0_00_00: memReadSized = {{24{memReadWord[7]}},memReadWord[7:0]};
150         // ~ half (01) : extend for 2 bytes and 2 bytes of data ~
151         5'b0_01_10: memReadSized = {{16{memReadWord[31]}},memReadWord[31:16]};
152         5'b0_01_01: memReadSized = {{16{memReadWord[23]}},memReadWord[23:8]};
153         5'b0_01_00: memReadSized = {{16{memReadWord[15]}},memReadWord[15:0]};
154         // ~ word (10) : no extension ~~
155         5'b0_10_00: memReadSized = memReadWord;
156
157
158         // ~~ unsigned (1) : no sign extensions ~~
159         // ~ byte (00) : 3 bytes of zero and 1 byte of data ~
160         5'b1_00_11: memReadSized = {24'd0,memReadWord[31:24]};
161         5'b1_00_10: memReadSized = {24'd0,memReadWord[23:16]};
162         5'b1_00_01: memReadSized = {24'd0,memReadWord[15:8]};
163         5'b1_00_00: memReadSized = {24'd0,memReadWord[7:0]};
164         // ~ half (01) : 2 bytes of zero and 2 bytes of data ~
165         5'b1_01_10: memReadSized = {16'd0,memReadWord[31:16]};
166         5'b1_01_01: memReadSized = {16'd0,memReadWord[23:8]};
167         5'b1_01_00: memReadSized = {16'd0,memReadWord[15:0]};
168         // ~ unsigned word is missing? ~
169
170         default: memReadSized = 32'b0;                // unsupported size, byte offset combination
171     endcase
172 end
173
174 // ~~~~ MemoryMapped IO ~~~~
175 always_comb
176 begin
177     if(MEM_ADDR2 >= MMIO_ADDR)                        // external address range (beyond stack)
178     begin

```

```

179     IO_WR = MEM_WE2;           // IO Write
180     MEM_DOUT2 = ioBuffer;      // IO read from buffer
181     weAddrValid = 0;           // address beyond memory range
182 end
183
184 else
185 begin
186     IO_WR = 0;                 // not MMIO
187     MEM_DOUT2 = memReadSized;  // output sized and sign extended data
188     weAddrValid = MEM_WE2;     // address in valid memory range
189 end
190 end
191
192 endmodule

```

0.3.7 Branch Address Generator

Listing 31: Branch Address Generator BranchAddrGen.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/28/2026 07:35:34 PM
7  // Design Name:
8  // Module Name: BranchAddrGen
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module BranchAddrGen(
24     input  logic [31:0] RS1,
25     input  logic [31:0] I_TYPE, J_TYPE, B_TYPE,
26     input  logic [31:0] PC,
27     output logic [31:0] JAL, JALR, BRANCH
28 );
29     // ~~~~ combinational logic ~~~~
30     assign JAL      = PC      + J_TYPE;
31     assign JALR     = RS1    + I_TYPE;
32     assign BRANCH   = PC      + B_TYPE;
33
34 endmodule
```

Listing 32: TestSim for Branch Addr Gen BranchAddrGen_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/05/2026 08:23:37 AM
7  // Design Name:
8  // Module Name: BranchAddrGen_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module BranchAddrGen_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic[31:0] rs, i, j, b, pc;    // input
27     logic[31:0] jal, jalr, bran;    // output
28
29     logic[31:0] testcase;           // testing var
30
31     // ~~~~ instances ~~~~
32     BranchAddrGen UUT
33     (
34         // ~~ inputs ~~
35         .RS1(rs),
36         .I_TYPE(i),
37         .J_TYPE(j),
38         .B_TYPE(b),
39         .PC(pc),
40         // ~~ outputs ~~
41         .JAL(jal),                // = pc + j
42         .JALR(jalr),              // = rs1 + i
43         .BRANCH(bran)             // = pc + b
44     );
45
46     // ~~~~ testing ~~~~
47     initial
48     begin
49
```

```

50 // ~~ TC1 : all zero ~~
51 #5;
52 rs = 32'h0000_0000;
53 i = 32'h0000_0000;
54 j = 32'h0000_0000;
55 b = 32'h0000_0000;
56 pc = 32'h0000_0000;
57 testcase = 1;
58 #5;
59 $display("TC%d:", testcase);
60 assert(jalr === 32'h0000_0000) else $error("JALR failed"); // = rs + i
61 assert(jal === 32'h0000_0000) else $error("JAL failed"); // = pc + j
62 assert(bran === 32'h0000_0000) else $error("BRANCH failed"); // = pc + b
63
64
65 // ~~ TC2 : only types ~~
66 #5;
67 rs = 32'h0000_0000;
68 i = 32'h0101_0101;
69 j = 32'hF0F0_A050;
70 b = 32'h1234_5678;
71 pc = 32'h0000_0000;
72 testcase++;
73 #5;
74 $display("TC%d:", testcase);
75 assert(jalr === 32'h0101_0101) else $error("JALR failed"); // = rs + i
76 assert(jal === 32'hF0F0_A050) else $error("JAL failed"); // = pc + j
77 assert(bran === 32'h1234_5678) else $error("BRANCH failed"); // = pc + b
78
79
80 // ~~ TC3 : only pc ~~
81 #5;
82 rs = 32'h0000_0000;
83 i = 32'h0000_0000;
84 j = 32'h0000_0000;
85 b = 32'h0000_0000;
86 pc = 32'h5EC3_25AF;
87 testcase++;
88 #5;
89 $display("TC%d:", testcase);
90 assert(jalr === 32'h0000_0000) else $error("JALR failed"); // = rs + i
91 assert(jal === 32'h5EC3_25AF) else $error("JAL failed"); // = pc + j
92 assert(bran === 32'h5EC3_25AF) else $error("BRANCH failed"); // = pc + b
93
94
95
96 // ~~ TC4 : only rs ~~
97 #5;
98 rs = 32'h9876_6789;
99 i = 32'h0000_0000;
100 j = 32'h0000_0000;
101 b = 32'h0000_0000;
102 pc = 32'h0000_0000;
103 testcase++;
104 #5;
105 $display("TC%d:", testcase);
106 assert(jalr === 32'h9876_6789) else $error("JALR failed"); // = rs + i
107 assert(jal === 32'h0000_0000) else $error("JAL failed"); // = pc + j
108 assert(bran === 32'h0000_0000) else $error("BRANCH failed"); // = pc + b
109
110
111
112 // ~~ TC5 : rollover + ceiling 1 ~~
113 #5;
114 rs = 32'hFFFF_FFFE;
115 i = 32'h0000_0001;
116 j = 32'h0000_0001;
117 b = 32'h0000_0001;
118 pc = 32'hFFFF_FFFF;
119 testcase++;
120 #5;
121 $display("TC%d:", testcase);
122 assert(jalr === 32'hFFFF_FFFF) else $error("JALR failed"); // = rs + i
123 assert(jal === 32'h0000_0000) else $error("JAL failed"); // = pc + j
124 assert(bran === 32'h0000_0000) else $error("BRANCH failed"); // = pc + b
125
126
127 // ~~ TC6 : rollover + ceiling 2 ~~
128 #5;
129 rs = 32'h0000_0002;
130 i = 32'hFFFF_FFFE;
131 j = 32'hFFFF_FFFF;
132 b = 32'hFFFF_FFFE;
133 pc = 32'h0000_0003;
134 testcase++;
135 #5;
136 $display("TC%d:", testcase);
137 assert(jalr === 32'h0000_0000) else $error("JALR failed"); // = rs + i
138 assert(jal === 32'h0000_0002) else $error("JAL failed"); // = pc + j
139 assert(bran === 32'h0000_0001) else $error("BRANCH failed"); // = pc + b
140

```

```
141
142     end
143
144 endmodule
```

0.3.8 Branch Condition Generator

Listing 33: Branch Condition Generator BranchConditionGen.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 07/28/2026 07:21:37 PM
7  // Design Name:
8  // Module Name: BranchConditionGen
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //      Determines the branch condition instructions
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module BranchConditionGen(
24     input    logic [31:0] RS1, RS2,
25     output   logic      BR_EQ,      // equal branch
26             logic      BR_LT,      // less than branch
27             logic      BR_LTU     // less then branch (unsigned)
28 );
29
30     // ~~~~ combinational logic ~~~~
31     assign BR_EQ    = (RS1 == RS2);
32     assign BR_LT    = ($signed(RS1) < $signed(RS2));
33     assign BR_LTU   = (RS1 < RS2);
34
35 endmodule
```

Listing 34: TestSim for Branch Condition Gen BranchConditionGen_tb.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/04/2026 10:49:22 AM
7  // Design Name:
8  // Module Name: BranchConditionGen_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module BranchConditionGen_tb();
24
25     // ~~~~ init local vars ~~~~
26     logic[31:0] a, b;
27     logic      eq, lts, ltu;
28
29     logic[31:0] testcase;      // testing var
30
31     // ~~~~ instances ~~~~
32     BranchConditionGen UUT
33     (
34         .RS1(a),
35         .RS2(b),
36         .BR_EQ(eq),
37         .BR_LT(lts),
38         .BR_LTU(ltu)
39     );
40
41     // ~~~~ testing ~~~~
42     initial
43     begin
44
45         // ~~ TC1 : equal are not less (simple) ~~
46         #5;
47         a = 32'h1234_5678;
48         b = 32'h1234_5678;
```

```

49     testcase = 1;
50     #5;
51     $display("TC%d:", testcase);
52     assert(eq === 1'b1) else $error("Equality failed");
53     assert(lts === 1'b0) else $error("LessThan Signed failed");
54     assert(ltu === 1'b0) else $error("LessThan Unsigned failed");
55
56
57     // ~~ TC2 : signed vs unsigned (simple) ~~
58     #5;
59     a = 32'hA50F_96C3; // is less than b if signed
60     b = 32'h5AF0_693C;
61     testcase++;
62     #5;
63     $display("TC%d:", testcase);
64     assert(eq === 1'b0) else $error("Equality failed");
65     assert(lts === 1'b1) else $error("LessThan Signed failed");
66     assert(ltu === 1'b0) else $error("LessThan Unsigned failed");
67
68
69     // ~~ TC3 : equal are not less (ceiling) ~~
70     #5;
71     a = 32'hFFFF_FFFF;
72     b = 32'hFFFF_FFFF;
73     testcase++;
74     #5;
75     $display("TC%d:", testcase);
76     assert(eq === 1'b1) else $error("Equality failed");
77     assert(lts === 1'b0) else $error("LessThan Signed failed");
78     assert(ltu === 1'b0) else $error("LessThan Unsigned failed");
79
80
81     // ~~ TC4 : equal are not less (floor) ~~
82     #5;
83     a = 32'h0000_0000;
84     b = 32'h0000_0000;
85     testcase++;
86     #5;
87     $display("TC%d:", testcase);
88     assert(eq === 1'b1) else $error("Equality failed");
89     assert(lts === 1'b0) else $error("LessThan Signed failed");
90     assert(ltu === 1'b0) else $error("LessThan Unsigned failed");
91
92
93     // ~~ TC5 : signed vs unsigned (simple reversed) ~~
94     #5;
95     a = 32'h5AF0_693C;
96     b = 32'hA50F_96C3;
97     testcase++;
98     #5;
99     $display("TC%d:", testcase);
100    assert(eq === 1'b0) else $error("Equality failed");
101    assert(lts === 1'b0) else $error("LessThan Signed failed");
102    assert(ltu === 1'b1) else $error("LessThan Unsigned failed");
103
104
105    // ~~ TC6 : signed vs unsigned (floor and ceiling 1) ~~
106    #5;
107    a = 32'h0000_0000;
108    b = 32'hFFFF_FFFF;
109    testcase++;
110    #5;
111    $display("TC%d:", testcase);
112    assert(eq === 1'b0) else $error("Equality failed");
113    assert(lts === 1'b0) else $error("LessThan Signed failed");
114    assert(ltu === 1'b1) else $error("LessThan Unsigned failed");
115
116
117    // ~~ TC7 : signed vs unsigned (floor and ceiling 2) ~~
118    #5;
119    a = 32'hFFFF_FFFF;
120    b = 32'h0000_0000;
121    testcase++;
122    #5;
123    $display("TC%d:", testcase);
124    assert(eq === 1'b0) else $error("Equality failed");
125    assert(lts === 1'b1) else $error("LessThan Signed failed");
126    assert(ltu === 1'b0) else $error("LessThan Unsigned failed");
127    end
128
129 endmodule

```

0.3.9 Control Unit Decoder

Listing 35: Control Unit Decoder CUnit_Decoder.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/14/2026 02:38:59 PM
7  // Design Name:
8  // Module Name: CUnit_Decoder
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module CUnit_Decoder(
24
25     // ~~~~ obtain inputs during/after EXEC ~~~~
26     input logic [6:0]    OPCODE,
27     input logic [2:0]    FUNC3,
28     input logic          FUNC7,
29     input logic          BR_EQ, BR_LT, BR_LTU,
30     input logic          INT_TAKEN,                // interrupt in process
31
32     // ~~~~ MUX sel outs ~~~~
33     output logic [3:0]   ALU_FUNC,                // ALU operation sel
34     output logic [1:0]   ALU_SRC_A,                // ALU srcA sel
35     output logic [2:0]   ALU_SRC_B,                // ALU srcB sel
36     output logic [2:0]   PC_SOURCE,                // input sel for program counter
37     output logic [1:0]   RF_WR_SEL                // register file write (data) source sel
38 );
39
40
41 always_comb
42 begin
43
44     // ~~ defaults ~~
45     ALU_FUNC    = 4'b0000;
46     ALU_SRC_A   = 2'b00;
47     ALU_SRC_B   = 3'b000;
48     PC_SOURCE   = 3'b000;
49     RF_WR_SEL   = 2'b00;
50
51     // if interrupt is requested, select mtvec for pc
52     if(INT_TAKEN == 1'b1) begin PC_SOURCE = 3'b100; end
53
54     // else interrupt is not requested, process by opcode instead
55     else begin
56
57         case(OPCODE)
58
59             // ~~ loads (OP=3) ~~
60             7'b0000011:
61                 begin
62                     ALU_SRC_B = 3'b001;                // Select : immed (I-type)
63                     RF_WR_SEL = 2'b10;                // Select : rd = DOUT2 = ram[r]
64                 end
65
66             // ~~ immed value (OP=19) ~~
67             7'b0010011:
68                 begin
69                     ALU_FUNC = ((FUNC3 == 3'b101) && (FUNC7 == 1'b1)) ? ({1'b0, FUNC3} + 4'b1000) : {1'b0,
70                     ALU_SRC_B = 3'b001;                // Select immed I
71                     RF_WR_SEL = 2'b11;                // Select rd = result
72                 end
73
74             // ~~ add upper imm to pc (auipc) (OP=23) ~~
75             7'b0010111:
76                 begin
77
78                     ALU_SRC_A = 2'b01;                // Select : imm (U-type)
79                     ALU_SRC_B = 3'b011;                // Select : PC
80                     RF_WR_SEL = 2'b11;                // Select : result
81                 end
82
83             // ~~ store (OP=35) ~~
84             7'b0100011:
85                 begin
86                     ALU_SRC_B = 3'b010;                // Select : immed (S-type)
87                 end
88         end
89     end
90 end
```

```

88
89 // ~~ Registers as value (OP=51) ~~
90 7'b0110011:
91 begin
92
93     if((FUNC3 == 3'b000) || (FUNC3 == 3'b101))
94     begin
95         ALU_FUNC = (FUNC7 == 1'b1) ? ({1'b0,FUNC3}+4'b1000) : {1'b0,FUNC3};
96     end
97
98     else begin ALU_FUNC = {1'b0,FUNC3}; end
99
100     RF_WR_SEL    = 2'b11; // Select rd = result
101 end
102
103 // ~~ lui (OP=55) ~~
104 7'b0110111:
105 begin
106     ALU_FUNC      = 4'b1001; // Select : copy srcA
107     ALU_SRC_A     = 2'b01;   // Select : imm (U-type)
108     RF_WR_SEL     = 2'b11;   // Select : result
109 end
110
111 // ~~ Branch (OP=99) ~~
112 7'b1100011:
113 begin
114     case(FUNC3)
115         3'b000: begin PC_SOURCE = (BR_EQ) ? 3'b010 : 3'b000; end // BEQ : branch
116         3'b001: begin PC_SOURCE = (~BR_EQ) ? 3'b010 : 3'b000; end // BNE : branch
117         3'b100: begin PC_SOURCE = (BR_LT) ? 3'b010 : 3'b000; end // BLT : branch
118         3'b101: begin PC_SOURCE = (~BR_LT) ? 3'b010 : 3'b000; end // BGE : branch
119         3'b110: begin PC_SOURCE = (BR_LTU) ? 3'b010 : 3'b000; end // BLTU : branch
120         3'b111: begin PC_SOURCE = (~BR_LTU) ? 3'b010 : 3'b000; end // BGEU : branch
121         default: begin PC_SOURCE = 3'b000; end
122     endcase
123 end
124
125 // ~~ Jump and link register (OP=103) ~~
126 7'b1100111:
127 begin
128     PC_SOURCE = 3'b001; // Select JALR
129 end
130
131 // ~~ Jump and link (OP=111) ~~
132 7'b110_1111:
133 begin
134     PC_SOURCE = 3'b011; // Select JAL
135 end
136
137 // ~~ CSR (OP=115) ~~
138 7'b111_0011:
139 begin
140
141     RF_WR_SEL    = 2'b01; // sel CSR[csr] for regfile mu
142     ALU_SRC_B     = 3'b100; // sel CSR[csr] for ALU-B
143
144     case(FUNC3)
145
146         // MRET
147         3'b000: begin PC_SOURCE = 3'b101; end // MEPC for PC
148
149         // CSRRW
150         3'b001:
151         begin
152             ALU_FUNC      = 4'b1001; // mv rs1 to result
153         end
154
155         // CSRRS
156         3'b010:
157         begin
158             ALU_FUNC      = 4'b0110; // rs1 | CSR[csr]
159         end
160
161         // CSRRC
162         3'b011:
163         begin
164             ALU_SRC_A     = 2'b10; // sel ~rs1
165             ALU_FUNC      = 4'b0111; // ~rs1 & CSR[csr]
166         end
167
168         default: begin end
169     endcase
170 end
171
172 endcase
173 end
174 end
175 endmodule

```

Listing 36: TestSim for Control Unit Decoder CUnit_Decoder_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/18/2026 09:18:24 AM
7  // Design Name:
8  // Module Name: CUnit_Decoder_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module CUnit_Decoder_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ local vars ~~~~
29     logic [6:0] opcode;
30     logic [2:0] func3;
31     logic      func7;
32     logic      br_eq, br_lt, br_ltu;
33     logic      int_taken;
34
35     logic [3:0] alu_func;
36     logic [1:0] src_a;
37     logic [2:0] src_b;
38     logic [2:0] pc_src;
39     logic [1:0] rf_wr_sel;
40
41     // ~~ testing ~~
42     testcase tc;
43     string message;
44     logic [31:0] tnum;
45
46     logic [2:0] func3_arr [0:9];
47     logic [3:0] alu_arr [0:9];
48     int         opcodeCNT;
49
50
51     // ~~~~ DUT instance ~~~~
52     CUnit_Decoder UUT
53     (
54         // ~~ inputs ~~
55         .OPCODE(opcode),           // 7'b
56         .FUNC3(func3),             // 3'b
57         .FUNC7(func7),             // 1'b
58         .BR_EQ(br_eq),             // 1'b
59         .BR_LT(br_lt),             // 1'b
60         .BR_LTU(br_ltu),           // 1'b
61         .INT_TAKEN(int_taken),     // 1'b
62
63         // ~~ outputs ~~
64         .ALU_FUNC(alu_func),        // 4'b
65         .ALU_SRC_A(src_a),          // 1'b
66         .ALU_SRC_B(src_b),          // 2'b
67         .PC_SOURCE(pc_src),         // 2'b
68         .RF_WR_SEL(rf_wr_sel)      // 2'b
69     );
70
71
72     // ~~~~ testing ~~~~
73     initial
74     begin
75
76
77         // ~~ class instances ~~
78         tc = new();
79
80         // ~~ TC1 : unknown opcode is defaulted ~~
81         tc.new_test("unknown opcode defaults");
82         tnum = tc.get_testnum();
83
84         opcode      = 7'b000_0000;
85         func3       = 3'bxxx;
86         func7       = 1'bx;
87         br_eq       = 1'bx;
88         br_lt       = 1'bx;
89         br_ltu      = 1'bx;
90         #5;
91         assert(alu_func == 4'b0000) else tc.err("ALU_FUNC failed");

```

```

92         assert(src_a == 2'b00)           else tc.err("ALU_SRC_A failed");
93         assert(src_b == 3'b000)         else tc.err("ALU_SRC_B failed");
94         assert(pc_src == 3'b000)         else tc.err("PC_SOURCE failed");
95         assert(rf_wr_sel == 2'b00)       else tc.err("RF_WR_SEL failed");
96         #5;
97     tc.test_done();
98
99
100
101 // ~~ TC2 : loading ~~
102 tc.new_test("load instr");
103 tnum = tc.get_testnum();
104
105 opcode = 7'b000_0011;
106 opcodeCNT = 5;
107 func3_arr =
108 '{
109     default : 3'b000,
110     0: 3'b000,           // lb
111     1: 3'b001,           // lh
112     2: 3'b010,           // lw
113     3: 3'b100,           // lbu
114     4: 3'b101           // lhu
115 };
116
117 for(int i = 0; i < opcodeCNT; i++)
118 begin
119
120     func3 = func3_arr[i];
121     func7 = 1'bx;
122     br_eq = 1'bx;
123     br_lt = 1'bx;
124     br_ltu = 1'bx;
125     #5;
126     message = $sformatf("FUNC3 = %0d", func3);
127     tc.print_subtest(message);
128     assert(alu_func == 4'b0000)         else tc.err("ALU_FUNC failed");
129     assert(src_a == 2'b00)             else tc.err("ALU_SRC_A failed");
130     assert(src_b == 3'b001)           else tc.err("ALU_SRC_B failed");
131     assert(pc_src == 3'b000)           else tc.err("PC_SOURCE failed");
132     assert(rf_wr_sel == 2'b10)         else tc.err("RF_WR_SEL failed");
133     #5;
134 end
135 tc.test_done();
136
137
138 // ~~ TC3 : immed ops ~~
139 tc.new_test("immed instr");
140 tnum = tc.get_testnum();
141
142 opcode = 7'b001_0011;
143 opcodeCNT = 9;
144 func3_arr =
145 '{
146     default : 3'b000,
147     0: 3'b000,           // addi
148     1: 3'b001,           // slli
149     2: 3'b010,           // slti
150     3: 3'b011,           // sltiu
151     4: 3'b100,           // xori
152     5: 3'b101,           // srli
153     6: 3'b101,           // srai
154     7: 3'b110,           // ori
155     8: 3'b111           // andi
156 };
157
158 alu_arr =
159 '{
160     default : 4'b0000,
161     0: 4'b0000,           // addi
162     1: 4'b0001,           // slli
163     2: 4'b0010,           // slti
164     3: 4'b0011,           // sltiu
165     4: 4'b0100,           // xori
166     5: 4'b0101,           // srli
167     6: 4'b1101,           // srai
168     7: 4'b0110,           // ori
169     8: 4'b0111           // andi
170 };
171
172 for(int i = 0; i < opcodeCNT; i++)
173 begin
174
175     func3 = func3_arr[i];
176     func7 = (i==6) ? 1'b1 : 1'b0;      //srai needs func7
177     br_eq = 1'bx;
178     br_lt = 1'bx;
179     br_ltu = 1'bx;
180     #5;
181     message = $sformatf("FUNC3 = %0d", func3);
182     tc.print_subtest(message);

```

```

183         assert(alu_func === alu_arr[i])           else tc.err("ALU_FUNC failed");
184         assert(src_a === 2'b00)                   else tc.err("ALU_SRC_A failed");
185         assert(src_b === 3'b001)                   else tc.err("ALU_SRC_B failed");
186         assert(pc_src === 3'b000)                   else tc.err("PC_SOURCE failed");
187         assert(rf_wr_sel === 2'b11)                 else tc.err("RF_WR_SEL failed");
188         #5;
189     end
190     tc.test_done();
191
192
193
194     // ~~ TC4 : auipc ~~
195     tc.new_test("auipc instr");
196     tnum = tc.get_testnum();
197
198     opcode = 7'b001_0111;
199     func3 = 3'bxxx;
200     func7 = 1'bx;
201     br_eq = 1'bx;
202     br_lt = 1'bx;
203     br_ltu = 1'bx;
204     #5;
205     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
206     assert(src_a === 2'b01)                 else tc.err("ALU_SRC_A failed");
207     assert(src_b === 3'b011)                 else tc.err("ALU_SRC_B failed");
208     assert(pc_src === 3'b000)                 else tc.err("PC_SOURCE failed");
209     assert(rf_wr_sel === 2'b11)                 else tc.err("RF_WR_SEL failed");
210     #5;
211     tc.test_done();
212
213
214
215     // ~~ TC5 : store ~~
216     tc.new_test("store instr");
217     tnum = tc.get_testnum();
218     opcode = 7'b010_0011;
219     opcodeCNT = 3;
220     func3_arr =
221     '{
222         default : 3'b000,
223         0: 3'b000,           // sb
224         1: 3'b001,           // sh
225         2: 3'b010           // sw
226     };
227
228     for(int i = 0; i < opcodeCNT; i++)
229     begin
230
231         func3 = func3_arr[i];
232         func7 = 1'bx;
233         br_eq = 1'bx;
234         br_lt = 1'bx;
235         br_ltu = 1'bx;
236         #5;
237         message = $sformatf("FUNC3 = %0d", func3);
238         tc.print_subtest(message);
239         assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
240         assert(src_a === 2'b00)                 else tc.err("ALU_SRC_A failed");
241         assert(src_b === 3'b010)                 else tc.err("ALU_SRC_B failed");
242         assert(pc_src === 3'b000)                 else tc.err("PC_SOURCE failed");
243         assert(rf_wr_sel === 2'b00)                 else tc.err("RF_WR_SEL failed");
244         #5;
245     end
246     tc.test_done();
247
248
249
250     // ~~ TC6 : dual register ops ~~
251     tc.new_test("dual src register instr");
252     tnum = tc.get_testnum();
253     opcode = 7'b011_0011;
254     opcodeCNT = 10;
255     func3_arr =
256     '{
257         default : 3'b000,
258         0: 3'b000,           // add
259         1: 3'b000,           // sub "
260         2: 3'b001,           // sll
261         3: 3'b010,           // slt
262         4: 3'b011,           // sltu
263         5: 3'b100,           // xor
264         6: 3'b101,           // srl
265         7: 3'b101,           // sra "
266         8: 3'b110,           // or
267         9: 3'b111           // and
268     };
269
270     alu_arr =
271     '{
272         default : 4'b0000,
273         0: 4'b0000,           // add

```

```

274         1: 4'b1000,           // sub
275         2: 4'b0001,           // sll
276         3: 4'b0010,           // slt
277         4: 4'b0011,           // sltu
278         5: 4'b0100,           // xor
279         6: 4'b0101,           // srl
280         7: 4'b1101,           // sra
281         8: 4'b0110,           // or
282         9: 4'b0111           // and
283     };
284
285
286     for(int i = 0; i < opcodeCNT; i++)
287     begin
288
289         func3    = func3_arr[i];
290         func7    = (i==1 || i==7) ? 1'b1 : 1'b0;    //sub and sra need func7
291         br_eq    = 1'bx;
292         br_lt    = 1'bx;
293         br_ltu   = 1'bx;
294         #5;
295         message = $sformatf("FUNC3 = %0d", func3);
296         tc.print_subtest(message);
297         assert(alu_func == alu_arr[i])           else tc.err("ALU_FUNC failed");
298         assert(src_a == 2'b00)                   else tc.err("ALU_SRC_A failed");
299         assert(src_b == 3'b000)                 else tc.err("ALU_SRC_B failed");
300         assert(pc_src == 3'b000)                 else tc.err("PC_SOURCE failed");
301         assert(rf_wr_sel == 2'b11)               else tc.err("RF_WR_SEL failed");
302         #5;
303     end
304     tc.test_done();
305
306
307     // ~~ TC7 : lui ~~
308     tc.new_test("lui instr");
309     tnum = tc.get_testnum();
310
311     opcode = 7'b011_0111;
312     func3  = 3'bxxx;
313     func7  = 1'bx;
314     br_eq  = 1'bx;
315     br_lt  = 1'bx;
316     br_ltu = 1'bx;
317     #5;
318     assert(alu_func == 4'b1001) else tc.err("ALU_FUNC failed");
319     assert(src_a == 2'b01)      else tc.err("ALU_SRC_A failed");
320     assert(src_b == 3'b000)     else tc.err("ALU_SRC_B failed");
321     assert(pc_src == 3'b000)     else tc.err("PC_SOURCE failed");
322     assert(rf_wr_sel == 2'b11)   else tc.err("RF_WR_SEL failed");
323     #5;
324     tc.test_done();
325
326
327     // ~~ TC8 : branch ~~
328     tc.new_test("branch instr");
329     tnum = tc.get_testnum();
330     opcode = 7'b110_0011;
331     opcodeCNT = 6;
332     func3_arr =
333     '{
334         default : 3'b000,
335         0: 3'b000,           // beq
336         1: 3'b001,           // bne
337         2: 3'b100,           // blt
338         3: 3'b101,           // bge
339         4: 3'b110,           // bltu
340         5: 3'b111           // bgeu
341     };
342
343     func7    = 1'bx;
344
345     // ~ beq (0) ~
346     func3    = func3_arr[0];
347     br_eq    = 1'b0;
348     br_lt    = 1'bx;
349     br_ltu   = 1'bx;
350     #5;
351     tc.print_subtest("before BEQ");
352     assert(alu_func == 4'b0000) else tc.err("ALU_FUNC failed");
353     assert(src_a == 2'b00)      else tc.err("ALU_SRC_A failed");
354     assert(src_b == 3'b000)     else tc.err("ALU_SRC_B failed");
355     assert(pc_src == 3'b000)     else tc.err("PC_SOURCE failed");
356     assert(rf_wr_sel == 2'b00)   else tc.err("RF_WR_SEL failed");
357     #5;
358     br_eq    = 1'b1;
359     #5;
360     tc.print_subtest("after BEQ");
361     assert(alu_func == 4'b0000) else tc.err("ALU_FUNC failed");
362     assert(src_a == 2'b00)      else tc.err("ALU_SRC_A failed");
363     assert(src_b == 3'b000)     else tc.err("ALU_SRC_B failed");
364     assert(pc_src == 3'b010)     else tc.err("PC_SOURCE failed");

```

```

365     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
366     #5;
367
368     // ~ bne (1) ~
369     func3      = func3_arr[1];
370     br_eq      = 1'b0;
371     br_lt      = 1'bx;
372     br_ltu     = 1'bx;
373     #5;
374     tc.print_subtest("before BNE");
375     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
376     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
377     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
378     assert(pc_src === 3'b010)             else tc.err("PC_SOURCE failed");
379     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
380     #5;
381     br_eq      = 1'b1;
382     #5;
383     tc.print_subtest("after BNE");
384     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
385     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
386     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
387     assert(pc_src === 3'b000)             else tc.err("PC_SOURCE failed");
388     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
389     #5;
390
391     // ~ blt (2) ~
392     func3      = func3_arr[2];
393     br_eq      = 1'bx;
394     br_lt      = 1'b0;
395     br_ltu     = 1'bx;
396     #5;
397     tc.print_subtest("before BLT");
398     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
399     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
400     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
401     assert(pc_src === 3'b000)             else tc.err("PC_SOURCE failed");
402     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
403     #5;
404     br_lt      = 1'b1;
405     #5;
406     tc.print_subtest("after BLT");
407     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
408     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
409     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
410     assert(pc_src === 3'b010)             else tc.err("PC_SOURCE failed");
411     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
412     #5;
413
414     // ~ bge (3) ~
415     func3      = func3_arr[3];
416     br_eq      = 1'bx;
417     br_lt      = 1'b0;
418     br_ltu     = 1'bx;
419     #5;
420     tc.print_subtest("before BGE");
421     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
422     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
423     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
424     assert(pc_src === 3'b010)             else tc.err("PC_SOURCE failed");
425     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
426     #5;
427     br_lt      = 1'b1;
428     #5;
429     tc.print_subtest("after BGE");
430     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
431     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
432     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
433     assert(pc_src === 3'b000)             else tc.err("PC_SOURCE failed");
434     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
435     #5;
436
437     // ~ bltu (4) ~
438     func3      = func3_arr[4];
439     br_eq      = 1'bx;
440     br_lt      = 1'bx;
441     br_ltu     = 1'b0;
442     #5;
443     tc.print_subtest("before BLTU");
444     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
445     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
446     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");
447     assert(pc_src === 3'b000)             else tc.err("PC_SOURCE failed");
448     assert(rf_wr_sel === 2'b00)           else tc.err("RF_WR_SEL failed");
449     #5;
450     br_ltu     = 1'b1;
451     #5;
452     tc.print_subtest("after BLTU");
453     assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
454     assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
455     assert(src_b  === 3'b000)             else tc.err("ALU_SRC_B failed");

```

```

456         assert(pc_src == 3'b010)           else tc.err("PC_SOURCE failed");
457         assert(rf_wr_sel == 2'b00)         else tc.err("RF_WR_SEL failed");
458         #5;
459
460         // ~ bgeu (5) ~
461         func3 = func3_arr[5];
462         br_eq  = 1'bx;
463         br_lt  = 1'bx;
464         br_ltu = 1'b0;
465         #5;
466         tc.print_subtest("before BGEU");
467         assert(alu_func == 4'b0000)         else tc.err("ALU_FUNC failed");
468         assert(src_a == 2'b00)              else tc.err("ALU_SRC_A failed");
469         assert(src_b == 3'b000)             else tc.err("ALU_SRC_B failed");
470         assert(pc_src == 3'b010)           else tc.err("PC_SOURCE failed");
471         assert(rf_wr_sel == 2'b00)         else tc.err("RF_WR_SEL failed");
472         #5;
473         br_ltu = 1'b1;
474         #5;
475         tc.print_subtest("after BGEU");
476         assert(alu_func == 4'b0000)         else tc.err("ALU_FUNC failed");
477         assert(src_a == 2'b00)              else tc.err("ALU_SRC_A failed");
478         assert(src_b == 3'b000)             else tc.err("ALU_SRC_B failed");
479         assert(pc_src == 3'b000)           else tc.err("PC_SOURCE failed");
480         assert(rf_wr_sel == 2'b00)         else tc.err("RF_WR_SEL failed");
481         #5;
482         tc.test_done();
483
484         // ~~ TC9 : jalr ~~
485         tc.new_test("jalr instr");
486         tnum = tc.get_testnum();
487         opcode = 7'b110_0111;
488         func3  = 3'bxxx;
489         func7  = 1'bx;
490         br_eq  = 1'bx;
491         br_lt  = 1'bx;
492         br_ltu = 1'bx;
493         #5;
494         assert(alu_func == 4'b0000)         else tc.err("ALU_FUNC failed");
495         assert(src_a == 2'b00)              else tc.err("ALU_SRC_A failed");
496         assert(src_b == 3'b000)             else tc.err("ALU_SRC_B failed");
497         assert(pc_src == 3'b001)           else tc.err("PC_SOURCE failed");
498         assert(rf_wr_sel == 2'b00)         else tc.err("RF_WR_SEL failed");
499         #5;
500         tc.test_done();
501
502         // ~~ TC10 : jal ~~
503         tc.new_test("jal instr");
504         tnum = tc.get_testnum();
505         opcode = 7'b110_1111;
506         func3  = 3'bxxx;
507         func7  = 1'bx;
508         br_eq  = 1'bx;
509         br_lt  = 1'bx;
510         br_ltu = 1'bx;
511         #5;
512         assert(alu_func == 4'b0000)         else tc.err("ALU_FUNC failed");
513         assert(src_a == 2'b00)              else tc.err("ALU_SRC_A failed");
514         assert(src_b == 3'b000)             else tc.err("ALU_SRC_B failed");
515         assert(pc_src == 3'b011)           else tc.err("PC_SOURCE failed");
516         assert(rf_wr_sel == 2'b00)         else tc.err("RF_WR_SEL failed");
517         #5;
518         tc.test_done();
519
520         // ~~ TC11 : interrupt request dominates ~~
521         tc.new_test("requested intrr");
522         tnum = tc.get_testnum();
523
524         int_taken = 1'b1;
525         opcode    = 7'bxxx_xxxx;
526         func3     = 3'bxxx;
527         func7     = 1'bx;
528         br_eq     = 1'bx;
529         br_lt     = 1'bx;
530         br_ltu    = 1'bx;
531         #5;
532         assert(alu_func == 4'b0000)         else tc.err("ALU_FUNC failed");
533         assert(src_a == 2'b00)              else tc.err("ALU_SRC_A failed");
534         assert(src_b == 3'b000)             else tc.err("ALU_SRC_B failed");
535         assert(pc_src == 3'b100)           else tc.err("PC_SOURCE failed");
536         assert(rf_wr_sel == 2'b00)         else tc.err("RF_WR_SEL failed");
537         #5;
538         tc.test_done();
539
540         // ~~ TC11 : csr ~~
541         tc.new_test("csr instrs");
542         tnum = tc.get_testnum();
543
544         int_taken = 1'b0;

```

```

547         opcode      = 7'b111_0011;
548         func3        = 3'b000;
549         func7        = 1'bx;
550         br_eq        = 1'bx;
551         br_lt        = 1'bx;
552         br_ltu       = 1'bx;
553         #5;
554         tc.print_subtest("MRET");
555         assert(alu_func === 4'b0000)           else tc.err("ALU_FUNC failed");
556         assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
557         assert(src_b  === 3'b100)              else tc.err("ALU_SRC_B failed");
558         assert(pc_src === 3'b101)              else tc.err("PC_SOURCE failed");
559         assert(rf_wr_sel === 2'b01)            else tc.err("RF_WR_SEL failed");
560         #5;
561
562         func3        = 3'b001;
563         #5;
564         tc.print_subtest("CSRRW");
565         assert(alu_func === 4'b1001)           else tc.err("ALU_FUNC failed");
566         assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
567         assert(src_b  === 3'b100)              else tc.err("ALU_SRC_B failed");
568         assert(pc_src === 3'b000)              else tc.err("PC_SOURCE failed");
569         assert(rf_wr_sel === 2'b01)            else tc.err("RF_WR_SEL failed");
570         #5;
571
572         func3        = 3'b010;
573         #5;
574         tc.print_subtest("CSRRS");
575         assert(alu_func === 4'b0110)           else tc.err("ALU_FUNC failed");
576         assert(src_a  === 2'b00)               else tc.err("ALU_SRC_A failed");
577         assert(src_b  === 3'b100)              else tc.err("ALU_SRC_B failed");
578         assert(pc_src === 3'b000)              else tc.err("PC_SOURCE failed");
579         assert(rf_wr_sel === 2'b01)            else tc.err("RF_WR_SEL failed");
580         #5;
581
582         func3        = 3'b011;
583         #5;
584         tc.print_subtest("CSRRC");
585         assert(alu_func === 4'b0111)           else tc.err("ALU_FUNC failed");
586         assert(src_a  === 2'b10)               else tc.err("ALU_SRC_A failed");
587         assert(src_b  === 3'b100)              else tc.err("ALU_SRC_B failed");
588         assert(pc_src === 3'b000)              else tc.err("PC_SOURCE failed");
589         assert(rf_wr_sel === 2'b01)            else tc.err("RF_WR_SEL failed");
590         #5;
591         tc.test_done();
592     end
593 endmodule

```

0.3.10 Control Unit Finite State Machine

Listing 37: Control Unit FSM CUnit_FSM.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/15/2026 11:47:24 AM
7  // Design Name:
8  // Module Name: CUnit_FSM
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////
21
22
23 module CUnit_FSM(
24     input logic      CLK,
25     input logic      RST,           // high-level input : reset
26     input logic      INTRR,        // high-level input : interrupt
27     input logic [6:0] OPCODE,      // shared with decoder + regfile
28     input logic [2:0] FUNC3,       //
29
30     output logic      PC_WRITE,    // update PC
31     output logic      REG_WRITE,   // update dest reg value
32     output logic      MEM_RDEN1,   // fetch value at addr1
33     output logic      MEM_RDEN2,   // fetch value at addr2 (occurs after update if enacted)
34     output logic      MEM_WE2,     // update value at addr2
35     output logic      RESET,       // program counter reset
36
37     output logic      CSR_WE,      // update csr[ADDR] = data
38     output logic      INT_TAKEN,   // interrupt desired
39     output logic      MRET_EXEC    // interrupt return desired
40 );
41
42
43 // ~~~~ local vars ~~~~
44 typedef enum logic [2:0] {INIT, FETCH, EXEC, WRITEBACK, INTRPT} e_state;
45 e_state curr_state, next_state;
46
47 typedef enum logic {LOAD, LOAD_BAR} e_lsignal;
48 e_lsignal curr_signal, next_signal;
49
50 logic start;
51
52 // ~~~~ initial conditions ~~~~
53 initial
54 begin
55     start      = 1'b1;
56 end
57
58 // ~~~~ state engine ~~~~
59 always_ff @(posedge CLK or posedge RST) begin
60     if(RST == 1'b1 || start == 1'b1) begin
61         curr_state      <= INIT;
62         curr_signal     <= LOAD;
63         start           <= 1'b0;
64     end
65     else begin
66         curr_state <= next_state;
67         curr_signal <= next_signal;
68     end
69 end
70
71 // ~~~~ state definitions ~~~~
72 always_comb begin
73
74     // ~~ defaults ~~
75     RESET      = 1'b0;
76     PC_WRITE   = 1'b0;
77     REG_WRITE  = 1'b0;
78     MEM_WE2    = 1'b0;
79     MEM_RDEN1  = 1'b0;
80     MEM_RDEN2  = 1'b0;
81     CSR_WE     = 1'b0;
82     INT_TAKEN  = 1'b0;
83     MRET_EXEC  = 1'b0;
84
85     next_state = INIT;
86     next_signal = LOAD;
87
```

```

88
89     case(curr_state)
90
91         // ~ INIT ~
92         //     - Defaults all signals
93         //     - Entry point for reset
94     INIT: begin
95         RESET          = 1'b1;           // reset PC
96
97         next_state     = FETCH;
98         next_signal    = LOAD;
99     end
100
101
102     // ~ FETCH ~
103     //     - Grabs instruction
104     //     - Set memory unit to grab instruction
105     FETCH: begin
106         // ~~ Repeat FETCH in extra cycle ~~
107         if(curr_signal == LOAD_BAR) begin
108             MEM_RDEN2   = 1'b1;
109             next_signal = LOAD_BAR;
110         end
111         // ~~ else, standard FETCH cycle ~~
112         else begin
113             MEM_RDEN1   = 1'b1;
114             next_signal = LOAD;
115         end
116
117         next_state = EXEC;
118     end
119
120     // ~ EXEC ~
121     //     - Set all control sigs to read memory
122     //     - may direct to FETCH for lengthy instr cycles
123     //     - OPCODE dictates what is toggled here
124     EXEC: begin
125
126         next_signal    = LOAD;
127         next_state     = WRITEBACK;
128
129         if(OPCODE == 7'b000_0011 && curr_signal == LOAD) begin // load needs an extra cycle
130             if(INTRR == 1'b1) begin next_state = INTRPT; end // take intrr rather than do extra
131
132             else begin // perform the extra cycle (no intrr)
133                 next_signal = LOAD_BAR;
134                 next_state = FETCH;
135             end
136         end
137
138         // mret discovered
139         if(OPCODE == 7'b1110011 && FUNC3 == 3'b000) begin
140             MRET_EXEC = 1'b1;
141         end
142     end
143
144 end
145
146
147
148     // ~ WRITEBACK ~
149     //     - Save contents to reg or memory
150     //     - update pc
151     WRITEBACK: begin
152
153         // common default
154         PC_WRITE    = 1'b1;
155         REG_WRITE   = 1'b1;
156
157         // special cases
158         case(OPCODE)
159             // store (35)
160             7'b0100011: begin
161                 REG_WRITE    = 1'b0;
162                 MEM_WE2      = 1'b1;
163             end
164
165             // branch (99)
166             7'b1100011: begin REG_WRITE    = 1'b0; end
167
168             // csr interrupt (115)
169             7'b1110011: begin
170                 if(FUNC3 == 3'b000) begin REG_WRITE = 1'b0; end // mret
171                 else begin CSR_WE = 1'b1; end // csrXX
172             end
173         endcase
174
175         next_state = (INTRR == 1'b1)? INTRPT : FETCH;
176         next_signal = LOAD;
177
178     end

```

```

179
180         // ~ INTRPT ~
181         //      - Signal to CU decoder and CSR that an intrr is occurring
182         //      - Load the ISR address into the PC
183         INTRPT: begin
184             INT_TAKEN    = 1'b1;           // signal other mods of interrupt taken
185             PC_WRITE     = 1'b1;           // update PC with MVTEC (pray to avoid a race condition)
186
187             next_state   = FETCH;
188             next_signal   = LOAD;
189         end
190
191     endcase
192
193 end
194
195 endmodule

```

Listing 38: TestSim for Control Unit FSM CUnit_FSM_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/18/2026 12:48:54 PM
7  // Design Name:
8  // Module Name: CUnit_FSM_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module CUnit_FSM_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ local vars ~~~~
29     // ~~ input ~~
30     logic      clk;
31     logic      rst;
32     logic      intrr;
33     logic [6:0] opcode;
34     logic [2:0] func3;
35     // ~~ output ~~
36     logic      pc_write;
37     logic      reg_write;
38     logic      mem_rden1;
39     logic      mem_rden2;
40     logic      mem_we2;
41     logic      pc_reset;
42     // ~~ isr (output) ~~
43     logic      csr_we;
44     logic      int_taken;
45     logic      mret_exec;
46
47     // ~~ testing ~~
48     testcase tc;
49     logic [31:0] tnum;
50
51     // ~~ encapsulations ~~
52     logic[6:0] opc_arr [0:9];
53     int opcCNT = 10;
54
55     // ~~~~ UUT instance ~~~~
56     CUnit_FSM UUT
57     (
58         .CLK(clk),           // 1'b I
59         .RST(rst),           // 1'b I
60         .INTRR(intrr),       // 1'b I
61         .OPCODE(opcode),     // 7'b I
62         .FUNC3(func3),       // 3'b I
63
64         .PC_WRITE(pc_write), // 1'b O
65         .REG_WRITE(reg_write), // 1'b O
66         .MEM_RDEN1(mem_rden1), // 1'b O
67         .MEM_RDEN2(mem_rden2), // 1'b O
68         .MEM_WE2(mem_we2),    // 1'b O
69         .RESET(pc_reset),     // 1'b O
70
71         .CSR_WE(csr_we),      // 1'b

```

```

72     .INT_TAKEN(int_taken),          // 1'b
73     .MRET_EXEC(mret_exec)         // 1'b
74 );
75
76
77 // ~~ action : CLK (10ns period) ~~
78 always
79 begin
80     #5;
81     clk <= !clk;
82 end
83
84
85 // ~~~~ action : data ~~~~
86 initial begin
87
88     // ~~ class instances ~~
89     tc          = new();
90
91     // ~~ init ~~
92     clk         = 1'b1;
93     rst         = 1'b0;
94     intrr       = 1'b0;
95     opcode      = 7'bxxx_xxxx;
96     func3       = 3'b000;
97     opc_arr     =
98     '{
99         default : 7'b000_0000,
100         0:       7'b000_0011,          // load
101         1:       7'b001_0011,          // imm ops
102         2:       7'b001_0111,          // auipc
103         3:       7'b010_0011,          // store
104         4:       7'b011_0011,          // dual reg ops
105         5:       7'b011_0111,          // lui
106         6:       7'b110_0011,          // branch
107         7:       7'b110_0111,          // jalr
108         8:       7'b110_1111,          // jal
109         9:       7'b111_0011          // interrupt
110     };
111
112 // ~~ TC1 : load(3) ~~
113 tc.new_test("load");
114 tnum = tc.get_testnum();
115
116 // ~ INIT ~
117 tc.print_subtest("INIT");
118 #5;
119 assert(pc_reset === 1'b1)   else tc.err("PC_RESET failed");
120 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
121 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
122 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
123 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
124 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
125 #5;
126
127 // ~ FETCH 1 ~
128 tc.print_subtest("FETCH 1");
129 #5;
130 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
131 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
132 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
133 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
134 assert(mem_rden1 === 1'b1)   else tc.err("MEM_RDEN1 failed");
135 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
136 #5;
137
138 // ~ EXEC 1 ~
139 tc.print_subtest("EXEC 1");
140 opcode = opc_arr[0];          // opcode
141 #5;
142 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
143 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
144 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
145 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
146 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
147 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
148 #5;
149
150 // ~ FETCH 2 ~
151 tc.print_subtest("FETCH 2");
152 #5;
153 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
154 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
155 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
156 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
157 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
158 assert(mem_rden2 === 1'b1)   else tc.err("MEM_RDEN2 failed");
159 #5;
160
161 // ~ EXEC 2 ~
162 tc.print_subtest("EXEC 2");

```

```

163         #5;
164         assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
165         assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
166         assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
167         assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
168         assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
169         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
170         #5;
171
172         // ~ WRITEBACK ~
173         tc.print_subtest("WRITEBACK");
174         #5;
175         assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
176         assert(pc_write  === 1'b1)    else tc.err("PC_WRITE failed");
177         assert(reg_write === 1'b1)    else tc.err("REG_WRITE failed");
178         assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
179         assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
180         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
181         #5;
182     tc.test_done();
183
184
185     // ~~ TC2 : imm ops (19) ~~
186     tc.new_test("immed");
187     tnum = tc.get_testnum();
188
189     // ~ FETCH ~
190     tc.print_subtest("FETCH");
191     #5;
192     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
193     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
194     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
195     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
196     assert(mem_rden1 === 1'b1)    else tc.err("MEM_RDEN1 failed");
197     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
198     #5;
199
200     // ~ EXEC ~
201     tc.print_subtest("EXEC");
202     opcode = opc_arr[1];
203     #5;
204     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
205     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
206     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
207     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
208     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
209     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
210     #5;
211
212     // ~ WRITEBACK ~
213     tc.print_subtest("WRITEBACK");
214     #5;
215     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
216     assert(pc_write  === 1'b1)    else tc.err("PC_WRITE failed");
217     assert(reg_write === 1'b1)    else tc.err("REG_WRITE failed");
218     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
219     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
220     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
221     #5;
222     tc.test_done();
223
224     // ~~ TC3 : auipc (23) ~~
225     tc.new_test("auipc");
226     tnum = tc.get_testnum();
227
228     // ~ FETCH ~
229     tc.print_subtest("FETCH");
230     #5;
231     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
232     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
233     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
234     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
235     assert(mem_rden1 === 1'b1)    else tc.err("MEM_RDEN1 failed");
236     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
237     #5;
238
239     // ~ EXEC ~
240     tc.print_subtest("EXEC");
241     opcode = opc_arr[2];
242     #5;
243     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
244     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
245     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
246     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
247     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
248     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
249     #5;
250
251     // ~ WRITEBACK ~
252     tc.print_subtest("WRITEBACK");
253     #5;

```

// opcode

// opcode

```

254         assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
255         assert(pc_write  === 1'b1)    else tc.err("PC_WRITE failed");
256         assert(reg_write === 1'b1)    else tc.err("REG_WRITE failed");
257         assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
258         assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
259         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
260         #5;
261     tc.test_done();
262
263     // ~~ TC4 : store (35) ~~
264     tc.new_test("store");
265     tnum = tc.get_testnum();
266
267     // ~ FETCH ~
268     tc.print_subtest("FETCH");
269     #5;
270     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
271     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
272     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
273     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
274     assert(mem_rden1 === 1'b1)    else tc.err("MEM_RDEN1 failed");
275     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
276     #5;
277
278     // ~ EXEC ~
279     tc.print_subtest("EXEC");
280     opcode = opc_arr[3];
281     #5;
282     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
283     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
284     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
285     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
286     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
287     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
288     #5;
289
290     // ~ WRITEBACK ~
291     tc.print_subtest("WRITEBACK");
292     #5;
293     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
294     assert(pc_write  === 1'b1)    else tc.err("PC_WRITE failed");
295     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
296     assert(mem_we2   === 1'b1)    else tc.err("MEM_WE2 failed");
297     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
298     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
299     #5;
300     tc.test_done();
301
302     // ~~ TC5 : dual reg ops (51) ~~
303     tc.new_test("dual reg ops");
304     tnum = tc.get_testnum();
305
306     // ~ FETCH ~
307     tc.print_subtest("FETCH");
308     #5;
309     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
310     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
311     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
312     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
313     assert(mem_rden1 === 1'b1)    else tc.err("MEM_RDEN1 failed");
314     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
315     #5;
316
317     // ~ EXEC ~
318     tc.print_subtest("EXEC");
319     opcode = opc_arr[4];
320     #5;
321     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
322     assert(pc_write  === 1'b0)    else tc.err("PC_WRITE failed");
323     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
324     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
325     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
326     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
327     #5;
328
329     // ~ WRITEBACK ~
330     tc.print_subtest("WRITEBACK");
331     #5;
332     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
333     assert(pc_write  === 1'b1)    else tc.err("PC_WRITE failed");
334     assert(reg_write === 1'b1)    else tc.err("REG_WRITE failed");
335     assert(mem_we2   === 1'b0)    else tc.err("MEM_WE2 failed");
336     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
337     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
338     #5;
339     tc.test_done();
340
341     // ~~ TC6 : lui (55) ~~
342     tc.new_test("lui");
343     tnum = tc.get_testnum();
344

```

```

345 // ~ FETCH ~
346 tc.print_subtest("FETCH");
347 #5;
348 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
349 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
350 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
351 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
352 assert(mem_rden1 === 1'b1)   else tc.err("MEM_RDEN1 failed");
353 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
354 #5;
355
356 // ~ EXEC ~
357 tc.print_subtest("EXEC");
358 opcode = opc_arr[5];
359 // opcode
360 #5;
361 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
362 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
363 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
364 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
365 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
366 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
367 #5;
368
369 // ~ WRITEBACK ~
370 tc.print_subtest("WRITEBACK");
371 #5;
372 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
373 assert(pc_write  === 1'b1)   else tc.err("PC_WRITE failed");
374 assert(reg_write === 1'b1)   else tc.err("REG_WRITE failed");
375 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
376 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
377 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
378 #5;
379 tc.test_done();
380
381 // ~~ TC7 : branch (99) ~~
382 tc.new_test("branch");
383 tnum = tc.get_testnum();
384
385 // ~ FETCH ~
386 tc.print_subtest("FETCH");
387 #5;
388 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
389 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
390 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
391 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
392 assert(mem_rden1 === 1'b1)   else tc.err("MEM_RDEN1 failed");
393 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
394 #5;
395
396 // ~ EXEC ~
397 tc.print_subtest("EXEC");
398 opcode = opc_arr[6];
399 // opcode
400 #5;
401 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
402 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
403 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
404 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
405 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
406 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
407 #5;
408
409 // ~ WRITEBACK ~
410 tc.print_subtest("WRITEBACK");
411 #5;
412 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
413 assert(pc_write  === 1'b1)   else tc.err("PC_WRITE failed");
414 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
415 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
416 assert(mem_rden1 === 1'b0)   else tc.err("MEM_RDEN1 failed");
417 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
418 #5;
419 tc.test_done();
420
421 // ~~ TC8 : jalr (103) ~~
422 tc.new_test("jalr");
423 tnum = tc.get_testnum();
424
425 // ~ FETCH ~
426 tc.print_subtest("FETCH");
427 #5;
428 assert(pc_reset === 1'b0)   else tc.err("PC_RESET failed");
429 assert(pc_write  === 1'b0)   else tc.err("PC_WRITE failed");
430 assert(reg_write === 1'b0)   else tc.err("REG_WRITE failed");
431 assert(mem_we2   === 1'b0)   else tc.err("MEM_WE2 failed");
432 assert(mem_rden1 === 1'b1)   else tc.err("MEM_RDEN1 failed");
433 assert(mem_rden2 === 1'b0)   else tc.err("MEM_RDEN2 failed");
434 #5;
435

```

```

436 // ~ EXEC ~
437 tc.print_subtest("EXEC");
438 opcode = opc_arr[7]; // opcode
439 #5;
440 assert(pc_reset === 1'b0) else tc.err("PC_RESET failed");
441 assert(pc_write === 1'b0) else tc.err("PC_WRITE failed");
442 assert(reg_write === 1'b0) else tc.err("REG_WRITE failed");
443 assert(mem_we2 === 1'b0) else tc.err("MEM_WE2 failed");
444 assert(mem_rden1 === 1'b0) else tc.err("MEM_RDEN1 failed");
445 assert(mem_rden2 === 1'b0) else tc.err("MEM_RDEN2 failed");
446 #5;
447
448 // ~ WRITEBACK ~
449 tc.print_subtest("WRITEBACK");
450 #5;
451 assert(pc_reset === 1'b0) else tc.err("PC_RESET failed");
452 assert(pc_write === 1'b1) else tc.err("PC_WRITE failed");
453 assert(reg_write === 1'b1) else tc.err("REG_WRITE failed");
454 assert(mem_we2 === 1'b0) else tc.err("MEM_WE2 failed");
455 assert(mem_rden1 === 1'b0) else tc.err("MEM_RDEN1 failed");
456 assert(mem_rden2 === 1'b0) else tc.err("MEM_RDEN2 failed");
457 #5;
458 tc.test_done();
459
460
461 // ~~ TC9 : jal (111) ~~
462 tc.new_test("jal");
463 tnum = tc.get_testnum();
464
465 // ~ FETCH ~
466 tc.print_subtest("FETCH");
467 #5;
468 assert(pc_reset === 1'b0) else tc.err("PC_RESET failed");
469 assert(pc_write === 1'b0) else tc.err("PC_WRITE failed");
470 assert(reg_write === 1'b0) else tc.err("REG_WRITE failed");
471 assert(mem_we2 === 1'b0) else tc.err("MEM_WE2 failed");
472 assert(mem_rden1 === 1'b1) else tc.err("MEM_RDEN1 failed");
473 assert(mem_rden2 === 1'b0) else tc.err("MEM_RDEN2 failed");
474 #5;
475
476 // ~ EXEC ~
477 tc.print_subtest("EXEC");
478 opcode = opc_arr[8]; // opcode
479 #5;
480 assert(pc_reset === 1'b0) else tc.err("PC_RESET failed");
481 assert(pc_write === 1'b0) else tc.err("PC_WRITE failed");
482 assert(reg_write === 1'b0) else tc.err("REG_WRITE failed");
483 assert(mem_we2 === 1'b0) else tc.err("MEM_WE2 failed");
484 assert(mem_rden1 === 1'b0) else tc.err("MEM_RDEN1 failed");
485 assert(mem_rden2 === 1'b0) else tc.err("MEM_RDEN2 failed");
486 #5;
487
488 // ~ WRITEBACK ~
489 tc.print_subtest("WRITEBACK");
490 #5;
491 assert(pc_reset === 1'b0) else tc.err("PC_RESET failed");
492 assert(pc_write === 1'b1) else tc.err("PC_WRITE failed");
493 assert(reg_write === 1'b1) else tc.err("REG_WRITE failed");
494 assert(mem_we2 === 1'b0) else tc.err("MEM_WE2 failed");
495 assert(mem_rden1 === 1'b0) else tc.err("MEM_RDEN1 failed");
496 assert(mem_rden2 === 1'b0) else tc.err("MEM_RDEN2 failed");
497 #5;
498 tc.test_done();
499
500 // ~~ TC10 : reset check ~~
501 tc.new_test("reset1");
502 tnum = tc.get_testnum();
503
504 // ~ FETCH ~
505 tc.print_subtest("FETCH");
506 rst = 1'b1;
507 #5;
508 assert(pc_reset === 1'b1) else tc.err("PC_RESET failed");
509 #5;
510 rst = 1'b0;
511 #5;
512 // we've reached fetch after reset
513 assert(pc_reset === 1'b0) else tc.err("PC_RESET failed");
514 assert(pc_write === 1'b0) else tc.err("PC_WRITE failed");
515 assert(reg_write === 1'b0) else tc.err("REG_WRITE failed");
516 assert(mem_we2 === 1'b0) else tc.err("MEM_WE2 failed");
517 assert(mem_rden1 === 1'b1) else tc.err("MEM_RDEN1 failed");
518 assert(mem_rden2 === 1'b0) else tc.err("MEM_RDEN2 failed");
519 #5;
520
521 // ~ EXEC ~
522 tc.print_subtest("EXEC");
523 rst = 1'b1;
524 #5;
525 assert(pc_reset === 1'b1) else tc.err("PC_RESET failed");
526 #5;

```

```

527     rst = 1'b0;
528     #15;
529     // we've reached exec after reset
530     assert(pc_reset === 1'b0)     else tc.err("PC_RESET failed");
531     assert(pc_write  === 1'b0)     else tc.err("PC_WRITE failed");
532     assert(reg_write === 1'b0)     else tc.err("REG_WRITE failed");
533     assert(mem_we2   === 1'b0)     else tc.err("MEM_WE2 failed");
534     assert(mem_rden1 === 1'b0)     else tc.err("MEM_RDEN1 failed");
535     assert(mem_rden2 === 1'b0)     else tc.err("MEM_RDEN2 failed");
536     #5;
537
538     // ~ WRITEBACK ~
539     tc.print_subtest("WRITEBACK");
540     rst = 1'b1;
541     #5;
542     assert(pc_reset === 1'b1)     else tc.err("PC_RESET failed");
543     #5;
544     rst = 1'b0;
545     #25;
546     assert(pc_reset === 1'b0)     else tc.err("PC_RESET failed");
547     assert(pc_write  === 1'b1)     else tc.err("PC_WRITE failed");
548     assert(reg_write === 1'b1)     else tc.err("REG_WRITE failed");
549     assert(mem_we2   === 1'b0)     else tc.err("MEM_WE2 failed");
550     assert(mem_rden1 === 1'b0)     else tc.err("MEM_RDEN1 failed");
551     assert(mem_rden2 === 1'b0)     else tc.err("MEM_RDEN2 failed");
552     #5;
553     tc.test_done();
554
555
556     // ~~ TC11 : awkward reset timing ~~
557     tc.new_test("reset2");
558     tnum = tc.get_testnum();
559
560     // 1 clk(pos) to register reset
561     #3;
562     rst = 1'b1;
563     #1;
564     rst = 1'b0;
565     #1;
566     assert(pc_reset === 1'b1)     else tc.err("PC_RESET failed");
567     #5;
568     tc.test_done();
569
570
571     // ~~ TC12 : mret opcode returns from interrupt ~~
572     tc.new_test("mret csr instr");
573     tnum = tc.get_testnum();
574
575     tc.print_subtest("FETCH");
576     opcode = opc_arr[9];           // intrr opcode
577     func3   = 3'b000;             // mret instr
578     #10;
579
580     tc.print_subtest("EXEC");
581     #8;
582     assert(pc_reset === 1'b0)     else tc.err("PC_RESET failed");
583     assert(pc_write  === 1'b0)     else tc.err("PC_WRITE failed");
584     assert(reg_write === 1'b0)     else tc.err("REG_WRITE failed");
585     assert(mem_we2   === 1'b0)     else tc.err("MEM_WE2 failed");
586     assert(mem_rden1 === 1'b0)     else tc.err("MEM_RDEN1 failed");
587     assert(mem_rden2 === 1'b0)     else tc.err("MEM_RDEN2 failed");
588     assert(csr_we    === 1'b0)     else tc.err("CSR_WE failed");
589     assert(int_taken === 1'b0)     else tc.err("INT_TAKEN failed");
590     assert(mret_exec === 1'b1)     else tc.err("MRET_EXEC failed");
591     #2;
592
593     tc.print_subtest("WB");
594     #8;
595     assert(pc_reset === 1'b0)     else tc.err("PC_RESET failed");
596     assert(pc_write  === 1'b1)     else tc.err("PC_WRITE failed");
597     assert(reg_write === 1'b0)     else tc.err("REG_WRITE failed");
598     assert(mem_we2   === 1'b0)     else tc.err("MEM_WE2 failed");
599     assert(mem_rden1 === 1'b0)     else tc.err("MEM_RDEN1 failed");
600     assert(mem_rden2 === 1'b0)     else tc.err("MEM_RDEN2 failed");
601     assert(csr_we    === 1'b0)     else tc.err("CSR_WE failed");
602     assert(int_taken === 1'b0)     else tc.err("INT_TAKEN failed");
603     assert(mret_exec === 1'b0)     else tc.err("MRET_EXEC failed");
604     #2;
605     tc.test_done();
606
607     // ~~ TC13 : csr opcode but not mret ~~
608     tc.new_test("non-mret csr instr");
609     tnum = tc.get_testnum();
610
611     tc.print_subtest("FETCH");
612     opcode = opc_arr[9];           // intrr opcode
613     func3   = 3'b001;             // NOT mret instr
614     #10;
615
616     tc.print_subtest("EXEC");
617     #8;

```

```

618         assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
619         assert(pc_write === 1'b0)    else tc.err("PC_WRITE failed");
620         assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
621         assert(mem_we2 === 1'b0)      else tc.err("MEM_WE2 failed");
622         assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
623         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
624         assert(csr_we === 1'b0)       else tc.err("CSR_WE failed");
625         assert(int_taken === 1'b0)    else tc.err("INT_TAKEN failed");
626         assert(mret_exec === 1'b0)    else tc.err("MRET_EXEC failed");
627         #2;
628
629         tc.print_subtest("WB");
630         #8;
631         assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
632         assert(pc_write === 1'b1)    else tc.err("PC_WRITE failed");
633         assert(reg_write === 1'b1)    else tc.err("REG_WRITE failed");
634         assert(mem_we2 === 1'b0)      else tc.err("MEM_WE2 failed");
635         assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
636         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
637         assert(csr_we === 1'b1)       else tc.err("CSR_WE failed");
638         assert(int_taken === 1'b0)    else tc.err("INT_TAKEN failed");
639         assert(mret_exec === 1'b0)    else tc.err("MRET_EXEC failed");
640         #2;
641     tc.test_done();
642
643     // ~~ TC14 : INTR on WB ~~
644     tc.new_test("interrupt on WB");
645     tnum = tc.get_testnum();
646
647     #5;
648     tc.print_subtest("FETCH1");
649     opcode = opc_arr[1];                // opcode not 'csr' or 'load' = no extra state cycles
650     intrr = 1'b1;
651     #5;
652
653     tc.print_subtest("EXEC");           // interrupt shouldn't trigger here
654     #10;
655
656     tc.print_subtest("WB");             // interrupt should trigger after WB
657     #10;
658
659     tc.print_subtest("INTRPT");         // interrupt taken
660     #2;
661     intrr = 1'b0;                      // CSR brings down intrr manually
662     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
663     assert(pc_write === 1'b1)    else tc.err("PC_WRITE failed");    // update PC with ISR
664     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
665     assert(mem_we2 === 1'b0)      else tc.err("MEM_WE2 failed");
666     assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
667     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
668     assert(csr_we === 1'b0)       else tc.err("CSR_WE failed");
669     assert(int_taken === 1'b1)    else tc.err("INT_TAKEN failed");    // interrupt flag is up
670     assert(mret_exec === 1'b0)    else tc.err("MRET_EXEC failed");
671     #8;
672
673     tc.print_subtest("FETCH2");
674     #2;
675     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
676     assert(pc_write === 1'b0)    else tc.err("PC_WRITE failed");
677     assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
678     assert(mem_we2 === 1'b0)      else tc.err("MEM_WE2 failed");
679     assert(mem_rden1 === 1'b1)    else tc.err("MEM_RDEN1 failed");    // typical fetch
680     assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
681     assert(csr_we === 1'b0)       else tc.err("CSR_WE failed");
682     assert(int_taken === 1'b0)    else tc.err("INT_TAKEN failed");    // interrupt flag is taken down
683     assert(mret_exec === 1'b0)    else tc.err("MRET_EXEC failed");
684     #8;
685
686     #20; // bring past WB
687     tc.test_done();
688
689     // ~~ TC14 : INTR on WB ~~
690     tc.new_test("interrupt on EXEC (load instr)");
691     tnum = tc.get_testnum();
692
693     #5;
694     tc.print_subtest("FETCH1");
695     opcode = opc_arr[0];                // opcode load = extra state cycles
696     intrr = 1'b1;
697     #5;
698
699
700     tc.print_subtest("EXEC1");          // interrupt should trigger here to preserve opcode
701     #10;
702
703     // delay FETCH2, EXEC2, WB ... straight to INTRPT first
704
705     tc.print_subtest("INTRPT");         // interrupt taken
706     #2;
707     intrr = 1'b0;                      // CSR brings down intrr manually
708     assert(pc_reset === 1'b0)    else tc.err("PC_RESET failed");
709     assert(pc_write === 1'b1)    else tc.err("PC_WRITE failed");    // update PC with ISR

```

```

709         assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
710         assert(mem_we2 === 1'b0)      else tc.err("MEM_WE2 failed");
711         assert(mem_rden1 === 1'b0)    else tc.err("MEM_RDEN1 failed");
712         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
713         assert(csr_we === 1'b0)       else tc.err("CSR_WE failed");
714         assert(int_taken === 1'b1)    else tc.err("INT_TAKEN failed");    // interrupt flag is up
715         assert(mret_exec === 1'b0)    else tc.err("MRET_EXEC failed");
716         #8;
717
718         tc.print_subtest("ISR FETCH");
719         #2;
720         assert(pc_reset === 1'b0)     else tc.err("PC_RESET failed");
721         assert(pc_write === 1'b0)     else tc.err("PC_WRITE failed");
722         assert(reg_write === 1'b0)    else tc.err("REG_WRITE failed");
723         assert(mem_we2 === 1'b0)      else tc.err("MEM_WE2 failed");
724         assert(mem_rden1 === 1'b1)    else tc.err("MEM_RDEN1 failed");    // typical fetch
725         assert(mem_rden2 === 1'b0)    else tc.err("MEM_RDEN2 failed");
726         assert(csr_we === 1'b0)       else tc.err("CSR_WE failed");
727         assert(int_taken === 1'b0)    else tc.err("INT_TAKEN failed");    // interrupt flag is taken down
728         assert(mret_exec === 1'b0)    else tc.err("MRET_EXEC failed");
729         #8;
730
731         #20; // bring past WB
732         tc.test_done();
733     end
734
735
736 endmodule

```

0.3.11 MicroController

Listing 39: Otter MCU Otter_MCU.sv

```
1  'timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/19/2026 08:57:52 AM
7  // Design Name:
8  // Module Name: Otter_MCU
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module Otter_MCU(
24     input logic [31:0] IOBUS_IN,
25     input logic RST,
26     input logic INTRR,
27     input logic CLK,
28
29     output logic [31:0] IOBUS_OUT,
30     output logic [31:0] IOBUS_ADDR,
31     output logic IOBUS_WR
32 );
33
34
35 // ~~~~ inits / wires ~~~~
36
37 // ~~ pc + mux ~~
38 logic reset;
39 logic pc_write;
40 logic [2:0] pc_source;
41 logic [31:0] pc;
42 logic [31:0] pc_p4;
43 logic [31:0] pcmux_out;
44
45 // ~~ memory ~~
46 logic mem_rden1;
47 logic mem_rden2;
48 logic mem_we2;
49 logic [31:0] ir, dout2;
50
51 // ~~ reg file + mux ~~
52 logic reg_write;
53 logic [31:0] reg_w_data;
54 logic [31:0] rs1, rs2;
55 logic [1:0] rf_wr_sel;
56
57 // ~~ immed gen ~~
58 logic [31:0] ut, it, st, jt, bt;
59
60 // ~~ branch addr gen ~~
61 logic [31:0] jalr, branch, jal;
62
63 // ~~ alu + mux ~~
64 logic [1:0] alu_src_a;
65 logic [2:0] alu_src_b;
66 logic [3:0] alu_func;
67 logic [31:0] src_a, src_b, result;
68
69 // ~~ csr ~~
70 logic mret_exec;
71 logic int_taken;
72 logic csr_we;
73 logic csr_mstatus_mie;
74 logic [31:0] csr_mepc;
75 logic [31:0] csr_mtvec;
76 logic [31:0] csr_rd;
77
78 // ~~ bcond + control decode and fsm ~~
79 logic br_eq, br_lt, br_ltu;
80
81
82 // ~~~~ modules (l1-2-r on diagram) ~~~~
83 PC_Reg PC(
84     .CLK(CLK), // 1'b I
85     .PC_RST(reset), // 1'b I
86     .PC_WE(pc_write), // 1'b I
87     .PC_DIN(pcmux_out), // 32'b I
```

```

88     .PC_COUNT(pc)           // 32'b 0
89 );
90
91 Mux_6N #(32) PC_MUX(
92     .D0(pc_p4),             // 32'b I
93     .D1(jalr),              // 32'b I
94     .D2(branch),           // 32'b I
95     .D3(jal),               // 32'b I
96     .D4(csr_mtvec),         // 32'b I
97     .D5(csr_mepc),          // 32'b I
98     .S(pc_source),          // 2'b I
99     .Y(pcmux_out)           // 32'b 0
100 );
101
102 Plus4_1N #(32) P4(
103     .IN(pc),                 // 32'b I
104     .OUT(pc_p4)              // 32'b 0
105 );
106
107 Memory MEMORY
108 (
109     .MEM_CLK(CLK),           // 1'b I
110     .MEM_RDEN1(mem_rden1),   // 1'b I
111     .MEM_RDEN2(mem_rden2),   // 1'b I
112     .MEM_WE2(mem_we2),       // 1'b I
113     .MEM_ADDR1(pc[15:2]),     // 14'b I
114     .MEM_ADDR2(result),      // 32'b I
115     .MEM_DIN2(rs2),          // 32'b I
116     .MEM_SIZE(ir[13:12]),     // 2'b I
117     .MEM_SIGN(ir[14]),        // 1'b I
118     .IO_IN(IOBUS_IN),        // 32'b I
119
120     .IO_WR(IOBUS_WR),        // 1'b 0
121     .MEM_DOUT1(ir),           // 32'b 0
122     .MEM_DOUT2(dout2)        // 32'b 0
123 );
124
125 Mux_4N #(32) REG_MUX(
126     .D0(pc_p4),             // 32'b I
127     .D1(csr_rd),            // 32'b I
128     .D2(dout2),             // 32'b I
129     .D3(result),            // 32'b I
130     .S(rf_wr_sel),          // 2'b I
131     .Y(reg_w_data)          // 32'b 0
132 );
133
134 RegFile REG_FILE(
135     .EN(reg_write),          // 1'b I
136     .CLK(CLK),               // 1'b I
137     .ADR1(ir[19:15]),        // 5'b I
138     .ADR2(ir[24:20]),        // 5'b I
139     .W_ADR(ir[11:7]),         // 5'b I
140     .W_DATA(reg_w_data),     // 32'b I
141     .RS1(rs1),               // 32'b 0
142     .RS2(rs2)                // 32'b 0
143 );
144 assign IOBUS_OUT = rs2;
145
146 ImmedGen IMMED_GEN(
147     .INSTR(ir),              // 32'b I but only 25 are used
148     .U_TYPE(ut),             // 32'b 0
149     .I_TYPE(it),             // 32'b 0
150     .S_TYPE(st),             // 32'b 0
151     .J_TYPE(jt),             // 32'b 0
152     .B_TYPE(bt)              // 32'b 0
153 );
154
155 BranchAddrGen BRANCH_ADDR_GEN(
156     .RS1(rs1),               // 32'b I
157     .I_TYPE(it),             // 32'b I
158     .J_TYPE(jt),             // 32'b I
159     .B_TYPE(bt),             // 32'b I
160     .PC(pc),                 // 32'b I
161     .JAL(jal),               // 32'b 0
162     .JALR(jalr),             // 32'b 0
163     .BRANCH(branch)          // 32'b 0
164 );
165
166 Mux_3N #(32) ALU_SRCA_MUX(
167     .D0(rs1),                // 32'b I
168     .D1(ut),                 // 32'b I
169     .D2(~rs1),               // 32'b I
170     .S(alu_src_a),           // 2'b I
171     .Y(src_a)                // 32'b 0
172 );
173
174 Mux_5N #(32) ALU_SRCB_MUX(
175     .D0(rs2),                // 32'b I
176     .D1(it),                 // 32'b I
177     .D2(st),                 // 32'b I
178     .D3(pc),                 // 32'b I

```

```

179     .D4(csr_rd),           // 32'b I
180     .S(alu_src_b),        // 2'b I
181     .Y(src_b)             // 32'b 0
182 );
183
184 ALU ALU(
185     .ALU_FUN(alu_func),    // 4'b I
186     .SRC_A(src_a),         // 32'b I
187     .SRC_B(src_b),         // 32'b I
188     .ALU_RESULT(result)    // 32'b 0
189 );
190 assign IOBUS_ADDR = result;
191
192 BranchConditionGen BRANCH_COND_GEN(
193     .RS1(rs1),             // 32'b I
194     .RS2(rs2),             // 32'b I
195     .BR_EQ(br_eq),         // 1'b 0
196     .BR_LT(br_lt),         // 1'b 0
197     .BR_LTU(br_ltu)        // 1'b 0
198 );
199
200 CUnit_Decoder CU_DCDR(
201     .OPCODE(ir[6:0]),       // 7'b I
202     .FUNC3(ir[14:12]),      // 3'b I
203     .FUNC7(ir[30]),         // 1'b I
204     .BR_EQ(br_eq),          // 1'b I
205     .BR_LT(br_lt),          // 1'b I
206     .BR_LTU(br_ltu),        // 1'b I
207     .INT_TAKEN(int_taken),  // 1'b I
208     .ALU_FUNC(alu_func),    // 4'b 0
209     .ALU_SRC_A(alu_src_a),  // 1'b 0
210     .ALU_SRC_B(alu_src_b),  // 2'b 0
211     .PC_SOURCE(pc_source),  // 2'b 0
212     .RF_WR_SEL(rf_wr_sel)   // 2'b 0
213 );
214
215 CUnit_FSM CU_FSM(
216     .CLK(CLK),              // 1'b I
217     .RST(RST),              // 1'b I
218     .INTRR(INTRR & csr_mstatus_mie), // 1'b I
219     .OPCODE(ir[6:0]),       // 7'b I
220     .FUNC3(ir[14:12]),      // 3'b I
221     .PC_WRITE(pc_write),    // 1'b 0
222     .REG_WRITE(reg_write),  // 1'b 0
223     .MEM_RDEN1(mem_rden1),  // 1'b 0
224     .MEM_RDEN2(mem_rden2),  // 1'b 0
225     .MEM_WE2(mem_we2),      // 1'b 0
226     .RESET(reset),          // 1'b 0
227     .CSR_WE(csr_we),        // 1'b 0
228     .INT_TAKEN(int_taken),  // 1'b 0
229     .MRET_EXEC(mret_exec)   // 1'b 0
230 );
231
232 ControlStatusRegister CSR(
233     .CLK(CLK),              // 1'b I
234     .RST(reset),            // 1'b I
235     .INTR_TAKEN(int_taken), // 1'b I
236     .INTR_RET(mret_exec),   // 1'b I
237     .WR_EN(csr_we),         // 1'b I
238     .ADDR(ir[31:20]),        // 12'b I
239     .PC(pc),                 // 32'b I
240     .WD(result),             // 32'b I
241     .MSTATUS_MIE(csr_mstatus_mie), // 1'b 0
242     .MTVEC(csr_mtvec),       // 32'b 0
243     .MEPC(csr_mepc),         // 32'b 0
244     .RD(csr_rd)              // 32'b 0
245 );
246
247 endmodule

```

Listing 40: TestSim for MCU, CUnit_FSM_tb.sv, using RARS from demo.asm

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/19/2026 11:23:25 AM
7  // Design Name:
8  // Module Name: Otter_MCU_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //

```

```

20 // -----> Ensure mem file is using sample code from hw6 "otter_hw6sample_test.mem"
21 //
22 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
23
24
25 module Otter_MCU_tb();
26
27     // ~~~~ local vars ~~~~
28
29     // ~~ inputs ~~
30     logic          clk;
31     logic          rst;
32     logic          intrr;
33     logic [31:0]   din;
34
35     // ~~ outputs ~~
36     logic          write;
37     logic [31:0]   rs2;
38     logic [31:0]   alu_result;
39
40     // ~~ testing ~~
41     int testcase;
42     typedef enum {LUI, ADDI, SLLI, SLT, XOR_, BEQ} e_instr;
43     e_instr curr_instr;
44
45     // ~~~~ instances ~~~~
46     Otter_MCU UUT (
47         .IOBUS_IN(din),           // 32'b I
48         .RST(rst),               // 1'b I
49         .INTRR(intrr),           // 1'b I
50         .CLK(clk),               // 1'b I
51         .IOBUS_OUT(rs2),         // 32'b O
52         .IOBUS_ADDR(alu_result), // 32'b O
53         .IOBUS_WR(write)         // 1'b O // 0 for non-MMIO
54     );
55
56     // ~~ action : CLK (10ns period) ~~
57     always
58     begin
59         #5;
60         clk <= !clk;
61     end
62
63     initial begin
64
65         // ~~ init ~~
66         testcase = 0;
67         clk = 1'b1;
68         rst = 1'b0;
69         intrr = 1'b0;
70         din = 32'h0000_0000;
71
72         // ~~ TC1 : 4x walkthrough ~~
73         testcase++;
74         #10; // INIT complete
75         testall();
76
77         // ~~ TC2 : reset aand repeat TC1 ~~
78         testcase++;
79         rst = 1'b1;
80         #30;
81         rst = 1'b0; // INIT already complete here
82         testall();
83     end
84
85     task testall();
86         for(int i = 0; i < 5; i++) begin
87             $display("%d", i);
88
89             // ~~ instr 1 : lui ~~
90             $display("lui"); curr_instr = LUI;
91             #10; // FETCH complete
92             #10; // EXEC complete
93             assert(alu_result == 32'hAA05_5000) else $error("ALU_RESULT failed");
94             #10; // WRITEBACK complete
95
96             // ~~ instr 2 : addi ~~
97             $display("addi"); curr_instr = ADDI;
98             #10; // FETCH complete
99             #10; // EXEC complete
100            assert(alu_result == 32'hAA05_5765) else $error("ALU_RESULT failed");
101            assert(rs2 == 32'hAA05_5000) else $error("RS2 failed");
102            #10; // WRITEBACK complete
103
104            // ~~ instr 3 : slli ~~
105            $display("slli"); curr_instr = SLLI;
106            #10; // FETCH complete
107            #10; // EXEC complete
108            assert(alu_result == 32'h502A_BB28) else $error("ALU_RESULT failed");
109            #10; // WRITEBACK complete
110

```

```

111 // ~~ instr 4 : slt ~~
112 $display("slt"); curr_instr = SLT;
113 #10; // FETCH complete
114 #10; // EXEC complete
115 assert(alu_result == 32'h0000_0001) else $error("ALU_RESULT failed");
116 assert(rs2 == 32'hAA05_5765) else $error("RS2 failed");
117 #10; // WRITEBACK complete
118
119 // ~~ instr 5 : xor ~~
120 $display("xor"); curr_instr = XOR_;
121 #10; // FETCH complete
122 #10; // EXEC complete
123 assert(alu_result == 32'hFA2F_EC4D) else $error("ALU_RESULT failed");
124 assert(rs2 == 32'h502A_BB28) else $error("RS2 failed");
125 #10; // WRITEBACK complete
126
127 // ~~ instr 6 : beq ~~
128 $display("beq"); curr_instr = BEQ;
129 #10; // FETCH complete
130 #10; // EXEC complete
131 assert(rs2 == 32'h0000_0000) else $error("RS2 failed");
132 #10; // WRITEBACK complete
133 end
134 endtask
135
136
137 endmodule

```

0.3.12 Seven Segment

Listing 41: Binary-Coded DecimalBCD.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company: Cal Poly
4  // Engineer: Paul Hummel
5  //
6  // Module Name: BCD
7  // Target Devices: Used for displaying decimal on a 4 digit 7 segment display
8  // Description: Conversts a 16-bit value into 4 digit binary coded decimal
9  //
10 //             The full 16-bit range intput cannot be converted because the
11 //             output is limited to 4 digiets. Anything above 4 digiets gets
12 //             truncated so 12345 is output as 2345
13 // Revision:
14 // Revision 0.01 - File Created
15 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
16
17
18 module BCD(
19     input  [15:0] HEX,
20     output logic [3:0] THOUSANDS,
21     output logic [3:0] HUNDREDS,
22     output logic [3:0] TENS,
23     output logic [3:0] ONES
24 );
25
26     int i;
27
28     /*
29     logic block to convert binary to 4 digit BCD (0000 - 1001)
30     start with MSB and shift each bit left to fit into each place value.
31     THOUSANDS, HUNDREDS, TENS, and ONES.
32
33     Each shift left is the equivalent of x 2, so if current value is 5 or
34     greater, the next shift should increment the next place value.
35     By adding 3 to the current place value will set this to occur on the
36     next shift. See 2 examples below
37
38     5 will shift a 1 to next place value and put 0 in the current place value
39     to create 10 (5 x 2)
40     0101 -> 1000 -> shift -> xxx1 000x
41
42     7 will shift a 1 to the next place value and put 4 in current place value
43     to create 14 (7 x 2)
44     0111 -> 1010 -> shift -> xxx1 010x
45     */
46
47     always_comb begin
48         THOUSANDS = 4'h0; // default all digits to 0
49         HUNDREDS  = 4'h0;
50         TENS      = 4'h0;
51         ONES      = 4'h0;
52
53         // iterate through each bit, starting with MSB (bit 15)
54         for (i=15; i>=0; i=i-1) begin
55
56             // check for place values of 5 or greater
57             if (THOUSANDS >= 5)
58                 THOUSANDS = THOUSANDS + 3;
59             if (HUNDREDS >= 5)
60                 HUNDREDS = HUNDREDS + 3;
61             if (TENS >= 5)
62                 TENS = TENS + 3;
63             if (ONES >= 5)
64                 ONES = ONES + 3;
65
66             // shift bits to the left
67             THOUSANDS = {THOUSANDS[2:0],HUNDREDS[3]};
68             HUNDREDS  = {HUNDREDS[2:0],TENS[3]};
69             TENS      = {TENS[2:0],ONES[3]};
70             ONES      = {ONES[2:0],HEX[i]};
71         end
72     end
73
74 endmodule
```

Listing 42: CathodeDriver.sv

```
1  `timescale 1ns / 1ps
2  //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3  // Company: Cal Poly
4  // Engineer: Paul Hummel
5  //
6  // Create Date: 06/28/2018 11:50:35 PM
7  // Module Name: CathodeDriver
8  // Target Devices: Basys3, 4 digit 7 segment display
9  // Description: Display hex values on a 4 digit 7 segment display with common
```

```

10 //          anode and common cathode, both configured as negative logic so
11 //          0s in the CATHODE light up the segments for any digit
12 //          corresponding with a 0 in the ANODE
13 //
14 //          CATHODES = {dp,a,b,c,d,e,f,g}
15 //          ANODES = {d4, d3, d2, d1}
16 //
17 // Revision:
18 // Revision 0.01 - File Created
19 ///////////////////////////////////////////////////////////////////
20
21
22 module CathodeDriver(
23     input CLK,
24     input [15:0] HEX,
25     output logic [7:0] CATHODES,
26     output logic [3:0] ANODES
27 );
28
29     logic s_clk_500 = 1'b0;          // 250Hz refresh clock
30     logic [1:0] r_disp_digit = 2'b00; // current digit being displayed
31     logic [19:0] clk_div_counter = 20'h00000;
32
33     // Clock Divider to create 500 Hz refresh from 100 MHz clock
34     always_ff @(posedge CLK) begin
35         clk_div_counter = clk_div_counter + 1;
36
37         // x186A0 = 1*10^5 = 1 ms toggle (x30D40)
38         if ( clk_div_counter == 20'h186A0) begin
39             clk_div_counter = 20'h00000;
40             s_clk_500 = ~s_clk_500; // toggle every 1 ms creates 500 Hz clock
41         end
42     end
43
44     // Refresh Seven Segment Display every 240 Hz
45     always_ff @(posedge s_clk_500) begin
46         case (r_disp_digit)
47             2'b00: begin
48                 ANODES= 4'b1110;
49                 case (HEX[3:0])
50                     4'b0000: CATHODES = 8'b10000001; //0
51                     4'b0001: CATHODES = 8'b11001111; //1
52                     4'b0010: CATHODES = 8'b10010010; //2
53                     4'b0011: CATHODES = 8'b10000110; //3
54                     4'b0100: CATHODES = 8'b11001100; //4
55                     4'b0101: CATHODES = 8'b10100100; //5
56                     4'b0110: CATHODES = 8'b10100000; //6
57                     4'b0111: CATHODES = 8'b10001111; //7
58                     4'b1000: CATHODES = 8'b10000000; //8
59                     4'b1001: CATHODES = 8'b10001100; //9
60                     4'b1010: CATHODES = 8'b10001000; //a
61                     4'b1011: CATHODES = 8'b11100000; //b
62                     4'b1100: CATHODES = 8'b10110001; //c
63                     4'b1101: CATHODES = 8'b11000010; //d
64                     4'b1110: CATHODES = 8'b10110000; //e
65                     4'b1111: CATHODES = 8'b10111000; //f
66                     default: CATHODES = 8'b11111111; // failsafe turn off
67                 endcase
68             end
69             2'b01: begin
70                 ANODES= 4'b1101;
71                 case (HEX[7:4])
72                     4'b0000: CATHODES = 8'b10000001;
73                     4'b0001: CATHODES = 8'b11001111;
74                     4'b0010: CATHODES = 8'b10010010;
75                     4'b0011: CATHODES = 8'b10000110;
76                     4'b0100: CATHODES = 8'b11001100;
77                     4'b0101: CATHODES = 8'b10100100;
78                     4'b0110: CATHODES = 8'b10100000;
79                     4'b0111: CATHODES = 8'b10001111;
80                     4'b1000: CATHODES = 8'b10000000;
81                     4'b1001: CATHODES = 8'b10001100;
82                     4'b1010: CATHODES = 8'b10001000; //a
83                     4'b1011: CATHODES = 8'b11100000;
84                     4'b1100: CATHODES = 8'b10110001;
85                     4'b1101: CATHODES = 8'b11000010;
86                     4'b1110: CATHODES = 8'b10110000;
87                     4'b1111: CATHODES = 8'b10111000;
88                     default: CATHODES = 8'b11111111; // all off on error
89                 endcase
90             end
91             2'b10: begin
92                 ANODES= 4'b1011;
93                 case (HEX[11:8])
94                     4'b0000: CATHODES = 8'b10000001;
95                     4'b0001: CATHODES = 8'b11001111;
96                     4'b0010: CATHODES = 8'b10010010;
97                     4'b0011: CATHODES = 8'b10000110;
98                     4'b0100: CATHODES = 8'b11001100;
99                     4'b0101: CATHODES = 8'b10100100;
100                    4'b0110: CATHODES = 8'b10100000;

```

```

101         4'b0111: CATHODES = 8'b10001111;
102         4'b1000: CATHODES = 8'b10000000;
103         4'b1001: CATHODES = 8'b10001100;
104         4'b1010: CATHODES = 8'b10001000; //a
105         4'b1011: CATHODES = 8'b11100000;
106         4'b1100: CATHODES = 8'b10110001;
107         4'b1101: CATHODES = 8'b11000010;
108         4'b1110: CATHODES = 8'b10110000;
109         4'b1111: CATHODES = 8'b10111000;
110         default: CATHODES = 8'b11111111; // all off on error
111     endcase
112 end
113 2'b11: begin
114     ANODES= 4'b0111;
115     case (HEX[15:12])
116         4'b0000: CATHODES = 8'b10000001;
117         4'b0001: CATHODES = 8'b11001111;
118         4'b0010: CATHODES = 8'b10010010;
119         4'b0011: CATHODES = 8'b10000110;
120         4'b0100: CATHODES = 8'b11001100;
121         4'b0101: CATHODES = 8'b10100100;
122         4'b0110: CATHODES = 8'b10100000;
123         4'b0111: CATHODES = 8'b10001111;
124         4'b1000: CATHODES = 8'b10000000;
125         4'b1001: CATHODES = 8'b10001100;
126         4'b1010: CATHODES = 8'b10001000; //a
127         4'b1011: CATHODES = 8'b11100000;
128         4'b1100: CATHODES = 8'b10110001;
129         4'b1101: CATHODES = 8'b11000010;
130         4'b1110: CATHODES = 8'b10110000;
131         4'b1111: CATHODES = 8'b10111000;
132         default: CATHODES = 8'b11111111; // all off on error
133     endcase
134 end
135 default: begin // digit error turn everything off
136     ANODES = 4'hF;
137     CATHODES = 8'hFF;
138     r_disp_digit = 2'b00;
139 end
140 endcase
141
142     r_disp_digit = r_disp_digit + 1;
143 end
144
145
146
147 endmodule

```

Listing 43: Seven Segment Display Driver SevSegDisp.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company: Cal Poly
4  // Engineer: Paul Hummel
5  //
6  // Create Date: 06/29/2018 12:58:25 AM
7  // Module Name: SevSegDisp
8  // Target Devices: Basys3 - 4 digit 7 segment, common anode, common cathode
9  // Description: Display driver for a 4 digit 7 segment with common anode and
10 //               common cathode. The 16-bit input can be displayed as decimal or
11 //               hex value using the mode input, 0 for hex, 1 for decimal.
12 //               In decimal mode, the full 16-bit range of values cannot be
13 //               displayed due to the output being limited to 4 digits. Decimal
14 //               values over 4 digits are truncated, 12345 is displayed as 2345
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 ///////////////////////////////////////////////////////////////////
19
20
21 module SevSegDisp(
22     input CLK, // 100 MHz
23     input MODE, // 0 - Hex, 1 - Decimal
24     input [15:0] DATA_IN,
25     output [7:0] CATHODES,
26     output [3:0] ANODES
27 );
28
29     logic [15:0] BCD_Val;
30     logic [15:0] Hex_Val;
31
32     // decimal values are converted to binary code decimal for displaying
33     BCD BCDMod (.HEX(DATA_IN), .THOUSANDS(BCD_Val[15:12]),
34               .HUNDREDS(BCD_Val[11:8]), .TENS(BCD_Val[7:4]),
35               .ONES(BCD_Val[3:0]));
36
37     // Cathode driver displays all values as 4-bit hex
38     CathodeDriver CathMod (.HEX(Hex_Val), .CLK(CLK), .CATHODES(CATHODES),
39                           .ANODES(ANODES));
40
41     // MUX to switch between HEX and BCD input for Hex and Decimal display

```

```
42     always_comb begin
43         if (MODE == 1'b1)
44             Hex_Val = BCD_Val;
45         else
46             Hex_Val = DATA_IN;
47     end
48
49
50 endmodule
```

0.3.13 Peripheral Fabric

Listing 44: Otter Wrapper OTTER_Wrapper_v1_02.sv

```
1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: J. Calllenes
5  //          P. Hummel
6  //
7  // Create Date: 01/20/2019 10:36:50 AM
8  // Module Name: OTTER_Wrapper
9  // Target Devices: OTTER MCU on Basys3
10 // Description: OTTER_WRAPPER with Switches, LEDs, and 7-segment display
11 //
12 // Revision:
13 // Revision 0.01 - File Created
14 // Revision 0.02 - Updated MMIO Addresses, signal names
15 ///////////////////////////////////////////////////////////////////
16
17 module OTTER_Wrapper(
18     input CLK,
19     input BTNL,
20     input BTNC,
21     input [15:0] SWITCHES,
22     output logic [15:0] LEDS,
23     output [7:0] CATHODES,
24     output [3:0] ANODES
25 );
26
27     // INPUT PORT IDS ///////////////////////////////////////////////////////////////////
28     // Right now, the only possible inputs are the switches
29     // In future labs you can add more MMIO, and you'll have
30     // to add constants here for the mux below
31     localparam SWITCHES_AD = 32'h1100_0000;
32
33     // OUTPUT PORT IDS ///////////////////////////////////////////////////////////////////
34     // In future labs you can add more MMIO
35     localparam LEDS_AD = 32'h1100_0020; //32'h11000020
36     localparam SSEG_AD = 32'h1100_0040; //32'h11000040
37
38     // Signals for connecting OTTER_MCU to OTTER_wrapper ///////////////////////////////////////////////////////////////////
39     logic clk_50 = 0;
40
41     logic [31:0] IOBUS_out, IOBUS_in, IOBUS_addr;
42     logic s_reset, IOBUS_wr;
43     logic s_intrr, s_intrr_oneshot, s_reset_oneshot;
44
45     // Registers for buffering outputs ///////////////////////////////////////////////////////////////////
46     logic [15:0] r_SSEG;
47
48     // OneShot Debounce (80ns pulse)
49     OneShot #(80) ONESHOT_INTRR(
50         .IN(s_intrr),
51         .CLK(clk_50),
52         .OUT(s_intrr_oneshot)
53     );
54
55     OneShot #(80) ONESHOT_RESET(
56         .IN(s_reset),
57         .CLK(clk_50),
58         .OUT(s_reset_oneshot)
59     );
60
61     // Declare OTTER_CPU ///////////////////////////////////////////////////////////////////
62     Otter_MCU OTTER_MCU(
63         .RST(s_reset_oneshot),
64         .CLK(clk_50),
65         .INTRR(s_intrr_oneshot),
66         .IOBUS_OUT(IOBUS_out),
67         .IOBUS_IN(IOBUS_in),
68         .IOBUS_ADDR(IOBUS_addr),
69         .IOBUS_WR(IOBUS_wr)
70     );
71
72     // Declare Seven Segment Display ///////////////////////////////////////////////////////////////////
73     SevSegDisp SSG_DISP (
74         .DATA_IN(r_SSEG),
75         .CLK(clk_50),
76         .MODE(1'b0),
77         .CATHODES(CATHODES),
78         .ANODES(ANODES)
79     );
80
81
82     // Clock Divider to create 50 MHz Clock ///////////////////////////////////////////////////////////////////
83     always_ff @(posedge CLK) begin
84         clk_50 <= ~clk_50;
85     end
86
87     // Connect Signals ///////////////////////////////////////////////////////////////////
```

```

88     assign s_reset = BTNC;
89     assign s_intrr = BTNL;
90
91     // Connect Board input peripherals (Memory Mapped IO devices) to IOBUS
92     always_comb begin
93         case(IOBUS_addr)
94             SWITCHES_AD: IOBUS_in = {16'b0,SWITCHES};
95             default:     IOBUS_in = 32'b0;    // default bus input to 0
96         endcase
97     end
98
99
100    // Connect Board output peripherals (Memory Mapped IO devices) to IOBUS
101    always_ff @ (posedge clk_50) begin
102        if(IOBUS_wr)
103            case(IOBUS_addr)
104                LEDS_AD: LEDS    <= IOBUS_out[15:0];
105                SSEG_AD: r_SSEG <= IOBUS_out[15:0];
106            endcase
107        end
108
109    endmodule

```

Listing 45: Visual testsim for Otter Wrapper OTTER_Wrapper_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/21/2026 10:09:19 AM
7  // Design Name:
8  // Module Name: OTTER_Wrapper_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //          ----> pair with cpe233 hw8 given testcode
20 ///////////////////////////////////////////////////////////////////
21
22
23 module OTTER_Wrapper_tb();
24
25     // ~~~~ imports ~~~~
26     import tb_utils_pkg::*;
27
28     // ~~~~ local vars ~~~~
29     // ~~ inputs ~~
30     logic          clk;
31     logic          reset;
32     logic          interrupt;
33     logic [15:0]   switches;
34     // ~~ outputs ~~
35     logic [15:0]   leds;
36     logic [7:0]    catho;
37     logic [3:0]    ano;
38
39     // ~~ testing ~~
40     testcase tc;
41     logic [31:0] tnum;
42
43     // ~~~~ module instances ~~~~
44     OTTER_Wrapper UUT(
45         .CLK(clk),           // 1'b I
46         .BTNL(interrupt),    // 1'b I
47         .BTNC(reset),        // 1'b I
48         .SWITCHES(switches), // 16'b I
49         .LEDS(leds),         // 16'b O
50         .CATHODES(catho),    // 8'b O
51         .ANODES(ano)         // 4'b O
52     );
53
54     // ~~~~ heartbeat (10ns) ~~~~
55     always begin
56         #5;
57         clk <= !clk;
58     end
59
60     // ~~~~ testing ~~~~
61     initial begin
62
63         // ~~ class instances ~~
64         tc = new();
65
66         // ~~ defaults ~~

```

```

67         clk          = 1'b1;
68         reset        = 1'b0;
69         interrupt     = 1'b0;
70         switches      = 16'h01;          // mem file demands SW[0] high to write 1 to mie bit
71         #3000;                          // enough to start LOOP in asm
72
73
74         // ~~ TC1 ~~
75         tc.new_test("First Interrupt");
76         tnum = tc.get_testnum();
77         interrupt   = 1'b1;
78         #500;
79         interrupt   = 1'b0;              // OneShot and intrr actives on falling edge
80         #2050;
81         tc.test_done();
82
83
84         // ~~ TC2 ~~
85         tc.new_test("Second Interrupt");
86         tnum = tc.get_testnum();
87         interrupt   = 1'b1;
88         #500;
89         interrupt   = 1'b0;              // OneShot and intrr actives on falling edge
90         #2000;
91         tc.test_done();
92     end
93
94 endmodule

```

Listing 46: Peripheral Bindings Basys3_Master.xdc

```

1 ## This file is a general .xdc for the Basys3 rev B board
2 ## To use it in a project:
3 ## - uncomment the lines corresponding to used pins
4 ## - rename the used ports (in each line, after get_ports) according to the top level signal
   names in the project
5
6 ## Clock signal
7 set_property PACKAGE_PIN W5 [get_ports CLK]
8 set_property IOSTANDARD LVCMOS33 [get_ports CLK]
9 create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK]
10
11 ## Switches
12 set_property PACKAGE_PIN V17 [get_ports {SWITCHES[0]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[0]}]
14 set_property PACKAGE_PIN V16 [get_ports {SWITCHES[1]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[1]}]
16 set_property PACKAGE_PIN W16 [get_ports {SWITCHES[2]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[2]}]
18 set_property PACKAGE_PIN W17 [get_ports {SWITCHES[3]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[3]}]
20 set_property PACKAGE_PIN W15 [get_ports {SWITCHES[4]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[4]}]
22 set_property PACKAGE_PIN V15 [get_ports {SWITCHES[5]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[5]}]
24 set_property PACKAGE_PIN W14 [get_ports {SWITCHES[6]}]
25 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[6]}]
26 set_property PACKAGE_PIN W13 [get_ports {SWITCHES[7]}]
27 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[7]}]
28 set_property PACKAGE_PIN V2 [get_ports {SWITCHES[8]}]
29 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[8]}]
30 set_property PACKAGE_PIN T3 [get_ports {SWITCHES[9]}]
31 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[9]}]
32 set_property PACKAGE_PIN T2 [get_ports {SWITCHES[10]}]
33 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[10]}]
34 set_property PACKAGE_PIN R3 [get_ports {SWITCHES[11]}]
35 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[11]}]
36 set_property PACKAGE_PIN W2 [get_ports {SWITCHES[12]}]
37 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[12]}]
38 set_property PACKAGE_PIN U1 [get_ports {SWITCHES[13]}]
39 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[13]}]
40 set_property PACKAGE_PIN T1 [get_ports {SWITCHES[14]}]
41 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[14]}]
42 set_property PACKAGE_PIN R2 [get_ports {SWITCHES[15]}]
43 set_property IOSTANDARD LVCMOS33 [get_ports {SWITCHES[15]}]
44
45
46 ## LEDs
47 set_property PACKAGE_PIN U16 [get_ports {LEDS[0]}]
48 set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[0]}]
49 set_property PACKAGE_PIN E19 [get_ports {LEDS[1]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[1]}]
51 set_property PACKAGE_PIN U19 [get_ports {LEDS[2]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[2]}]
53 set_property PACKAGE_PIN V19 [get_ports {LEDS[3]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[3]}]
55 set_property PACKAGE_PIN W18 [get_ports {LEDS[4]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[4]}]
57 set_property PACKAGE_PIN U15 [get_ports {LEDS[5]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[5]}]
59 set_property PACKAGE_PIN U14 [get_ports {LEDS[6]}]

```

```

60  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[6]}]
61 set_property PACKAGE_PIN V14 [get_ports {LEDS[7]}]
62  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[7]}]
63 set_property PACKAGE_PIN V13 [get_ports {LEDS[8]}]
64  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[8]}]
65 set_property PACKAGE_PIN V3 [get_ports {LEDS[9]}]
66  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[9]}]
67 set_property PACKAGE_PIN W3 [get_ports {LEDS[10]}]
68  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[10]}]
69 set_property PACKAGE_PIN U3 [get_ports {LEDS[11]}]
70  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[11]}]
71 set_property PACKAGE_PIN P3 [get_ports {LEDS[12]}]
72  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[12]}]
73 set_property PACKAGE_PIN N3 [get_ports {LEDS[13]}]
74  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[13]}]
75 set_property PACKAGE_PIN P1 [get_ports {LEDS[14]}]
76  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[14]}]
77 set_property PACKAGE_PIN L1 [get_ports {LEDS[15]}]
78  set_property IOSTANDARD LVCMOS33 [get_ports {LEDS[15]}]
79
80
81 ##7 segment display
82 set_property PACKAGE_PIN W7 [get_ports {CATHODES[6]}]
83  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[6]}]
84 set_property PACKAGE_PIN W6 [get_ports {CATHODES[5]}]
85  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[5]}]
86 set_property PACKAGE_PIN U8 [get_ports {CATHODES[4]}]
87  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[4]}]
88 set_property PACKAGE_PIN V8 [get_ports {CATHODES[3]}]
89  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[3]}]
90 set_property PACKAGE_PIN U5 [get_ports {CATHODES[2]}]
91  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[2]}]
92 set_property PACKAGE_PIN V5 [get_ports {CATHODES[1]}]
93  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[1]}]
94 set_property PACKAGE_PIN U7 [get_ports {CATHODES[0]}]
95  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[0]}]
96
97 set_property PACKAGE_PIN V7 [get_ports {CATHODES[7]}]
98  set_property IOSTANDARD LVCMOS33 [get_ports {CATHODES[7]}]
99
100 set_property PACKAGE_PIN U2 [get_ports {ANODES[0]}]
101  set_property IOSTANDARD LVCMOS33 [get_ports {ANODES[0]}]
102 set_property PACKAGE_PIN U4 [get_ports {ANODES[1]}]
103  set_property IOSTANDARD LVCMOS33 [get_ports {ANODES[1]}]
104 set_property PACKAGE_PIN V4 [get_ports {ANODES[2]}]
105  set_property IOSTANDARD LVCMOS33 [get_ports {ANODES[2]}]
106 set_property PACKAGE_PIN W4 [get_ports {ANODES[3]}]
107  set_property IOSTANDARD LVCMOS33 [get_ports {ANODES[3]}]
108
109
110 ##Buttons
111 set_property PACKAGE_PIN U18 [get_ports BTNC]
112  set_property IOSTANDARD LVCMOS33 [get_ports BTNC]
113 #set_property PACKAGE_PIN T18 [get_ports btnU]
114 # set_property IOSTANDARD LVCMOS33 [get_ports btnU]
115 set_property PACKAGE_PIN W19 [get_ports BTNL]
116  set_property IOSTANDARD LVCMOS33 [get_ports BTNL]
117 #set_property PACKAGE_PIN T17 [get_ports btnR]
118 #set_property IOSTANDARD LVCMOS33 [get_ports btnR]
119 #set_property PACKAGE_PIN U17 [get_ports btnD]
120 #set_property IOSTANDARD LVCMOS33 [get_ports btnD]
121
122
123
124 ##Pmod Header JA
125 ##Sch name = JA1
126 #set_property PACKAGE_PIN J1 [get_ports {JA[0]}]
127  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[0]}]
128 ##Sch name = JA2
129 #set_property PACKAGE_PIN L2 [get_ports {JA[1]}]
130  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[1]}]
131 ##Sch name = JA3
132 #set_property PACKAGE_PIN J2 [get_ports {JA[2]}]
133  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[2]}]
134 ##Sch name = JA4
135 #set_property PACKAGE_PIN G2 [get_ports {JA[3]}]
136  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[3]}]
137 ##Sch name = JA7
138 #set_property PACKAGE_PIN H1 [get_ports {JA[4]}]
139  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[4]}]
140 ##Sch name = JA8
141 #set_property PACKAGE_PIN K2 [get_ports {JA[5]}]
142  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[5]}]
143 ##Sch name = JA9
144 #set_property PACKAGE_PIN H2 [get_ports {JA[6]}]
145  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[6]}]
146 ##Sch name = JA10
147 #set_property PACKAGE_PIN G3 [get_ports {JA[7]}]
148  #set_property IOSTANDARD LVCMOS33 [get_ports {JA[7]}]
149
150

```

```

151
152 ##Pmod Header JB
153 ##Sch name = JB1
154 #set_property PACKAGE_PIN A14 [get_ports {JB[0]}]
155 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[0]}]
156 ##Sch name = JB2
157 #set_property PACKAGE_PIN A16 [get_ports {JB[1]}]
158 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[1]}]
159 ##Sch name = JB3
160 #set_property PACKAGE_PIN B15 [get_ports {JB[2]}]
161 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[2]}]
162 ##Sch name = JB4
163 #set_property PACKAGE_PIN B16 [get_ports {JB[3]}]
164 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[3]}]
165 ##Sch name = JB7
166 #set_property PACKAGE_PIN A15 [get_ports {JB[4]}]
167 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[4]}]
168 ##Sch name = JB8
169 #set_property PACKAGE_PIN A17 [get_ports {JB[5]}]
170 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[5]}]
171 ##Sch name = JB9
172 #set_property PACKAGE_PIN C15 [get_ports {JB[6]}]
173 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[6]}]
174 ##Sch name = JB10
175 #set_property PACKAGE_PIN C16 [get_ports {JB[7]}]
176 #set_property IOSTANDARD LVCMOS33 [get_ports {JB[7]}]
177
178
179
180 ##Pmod Header JC
181 ##Sch name = JC1
182 #set_property PACKAGE_PIN K17 [get_ports {JC[0]}]
183 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[0]}]
184 ##Sch name = JC2
185 #set_property PACKAGE_PIN M18 [get_ports {JC[1]}]
186 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[1]}]
187 ##Sch name = JC3
188 #set_property PACKAGE_PIN N17 [get_ports {JC[2]}]
189 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[2]}]
190 ##Sch name = JC4
191 #set_property PACKAGE_PIN P18 [get_ports {JC[3]}]
192 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[3]}]
193 ##Sch name = JC7
194 #set_property PACKAGE_PIN L17 [get_ports {JC[4]}]
195 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[4]}]
196 ##Sch name = JC8
197 #set_property PACKAGE_PIN M19 [get_ports {JC[5]}]
198 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[5]}]
199 ##Sch name = JC9
200 #set_property PACKAGE_PIN P17 [get_ports {JC[6]}]
201 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[6]}]
202 ##Sch name = JC10
203 #set_property PACKAGE_PIN R18 [get_ports {JC[7]}]
204 #set_property IOSTANDARD LVCMOS33 [get_ports {JC[7]}]
205
206
207 ##Pmod Header JXADC
208 ##Sch name = XA1_P
209 #set_property PACKAGE_PIN J3 [get_ports {JXADC[0]}]
210 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[0]}]
211 ##Sch name = XA2_P
212 #set_property PACKAGE_PIN L3 [get_ports {JXADC[1]}]
213 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[1]}]
214 ##Sch name = XA3_P
215 #set_property PACKAGE_PIN M2 [get_ports {JXADC[2]}]
216 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[2]}]
217 ##Sch name = XA4_P
218 #set_property PACKAGE_PIN N2 [get_ports {JXADC[3]}]
219 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[3]}]
220 ##Sch name = XA1_N
221 #set_property PACKAGE_PIN K3 [get_ports {JXADC[4]}]
222 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[4]}]
223 ##Sch name = XA2_N
224 #set_property PACKAGE_PIN M3 [get_ports {JXADC[5]}]
225 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[5]}]
226 ##Sch name = XA3_N
227 #set_property PACKAGE_PIN M1 [get_ports {JXADC[6]}]
228 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[6]}]
229 ##Sch name = XA4_N
230 #set_property PACKAGE_PIN N1 [get_ports {JXADC[7]}]
231 #set_property IOSTANDARD LVCMOS33 [get_ports {JXADC[7]}]
232
233
234
235 ##VGA Connector
236 #set_property PACKAGE_PIN G19 [get_ports {VGA_RGB[5]}]
237 #set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[5]}]
238 #set_property PACKAGE_PIN H19 [get_ports {VGA_RGB[5]}]
239 #set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[5]}]
240 #set_property PACKAGE_PIN J19 [get_ports {VGA_RGB[6]}]
241 #set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[6]}]

```

```

242 #set_property PACKAGE_PIN N19 [get_ports {VGA_RGB[7]}]
243 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[7]}]
244 #set_property PACKAGE_PIN N18 [get_ports {VGA_RGB[0]}]
245 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[0]}]
246 #set_property PACKAGE_PIN L18 [get_ports {VGA_RGB[0]}]
247 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[0]}]
248 #set_property PACKAGE_PIN K18 [get_ports {VGA_RGB[1]}]
249 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[1]}]
250 #set_property PACKAGE_PIN J18 [get_ports {VGA_RGB[1]}]
251 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[1]}]
252 #set_property PACKAGE_PIN J17 [get_ports {VGA_RGB[2]}]
253 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[2]}]
254 #set_property PACKAGE_PIN H17 [get_ports {VGA_RGB[2]}]
255 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[2]}]
256 #set_property PACKAGE_PIN G17 [get_ports {VGA_RGB[3]}]
257 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[3]}]
258 #set_property PACKAGE_PIN D17 [get_ports {VGA_RGB[4]}]
259 # set_property IOSTANDARD LVCMOS33 [get_ports {VGA_RGB[4]}]
260 #set_property PACKAGE_PIN P19 [get_ports VGA_HS]
261 # set_property IOSTANDARD LVCMOS33 [get_ports VGA_HS]
262 #set_property PACKAGE_PIN R19 [get_ports VGA_VS]
263 # set_property IOSTANDARD LVCMOS33 [get_ports VGA_VS]
264
265
266 ##USB-RS232 Interface
267 #set_property PACKAGE_PIN B18 [get_ports RX]
268 # set_property IOSTANDARD LVCMOS33 [get_ports RX]
269 #set_property PACKAGE_PIN A18 [get_ports TX]
270 # set_property IOSTANDARD LVCMOS33 [get_ports TX]
271
272
273 ##USB HID (PS/2)
274 #set_property PACKAGE_PIN C17 [get_ports PS2Clk]
275 # set_property IOSTANDARD LVCMOS33 [get_ports PS2Clk]
276 # set_property PULLUP true [get_ports PS2Clk]
277 #set_property PACKAGE_PIN B17 [get_ports PS2Data]
278 # set_property IOSTANDARD LVCMOS33 [get_ports PS2Data]
279 # set_property PULLUP true [get_ports PS2Data]
280
281
282 ##Quad SPI Flash
283 ##Note that CCLK_0 cannot be placed in 7 series devices. You can access it using the
284 ##STARTUPE2 primitive.
285 #set_property PACKAGE_PIN D18 [get_ports {QspiDB[0]}]
286 #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[0]}]
287 #set_property PACKAGE_PIN D19 [get_ports {QspiDB[1]}]
288 #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[1]}]
289 #set_property PACKAGE_PIN G18 [get_ports {QspiDB[2]}]
290 #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[2]}]
291 #set_property PACKAGE_PIN F18 [get_ports {QspiDB[3]}]
292 #set_property IOSTANDARD LVCMOS33 [get_ports {QspiDB[3]}]
293 #set_property PACKAGE_PIN K19 [get_ports QspiCSn]
294 #set_property IOSTANDARD LVCMOS33 [get_ports QspiCSn]

```

0.3.14 Interrupts

Listing 47: Debounce OneShot™ OneShot.sv

```
1 `timescale 1ns / 1ps
2 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
3 // Company:
4 // Engineer:
5 //
6 // Create Date: 08/21/2026 03:38:23 PM
7 // Design Name:
8 // Module Name: OneShot
9 // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21
22
23 module OneShot
24 #(parameter DUR_CNT = 80)
25 (
26     input logic    IN,                // active on falling edge
27     input logic    CLK,
28     output logic    OUT
29 );
30
31     // ~~~~ local params ~~~~
32     // ~~ state ~~
33     typedef enum logic [1:0] {
34         INACTIVE,
35         PULSE,
36         INC
37     } e_state;
38     e_state curr_state, next_state;
39
40     // ~~ timing ~~
41     localparam INC_CNT = 10;
42     logic [31:0] t_elap_ns, next_t_elap_ns;
43
44     // ~~ button processing ~~
45     logic in_meta;
46     logic in_sync;
47     logic in_sync_d;
48     logic button_press;
49
50     // ~~~~ synch for falling button : two-flop ~~~~
51     /* Note : we introduce some delay in waiting for the CLK and
52     //         flops BUT we avoid (re)assigning the same logic synch
53     //         and asynch for the button (as before).
54     */
55     always_ff @(posedge CLK) begin
56         in_meta      <= IN;
57         in_sync      <= in_meta;
58         in_sync_d    <= in_sync;
59     end
60     assign button_press = in_sync_d && !in_sync; // HIGH during falling edge
61
62     // ~~~~ synch state progression ~~~~
63     always_ff @(posedge CLK) begin
64         curr_state <= next_state;
65         t_elap_ns <= next_t_elap_ns;
66     end
67
68     always_comb begin
69
70         // ~~ defaults ~~
71         next_state      = curr_state;
72         next_t_elap_ns  = t_elap_ns;
73         OUT              = 1'b0;
74
75         case(curr_state)
76
77             INACTIVE: begin
78                 next_t_elap_ns = 0;
79                 if(button_press) begin next_state = PULSE; end
80             end
81
82             PULSE: begin
83
84                 OUT = 1'b1;
85
86                 if(t_elap_ns >= (DUR_CNT - INC_CNT)) begin
```

```

88         next_state      = INACTIVE;
89         next_t_elap_ns   = 0;
90     end
91
92     else begin
93         next_state      = PULSE;
94         next_t_elap_ns   = t_elap_ns + INC_CNT;
95     end
96 end
97
98 default:    begin
99     OUT      = 1'b0;
100    next_state = INACTIVE;
101    next_t_elap_ns = 0;
102 end
103 endcase
104 end
105
106 endmodule

```

Listing 48: Testsim for debounce OneShot™ OneShot_tb.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/21/2026 06:24:25 PM
7  // Design Name:
8  // Module Name: OneShot_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 ///////////////////////////////////////////////////////////////////
21
22
23 module OneShot_tb();
24
25     // ~~~~ local vars ~~~~
26
27     // ~~ input ~~
28     logic          in, clk;
29
30     // ~~ output ~~
31     logic          out;
32
33     // ~~ testing ~~
34     logic [31:0]    testcase;
35     localparam      p_dur = 80;
36
37
38
39     // ~~~~ module instances ~~~~
40     OneShot #(p_dur) UUT(
41         .IN(in),           // 1'b I
42         .CLK(clk),         // 1'b I
43         .OUT(out)          // 1'b 0
44     );
45
46
47
48     // ~~~~ heartbeat (10ns) ~~~~
49     always begin
50         #5;
51         clk <= !clk;
52     end
53
54     // ~~~~ testing ~~~~
55     initial begin
56         testcase      = 0;
57         clk            = 1'b1;
58         in             = 1'b0;
59
60         #100;          // get the poo out
61
62
63         // ~~ TC1 : No button press = no shooting ~~
64         testcase++;
65         $display("TC %d", testcase);
66         for(int i = 0; i < 30; i++) begin
67             assert(out == 1'b0) else $error("OUT HIGH but expected LOW");
68         end
69

```

```

70
71 // ~~ TC2 : mid-clk cycle button press ~~
72 testcase++;
73 $display("TC %d", testcase);
74 in = 1'b1;
75 #1;
76 in = 1'b0;
77 #29; // wait the 30ns for button processing delay
78
79 for(int i = 0; i < p_dur; i++) begin
80     #1;
81     $display("%d", i);
82     assert(out === 1'b1) else $error("OUT LOW but expected HIGH");
83
84 end
85 #1;
86 assert(out === 1'b0) else $error("OUT HIGH but expected LOW");
87
88
89
90 // ~~ TC3 : small blip (not on clk) is ignored as noise) ~~
91 testcase++;
92 $display("TC %d", testcase);
93
94 // ~ first button ~
95 in = 1'b1;
96 #1;
97 in = 1'b0;
98 #29; // wait the 30ns for button processing delay
99
100 // ~ first half of pulse is on ~
101 for(int i = 0; i < 40; i++) begin
102     #1;
103     $display("%d", i);
104     assert(out === 1'b0) else $error("OUT HIGH but expected LOW");
105 end
106 #9; //readjustment to clk
107
108
109 // ~~ TC4 : retry button during pulse firing ~~
110 testcase++;
111 $display("TC %d", testcase);
112
113 // ~ first button ~
114 in = 1'b1;
115 #1;
116 in = 1'b0;
117 #29; // wait the 30ns for button processing delay
118
119 // ~ first half of pulse is on ~
120 for(int i = 0; i < 14; i++) begin
121     #1;
122     $display("%d", i);
123     assert(out === 1'b1) else $error("OUT LOW but expected HIGH");
124 end
125 // ~ second half of pulse is still on (regardless of button) ~
126 for(int i = 0; i < 14; i++) begin
127     // ~ second pulse ~
128     in = 1'b1;
129     #2;
130
131     $display("%d", i);
132     assert(out === 1'b1) else $error("OUT LOW but expected HIGH");
133     in = 1'b0;
134     #1;
135 end
136 #24;
137 assert(out === 1'b1) else $error("OUT LOW but expected HIGH");
138 #1;
139 assert(out === 1'b0) else $error("OUT HIGH but expected LOW");
140 #200;
141
142
143 // ~~ TC4 : long press ~~
144 testcase++;
145 $display("TC %d", testcase);
146 in = 1'b1;
147 #300;
148 in = 1'b0;
149 #30; // recognize button propagation
150 for(int i = 0; i < 80; i++) begin
151     $display("%d", i);
152     assert(out === 1'b1) else $error("OUT LOW but expected HIGH");
153     #1;
154 end
155 #80;
156 end
157
158
159 endmodule

```

Listing 49: Control Status Register ControlStatusRegister.sv

```

1  `timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer: WhisperingRock
5  //
6  // Create Date: 08/24/2026 12:36:46 PM
7  // Design Name:
8  // Module Name: ControlStatusRegister
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description: Special purpose registers
13 //               + logic to save PC
14 //               + disable interrupts during interrupt
15 //
16 //               The 7th (and sys) type of instruction
17 // Dependencies:
18 //
19 // Revision:
20 // Revision 0.01 - File Created
21 // Additional Comments:
22 //
23 ///////////////////////////////////////////////////////////////////
24
25
26 module ControlStatusRegister(
27     input logic          CLK,
28     input logic          RST,          // reset
29     input logic          INTR_TAKEN,   // request : go into interrupt
30     input logic          INTR_RET,     // request : return from ISR subroutine
31     input logic          WR_EN,        // request : overwrite contents in csr[ADDR]
32     input logic [11:0]   ADDR,         // requested register addresses
33     input logic [31:0]   PC,           // program counter
34     input logic [31:0]   WD,           // data to overwrite csr[ADDR]
35     output logic         MSTATUS_MIE,  // Machine Interrupt Status Reg - Machine Interrupt Enab
36     output logic [31:0]  MTVEC,        // Machine Trap Vector : instr for PC for particular ISR
37     output logic [31:0]  MEPC,         // Machine Exception PC : stores/restores PC around inte
38     output logic [31:0]  RD            // data contents of csr[ADDR]
39 );
40
41 // ~~~~ local allocs, params, and wires ~~~~
42
43 // DONT DO THIS! THIS REGISTER IS SPARSE AND IS GOING TO LARGELY REMAIN THAT WAY
44 //logic [31:0]          csr [0:4095];
45
46 logic [31:0]           mstatus;
47 localparam logic [11:0] mstatus_addr    = 12'h300;
48 localparam int         mstatus_mie_bit  = 3;
49 localparam int         mstatus_mpie_bit = 7;
50
51 logic [31:0]           mtvec;
52 localparam logic [11:0] mtvec_addr      = 12'h305;
53
54 logic [31:0]           mepc;
55 localparam logic [11:0] mepc_addr       = 12'h341;
56
57
58
59 // ~~~~ initial conditions ~~~~
60 initial begin
61     mstatus      = 32'h0000_0000;    // disable interrupts by default
62     mtvec         = 32'h0000_0000;    // clear intr vecter
63     mepc          = 32'h0000_0000;    // clear PC
64 end
65
66
67 // ~~~~ combinational logic ~~
68 always_comb begin
69
70     // ~~ assign port outputs ~~
71     MSTATUS_MIE = mstatus[mstatus_mie_bit];
72     MTVEC        = mtvec;
73     MEPC         = mepc;
74
75     case (ADDR)
76         mstatus_addr: begin RD = mstatus; end
77
78         mtvec_addr:   begin RD = mtvec; end
79
80         mepc_addr:    begin RD = mepc; end
81
82         default:      begin RD = 32'hXXXX_XXXX; end
83     endcase
84 end
85
86
87
88 // ~~~~ synch logic ~~~~
89 always_ff @(posedge CLK or posedge RST) begin

```

```

90
91     // ~~ priority 0 : reset clears and disables intrr ~~
92     if(RST == 1'b1) begin
93         mstatus               <= 32'h0000_0000;           // disable interrupts by default
94         mtvec                 <= 32'h0000_0000;           // clear intr vector
95         mepc                  <= 32'h0000_0000;           // clear PC
96     end
97
98     // ~~ priority 1: intrr requested ~~
99     else if(INTR_TAKEN == 1'b1) begin
100         mepc                  <= PC;                       // save non-intrr PC
101         mstatus[mstatus_mpie_bit] <= mstatus[mstatus_mpie_bit]; // store intr setting
102         mstatus[mstatus_mie_bit]  <= 1'b0;                 // disable other intr during this
103     end
104
105
106     // ~~ priority 2 : return from intrr requested ~~
107     else if(INTR_RET == 1'b1) begin
108         mstatus[mstatus_mpie_bit] <= mstatus[mstatus_mpie_bit]; // restore intrr setting
109         mstatus[mstatus_mpie_bit] <= 1'b0;                 // clear MPIE for sanitization
110     end
111
112     // ~~ priority 3 : update csr register contents ~~
113     else if(WR_EN == 1'b1) begin
114         case(ADDR)
115             mstatus_addr:    begin mstatus <= WD; end           // TODO : mask this guy??
116
117             mtvec_addr:      begin mtvec <= WD; end
118
119             mepc_addr:       begin mepc <= WD; end
120
121             default:        begin end // do nothing
122         endcase
123     end
124
125 end
126
127
128 endmodule

```

Listing 50: Testsim for Control Status Register ControlStatusRegister_tb.sv

```

1  'timescale 1ns / 1ps
2  ///////////////////////////////////////////////////////////////////
3  // Company:
4  // Engineer:
5  //
6  // Create Date: 08/24/2026 02:33:08 PM
7  // Design Name:
8  // Module Name: ControlStatusRegister_tb
9  // Project Name:
10 // Target Devices:
11 // Tool Versions:
12 // Description:
13 //
14 // Dependencies:
15 //
16 // Revision:
17 // Revision 0.01 - File Created
18 // Additional Comments:
19 //
20 //
21 //     ASSUMPTIONS:
22 //         - CSR is not checking whether intr is enabled, only storing
23 //           the mie bit. The CUFSM is gatekeeping the start of intr
24 ///////////////////////////////////////////////////////////////////
25
26
27 module ControlStatusRegister_tb();
28
29     // ~~~~ imports ~~~~
30     import tb_utils_pkg::*;
31
32
33     // ~~~~ local wires/params ~~~~
34
35     // ~~ inputs ~~
36     logic        clk;
37     logic        reset;
38     logic        intr_taken;
39     logic        intr_return;
40     logic        wr_en;
41     logic [11:0] csr_addr;
42     logic [31:0] pc;
43     logic [31:0] wd_word;
44
45     // ~~ outputs ~~
46     logic        mie;
47     logic [31:0] mtvec;
48     logic [31:0] mepc;
49     logic [31:0] rd_word;

```

```

50
51 // ~~ consts ~~
52 localparam [11:0] mstatus_addr = 12'h300;
53 const int mstatus_mie_bit = 3;
54 const int mie_word = 32'h0000_0000 | (1'b1 << mstatus_mie_bit);
55 const int mstatus_mpie_bit = 7;
56 const int mpie_word = 32'h0000_0000 | (1'b1 << mstatus_mpie_bit);
57 localparam [11:0] mtvec_addr = 12'h305;
58 localparam [11:0] mepc_addr = 12'h341;
59
60 // ~~ testing ~~
61 testcase tc;
62 logic [31:0] tnum;
63
64 // ~~~~ module instances ~~~~
65 ControlStatusRegister UUT(
66     .CLK(clk), // 1'b I
67     .RST(reset), // 1'b I
68     .INTR_TAKEN(intr_taken), // 1'b I
69     .INTR_RET(intr_return), // 1'b I
70     .WR_EN(wr_en), // 1'b I
71     .ADDR(csr_addr), // 12'b I
72     .PC(pc), // 32'b I
73     .WD(wd_word), // 32'b I
74     .MSTATUS_MIE(mie), // 1'b 0
75     .MTVEC(mtvec), // 32'b 0
76     .MEPC(mepc), // 32'b 0
77     .RD(rd_word) // 32'b 0
78 );
79
80 // ~~~~ heartbeat (10ns) ~~~~
81 always begin
82     #5;
83     clk <= !clk;
84 end
85
86
87 // ~~~~ testing ~~~~
88 initial begin
89
90     // ~~ class instances ~~
91     tc = new();
92
93     // ~~ defaults ~~
94     clk = 1'b0;
95     reset = 1'b0;
96     intr_taken = 1'b0;
97     intr_return = 1'b0;
98     wr_en = 1'b0;
99     csr_addr = 12'h000;
100     pc = 32'h0000_0000;
101     wd_word = 32'h0000_0000;
102     #25;
103
104     // ~~ TC1 ~~
105     tc.new_test("CSR initialized data");
106     tnum = tc.get_testnum();
107     csr_addr = mstatus_addr;
108     #8;
109     assert(rd_word === 32'h0000_0000) else tc.err("MSTATUS reg was not init to zero");
110     assert(mtvec === 32'h0000_0000) else tc.err("MTVEC reg was not init to zero");
111     assert(mepc === 32'h0000_0000) else tc.err("MEPC reg was not init to zero");
112     #2;
113     tc.test_done();
114
115
116     // ~~ TC2 ~~
117     tc.new_test("enter INTRPT state");
118     tnum = tc.get_testnum();
119
120     // enable intrr
121     csr_addr = mstatus_addr;
122     wd_word = mie_word;
123     #10;
124     wr_en = 1'b1;
125     #10;
126     wr_en = 1'b0;
127     assert(rd_word === mie_word) else tc.err("MEI not set");
128
129     // place ISR addr into mtvec
130     csr_addr = mtvec_addr;
131     wd_word = 32'hDEAD_BEEF;
132     #10;
133     wr_en = 1'b1;
134     #10;
135     wr_en = 1'b0;
136     assert(mtvec === 32'hDEAD_BEEF) else tc.err("MTVEC not set");
137
138     // enter INTRPT
139     csr_addr = mstatus_addr;
140     pc = 32'h1234_ABCD;

```

```

141         intr_taken = 1'b1;
142         #8;
143         assert(rd_word === mpie_word)           else tc.err("MPIE swap didn't occur");
144         assert(mtvec === 32'hDEAD_BEEF)         else tc.err("MTVEC changed");
145         assert(mepc === 32'h1234_ABCD)         else tc.err("MEPC reg not storing pc");
146         #2;
147         intr_taken = 1'b0;
148         #10;
149         tc.test_done();
150
151         // ~~ TC3 ~~
152         tc.new_test("exiting interrupt ISR");
153         tnum = tc.get_testnum();
154         csr_addr = mstatus_addr;
155         intr_return = 1'b1;
156         #8;
157         assert(rd_word === mie_word)           else tc.err("MPIE swap didn't occur");
158         assert(mie === 1'b1)                   else tc.err("MIE didn't restore");
159         assert(mepc === 32'h1234_ABCD)         else tc.err("MEPC did not protect pc");
160         #2;
161         intr_return = 1'b0;
162         #10;
163         assert(rd_word === mie_word)           else tc.err("MPIE swap didn't occur");
164         assert(mie === 1'b1)                   else tc.err("MIE didn't restore");
165         assert(mepc === 32'h1234_ABCD)         else tc.err("MEPC did not protect pc");
166         tc.test_done();
167
168         // ~~ TC4 ~~
169         tc.new_test("reset");
170         tnum = tc.get_testnum();
171         reset = 1'b1;
172         intr_taken = 1'b0;
173         intr_return = 1'b0;
174         wr_en = 1'b0;
175         csr_addr = mstatus_addr;
176         #8;
177         assert(rd_word === 32'h0000_0000)       else tc.err("MSTATUS didn't reset");
178         assert(mie === 1'b0)                   else tc.err("MIE didn't reset");
179         assert(mtvec === 32'h0000_0000)       else tc.err("MTVEC didn't reset");
180         assert(mepc === 32'h0000_0000)       else tc.err("MEPC didn't reset");
181         #2;
182         reset = 1'b0;
183         tc.test_done();
184
185     end
186 endmodule

```

Listing 51: Hardware Interrupt Test InterruptTestProg.asm

```

1 #####
2 # Main initializes registers, including CSR, clears the 7 seg display, checks
3 # the SWITCHES to either enable or disable interrupts (SW0), and checks a
4 # register (s1) acting as an interrupt flag. If the flag is set, it will update
5 # the 7 seg display with the updated interrupt count which is saved in the data
6 # segment to save register use.
7 #
8 # When changing any CSR register (MSTATUS, MTVEC) the register is read after
9 # writing and verified to change correctly. If a value doesn't match, an error
10 # code is set (s3) and the program goes to a fail loop that sets 7 Seg to
11 # 0xFFxx with xx being the error code (listed below).
12 #
13 # The ISR sets the interrupt flag register (s1). The current interrupt count
14 # is read from the data segment, incremented, and written back. MSTATUS is also
15 # read to verify the MIE pending bit is set accordingly. If an error is
16 # detected an interrupt error flag register (s2) is set.
17 #
18 # Fail Codes (s3)
19 # 1 - Error setting MTVEC
20 # 2 - Error setting MSTATUS
21 # 3 - Error clearing MSTATUS
22 # 4 - Error with MSTATUS in the ISR
23 #####
24 # Register Use Table
25 # s0 : MMIO
26 # s1 : Interrupt Flag
27 # s2 : Interrupt error Flag
28 # s3 : Fail Code
29 #####
30 .equ MMIO,      0x11000000      # MMIO address and offsets
31 .equ LEDS,      0x20
32 .equ SEV_SEG,   0x40
33 .equ STACK,     0x10000        # stack grows downward
34 .equ INT_EN,    8              # enable interrupts MSTATUS = 0x08
35
36 .data
37 INTR_COUNT: .half 0            # keep interrupt count in data segment
38
39 .text
40 INIT:
41     li    sp, STACK            # setup sp
42     li    s0, MMIO             # setup MMIO pointer

```

```

43  la    t0, INTR_COUNT
44  sh    x0, 0(t0)          # clear interrupt count
45  la    t0, ISR
46  csrrw x0, mtvec, t0      # setup ISR address
47  addi  t1, x0, 0
48  csrrw t1, mtvec, t0      # read mtvec
49  addi  s3, x0, 1          # ERROR Code 1
50  bne   t0, t1, FAIL       # check mtvec
51  addi  s1, x0, 0          # clear interrupt flag
52  addi  s2, x0, 0          # clear interrupt error flag
53  sw    x0, SEV_SEG(s0)    # clear 7 seg
54
55  LOOP:
56  lw    a0, 0(s0)          # read SWITCHES
57  andi  a0, a0, 0x01       # mask SWITCH 0
58  sw    a0, LEDS(s0)       # set LEDs
59  call  SET_INT            # subroutine to enable or disable interrupts
60  beqz  a0, NO_ERROR       # return 0 if no error
61  add   s3, a0, x0         # copy error code
62  j     FAIL
63
64  NO_ERROR:
65  beq   s1, x0, LOOP       # check for interrupt
66  la    t0, INTR_COUNT
67  lhu   t0, 0(t0)          # not zero, so read new interrupt
68  sh    t0, SEV_SEG(s0)    # update 7SEG with new count
69  addi  s1, x0, 0          # clear interrupt flag
70  beqz  s2, LOOP           # check interrupt error flag
71  addi  s3, x0, 4          # set error code for mstatus interrupt
72
73  FAIL:
74  li    t0, INT_EN         # Fail State
75  csrrc x0, mstatus, t0    # disable interrupts
76  li    t0, 0xFF00
77  or    s3, s3, t0         # ERROR Code
78  sw    s3, SEV_SEG(s0)    # display ERROR on 7 Seg
79
80  STOP:
81  j     STOP
82
83  #####
84  # Set/Clear Interrupts Subroutine - Subroutine to set or clear MIE bit of
85  #   MSTATUS. After updating MSTATUS, register is read back and checked. If the
86  #   values don't match, an error code is returned
87  #
88  # a0: 1 - enable, 0 - disable
89  # return a0: error code (0 = no error)
90  #####
91
92  SET_INT:
93  li    t0, INT_EN
94  beqz  a0, SET_INT_DIS    # check for enable / disable MIE
95  addi  a0, x0, 0          # clear return value (error code)
96  csrrs x0, mstatus, t0    # enable interrupts
97  addi  t1, x0, 0
98  csrrc t1, mstatus, x0    # read mstatus
99  and   t1, t1, t0         # mask MIE bit
100  beq   t0, t1, SET_INT_RET # check MIE bit for error
101  addi  a0, x0, 2          # set error code return value
102  ret
103
104  SET_INT_DIS:
105  addi  a0, x0, 0          # clear return value (error code)
106  csrrc x0, mstatus, t0    # disable interrupts
107  addi  t1, x0, 0
108  csrrs t1, mstatus, x0    # read mstatus
109  and   t1, t1, t0         # mask MIE bit
110  beqz  t1, SET_INT_RET    # check MIE bit for error
111  addi  a0, x0, 3          # set error code return value
112
113  SET_INT_RET:
114  ret
115
116  #####
117  # ISR - Sets interrupt flag (s1) and increments count saved in saved in data
118  #   segment. Also checks MPIE and MIE bit of MSTATUS and sets interrupt error
119  #   registers accordingly
120  #####
121
122  ISR:
123  addi  sp, sp, -8          # push t1, t2 to stack
124  sw    t1, 4(sp)
125  sw    t2, 0(sp)
126  addi  s1, x0, 1          # set interrupt flag
127  la    t1, INTR_COUNT
128  lhu   t2, 0(t1)          # read current interrupt count
129  addi  t2, t2, 1          # increment interrupt count
130  sh    t2, 0(t1)          # save new interrupt count
131  li    t2, 0x80           # MPIE and MIE bit mask
132  csrrs t1, mstatus, x0    # read mstatus
133  beq   t1, t2, ISR_RET

```

```
134     addi    s2, x0, 1           # set interrupt error flag
135
136 ISR_RET:
137     lw      t2, 0(sp)           # pop t1, t2 from stack
138     lw      t1, 4(sp)
139     addi    sp, sp, 8
140     mret
```